UNINASSAU

Ciência da Computação | Turma 8NA | 2026.2

RotaCerta
Documento técnico cumulativo
Sprints 01 a 11 | Grupo 04

Álvaro Jordão - 01748200

Arthur Sales - 01593811

Vinícius Trigueiro - 01794959

William Coelho de Morais - 01263977

Guilherme de Luna Barros - 01615925

https://github.com/alvccpj/rota-certa

Consolidação em 10/11/2026. O conteúdo das Sprints 01 a 10 permanece nas páginas 2 a 169. A Sprint 11 começa na página 170, com os manuais completos, a documentação final e a revisão de código. Onde houver diferença, prevalece a parte mais recente.

Guia de leitura da Parte XI

Conteúdo	Página inicial
Registro da Sprint 11	170
Manual do Usuário	174
Manual Técnico	189
Documentação final consolidada	198
Revisão de código	205

Versão 0.11.0-rc.1. Prazo: 21/11/2026 às 23h59, horário de Recife. Envio único no Teams pelo Scrum Master Álvaro Jordão, após revisão da equipe.

Apresentação

Este documento reúne as entregas das Sprints 01 a 07 do projeto RotaCerta. As seis primeiras partes registram planejamento, arquitetura, estrutura funcional, os módulos de pedidos e entregas, roteirização e desempenho e o painel operacional. A Parte VII consolida o sistema com relatórios pesquisáveis, filtros combináveis, exportação CSV e histórico operacional.

As Partes I a VI mantêm o conteúdo entregue anteriormente. A Parte VII demonstra a evolução prática da Sprint 07, com dados persistidos, integração entre os módulos, evidências de execução e validações técnicas. Todas as telas e respostas mostradas nas Partes III a VII foram capturadas do sistema em funcionamento.

Sumário

Parte I Sprint 01 Planejamento do projeto

1 Identificação da equipe

2 Tema, problema, objetivos e público alvo

3 Requisitos funcionais e não funcionais

4 Casos de uso e Product Backlog

5 Cronograma semanal revisado

Parte II Sprint 02 Arquitetura e modelagem

6 Arquitetura do sistema

7 Diagrama de classes

8 Modelo Entidade Relacionamento

9 Modelo relacional

10 Protótipos das telas

11 Banco de dados e projeto no GitHub

12 Avaliação sequencial e paralela

13 Aderência às orientações da disciplina

Parte III Sprint 03 Estrutura inicial funcionando

14 Estrutura implementada

15 Banco de dados conectado

16 Login funcional

17 Cadastro de usuários

18 Controle de perfis

19 CRUD principal: pedidos

20 Execução local

21 Testes automatizados

22 Ajustes no planejamento e na modelagem

23 Repositório GitHub

24 Dificuldades encontradas e próximos passos

Parte IV Sprint 04 Primeiro módulo completo

25 Módulo implementado: Pedidos e entregas

26 Evidências do módulo funcionando

27 Persistência de dados

28 Validações

29 Mensagens de erro

30 Navegação entre as telas

31 Commits organizados e repositório

32 Ajustes no planejamento, na arquitetura e na modelagem

33 Testes automatizados

34 Dificuldades encontradas e próximos passos

Parte V Sprint 05 Segundo módulo funcionando

35 Módulo implementado: Roteirização e desempenho

36 Evidências das funcionalidades

37 Integração com o banco de dados

38 Regras de negócio

39 Núcleo de otimização e paralelização com CUDA

40 Comparação sequencial, paralela e GPU

41 Testes das funcionalidades

42 Bugs encontrados e correções

43 Ajustes no planejamento, na arquitetura e na modelagem

44 Repositório GitHub

45 Dificuldades encontradas e próximos passos

Parte VI Sprint 06 Aprimoramento do sistema

46 Terceiro módulo: Painel operacional e relatórios

47 Integração entre os módulos

48 Evidências das funcionalidades

49 Melhorias na interface e na navegação

50 Testes e qualidade técnica

51 Ajustes no planejamento e na arquitetura

52 Repositório GitHub

53 Dificuldades encontradas e próximos passos

54 Situação da Sprint 06

Parte VII Sprint 07 Sistema quase completo

55 Dashboard e acompanhamento operacional

56 Relatórios desenvolvidos

57 Pesquisas e filtros

58 Exportação CSV
59 Logs e histórico de operações
60 Integração com os módulos anteriores
61 Testes e qualidade técnica
62 Ajustes no planejamento e na arquitetura
63 Repositório GitHub
64 Dificuldades encontradas e próximos passos
65 Situação da Sprint 07

Parte I Sprint 01 Planejamento do projeto

1 Identificação da equipe

Integrante	Matrícula	Responsabilidade inicial
Álvaro Jordão	01748200	Scrum Master e Product Owner; organização, requisitos e priorização
Arthur Sales	01593811	Backend e núcleo de otimização e paralelização
Vinícius Trigueiro	01794959	Frontend, banco de dados e documentação
William Coelho de Morais	01263977	Desenvolvimento e testes
Guilherme de Luna Barros	01615925	Desenvolvimento e validação do sistema

Projeto RotaCerta. Turma 8NA. As responsabilidades organizam o trabalho, mas todos os integrantes participam das decisões, implementação, testes e documentação.

2 Tema, problema, objetivos e público alvo

Tema. Logística e otimização de processos para pequenos negócios com operação própria de entrega.

Problema. Farmácias, mercados, restaurantes e pequenas distribuidoras frequentemente planejam entregas de forma manual. Essa prática aumenta a distância percorrida, o consumo de combustível e a possibilidade de atraso quando o volume de pedidos cresce.

Objetivo geral. Desenvolver um sistema web que distribua pedidos entre entregadores e determine uma sequência de paradas que reduza a distância e o tempo total das rotas.

Resultados esperados. Centralizar pedidos e entregadores, reduzir distância e tempo em relação à alocação manual, calcular rotas para vários entregadores e disponibilizar uma interface responsiva para a operação diária.

Público alvo. Administradores, atendentes e entregadores de pequenos negócios locais. Os clientes finais são beneficiados por entregas mais previsíveis.

3 Requisitos funcionais e não funcionais

Requisitos funcionais

ID	Descrição
RF01	Cadastrar e autenticar usuários com perfis de administrador, atendente e entregador.
RF02	Cadastrar pedidos com cliente, endereço, prioridade e horário desejado.
RF03	Cadastrar entregadores com disponibilidade e capacidade de carga.
RF04	Gerar automaticamente uma rota otimizada para cada entregador.
RF05	Atribuir pedidos a entregadores de forma manual ou automática.
RF06	Exibir a rota como lista ordenada de paradas e mapa.

ID	Descrição
RF07	Atualizar o status do pedido entre pendente, atribuído, em rota e entregue.
RF08	Manter o histórico de entregas realizadas.
RF09	Gerar relatório com tempo, distância e comparação da otimização.
RF10	Disponibilizar painel administrativo da operação do dia.

Requisitos não funcionais

ID	Categoria	Descrição
RNF01	Desempenho	Executar o cálculo em tempo aceitável para vários entregadores e permitir processamento paralelo.
RNF02	Segurança	Autenticar usuários, autorizar por perfil e armazenar senhas com hash seguro.
RNF03	Usabilidade	Manter a interface simples para usuários com diferentes níveis de letramento técnico.
RNF04	Portabilidade	Funcionar em navegadores e adaptar o layout para celulares.
RNF05	Escalabilidade	Suportar o crescimento de pedidos e entregadores sem degradação significativa.
RNF06	Confiabilidade	Preservar pedidos, rotas e histórico com integridade referencial.
RNF07	Manutenibilidade	Manter código organizado, documentado, testável e versionado.

4 Casos de uso e Product Backlog

ID	Ator	Caso de uso
UC01	Atendente ou administrador	Cadastrar pedido
UC02	Administrador	Cadastrar entregador
UC03	Administrador	Gerar rotas para os pedidos do dia
UC04	Entregador	Visualizar a rota atribuída
UC05	Entregador	Atualizar o status da entrega
UC06	Administrador	Consultar métricas das rotas
UC07	Todos os perfis	Autenticar-se no sistema

Product Backlog inicial

Alta prioridade. Autenticação e perfis, CRUD de pedidos, CRUD de entregadores, banco de dados e motor sequencial com vizinho mais próximo e 2-opt.

Média prioridade. Visualização das rotas, atualização de status e paralelização para múltiplos entregadores.

Evolução. Relatórios, painel consolidado, notificações e avaliação futura de C++ com OpenMP ou CUDA.

5 Cronograma semanal revisado

O cronograma abaixo substituiu, na Sprint 02, a periodicidade quinzenal apresentada inicialmente. A Sprint 02 manteve o prazo excepcional de 19 de setembro comunicado pela professora; as etapas seguintes usam ciclos semanais. A versão vigente, com os ajustes das Sprints 03 e 04, está na seção 32.

Sprint	Período	Entrega principal
1	03/09 a 05/09	Planejamento, requisitos e backlog
2	06/09 a 19/09	Arquitetura, modelos, protótipos, banco e estrutura do projeto
3	20/09 a 26/09	Ambiente executável e contrato inicial da API
4	27/09 a 03/10	Autenticação e autorização por perfil
5	04/10 a 10/10	CRUD de clientes e pedidos
6	11/10 a 17/10	CRUD de entregadores e disponibilidade
7	18/10 a 24/10	Atribuição de pedidos e visualização de rotas
8	25/10 a 31/10	Vizinho mais próximo em modo sequencial
9	01/11 a 07/11	2-opt e linha de base de desempenho
10	08/11 a 14/11	Paralelização para múltiplos entregadores
11	15/11 a 21/11	Experimentos sequencial versus paralelo
12	22/11 a 28/11	Relatórios, testes e documentação
Final	29/11 a 05/12	Correções, vídeos e preparação para a banca

Parte II Sprint 02 Arquitetura e modelagem

A Sprint 02 transforma os requisitos em uma base técnica verificável. A solução adota uma arquitetura web modular, um modelo de dados relacional e um componente de otimização isolado, permitindo implementar e medir versões sequencial e paralela sem duplicar regras de negócio.

6 Arquitetura do sistema

Camada	Tecnologia	Responsabilidade
Interface	React, TypeScript e Vite	Aplicação responsiva para administração e operação móvel
API	FastAPI e Python 3.12	Endpoints REST, validação, OpenAPI e integração com o otimizador
Persistência	PostgreSQL 16	Integridade, índices, histórico e métricas de execução
Mapa	Leaflet, OpenStreetMap e adaptador OSRM	Exibição de rotas e matriz de distâncias sem prender o domínio a um provedor
Otimização	Python, 2-opt e processos independentes	Linha de base sequencial e cálculo paralelo por entregador
Ambiente	Docker Compose	Inicialização reproduzível de frontend, API e banco

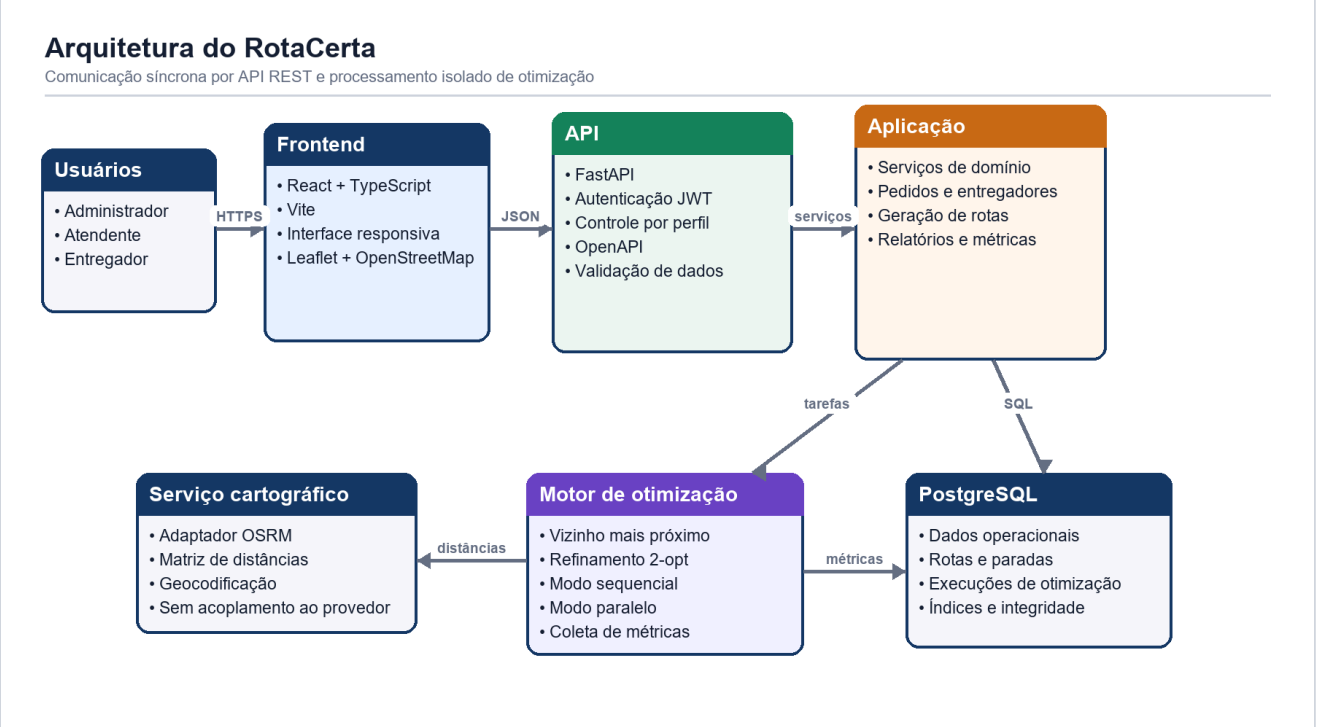


Figura 1 Arquitetura lógica e comunicação entre os componentes

O navegador envia requisições HTTPS em JSON para a API. A API executa regras de negócio, persiste dados no PostgreSQL e aciona o motor de otimização. O motor recebe pedidos, entregadores e uma matriz de distâncias por meio de uma interface estável. A mesma interface atende aos modos sequencial e paralelo, o que permite comparar desempenho sem alterar os dados de entrada.

A escolha de Python não pressupõe o uso de inteligência artificial. As orientações da disciplina aceitam inteligência artificial e/ou otimização de processamento, paralelização e alto desempenho. No RotaCerta,

o componente avançado é a otimização de rotas integrada ao fluxo principal. React e TypeScript permanecem restritos à interface, enquanto Python executa o núcleo computacional.

7 Diagrama de classes

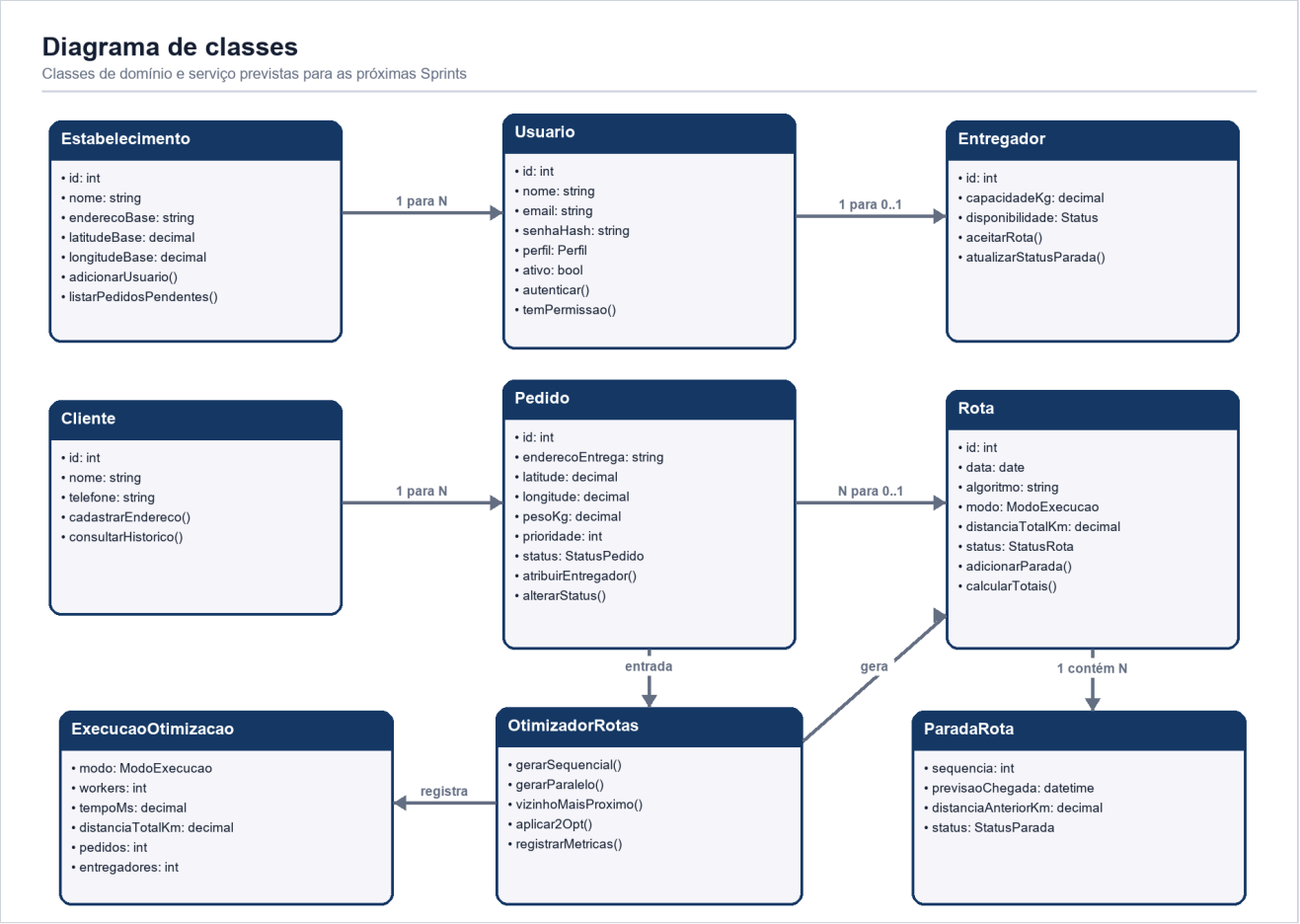


Figura 2 Classes de domínio e serviço previstas

Estabelecimento delimita os dados de cada negócio. Usuario representa autenticação e perfil; Entregador complementa um usuário operacional com capacidade e disponibilidade. Pedido registra destino, prioridade, janela de horário e status. Rota agrega paradas ordenadas. OtimizadorRotas executa o mesmo algoritmo nos modos sequencial e paralelo e registra cada ExecucaoOtimizacao.

8 Modelo Entidade Relacionamento

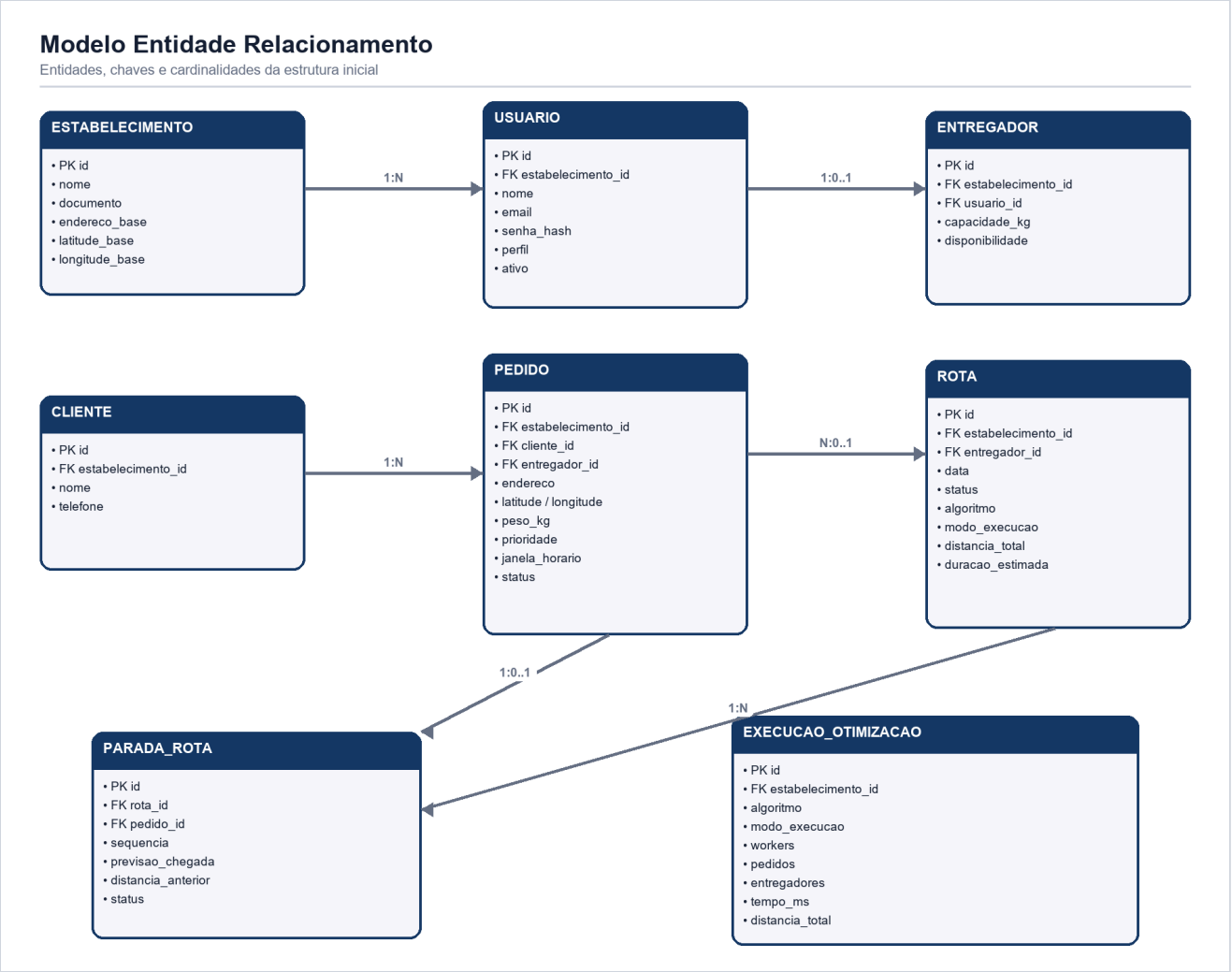


Figura 3 Modelo conceitual com chaves e cardinalidades

O modelo mantém o estabelecimento como limite organizacional. Um usuário pode possuir um cadastro de entregador. Cada pedido pertence a um cliente e pode ser atribuído a um entregador. Uma rota contém paradas ordenadas; cada parada referencia um pedido. As execuções de otimização registram quantidade de pedidos, entregadores, workers, tempo e distância total.

9 Modelo relacional

Tabela	PK	FK	Finalidade
establishments	id	-	Negócio, endereço e coordenadas do depósito
users	id	establishment_id	Credenciais, perfil e situação do usuário
couriers	id	establishment_id, user_id	Capacidade e disponibilidade do entregador
customers	id	establishment_id	Cliente associado ao estabelecimento
orders	id	establishment_id, customer_id, assigned_courier_id	Destino, peso, prioridade, janela e status
routes	id	establishment_id, courier_id	Rota diária, algoritmo, modo e totais
route_stops	id	route_id, order_id	Sequência e acompanhamento de cada parada

Tabela	PK	FK	Finalidade
optimization_runs	id	establishment_id	Métricas de execuções sequenciais e paralelas

As restrições CHECK limitam perfis, estados e modos de execução. As chaves estrangeiras preservam a integridade entre usuários, pedidos, entregadores e rotas. Índices cobrem as consultas mais frequentes por estabelecimento, status, entregador, data e modo de execução.

10 Protótipos das telas principais

Os wireframes priorizam navegação direta e leitura em dispositivos móveis. A primeira sequência cobre autenticação, painel diário e cadastro de pedido. A segunda cobre geração de rotas, consulta da rota pelo entregador e atualização de status.

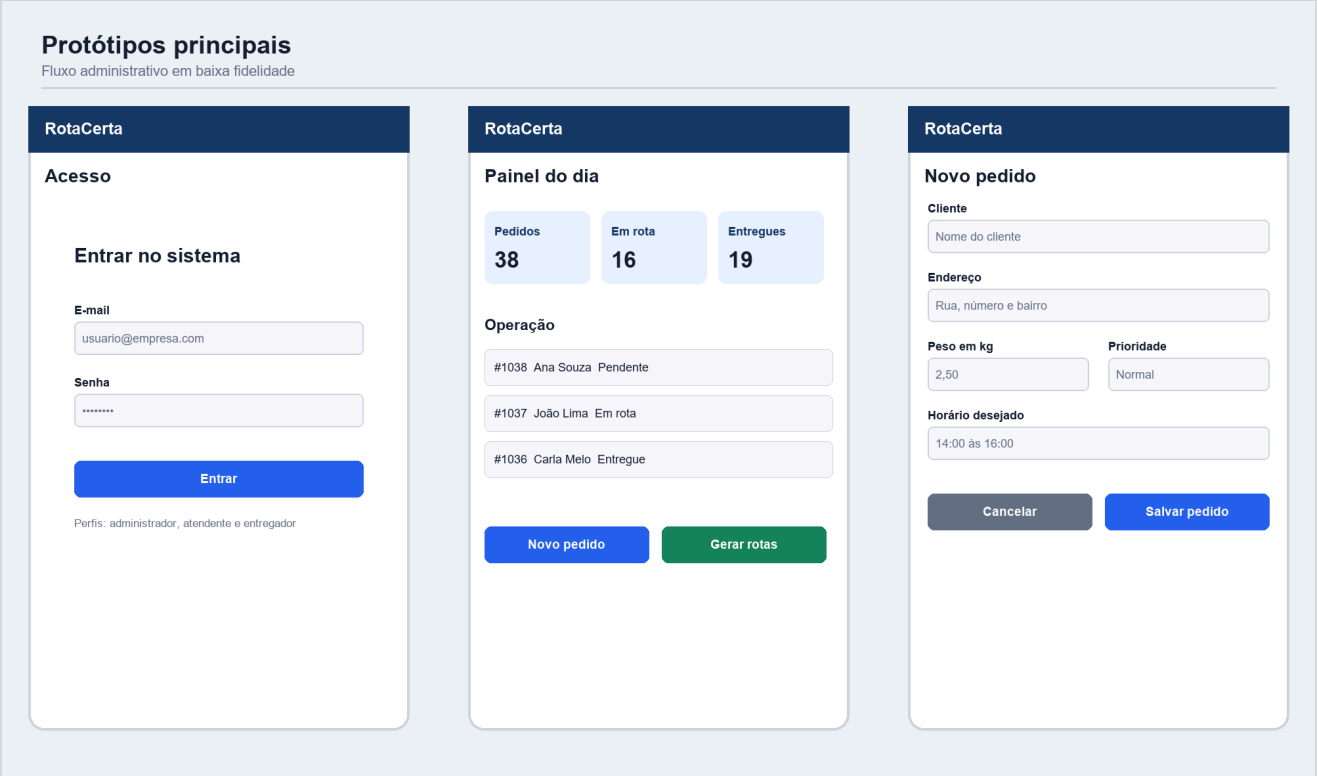


Figura 4 Acesso, painel administrativo e cadastro de pedido

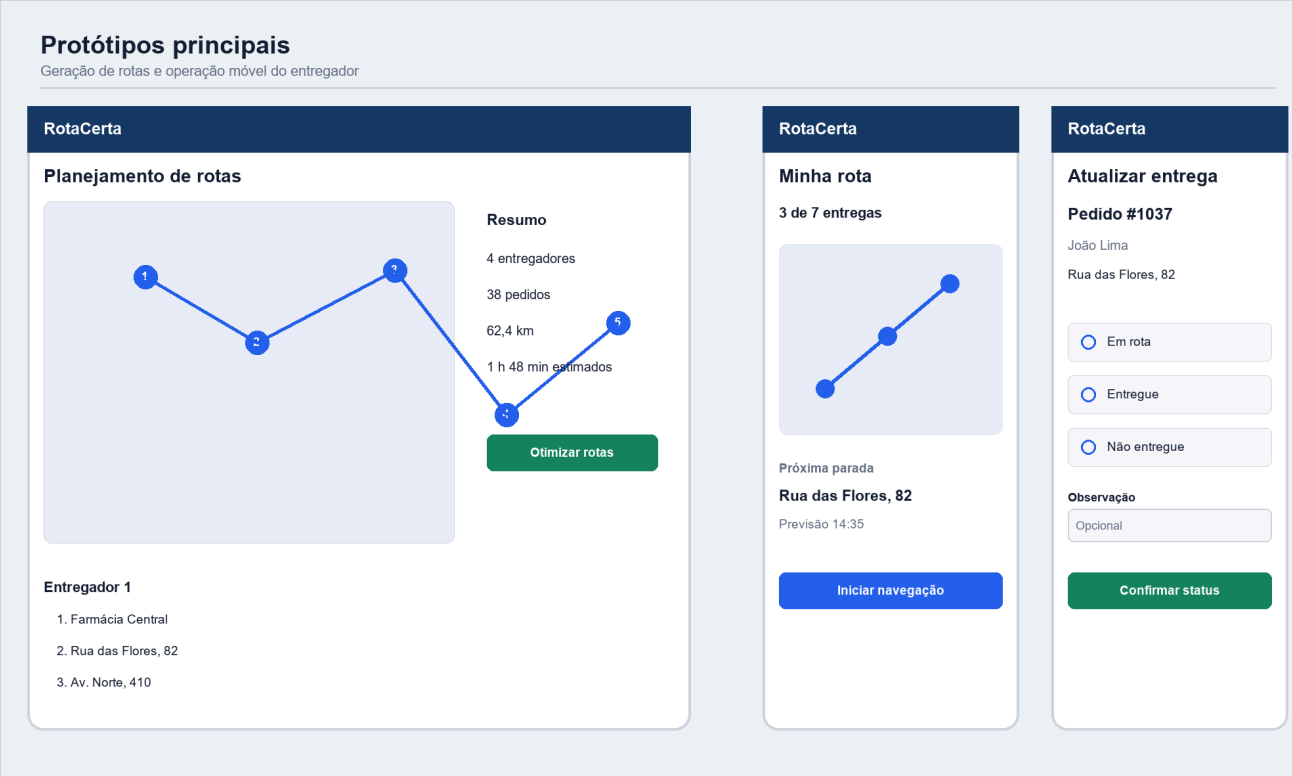


Figura 5 Planejamento de rotas e fluxo móvel do entregador

Tela	Requisito	Objetivo
Acesso	UC07	Login e identificação do perfil
Painel do dia	RF10	Pedidos, situação operacional e atalhos
Novo pedido	UC01 / RF02	Cliente, destino, carga, prioridade e janela
Planejamento	UC03 / RF04	Mapa, totais e comando de otimização
Minha rota	UC04 / RF06	Sequência, mapa e próxima parada
Atualizar entrega	UC05 / RF07	Transição de status e observação

11 Banco de dados e projeto no GitHub

Banco de dados criado

A estrutura inicial está implementada no arquivo database/schema.sql para PostgreSQL 16. O serviço db do arquivo compose.yaml executa o esquema automaticamente na primeira inicialização do volume. O modelo inclui oito tabelas, restrições de integridade e cinco índices iniciais.

Estrutura do repositório na Sprint 02

Caminho	Conteúdo
backend/	API FastAPI e limite do módulo de otimização
backend/app/optimizer/	Vizinho mais próximo, 2-opt e execução sequencial ou paralela
backend/tests/	Testes automatizados do núcleo de roteirização
frontend/	Aplicação React e TypeScript
database/	DDL, instruções e inicialização do PostgreSQL
docs/sprint-02/assets/	Diagramas e protótipos incorporados ao documento
tools/documentation/	Geração reproduzível dos materiais técnicos
compose.yaml	Orquestração local dos três serviços
CONTRIBUTING.md	Padrões de branch, commit e revisão

O fluxo recomendado usa branches curtas por tarefa, pull request e revisão de outro integrante. As branches pessoais dev/alvaro, dev/arthur e dev/vinicius identificam ambientes acadêmicos, mas não substituem branches de feature, correção ou documentação.

12 Avaliação sequencial e paralela

A orientação da Sprint 01 exige manter a comparação entre as versões sequencial e paralela. A primeira implementação já executa vizinho mais próximo e refinamento 2-opt nos dois modos. As versões usam os mesmos pedidos, entregadores e coordenadas; a única variável controlada é o modo de execução.

Métrica	Cálculo ou unidade	Critério
Tempo de execução	Milissegundos	Média, mediana e percentil 95 de dez repetições
Distância total	Quilômetros	Confirma que a paralelização não altera a qualidade da rota
Speedup	Tseq / Tpar	Ganho obtido com a execução paralela
Eficiência	Speedup / workers	Uso relativo dos processos disponíveis
Escala	Pedidos e entregadores	Cenários de 50, 100, 250 e 500 pedidos com 2, 4 e 8 entregadores

Cada cenário terá uma execução de aquecimento e dez repetições válidas. O relatório registrará processador, memória, sistema operacional, versão do código, número de workers e semente dos dados. A tabela optimization_runs armazenará cada medição para permitir auditoria e reprodução.

Evidência prática. O arquivo backend/app/optimizer/routing.py contém a heurística determinística, o cálculo de distância geodésica, o refinamento 2-opt e a execução com processos independentes. O endpoint POST /optimizer/compare retorna rotas, distâncias, tempos, workers, speedup e a confirmação de equivalência entre os modos. Cinco testes automatizados validam rota vazia, preservação das paradas, melhoria do 2-opt, resultado da otimização e equivalência sequencial/paralela.

13 Aderência às orientações da disciplina

Orientação	Situação na Sprint 02	Evidência
Projeto integrado	Atendido	Um único sistema reúne interface, API, banco e otimização
Componente avançado	Atendido	Otimização e paralelização fazem parte da geração de rotas
Tecnologia prioritária	Atendido	Python no núcleo; JavaScript e TypeScript apenas na interface
Integração não superficial	Atendido	O endpoint compara os modos sobre os mesmos dados
Evolução prática	Atendido	Código executável e cinco testes automatizados no repositório
GitHub atualizado	Atendido	Branch da Sprint 02 e Pull Request número 1
Participação da equipe	Contínuo	Cada integrante deve registrar contribuições próprias nas próximas tarefas

Parte III Sprint 03 Estrutura inicial funcionando

A Sprint 03 transforma a arquitetura e o modelo de dados da Sprint 02 em um sistema executável. Frontend, API e PostgreSQL passaram a funcionar integrados: o usuário faz login, recebe permissões de acordo com o seu perfil e cadastra, consulta, edita e exclui pedidos gravados no banco. Todas as telas e respostas mostradas nesta parte foram capturadas com o sistema em execução local.

14 Estrutura implementada

A aplicação mantém as três camadas definidas na Sprint 02. O frontend React consome a API REST com requisições JSON autenticadas por token. A API valida os dados, aplica as regras de cada perfil e grava as informações no PostgreSQL por meio do SQLAlchemy.

Componente	Arquivo	Responsabilidade
Configuração	backend/app/config.py	Lê a URL do banco, a chave do token, as origens permitidas e as opções de inicialização
Conexão	backend/app/database.py	Cria o pool de conexões, verifica o banco e aplica o esquema quando necessário
Modelos	backend/app/models.py	Mapeia as tabelas do schema.sql para classes Python
Segurança	backend/app/security.py	Hash Argon2, emissão e validação do JWT e verificação de perfil
Autenticação	backend/app/routers/auth.py	Login, cadastro de estabelecimento e dados do usuário logado
Usuários	backend/app/routers/users.py	Cadastro, edição e desativação de usuários; lista de entregadores
Pedidos	backend/app/routers/orders.py	CRUD de pedidos e mudança de situação
Dados iniciais	backend/app/seed.py	Estabelecimento, usuários e pedidos de demonstração
Interface	frontend/src/pages/	Login, pedidos, entregas do entregador e usuários
Testes	backend/tests/	16 testes da API e 5 do otimizador

Fluxo de uma requisição autenticada

1. O usuário informa e-mail e senha na tela de login.
2. A API compara a senha com o hash Argon2 gravado em users.password_hash e devolve um token JWT válido por oito horas.
3. O frontend envia o token no cabeçalho Authorization de cada requisição.
4. A API valida o token, carrega o usuário do banco e confere se o perfil tem permissão para a rota.
5. As consultas são sempre filtradas pelo estabelecimento do usuário; entregadores recebem apenas os pedidos atribuídos a eles.

15 Banco de dados conectado

A API se conecta ao PostgreSQL pela URL da variável DATABASE_URL, usando o driver psycopg 3 e o SQLAlchemy 2. O pool de conexões testa a conexão antes de cada uso. Na execução com Docker, o PostgreSQL aplica database/schema.sql na primeira inicialização; na execução sem Docker, a própria

API aplica o mesmo arquivo quando encontra o banco vazio. Nos dois casos a estrutura é exatamente a modelada na Sprint 02.

O endpoint GET /health executa uma consulta no banco e informa o tipo e a versão do servidor. Se o banco estiver indisponível, a resposta passa a ser HTTP 503 com database igual a unavailable.

Estilos de formatação ☐

{"status":"ok","database":"connected","dialect":"postgresql","server_version":"16.13"}

Figura 6 Endpoint de verificação confirmando a conexão da API com o PostgreSQL

```
-- Tabelas do banco rotacerta e quantidade de registros
SELECT 'establishments' AS tabela, count(*) AS registros FROM establishments UNION ALL ... (uma consulta por tabela)

tabela            | registros
------------------+----------
establishments    | 2
users             | 6
couriers          | 3
customers         | 7
orders            | 6
routes            | 0
route_stops       | 0
optimization_runs | 0
(8 linhas)
```

Consulta às oito tabelas do banco rotacerta com a quantidade de registros

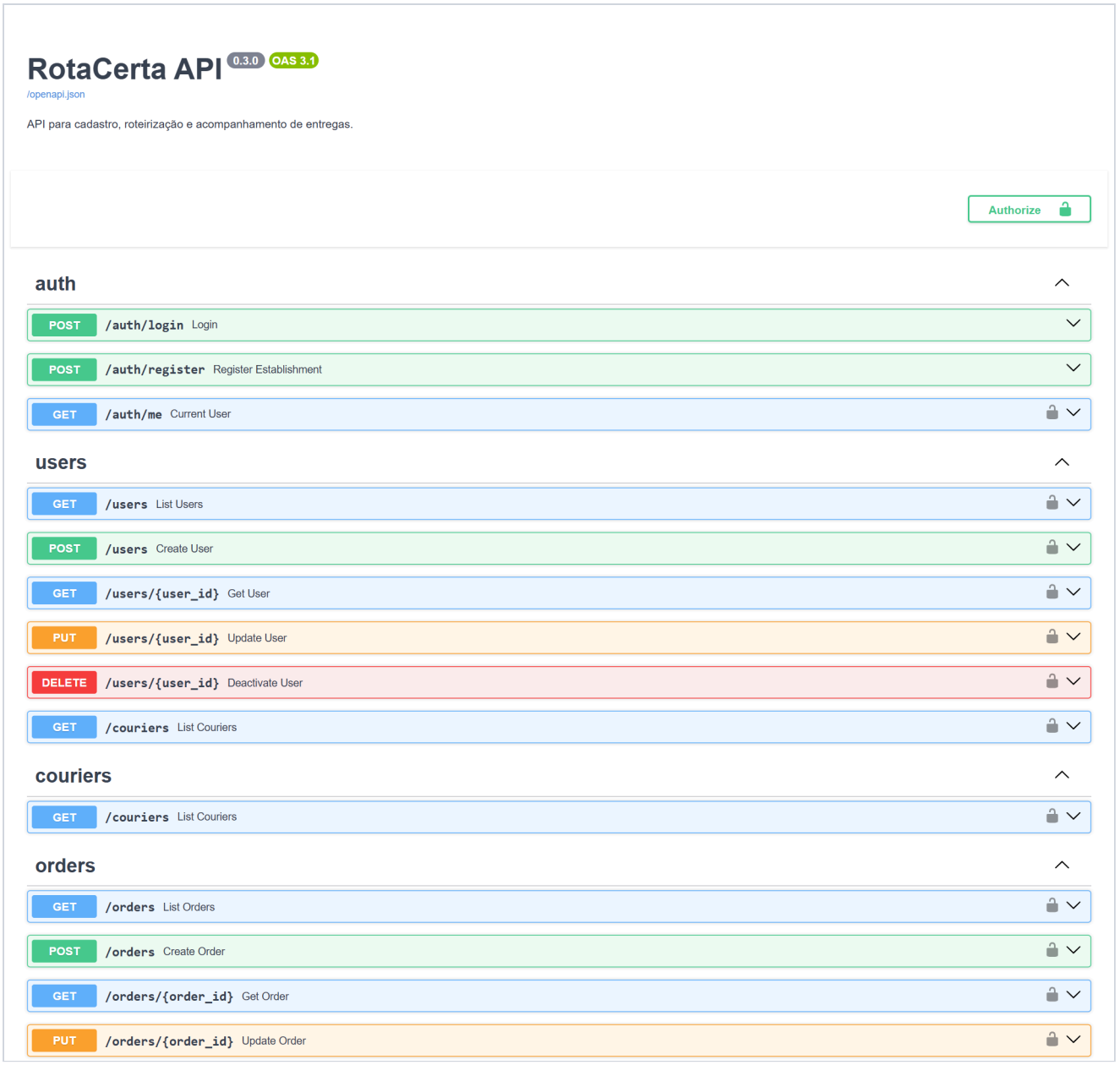


Figura 7 Documentação interativa da API gerada pelo FastAPI, com as rotas de autenticação, usuários e pedidos

16 Login funcional

A tela de login autentica os usuários cadastrados no banco. O e-mail é comparado sem diferenciar maiúsculas de minúsculas e a senha é verificada contra o hash Argon2; a senha em texto nunca é gravada. Credenciais incorretas retornam HTTP 401 com a mensagem E-mail ou senha inválidos. Usuários desativados recebem HTTP 403. A sessão fica salva no navegador e é validada novamente na API sempre que a página é aberta.

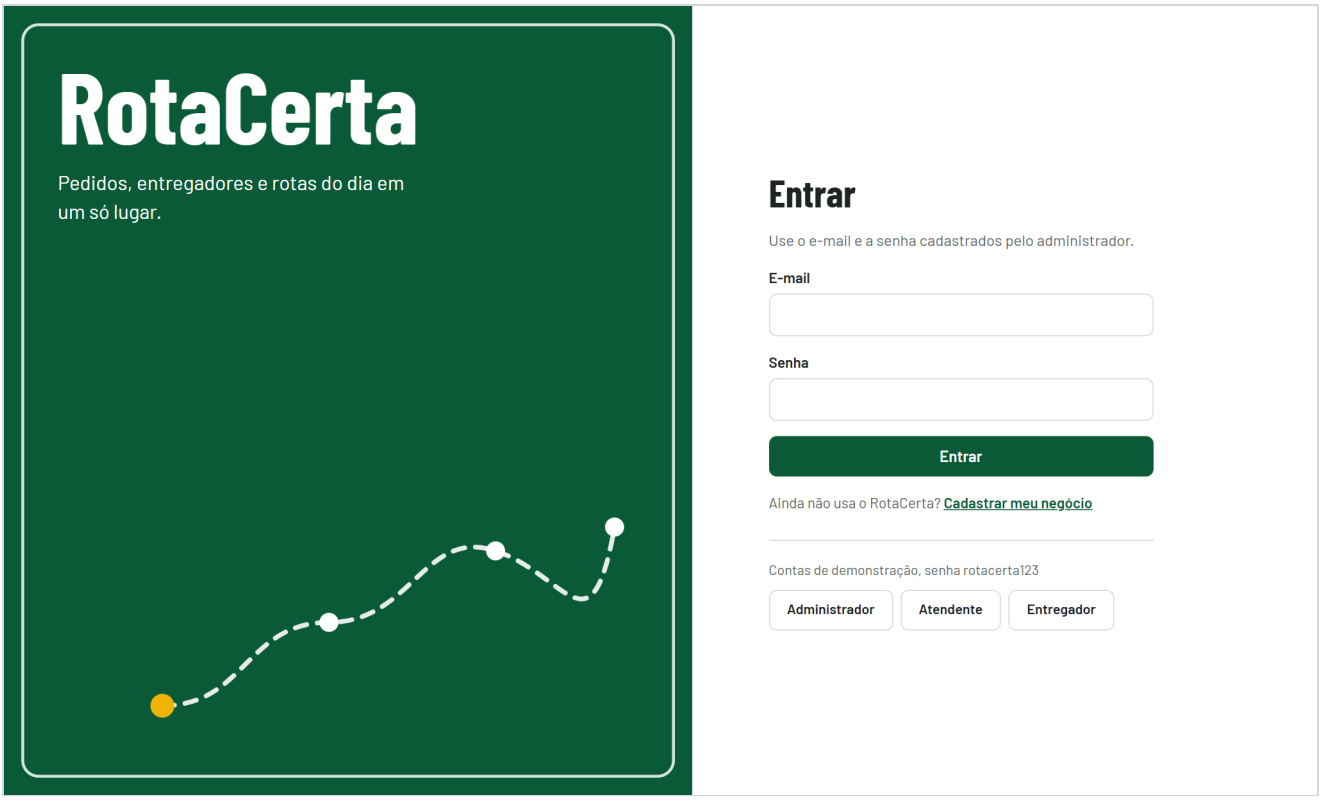


Figura 8 Tela de login com atalhos para as contas de demonstração

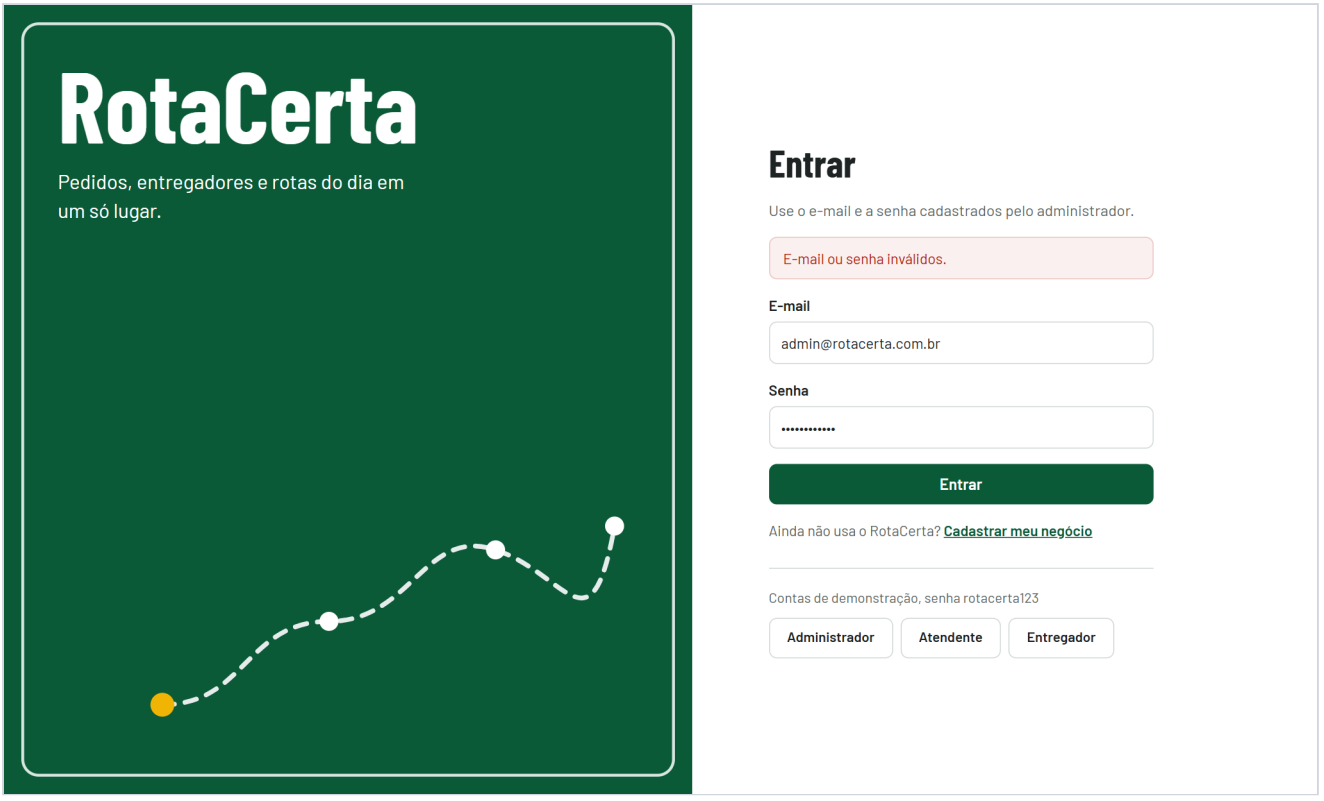


Figura 9 Tentativa de login com senha incorreta

```
-- Usuários cadastrados: a senha fica gravada apenas como hash Argon2
SELECT id, full_name AS nome, email, role AS perfil, active AS ativo, left(password_hash, 32) || '...' AS hash_da_senha
FROM users ORDER BY id

id | nome               | email                         | perfil    | ativo | hash_da_senha
---+--------------------+-------------------------------+-----------+-------+-----------------------------------
1  | Ana Administradora | admin@rotacerta.com.br        | ADMIN     | True  | $argon2id$v=19$m=65536,t=3,p=4$3...
2  | Bruno Atendente    | atendente@rotacerta.com.br    | ATTENDANT | True  | $argon2id$v=19$m=65536,t=3,p=4$z...
3  | Carla Entregadora  | entregador@rotacerta.com.br   | COURIER   | True  | $argon2id$v=19$m=65536,t=3,p=4$P...
4  | Diego Entregador   | entregador2@rotacerta.com.br  | COURIER   | True  | $argon2id$v=19$m=65536,t=3,p=4$D...
5  | José Almeida       | jose@mercadinhosaojose.com.br | ADMIN     | True  | $argon2id$v=19$m=65536,t=3,p=4$S...
6  | Eduardo Nascimento | eduardo@rotacerta.com.br      | COURIER   | True  | $argon2id$v=19$m=65536,t=3,p=4$g...
(6 linhas)
```

Usuários gravados no banco: apenas o hash da senha é armazenado

17 Cadastro de usuários

O cadastro acontece de duas formas. Um novo negócio cria a própria conta pela opção Cadastrar meu negócio, que grava o estabelecimento e o seu primeiro administrador. A partir daí, o administrador cadastra atendentes e entregadores na tela Usuários. Para entregadores, o sistema exige a capacidade de carga e cria também o registro na tabela couriers.

- O e-mail é único em todo o sistema; uma duplicidade retorna HTTP 409.
- A senha tem no mínimo oito caracteres e é gravada com hash Argon2.
- Usuários não são apagados: a desativação bloqueia o login e preserva o histórico de pedidos.
- O administrador não pode desativar a si mesmo nem remover o próprio perfil de administrador.
- Cada estabelecimento enxerga apenas os próprios usuários e pedidos.

Figura 10 Cadastro de um novo estabelecimento com o seu administrador

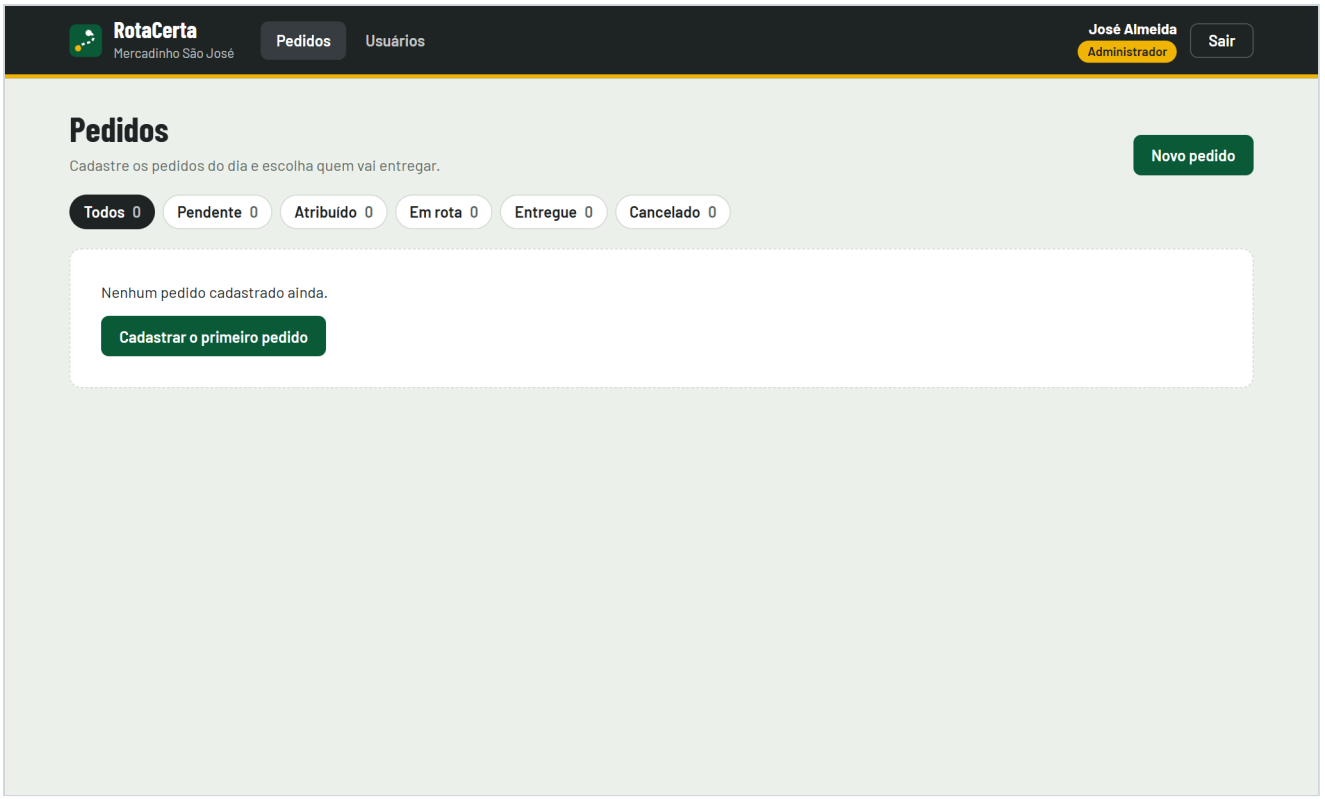


Figura 11 Estabelecimento recém-criado: os pedidos de outros negócios não aparecem

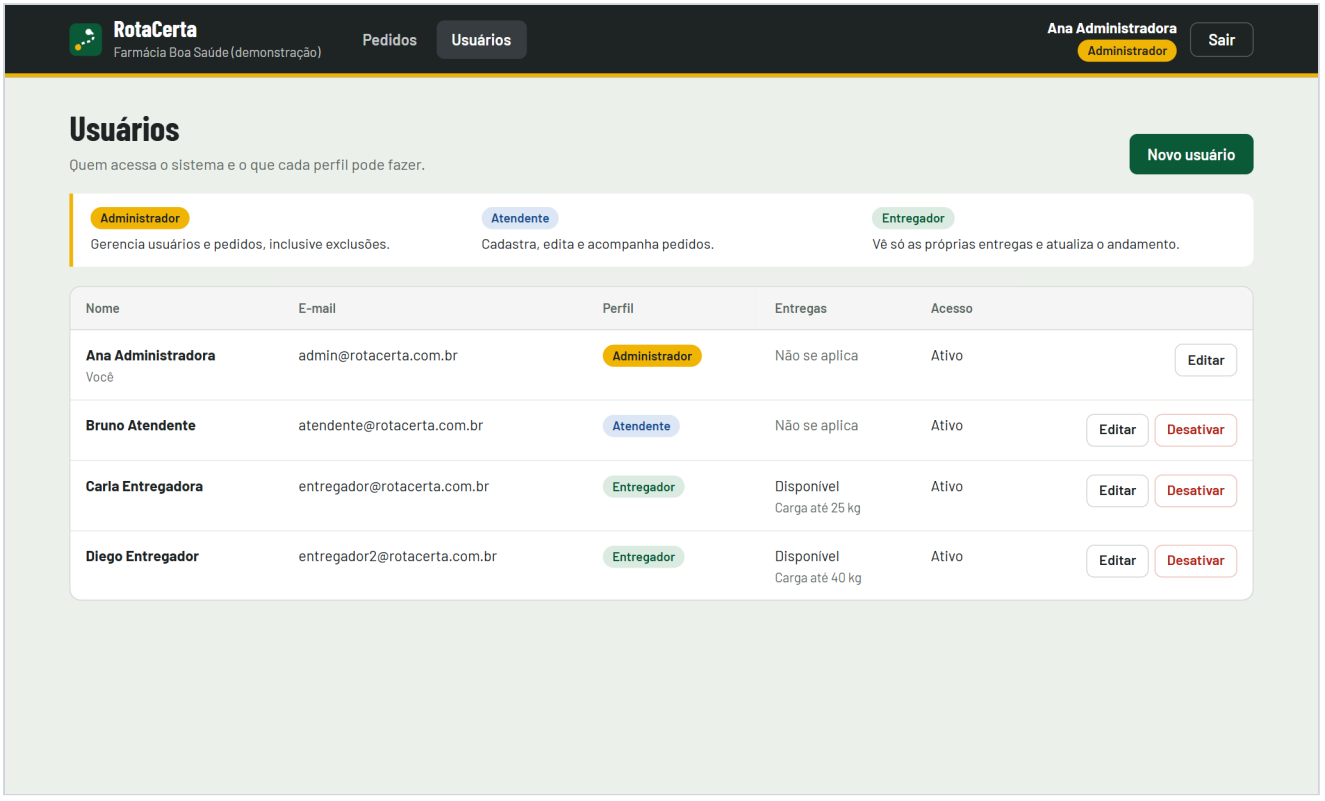


Figura 12 Tela de usuários do administrador, com o resumo das permissões de cada perfil

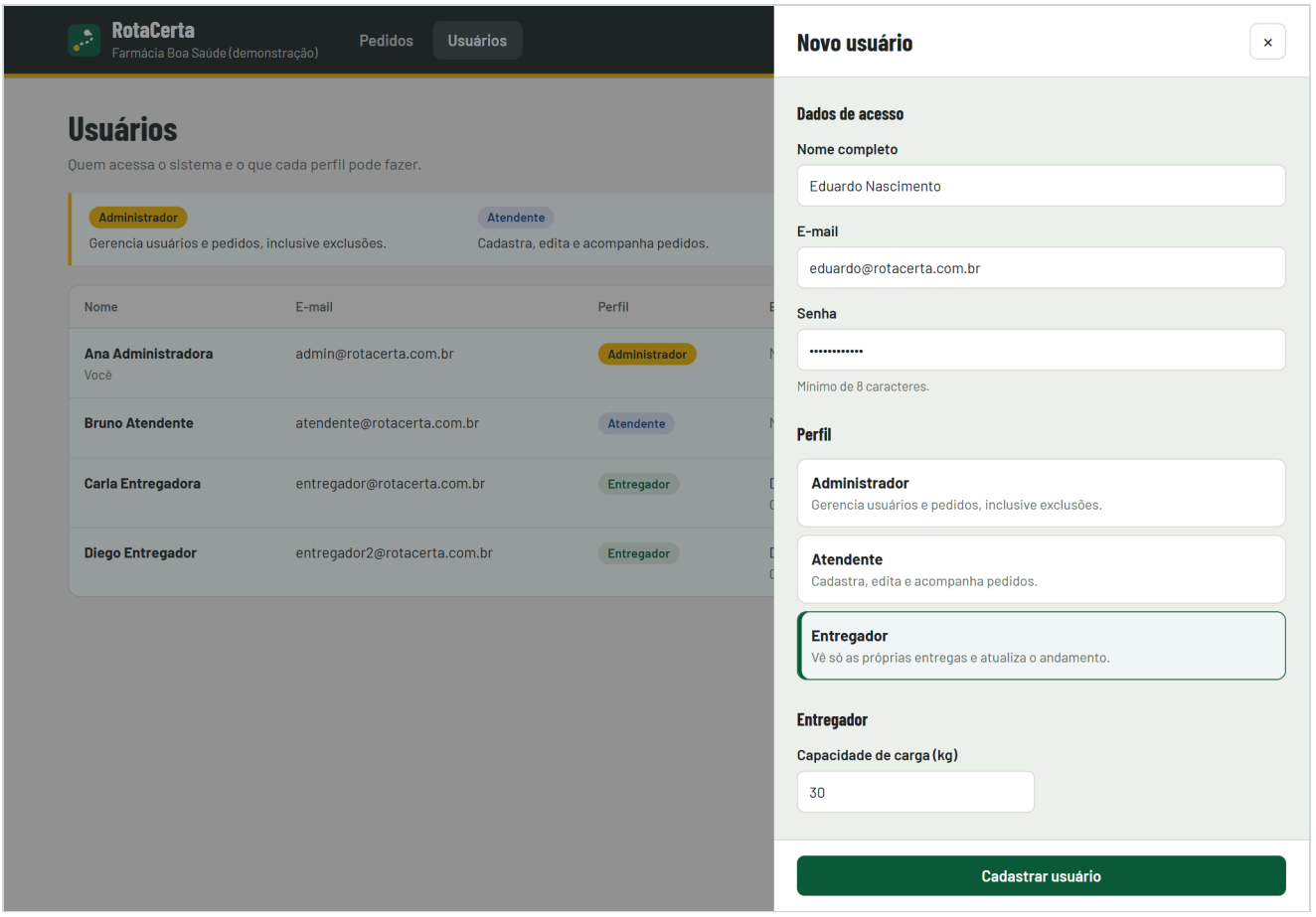


Figura 13 Cadastro de um entregador com capacidade de carga

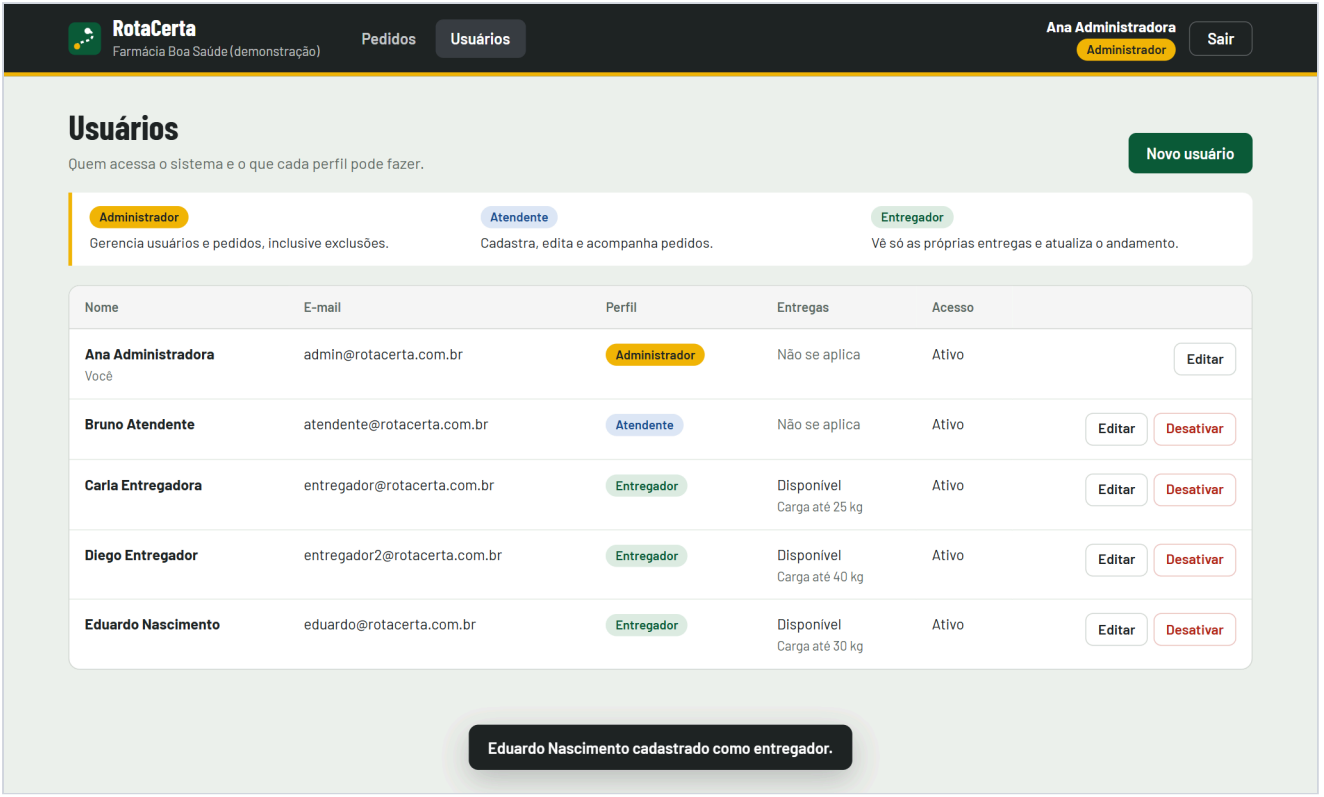


Figura 14 Entregador cadastrado e listado com disponibilidade e capacidade

18 Controle de perfis

Os três perfis previstos no RF01 foram implementados. A verificação acontece na API, em cada rota, e não apenas na interface: mesmo que alguém chame a API diretamente, a ação é negada para perfis sem permissão. O perfil é lido do banco a cada requisição, então uma alteração feita pelo administrador vale imediatamente.

Ação	Administrador	Atendente	Entregador
Entrar no sistema	Sim	Sim	Sim
Cadastrar, editar e desativar usuários	Sim	Não	Não
Consultar pedidos	Todos do estabelecimento	Todos do estabelecimento	Só os atribuídos a ele
Cadastrar e editar pedidos	Sim	Sim	Não
Atribuir entregador ao pedido	Sim	Sim	Não
Excluir pedidos	Sim	Não	Não
Atualizar a situação	Qualquer situação	Qualquer situação	De Atribuído para Em rota e de Em rota para Entregue

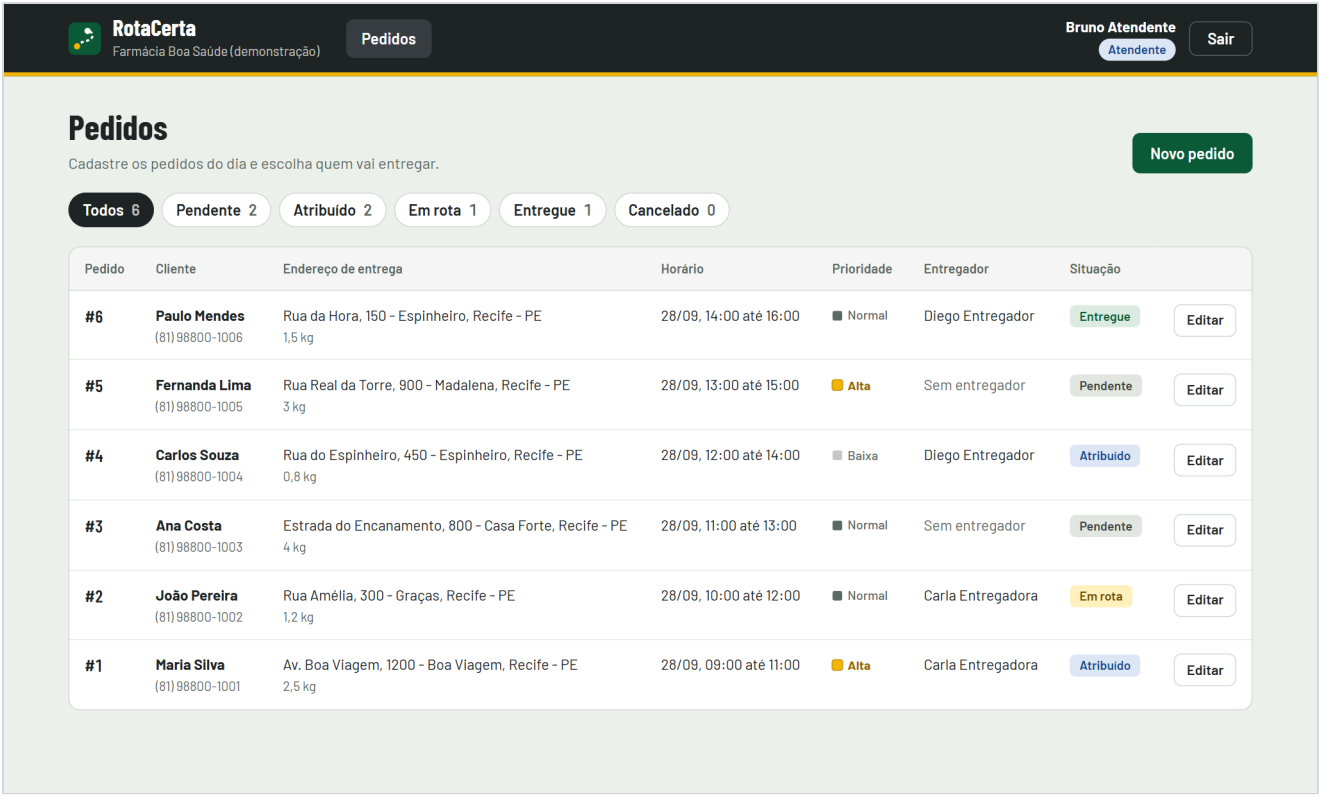


Figura 15 Visão do atendente: sem a aba Usuários e sem o botão Excluir

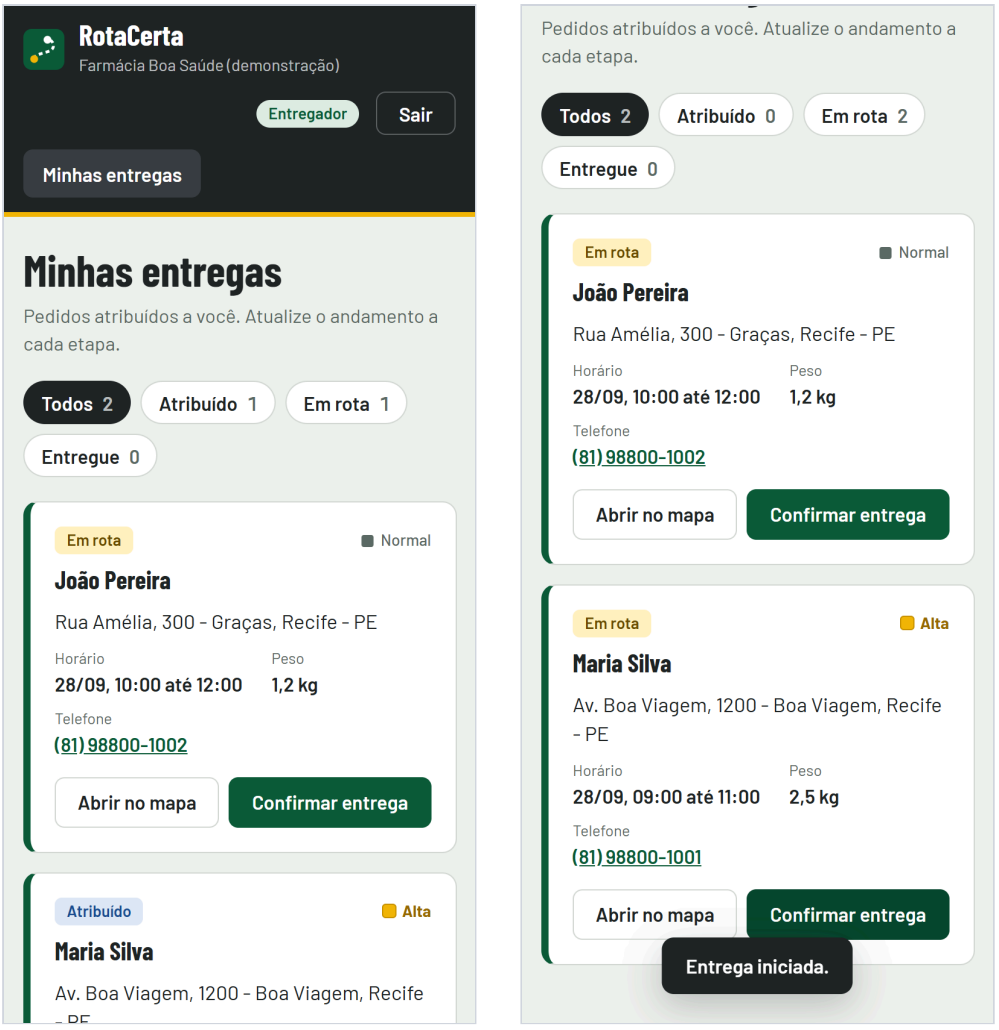


Figura 16 Visão do entregador no celular: apenas os próprios pedidos e o botão para avançar a entrega

```
GET /orders sem token
  -> HTTP 401 Sessão inválida ou expirada. Faça login novamente.
POST /auth/login com senha errada
  -> HTTP 401 E-mail ou senha inválidos.
POST /auth/login (Administrador)
  -> HTTP 200 token JWT emitido
POST /auth/login (Atendente)
  -> HTTP 200 token JWT emitido
POST /auth/login (Entregador)
  -> HTTP 200 token JWT emitido
GET /users como Administrador
  -> HTTP 200 5 registros
GET /users como Atendente
  -> HTTP 403 Seu perfil não tem permissão para esta ação.
GET /users como Entregador
  -> HTTP 403 Seu perfil não tem permissão para esta ação.
POST /orders como Entregador
  -> HTTP 403 Seu perfil não tem permissão para esta ação.
GET /orders como Entregador
  -> HTTP 200 2 pedidos, todos atribuídos a ele
DELETE /orders/6 como Atendente
  -> HTTP 403 Seu perfil não tem permissão para esta ação.
```

Respostas da API para cada perfil, obtidas com chamadas diretas

19 CRUD principal: pedidos

O pedido é a entidade principal do RotaCerta porque concentra o que será roteirizado: cliente, destino, carga, prioridade, janela de horário, entregador e situação. O CRUD usa as tabelas orders e customers do modelo da Sprint 02. O cliente é informado no próprio formulário do pedido; a API reaproveita o cadastro quando o mesmo nome e telefone já existem e cria um registro em customers quando não existem. O local de entrega é marcado com um clique no mapa, que preenche latitude e longitude.

Operação	Rota da API	Regra de negócio
Cadastrar	POST /orders	Coordenadas obrigatórias, fim da janela posterior ao início; com entregador o pedido já nasce Atribuído
Consultar	GET /orders e GET /orders/{id}	Lista com filtro por situação; o entregador vê só os próprios pedidos
Atualizar	PUT /orders/{id} e PATCH /orders/{id}/status	Pedidos entregues ou cancelados não podem ser editados; Em rota exige entregador
Excluir	DELETE /orders/{id}	Somente administrador; pedido que já faz parte de uma rota é protegido pela chave estrangeira

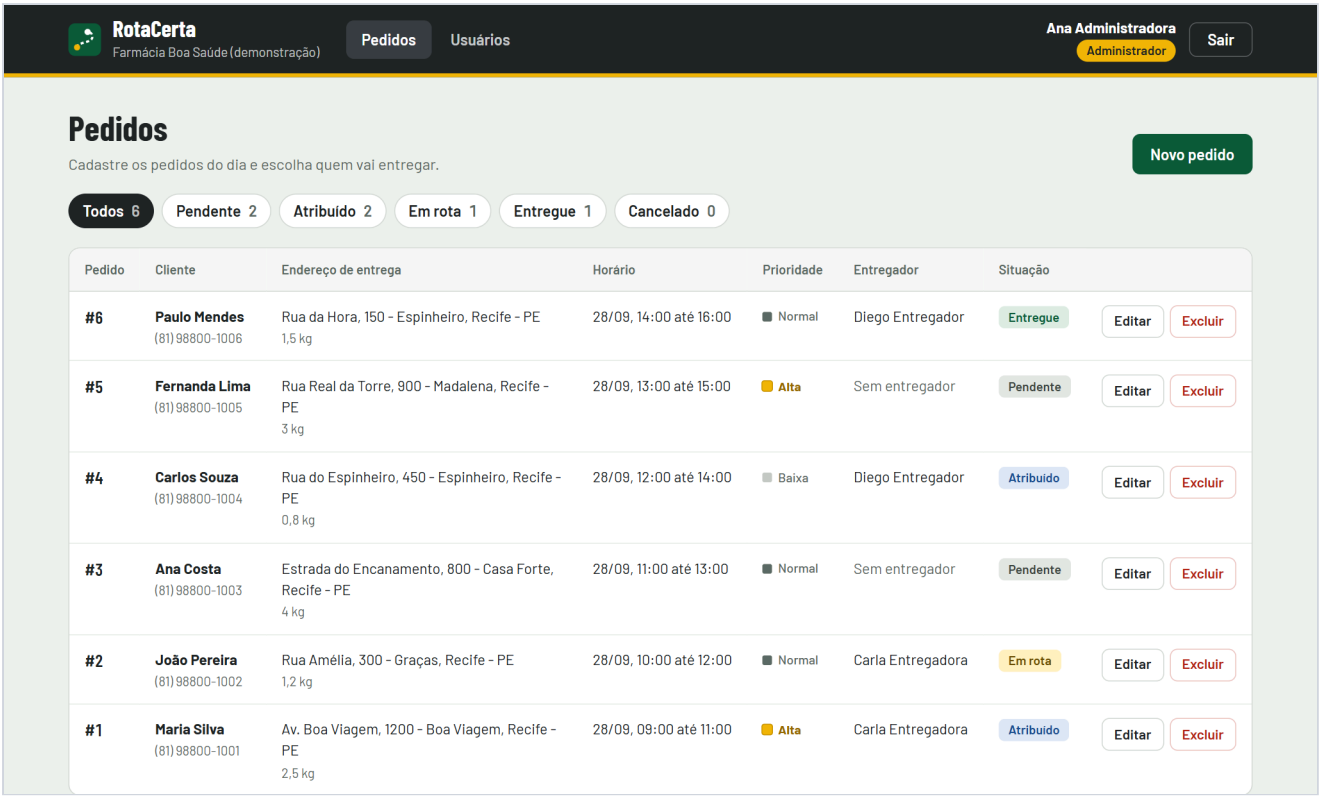


Figura 17 Lista de pedidos do administrador com filtros por situação

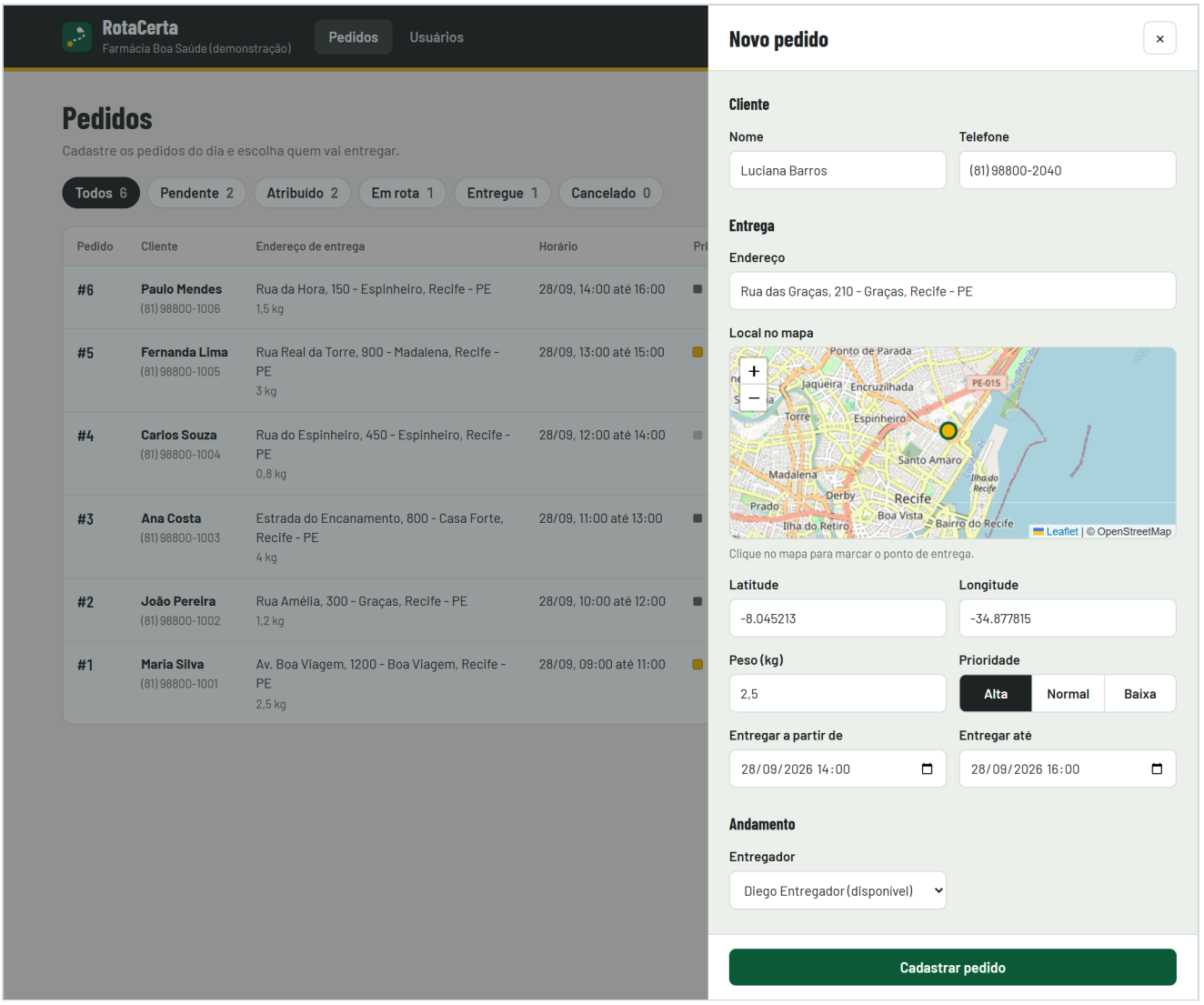


Figura 18 Cadastro de pedido com cliente, local no mapa, prioridade, janela e entregador

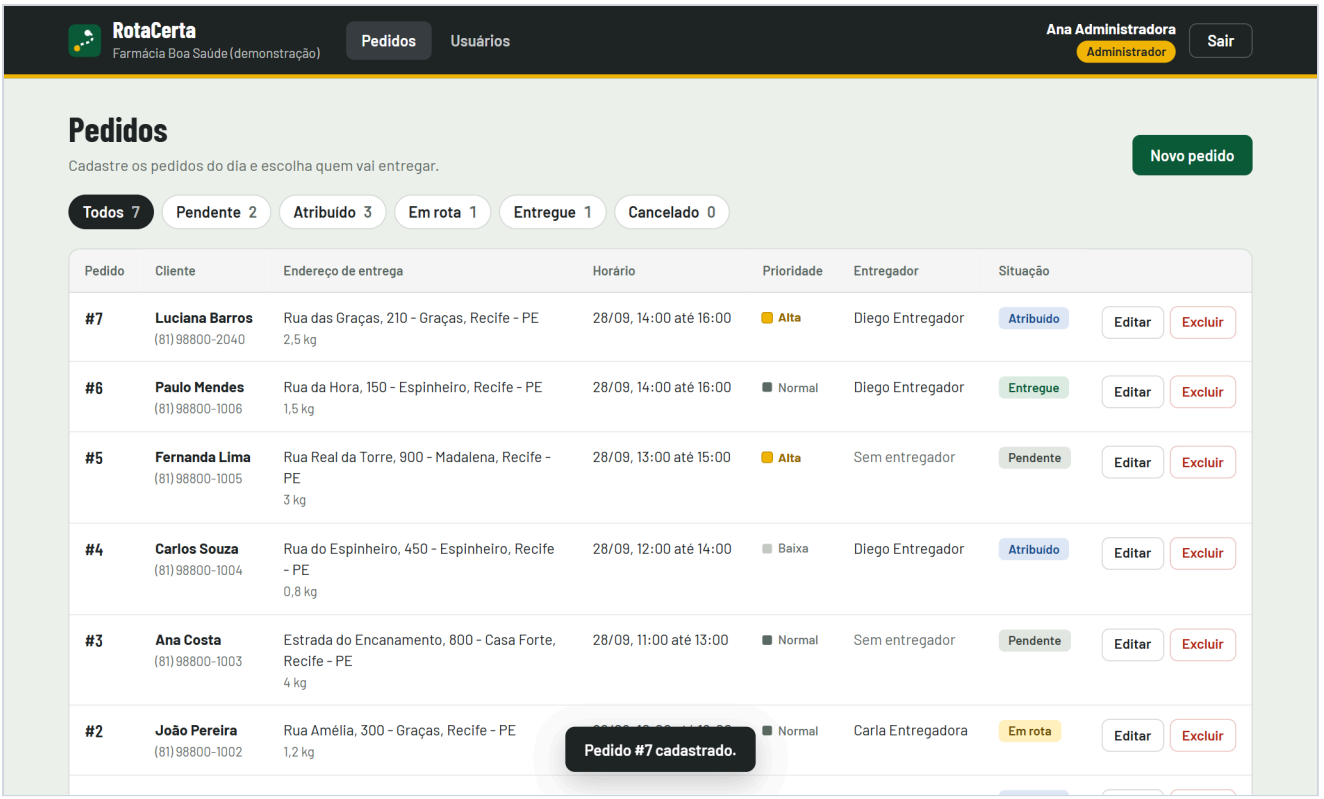


Figura 19 Pedido cadastrado aparecendo no topo da lista

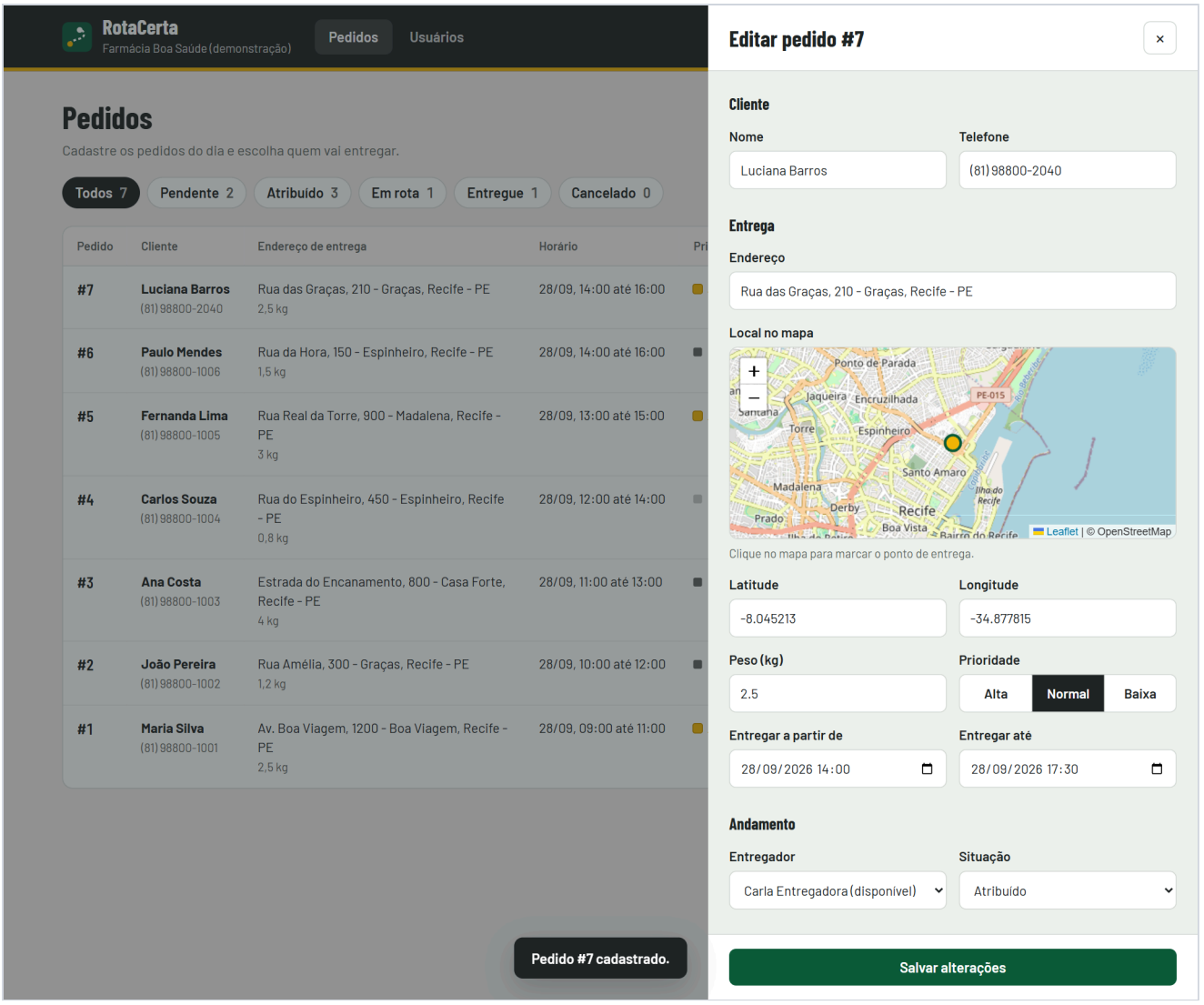


Figura 20 Edição do pedido: prioridade, horário e entregador alterados

```
-- Pedido #7 gravado no PostgreSQL após cadastro e edição pela interface
SELECT o.id, c.full_name AS cliente, o.priority AS prioridade, o.status, u.full_name AS entregador, o.weight_kg AS
peso_kg, to_char(o.desired_end AT TIME ZONE 'America/Recife', 'DD/MM HH24:MI') AS entregar_ate FROM orders o JOIN
customers c ON c.id = o.customer_id LEFT JOIN couriers co ON co.id = o.assigned_courier_id LEFT JOIN users u ON u.id =
co.user_id WHERE o.id = 7

id | cliente        | prioridade | status   | entregador        | peso_kg | entregar_ate
---+----------------+------------+----------+-------------------+---------+-------------
7  | Luciana Barros | 2          | ASSIGNED | Carla Entregadora | 2.50    | 28/09 17:30
(1 linha)
```

Pedido gravado no banco com os valores da edição

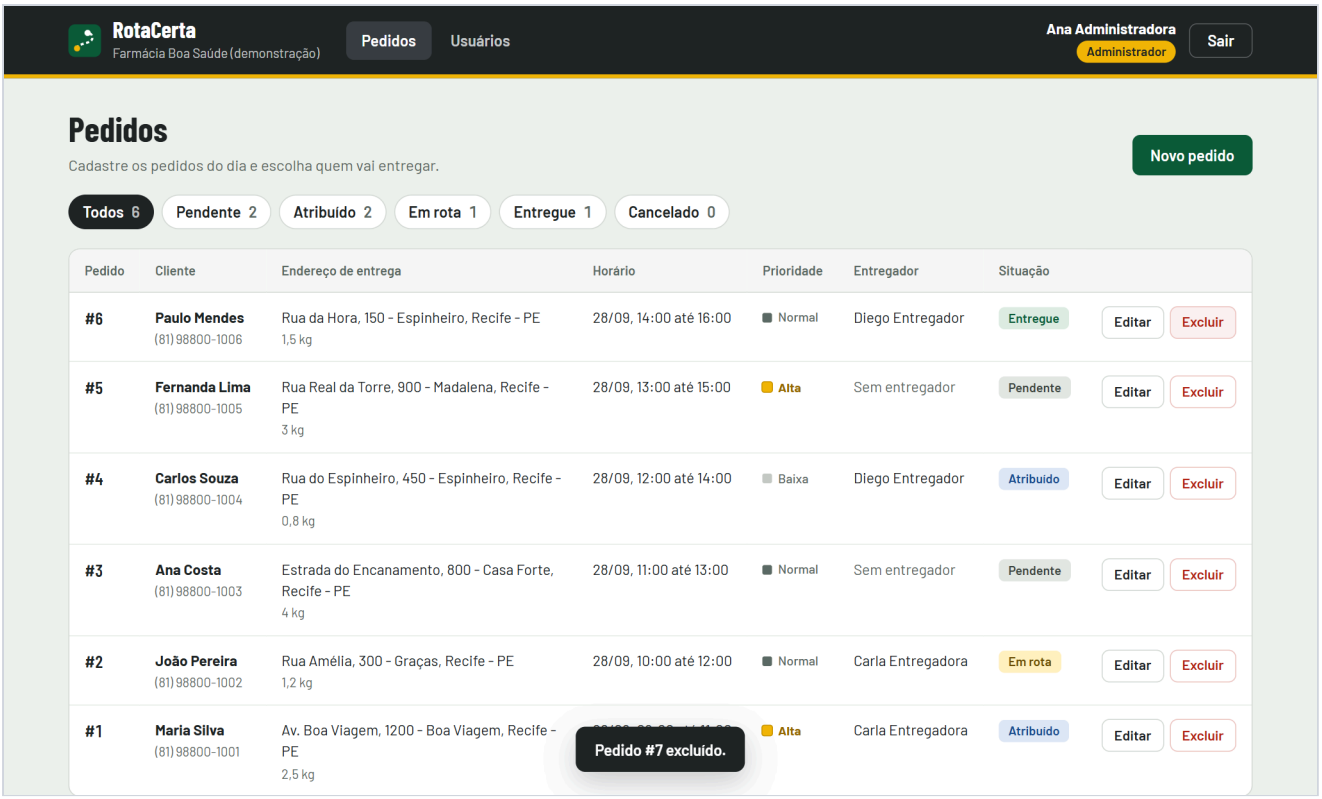


Figura 21 Pedido excluído pelo administrador

```
-- Consulta ao pedido #7 depois da exclusão pela interface
SELECT count(*) AS pedidos_encontrados FROM orders WHERE id = 7

pedidos_encontrados
-------------------
0
(1 linha)
```

O pedido excluído não existe mais no banco

20 Execução local

Com Docker

1. Instalar o Docker Desktop.
2. Na raiz do repositório, copiar .env.example para .env.
3. Executar docker compose up --build.
4. Se já existir um PostgreSQL usando a porta 5432, alterar POSTGRES_PORT no .env para 5433.

Sem Docker

1. Instalar Python 3.12 ou superior, Node.js 20 ou superior e PostgreSQL 16 ou superior.
2. No PostgreSQL, criar o usuário e o banco: CREATE ROLE rotacerta LOGIN PASSWORD 'rotacerta_dev'; CREATE DATABASE rotacerta OWNER rotacerta;
3. Na pasta backend, criar o ambiente virtual, instalar requirements.txt e executar uvicorn app.main:app --reload. Na primeira execução a API cria as tabelas e os dados de demonstração.
4. Na pasta frontend, executar npm install e npm run dev.

5. Para outra porta, usuário ou senha do banco, copiar backend/.env.example para backend/.env e ajustar DATABASE_URL.

Serviço	Endereço
Sistema	http://localhost:5173
Documentação interativa da API	http://localhost:8000/docs
Verificação da API e do banco	http://localhost:8000/health

Perfil	E-mail	Senha
Administrador	admin@rotacerta.com.br	rotacerta123
Atendente	atendente@rotacerta.com.br	rotacerta123
Entregador	entregador@rotacerta.com.br	rotacerta123

As contas acima são criadas automaticamente apenas quando o banco não possui nenhum estabelecimento, para facilitar a demonstração. O passo a passo completo também está no README do repositório.

21 Testes automatizados

Os testes da API usam um banco SQLite em memória, recriado a cada teste, e cobrem login, sessão, hash de senha, cadastro de usuários, isolamento entre estabelecimentos, permissões de cada perfil, CRUD de pedidos e regras de situação. Os testes do otimizador continuam validando os modos sequencial e paralelo.

```
$ python -m unittest discover backend/tests -v
test_admin_creates_user_who_can_log_in (test_api.ApiTestCase.test_admin_creates_user_who_can_log_in) ... ok
test_courier_advances_status_in_order (test_api.ApiTestCase.test_courier_advances_status_in_order) ... ok
test_courier_cannot_create_orders (test_api.ApiTestCase.test_courier_cannot_create_orders) ... ok
test_courier_requires_load_capacity (test_api.ApiTestCase.test_courier_requires_load_capacity) ... ok
test_courier_sees_only_own_orders (test_api.ApiTestCase.test_courier_sees_only_own_orders) ... ok
test_deactivated_user_cannot_log_in (test_api.ApiTestCase.test_deactivated_user_cannot_log_in) ... ok
test_delivered_order_cannot_be_edited (test_api.ApiTestCase.test_delivered_order_cannot_be_edited) ... ok
test_duplicate_email_is_rejected (test_api.ApiTestCase.test_duplicate_email_is_rejected) ... ok
test_login_rejects_wrong_password (test_api.ApiTestCase.test_login_rejects_wrong_password) ... ok
test_login_returns_token_and_profile (test_api.ApiTestCase.test_login_returns_token_and_profile) ... ok
test_only_admin_manages_users (test_api.ApiTestCase.test_only_admin_manages_users) ... ok
test_order_crud (test_api.ApiTestCase.test_order_crud) ... ok
test_order_window_must_be_valid (test_api.ApiTestCase.test_order_window_must_be_valid) ... ok
test_password_is_stored_as_hash (test_api.ApiTestCase.test_password_is_stored_as_hash) ... ok
test_protected_routes_require_token (test_api.ApiTestCase.test_protected_routes_require_token) ... ok
test_register_creates_isolated_establishment (test_api.ApiTestCase.test_register_creates_isolated_establishment) ... ok
test_empty_route_has_zero_distance (test_optimizer.RoutingTestCase.test_empty_route_has_zero_distance) ... ok
test_nearest_neighbor_keeps_every_stop_once
(test_optimizer.RoutingTestCase.test_nearest_neighbor_keeps_every_stop_once) ... ok
test_optimize_route_returns_order_and_distance
(test_optimizer.RoutingTestCase.test_optimize_route_returns_order_and_distance) ... ok
test_parallel_and_sequential_return_same_routes
(test_optimizer.RoutingTestCase.test_parallel_and_sequential_return_same_routes) ... ok
test_two_opt_never_worsens_route (test_optimizer.RoutingTestCase.test_two_opt_never_worsens_route) ... ok
Ran 21 tests in 4.576s
OK
```

Execução da suíte de testes

22 Ajustes no planejamento e na modelagem

Modelagem

Nenhuma tabela, coluna ou restrição do modelo relacional da Sprint 02 precisou ser alterada. A implementação tomou três decisões que não mudam o modelo:

- O cliente é cadastrado a partir do formulário do pedido, sem uma tela própria nesta etapa, reaproveitando a tabela customers.
- Usuários são desativados em vez de excluídos, porque pedidos e rotas referenciam entregadores por chave estrangeira.
- As coordenadas do pedido são obtidas com um clique no mapa Leaflet; a busca automática pelo endereço fica para a próxima etapa.

Planejamento

A Sprint 03 da disciplina exigiu login, perfis, cadastro de usuários e o CRUD principal, que no cronograma da Sprint 02 estavam previstos para as Sprints 04 e 05. Esses itens foram antecipados. As etapas seguintes foram reorganizadas para manter a otimização sequencial e paralela e os experimentos antes do período de relatórios e da banca.

Sprint	Período	Entrega principal	Situação
1	03/09 a 05/09	Planejamento, requisitos e backlog	Concluída
2	06/09 a 19/09	Arquitetura, modelos, protótipos, banco e estrutura	Concluída
3	20/09 a 26/09	Banco conectado, login, perfis, usuários e CRUD de pedidos	Concluída
4	27/09 a 03/10	Entregadores, disponibilidade, clientes e busca de endereço	Planejada
5	04/10 a 10/10	Atribuição de pedidos e visualização das rotas no mapa	Planejada
6	11/10 a 17/10	Vizinho mais próximo integrado aos pedidos do dia	Planejada
7	18/10 a 24/10	2-opt, persistência das rotas e linha de base	Planejada
8	25/10 a 31/10	Paralelização para múltiplos entregadores	Planejada
9	01/11 a 07/11	Experimentos sequencial versus paralelo	Planejada
10	08/11 a 14/11	Histórico, relatórios e painel do dia	Planejada
11	15/11 a 21/11	Testes, usabilidade e segurança	Planejada
12	22/11 a 28/11	Documentação e ajustes	Planejada
Final	29/11 a 05/12	Correções, vídeos e preparação para a banca	Planejada

23 Repositório GitHub

Repositório oficial. https://github.com/alvccpj/rota-certa

Branch	Finalidade
master	Versão estável e entregas aprovadas
sprint/02-arquitetura-modelagem	Integração da Sprint 02

Branch	Finalidade
sprint/03-estrutura-funcional	Integração da Sprint 03
dev/alvaro, dev/arthur, dev/vinicius	Ambientes pessoais dos integrantes

Caminho	Novidade da Sprint 03
backend/app/routers/	Rotas de autenticação, usuários e pedidos
backend/app/models.py e database.py	Conexão e mapeamento do PostgreSQL
backend/tests/test_api.py	Testes da API
frontend/src/pages/	Telas de login, pedidos e usuários
docs/sprint-03/	Capturas de tela e saídas usadas como evidência
tools/documentation/	Captura das evidências e geração deste documento

24 Dificuldades encontradas e próximos passos

Dificuldades

Dificuldade	Como foi tratada
Docker indisponível em máquina de desenvolvimento	Execução sem Docker documentada; a API cria as tabelas a partir do schema.sql quando o banco está vazio
PostgreSQL já instalado ocupando a porta 5432	Porta do banco no Compose passou a ser configurável por POSTGRES_PORT
psycopg 3.2.9 sem pacote para o Python 3.14	Atualização para psycopg 3.3.6, compatível com Python 3.12 a 3.14
Pedidos exigem latitude e longitude	Mapa no formulário para marcar o ponto; busca automática pelo endereço planejada
Paralelização mais lenta que a versão sequencial em entradas pequenas	Criar processos custa mais que o cálculo com poucos pedidos; os experimentos usarão cenários de 50 a 500 pedidos

Próximos passos

- Gerenciar entregadores e disponibilidade em tela própria e buscar coordenadas a partir do endereço.
- Integrar o otimizador aos pedidos do dia e gravar rotas em routes e route_stops.
- Registrar as execuções sequenciais e paralelas em optimization_runs.
- Distribuir as tarefas entre os integrantes com pull requests revisados, registrando a contribuição de cada um no GitHub.

Situação da Sprint 03

Entrega obrigatória	Situação	Evidência
Banco de dados conectado	Concluído	Seção 15: /health, consulta às tabelas e documentação da API
Login funcional	Concluído	Seção 16: tela de login, erro de senha e hash no banco

Entrega obrigatória	Situação	Evidência
Cadastro de usuários	Concluído	Seção 17: cadastro de estabelecimento e de usuários
Controle de perfis	Concluído	Seção 18: matriz de permissões, telas por perfil e respostas 401 e 403
CRUD principal	Concluído	Seção 19: cadastro, consulta, edição e exclusão de pedidos no banco
Deploy local	Concluído	Seção 20: Docker Compose ou execução manual
Repositório atualizado	Concluído	Seção 23: branch da Sprint 03

Parte IV Sprint 04 Primeiro módulo completo

A Sprint 04 consolida o primeiro módulo completo do RotaCerta: Pedidos e entregas. O módulo cobre o ciclo inteiro de um pedido, do cadastro do cliente até a confirmação da entrega pelo entregador, com dados gravados no PostgreSQL, validações na interface e na API, mensagens de erro que orientam o usuário e navegação por páginas com endereço próprio.

25 Módulo implementado: Pedidos e entregas

O módulo atende aos casos de uso UC01 (cadastrar pedido), UC04 (visualizar a rota atribuída) e UC05 (atualizar o status da entrega) e aos requisitos RF02, RF05, RF07 e RF08. Ele usa a estrutura da Sprint 03 (login, perfis e usuários) e acrescenta o cadastro de clientes, a busca de endereço, as regras de atribuição e o histórico de cada pedido.

Funcionalidade	Perfis	Tela
Cadastro, busca, edição e exclusão de clientes	Administrador e atendente	/clientes
Cadastro de pedido com cliente, endereço, local no mapa, peso, prioridade e horário	Administrador e atendente	/pedidos/novo
Busca de endereço no OpenStreetMap com preferência para a região do ponto de saída	Administrador e atendente	/pedidos/novo
Atribuição a um entregador respeitando disponibilidade e capacidade de carga	Administrador e atendente	/pedidos/novo e /pedidos/:id/editar
Lista de pedidos com busca e filtro por situação guardados no endereço da página	Administrador e atendente	/pedidos
Detalhe do pedido com mapa e histórico de cada mudança de situação	Todos	/pedidos/:id
Cancelamento com motivo obrigatório, reabertura e exclusão com confirmação	Administrador e atendente; exclusão só administrador	/pedidos/:id
Entregas do dia no celular: sair para entrega, confirmar entrega e informar disponibilidade	Entregador	/entregas

Fluxo completo de utilização

1. O atendente abre Novo pedido e, como a cliente ainda não tem cadastro, usa Cadastrar cliente. Ao salvar, o sistema volta ao pedido com a cliente já escolhida.
2. O atendente digita o endereço e usa Buscar endereço. O ponto escolhido aparece no mapa e pode ser ajustado com um clique.
3. O atendente informa peso, prioridade e horário e escolhe o entregador. A lista mostra a carga atual de cada um e o sistema impede ultrapassar a capacidade.
4. Ao salvar, o pedido é gravado como Atribuído e o sistema abre a página do pedido com o histórico.
5. No celular, a entregadora vê o pedido em Minhas entregas, toca em Sair para entrega e, ao chegar, em Confirmar entrega.
6. O administrador acompanha o histórico completo, com quem fez cada mudança e quando, e pode cancelar, reabrir ou excluir pedidos.

26 Evidências do módulo funcionando

As figuras a seguir seguem o fluxo acima com a cliente Luciana Barros, cadastrada durante a captura. Os dados de demonstração (Farmácia Boa Saúde, com dois entregadores e seis pedidos) são criados automaticamente quando o banco está vazio.

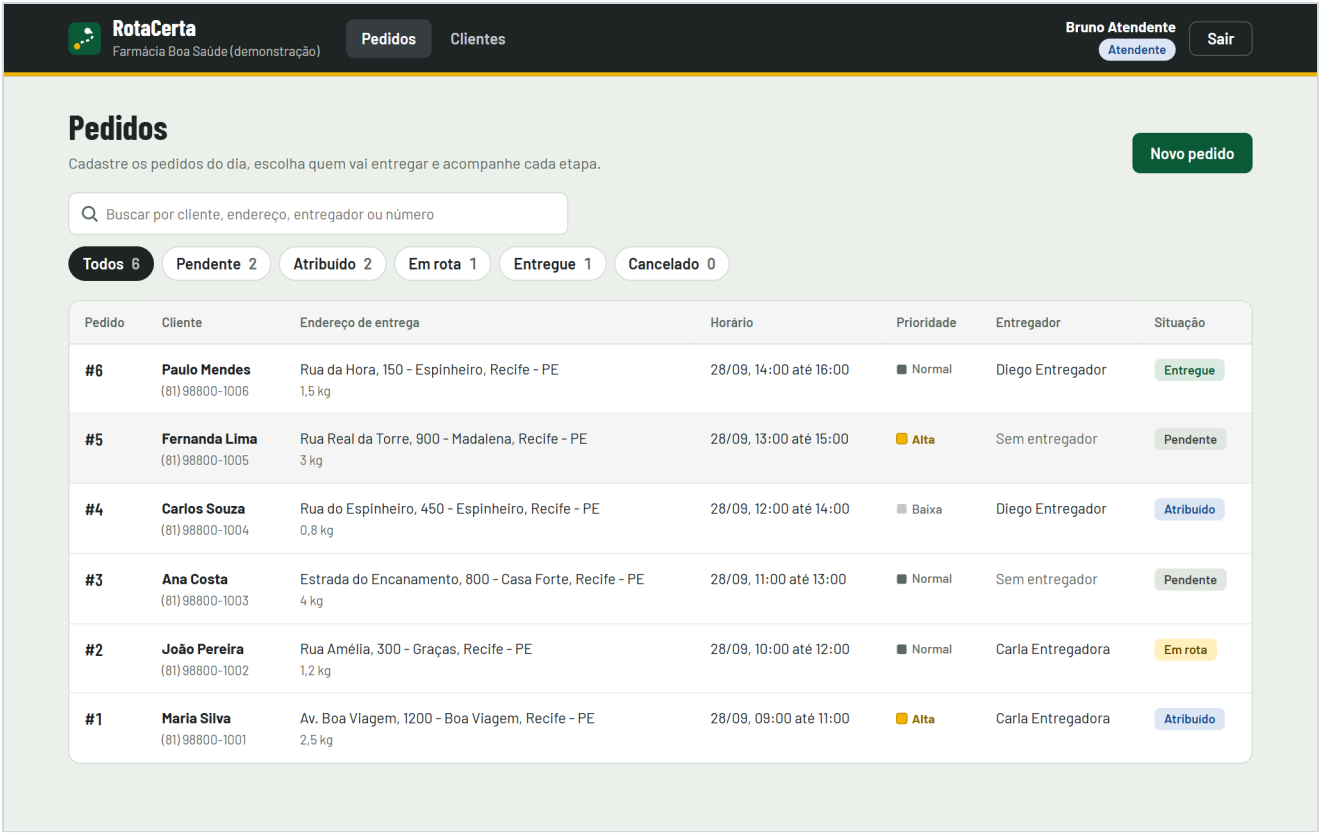


Figura 22 Lista de pedidos do atendente, com busca e filtros por situação

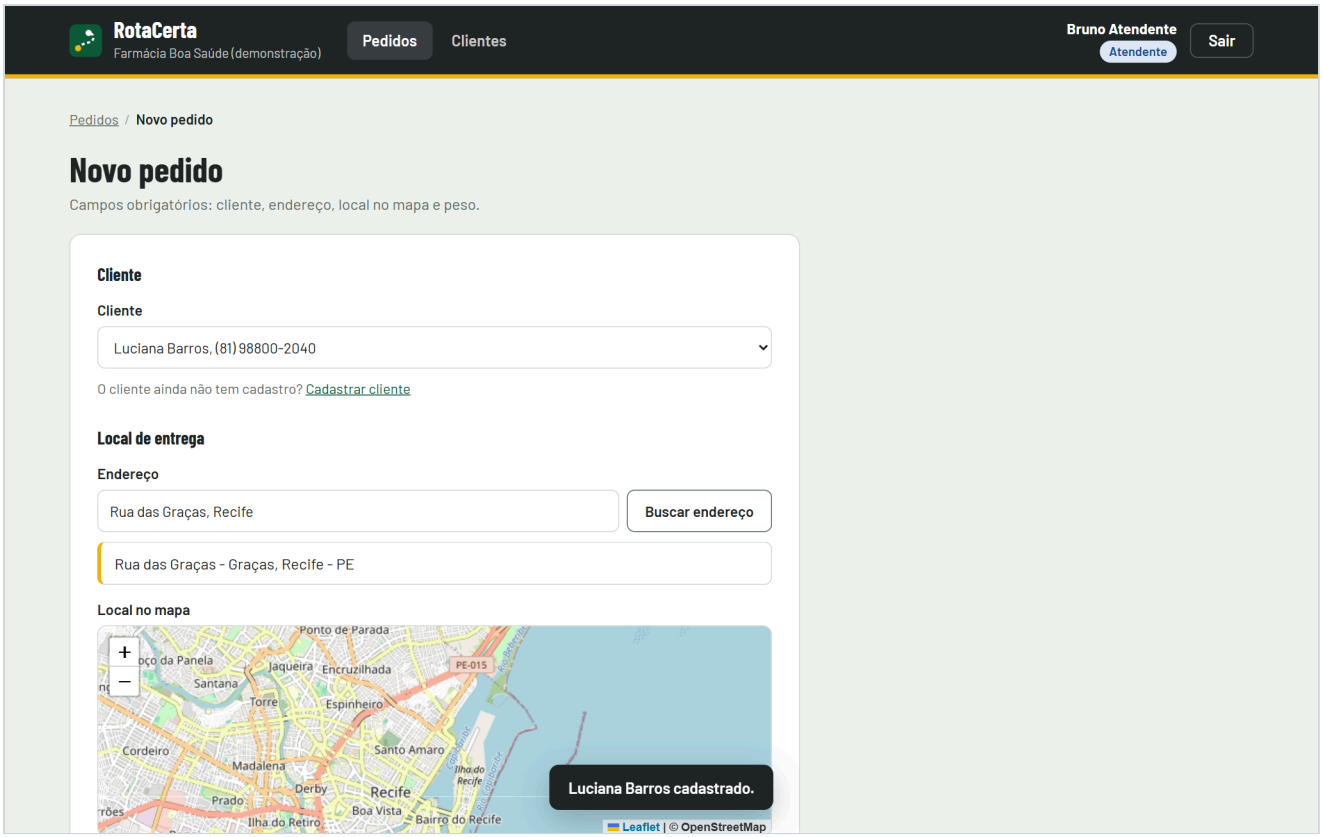


Figura 23 Pedido com a cliente recém-cadastrada já escolhida e o endereço encontrado pela busca

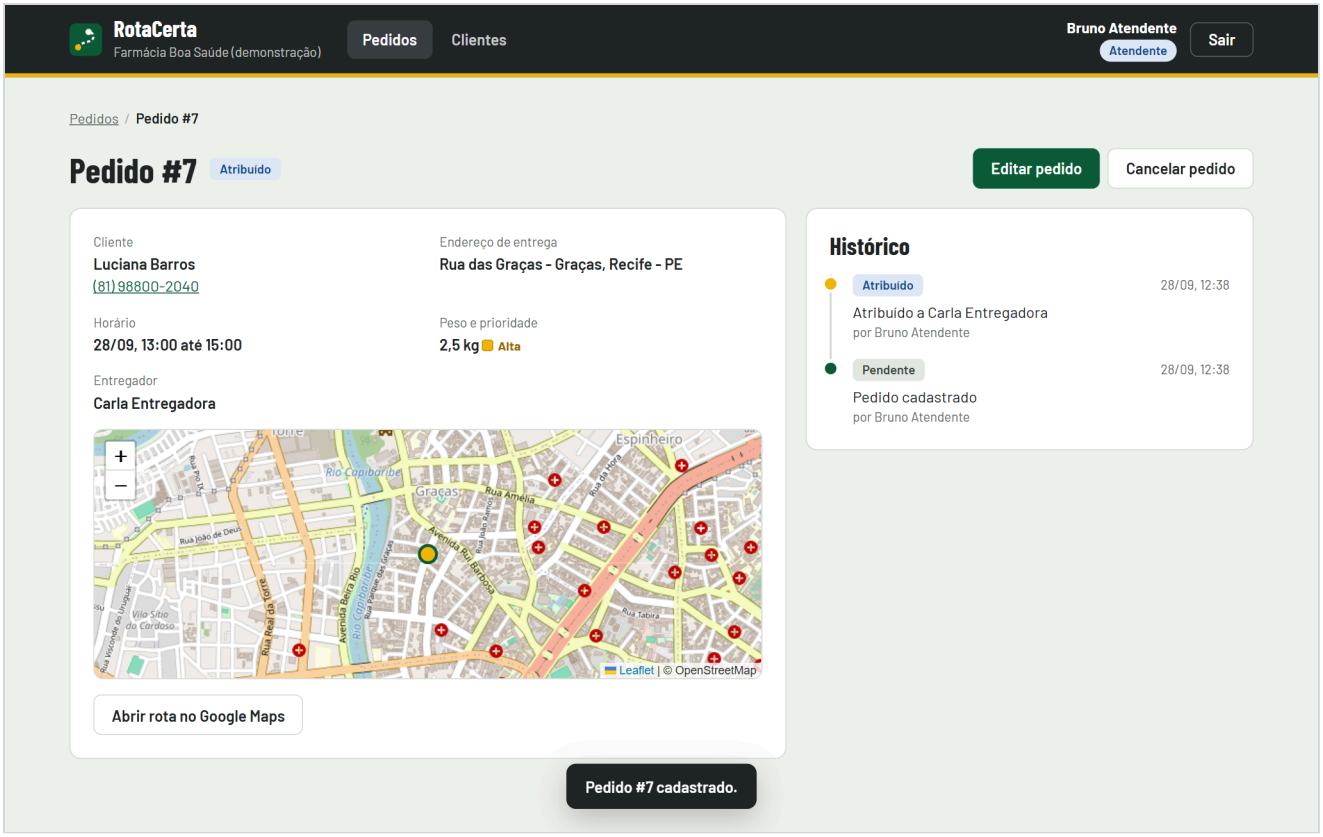


Figura 24 Página do pedido criado, com dados, mapa e histórico

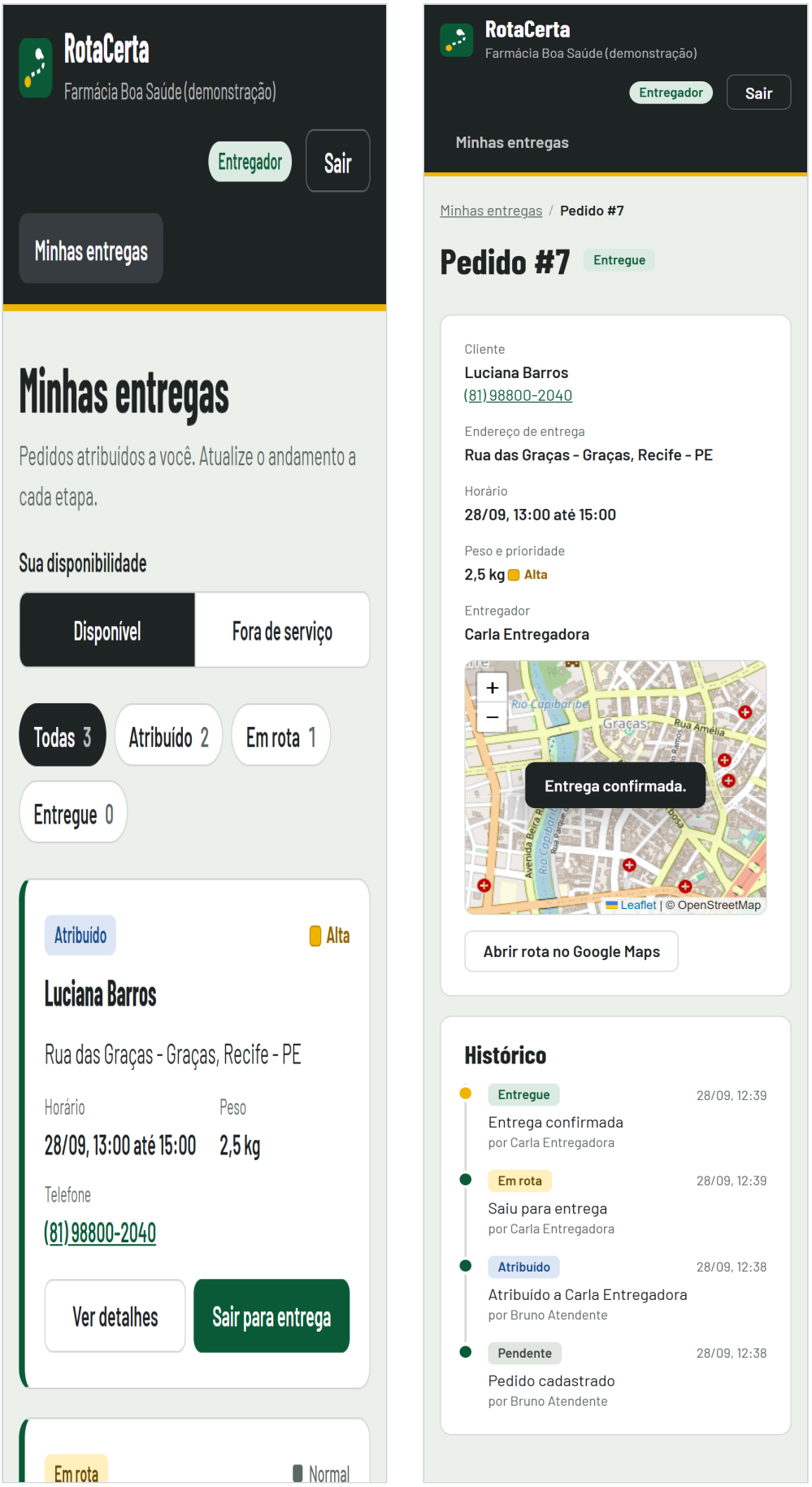


Figura 25 Entregadora no celular: lista de entregas e confirmação da entrega na página do pedido

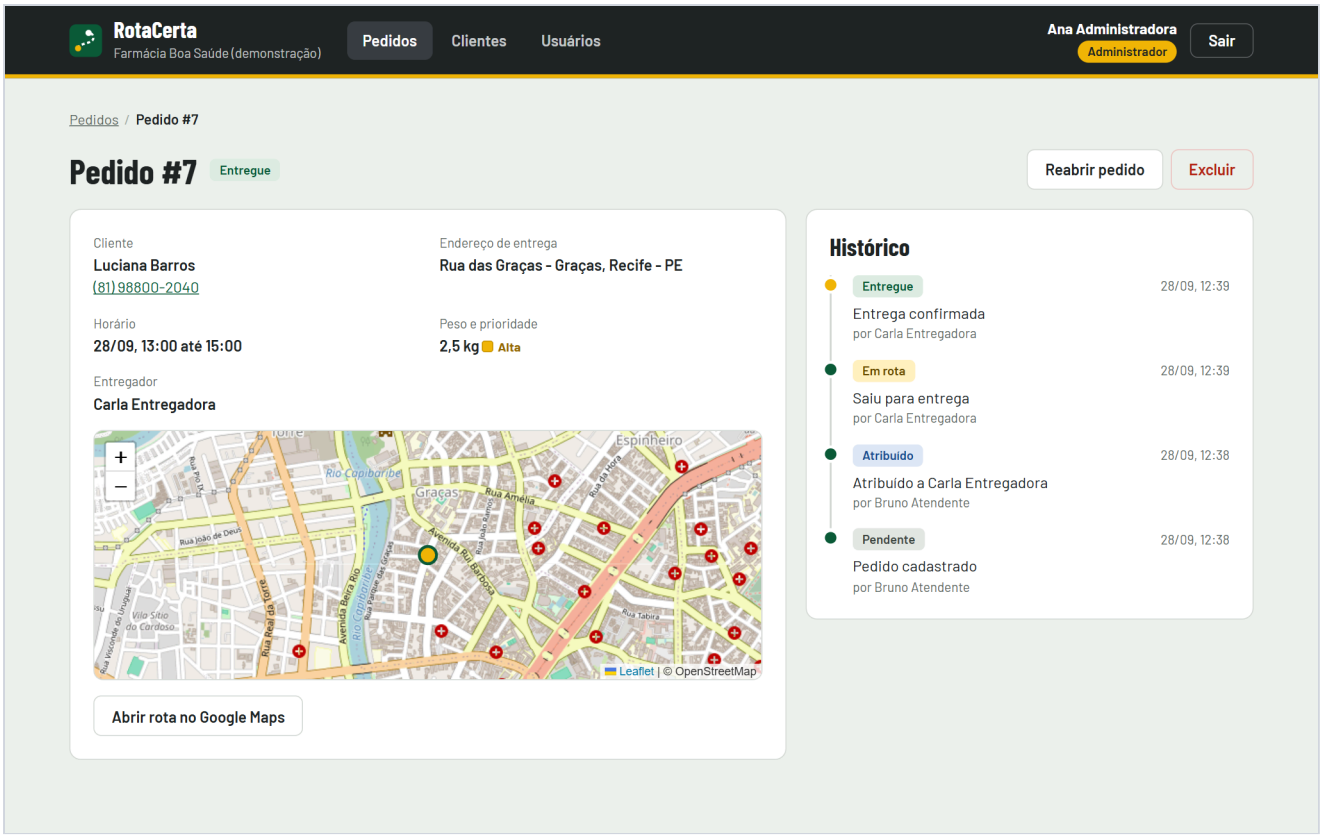


Figura 26 Visão do administrador: o histórico registra cadastro, atribuição, saída e entrega com autor e horário

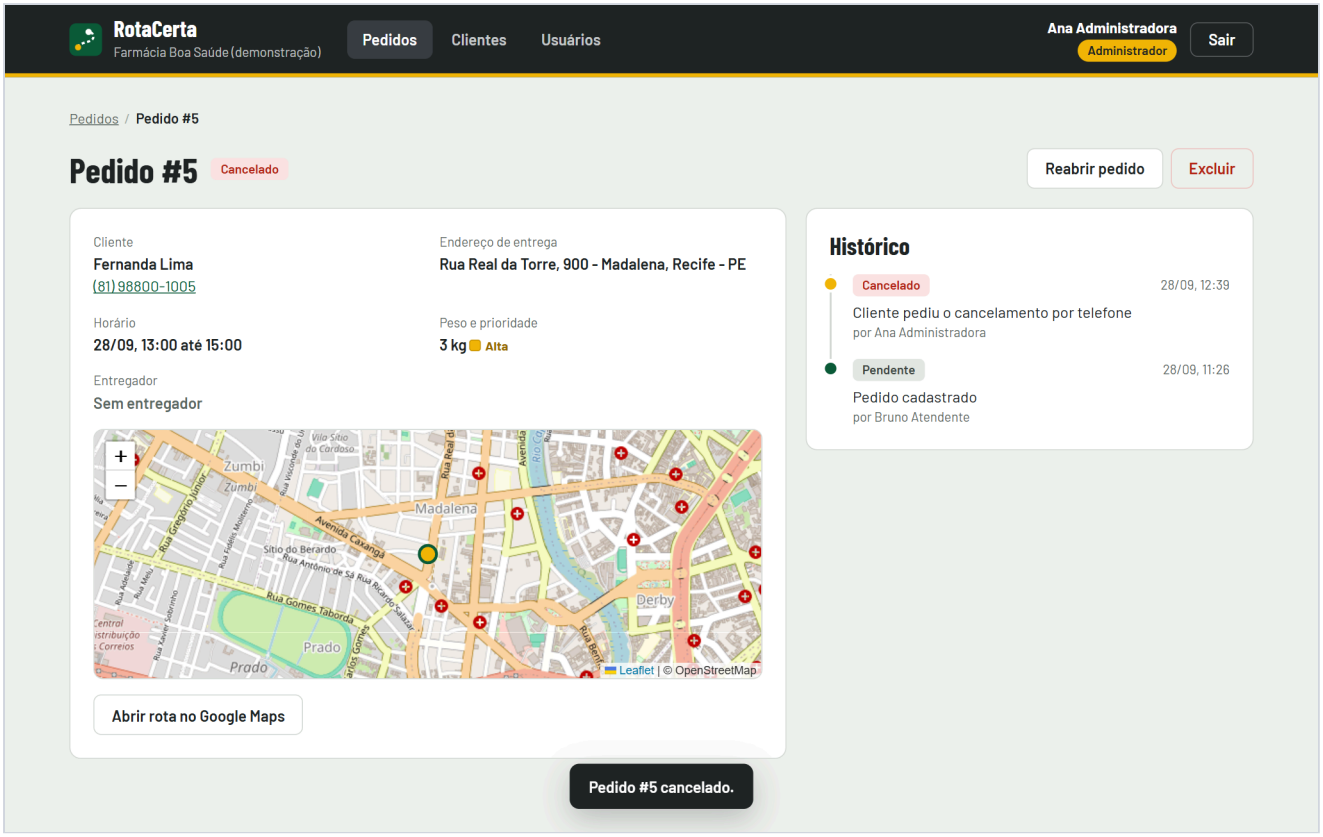


Figura 27 Pedido cancelado com o motivo registrado no histórico

27 Persistência de dados

Todos os dados do módulo ficam no PostgreSQL: clientes na tabela customers, pedidos em orders e cada mudança de situação na nova tabela order_status_history. Para demonstrar a persistência, depois do fluxo completo a API e o próprio PostgreSQL foram desligados. Com a API fora do ar, o sistema informou que o servidor não respondia. Em seguida o banco e a API foram religados e o mesmo pedido foi consultado, com os mesmos dados e o mesmo histórico.

```
[28/09/2026 12:38:49] Fase 1: fluxo completo executado com o sistema no ar.
[28/09/2026 12:39:35] API encerrada (processo uvicorn finalizado).
[28/09/2026 12:39:36] PostgreSQL encerrado com pg_ctl stop.
[28/09/2026 12:39:44] Fase 2: tentativa de uso com a API desligada.
[28/09/2026 12:40:02] PostgreSQL iniciado novamente.
[28/09/2026 12:40:17] API iniciada novamente (sem recriar tabelas nem dados de demonstração, pois o banco já tinha
registros).
[28/09/2026 12:40:17] Fase 3: sistema reaberto; consulta ao pedido #7.
GET /health -> {'status': 'ok', 'database': 'connected', 'dialect': 'postgresql', 'server_version': '16.13'}
```

Registro com horário de cada etapa do teste de persistência

```
-- Pedido #7 na tabela orders
id | cliente        | status   | entregador        | peso_kg | latitude  | longitude
---+----------------+----------+-------------------+---------+-----------+-----------
7  | Luciana Barros | ASSIGNED | Carla Entregadora | 2.50    | -8.047960 | -34.901177
(1 linha)

-- Histórico do pedido #7 na tabela order_status_history
status   | observacao                    | autor           | horario
---------+-------------------------------+-----------------+---------------
PENDING  | Pedido cadastrado             | Bruno Atendente | 28/09 12:38:56
ASSIGNED | Atribuído a Carla Entregadora | Bruno Atendente | 28/09 12:38:56
(2 linhas)
```

Pedido e histórico no banco logo depois do cadastro pela interface

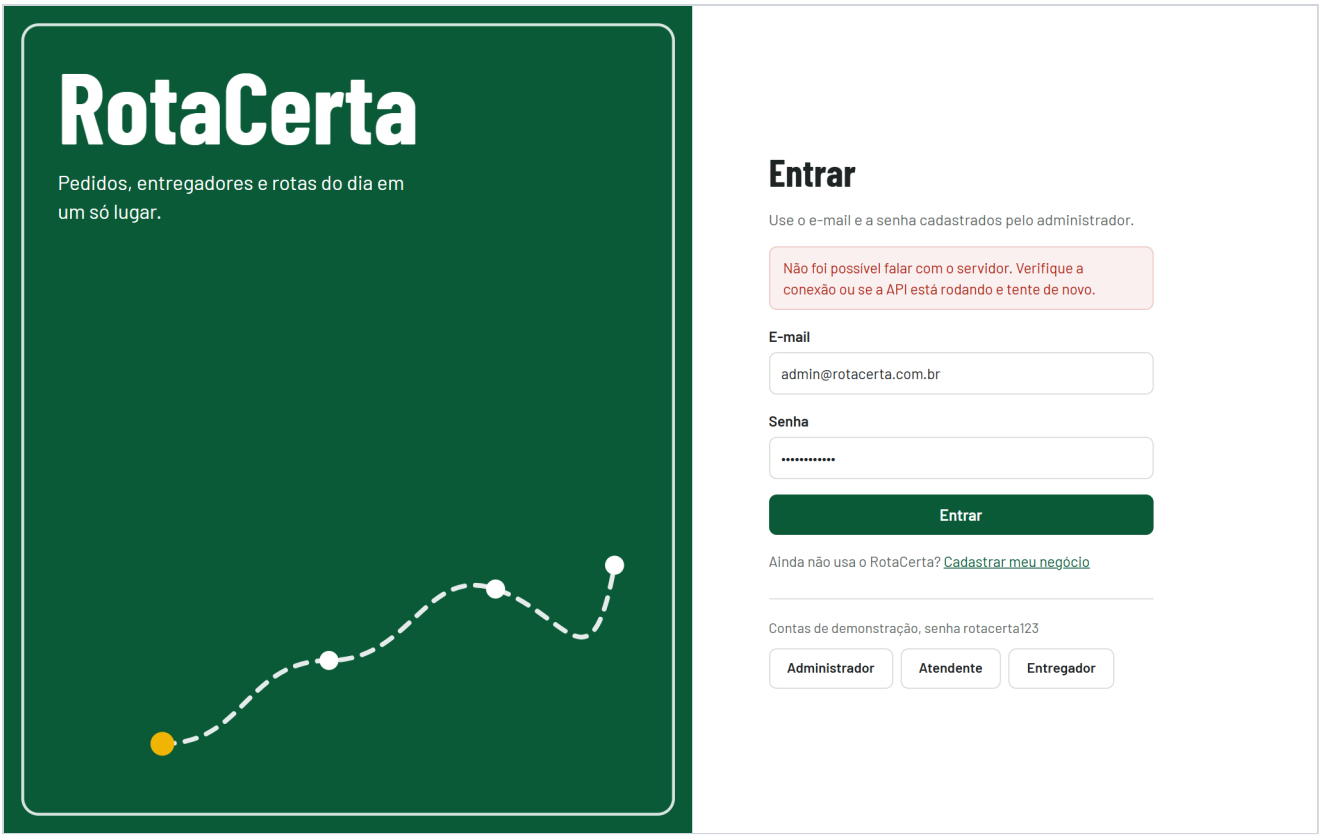


Figura 28 Tentativa de entrar com a API desligada: o sistema explica o problema em vez de travar

```
-- Pedido #7 na tabela orders
id | cliente        | status    | entregador        | peso_kg | latitude  | longitude
---+----------------+-----------+-------------------+---------+-----------+-----------
7  | Luciana Barros | DELIVERED | Carla Entregadora | 2.50    | -8.047960 | -34.901177
(1 linha)

-- Histórico do pedido #7 na tabela order_status_history
status    | observacao                    | autor             | horario
----------+-------------------------------+-------------------+--------------
PENDING   | Pedido cadastrado             | Bruno Atendente   | 28/09 12:38:56
ASSIGNED  | Atribuído a Carla Entregadora | Bruno Atendente   | 28/09 12:38:56
IN_ROUTE  | Saiu para entrega             | Carla Entregadora | 28/09 12:39:00
DELIVERED | Entrega confirmada            | Carla Entregadora | 28/09 12:39:01
(4 linhas)
```

O mesmo pedido consultado no banco depois de desligar e religar o PostgreSQL e a API

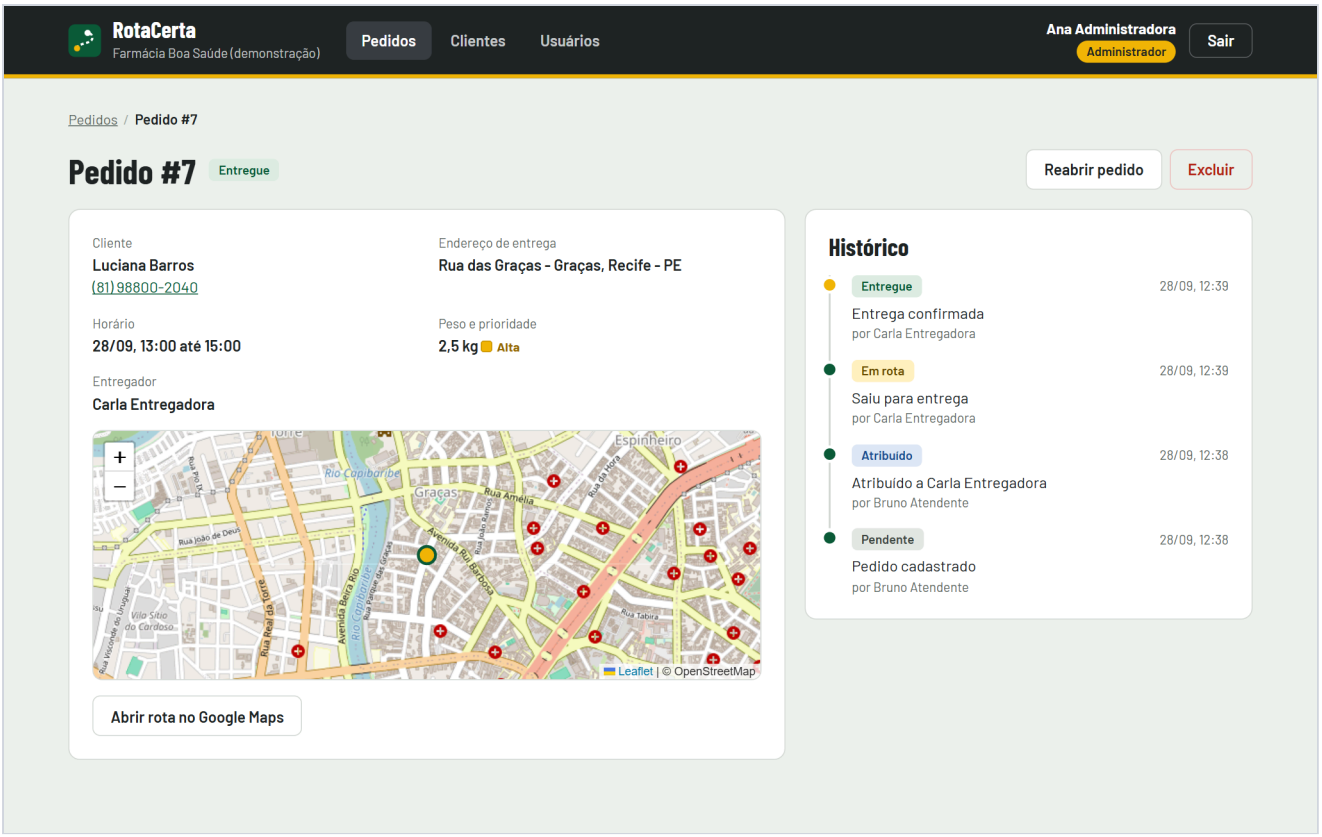


Figura 29 Página do pedido depois da reabertura do sistema, com o histórico preservado

Na execução com Docker Compose os dados ficam no volume postgres_data, que é mantido ao parar e subir os contêineres. Apenas docker compose down -v apaga o volume e recomeça o banco do zero.

28 Validações

As validações existem em duas camadas. A interface verifica os campos antes de enviar, mostra a mensagem junto ao campo e leva o foco ao primeiro campo com problema. A API repete as mesmas regras e aplica as regras de negócio que dependem do banco, de modo que nenhum dado inválido é gravado mesmo que alguém chame a API diretamente.

Campo ou regra	Onde	Regra
E-mail e senha no login	Interface	Obrigatórios; e-mail em formato válido
Senha de novos usuários	Interface e API	Mínimo de 8 caracteres, com letras e números
Nome de cliente e de usuário	Interface e API	Obrigatório, com pelo menos duas letras; espaços extras são removidos
Telefone do cliente	Interface e API	Opcional; se informado, precisa ter DDD. É gravado no formato (81) 98800-1001
Cliente duplicado	API	Não permite outro cliente com o mesmo nome e telefone no estabelecimento
Cliente, endereço e local do pedido	Interface e API	Cliente do próprio estabelecimento; endereço com pelo menos 5 caracteres; coordenadas marcadas no mapa
Peso do pedido	Interface e API	Maior que zero e até 500 kg

Campo ou regra	Onde	Regra
Janela de entrega	Interface e API	Fim depois do início; no cadastro, o horário final não pode estar no passado
Raio de entrega	API	O local precisa estar a até 30 km do ponto de saída do estabelecimento
Capacidade do entregador	Interface e API	A soma dos pedidos atribuídos e em rota não pode passar da capacidade de carga
Disponibilidade	Interface e API	Entregador fora de serviço não recebe pedidos e só fica fora de serviço sem entregas em rota
Mudança de situação	API	O entregador só avança de Atribuído para Em rota e para Entregue; pedidos entregues ou cancelados não são editados
Motivo do cancelamento	Interface	Obrigatório, com pelo menos 5 caracteres, e registrado no histórico

Figura 30 Login enviado sem preencher os campos

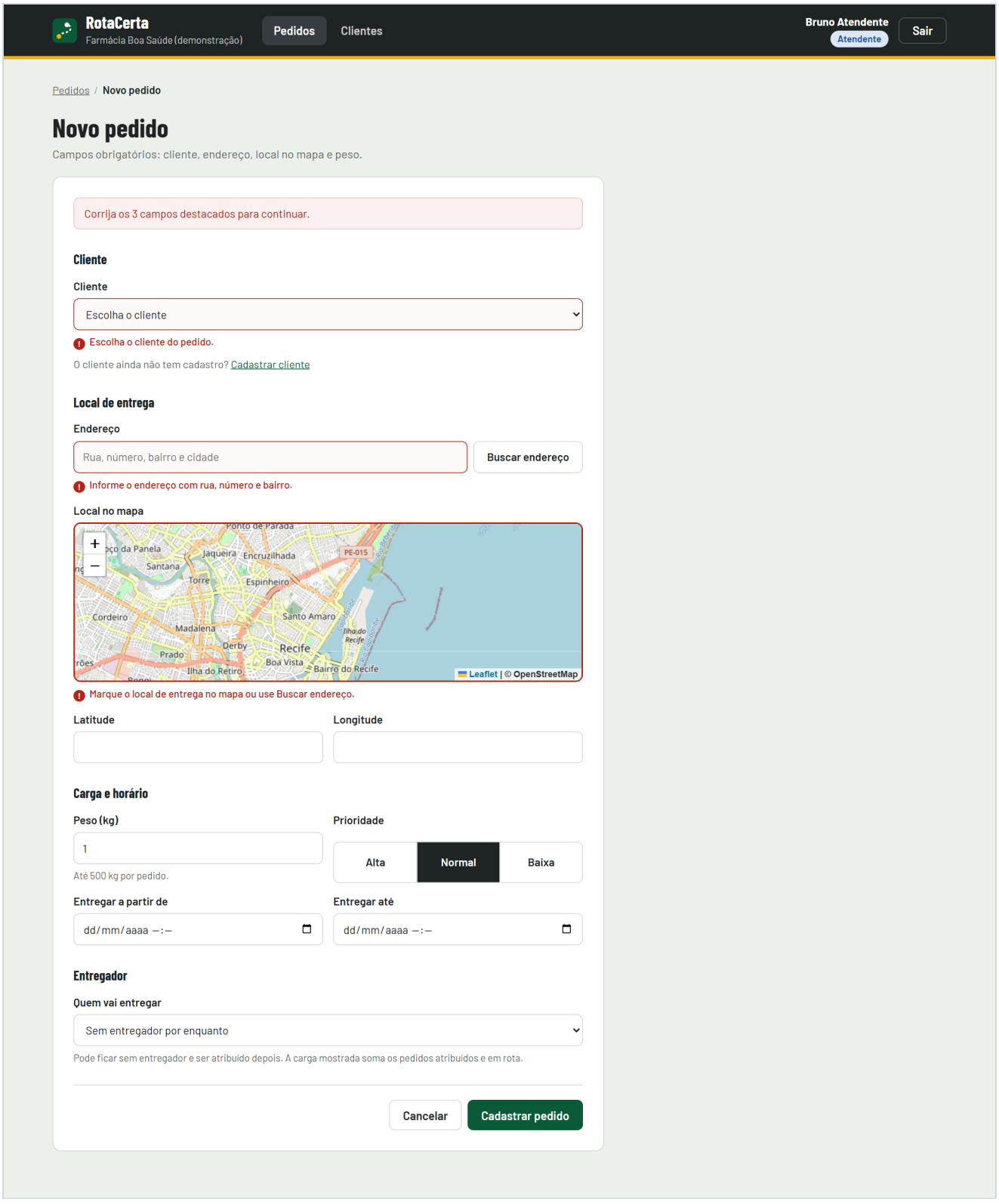


Figura 31 Pedido enviado vazio: cada campo obrigatório indica o que falta e o mapa fica destacado

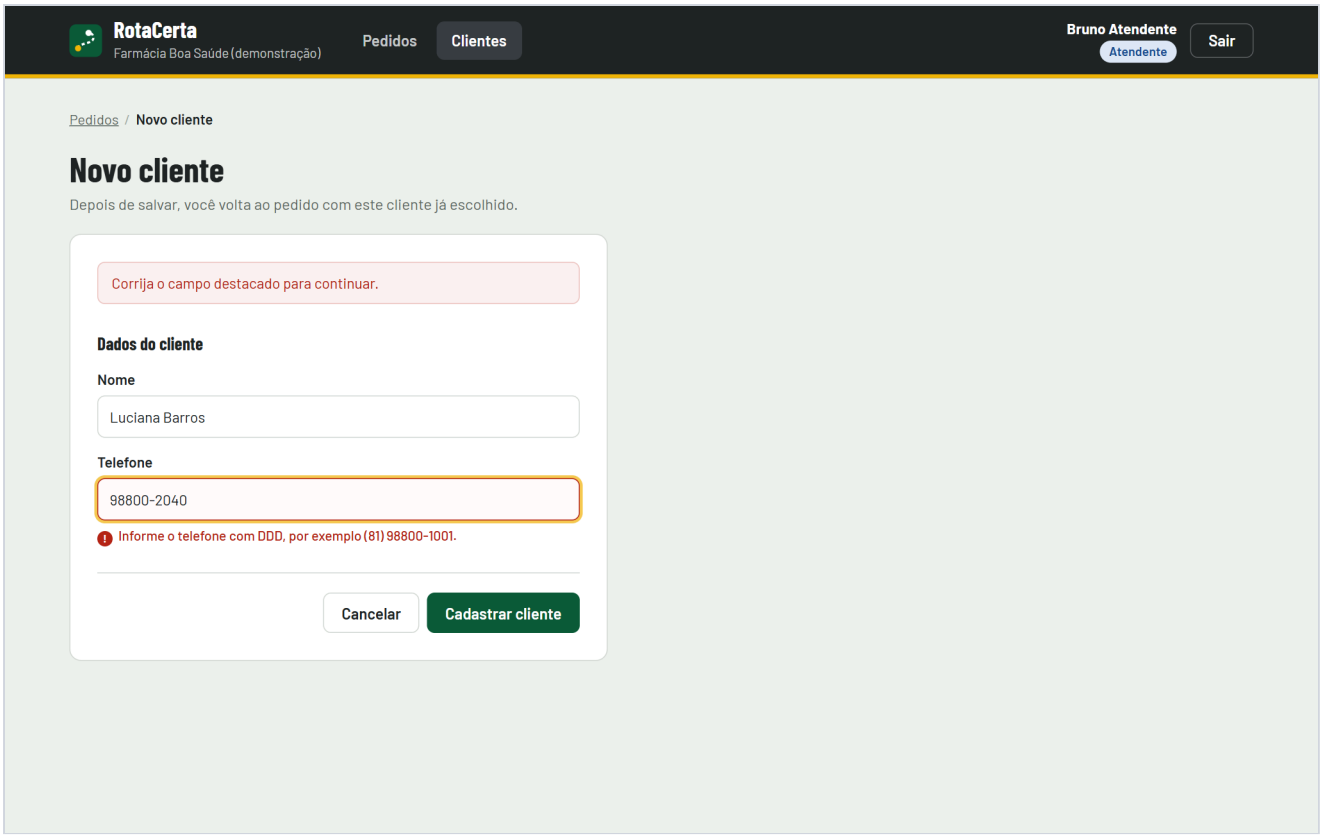


Figura 32 Telefone sem DDD recusado no cadastro de cliente

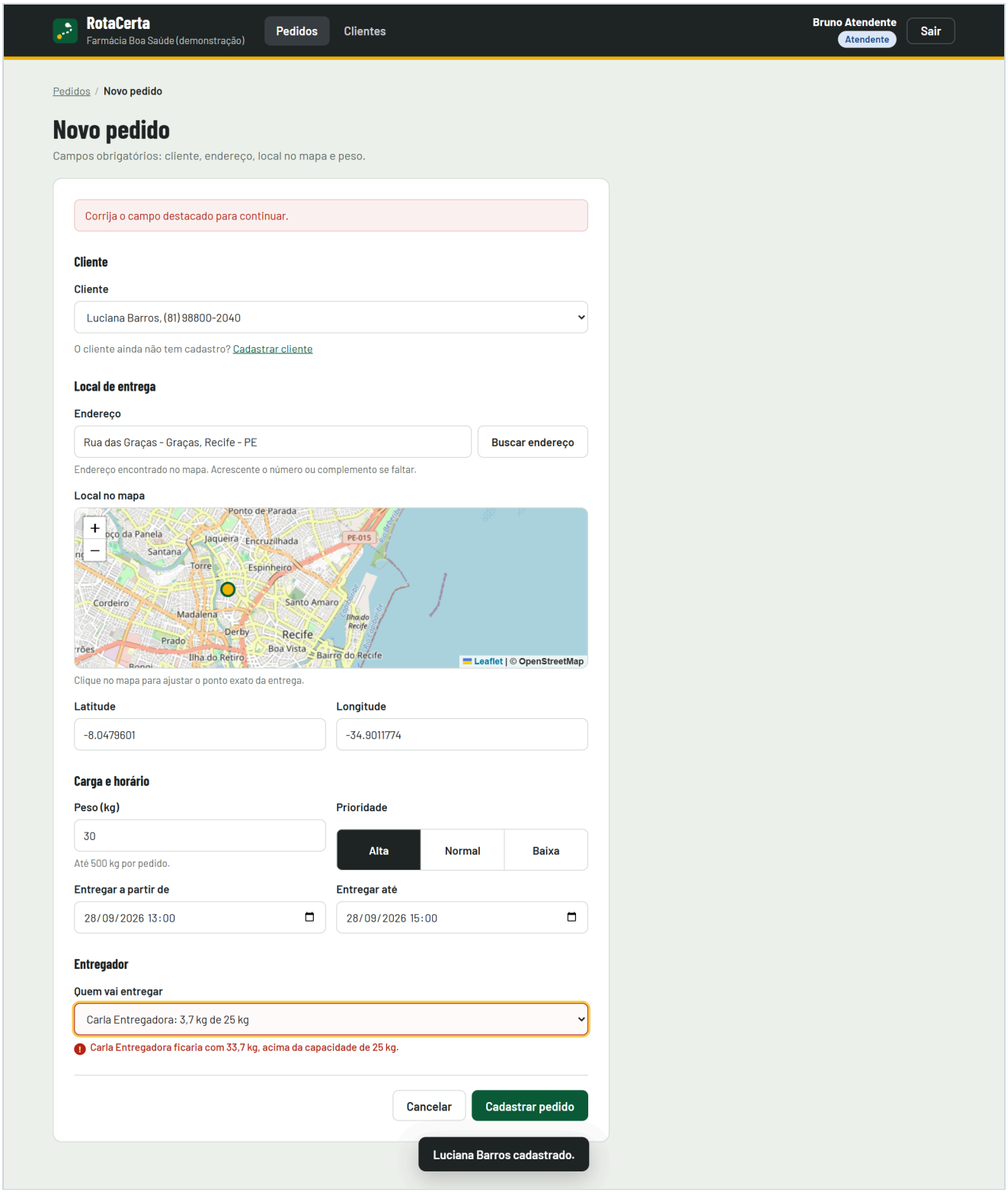


Figura 33 Pedido de 30 kg recusado para uma entregadora com capacidade de 25 kg

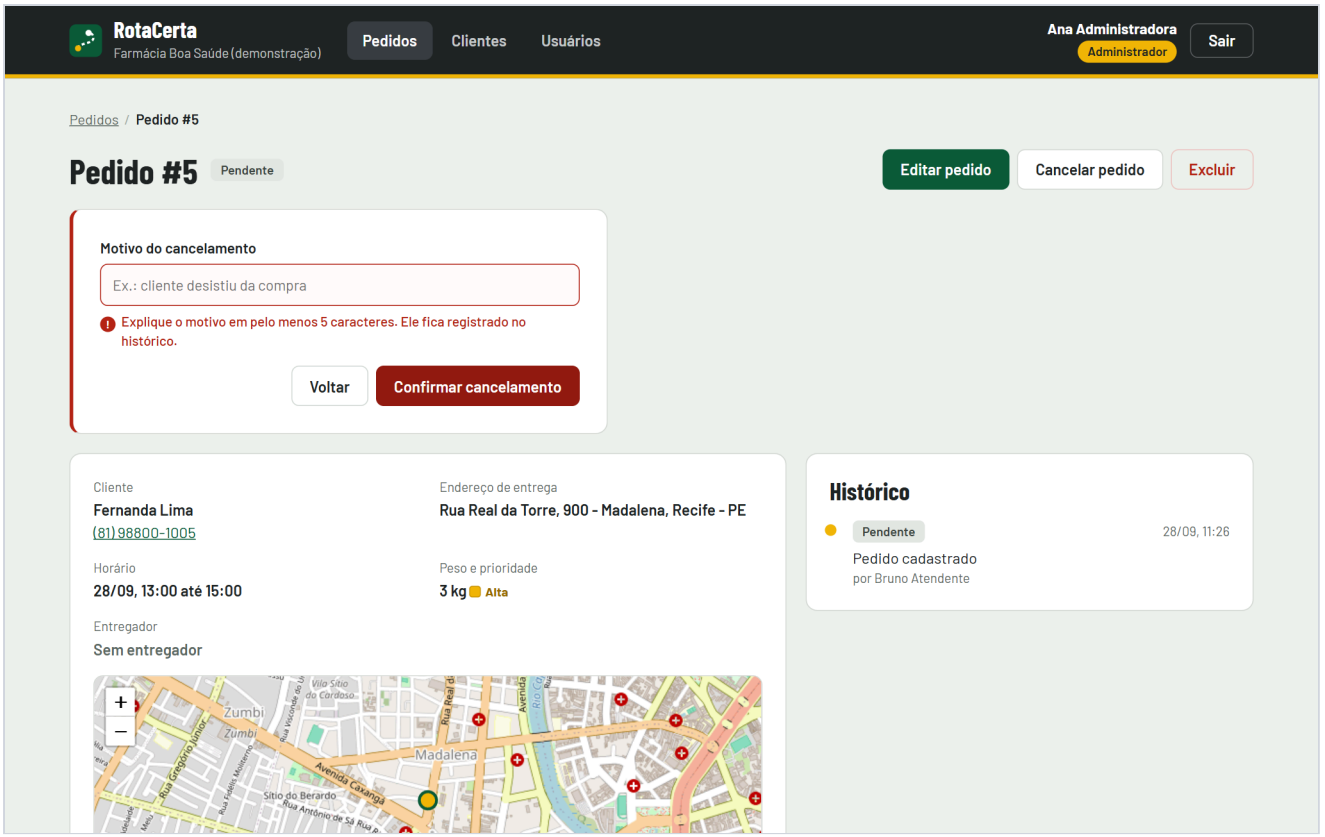


Figura 34 Cancelamento sem motivo não é aceito

```
POST /customers com telefone sem DDD
  -> HTTP 422 phone: Value error, Informe o telefone com DDD, por exemplo (81) 98800-1001.
POST /customers com nome sem letras
  -> HTTP 422 full_name: Value error, Informe um nome com pelo menos duas letras.
POST /customers repetindo nome e telefone
  -> HTTP 409 Já existe um cliente com este nome e telefone.
POST /users com senha sem números
  -> HTTP 422 password: Value error, A senha precisa ter letras e números.
POST /orders com peso de 600 kg
  -> HTTP 422 weight_kg: Input should be less than or equal to 500
POST /orders com fim da janela antes do início
  -> HTTP 422 body: Value error, O fim da janela de entrega deve ser depois do início.
POST /orders com horário final no passado
  -> HTTP 422 O horário final da entrega já passou. Informe um horário futuro.
POST /orders fora do raio de entrega
  -> HTTP 422 O local de entrega fica a 99,8 km do ponto de saída. O limite de entrega é 30 km.
POST /orders para Diego Entregador com 45 kg
  -> HTTP 409 Com este pedido, Diego Entregador ficaria com 45,8 kg em entregas, acima da capacidade de 40 kg. Escolha
outro entregador ou divida o pedido.
DELETE /customers de cliente com pedidos
  -> HTTP 409 Maria Silva tem 1 pedido e não pode ser excluído.
GET /geocode com texto curto
  -> HTTP 422 q: String should have at least 5 characters
```

Respostas da API às mesmas regras quando chamada diretamente

As mensagens de validação padrão da API aparecem em inglês quando ela é chamada diretamente, como no quadro acima. A interface traduz essas mensagens e as exibe em português junto ao campo correspondente.

29 Mensagens de erro

Cada tipo de falha tem uma mensagem própria, escrita para orientar o próximo passo do usuário. Nenhuma tela mostra erros técnicos, códigos internos ou páginas em branco.

Situação	Como o sistema responde
Campo inválido	Mensagem abaixo do campo, borda vermelha, resumo no topo do formulário e foco no primeiro campo com erro
Regra de negócio recusada pela API	A mensagem da API aparece junto ao campo relacionado, como entregador, local no mapa ou horário
Ação que não pode ser concluída	Aviso vermelho temporário explicando o motivo, como cliente com pedidos ou entrega em rota
Ação irreversível	Janela de confirmação antes de excluir pedidos, clientes ou desativar usuários
Página de outro perfil	Tela Acesso negado indicando quais perfis usam a página
Endereço inexistente	Tela Página não encontrada com o endereço digitado e link para o início
API ou internet fora do ar	Mensagem de servidor indisponível e, nas listas, a opção Tentar de novo
Busca de endereço indisponível	Orientação para marcar o local direto no mapa
Sessão expirada	Volta para o login com o aviso Sua sessão expirou
Falha inesperada ao exibir uma tela	Tela de erro com o botão Recarregar a página, sem afetar o restante do sistema

Figura 35 Entregadora com entrega em rota tentando ficar fora de serviço

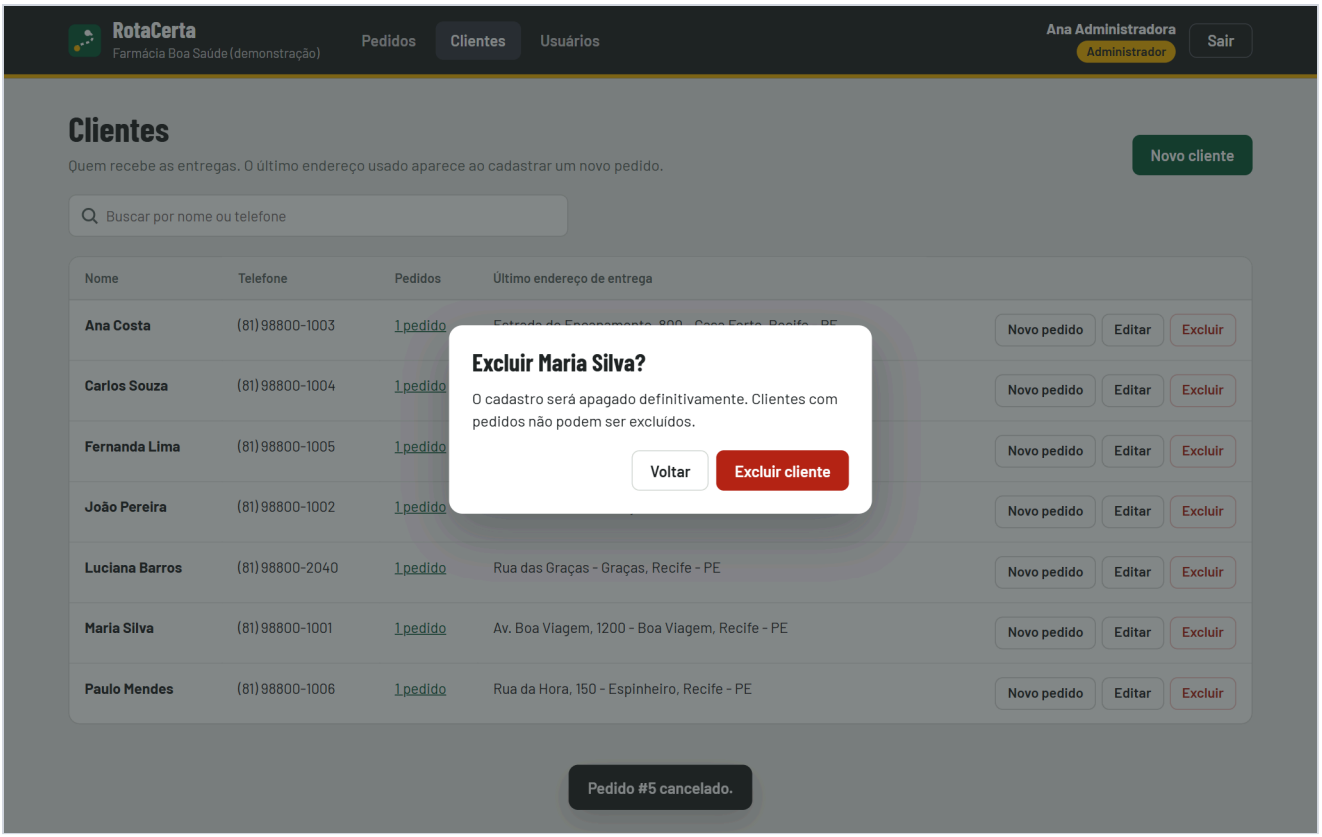


Figura 36 Confirmação antes de excluir um cliente

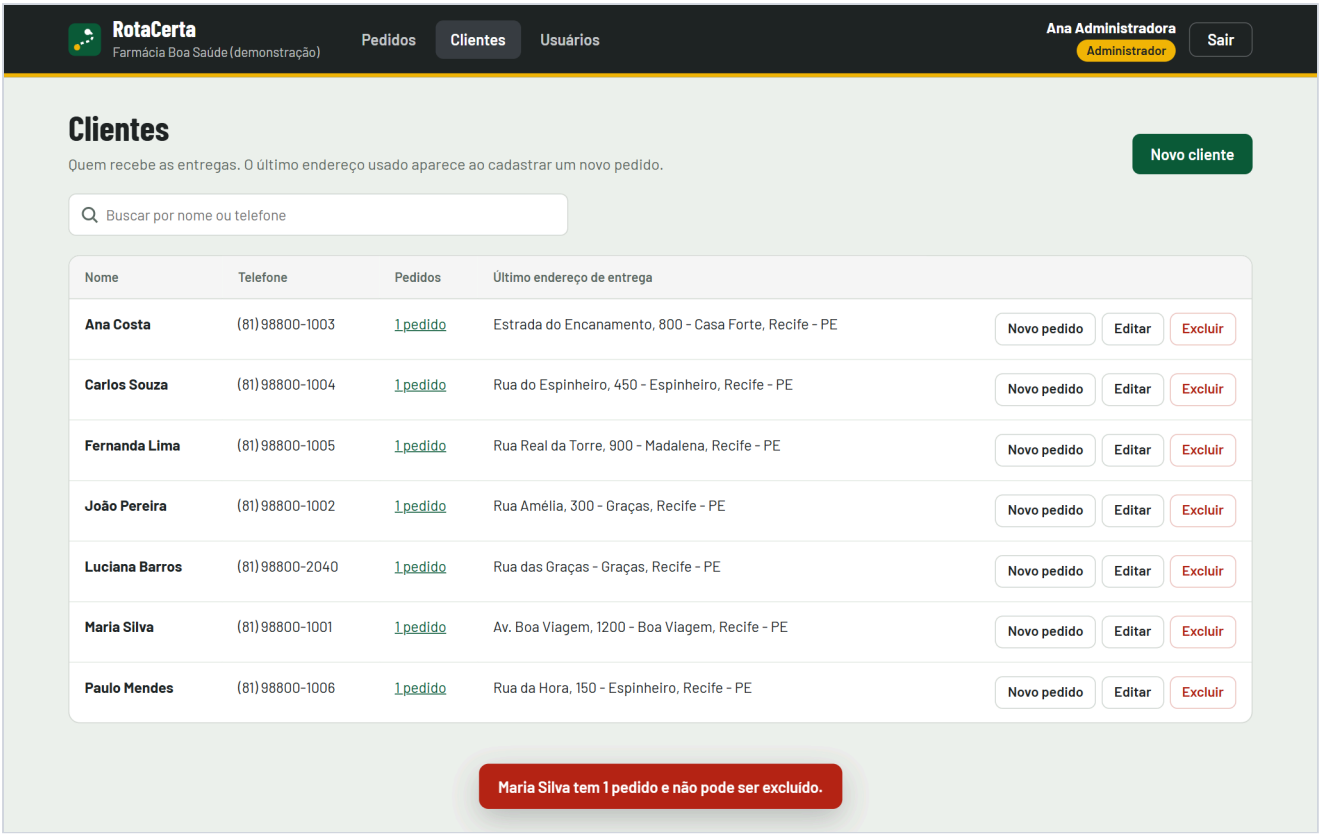


Figura 37 Cliente com pedidos não pode ser excluído

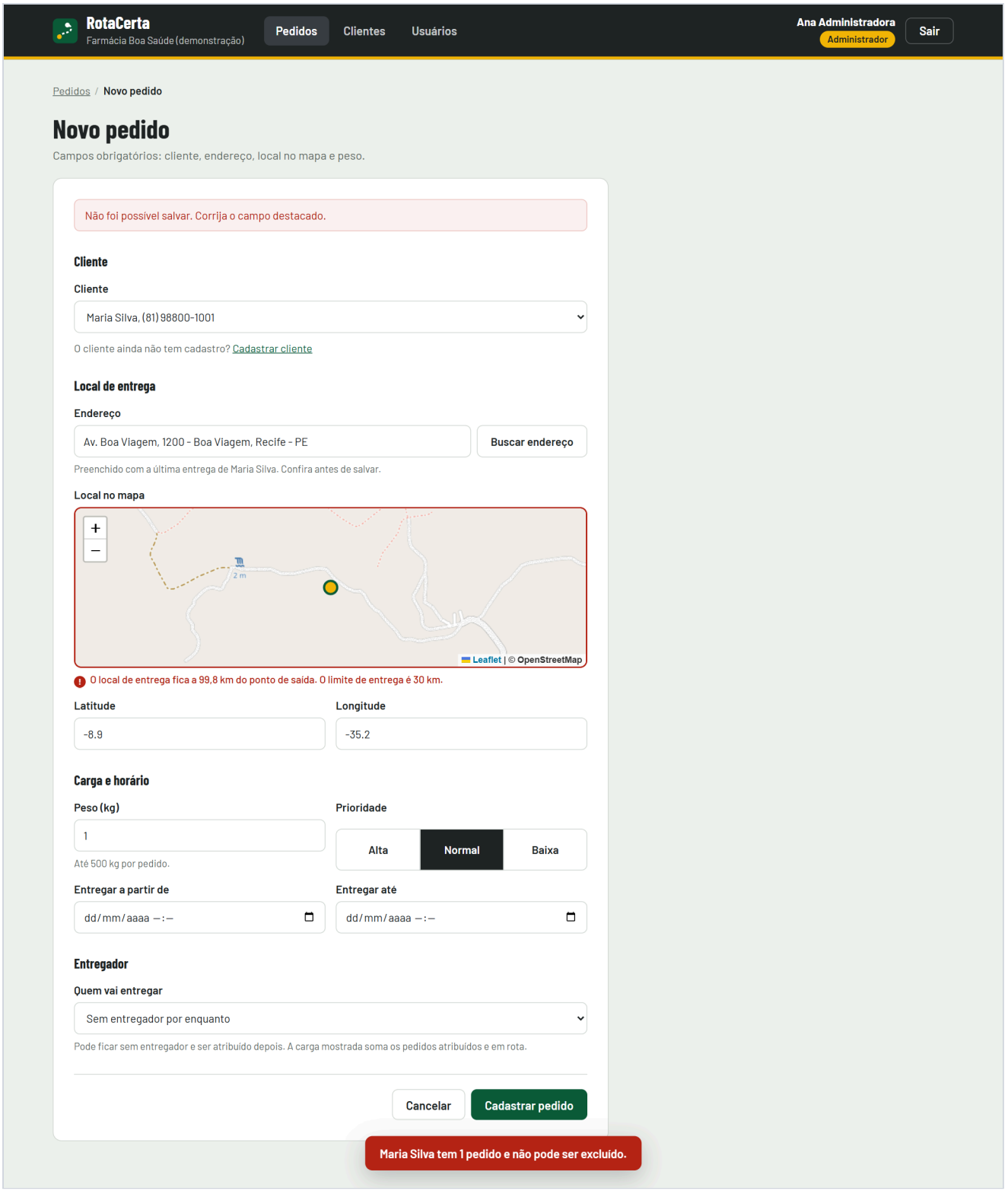


Figura 38 Local de entrega fora do raio de 30 km, recusado pela API e indicado no mapa

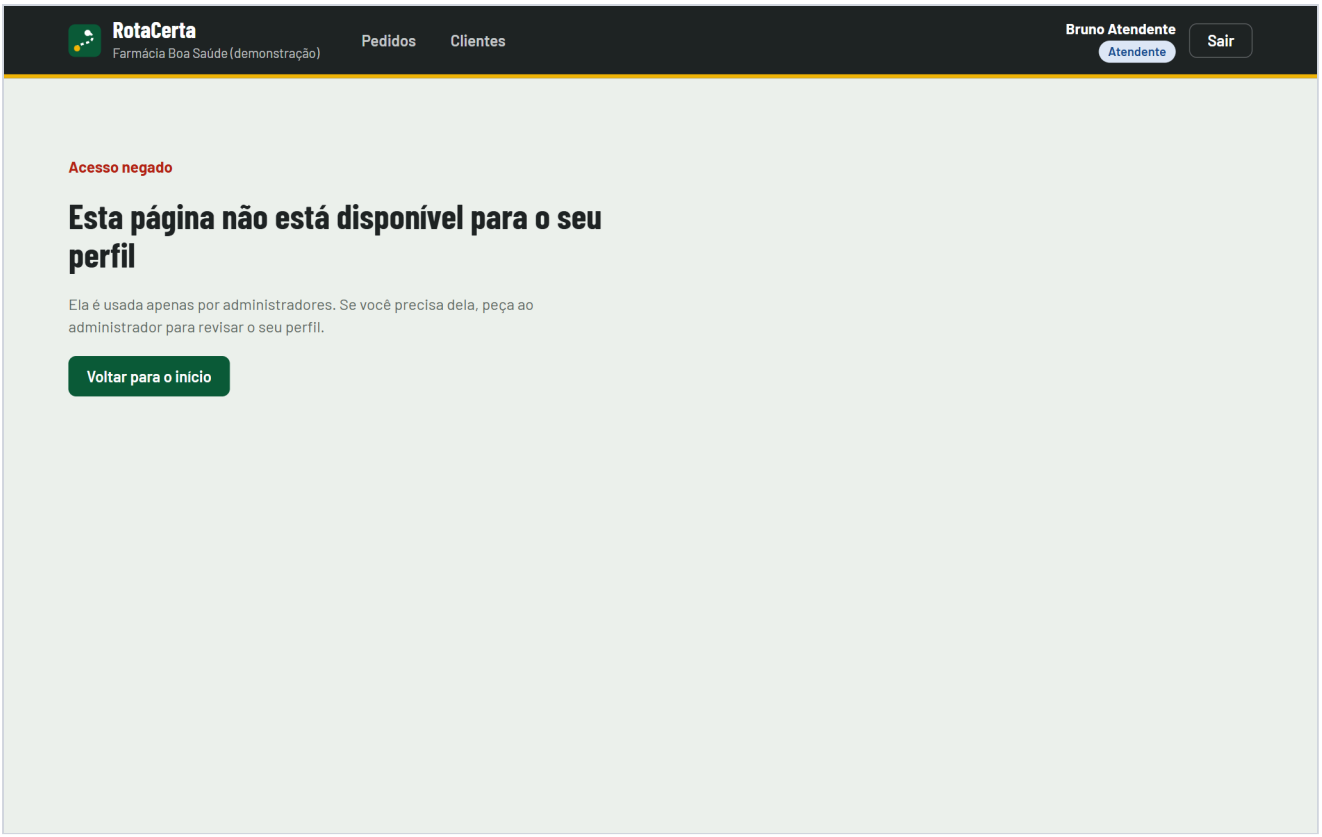


Figura 39 Atendente tentando abrir a página de usuários

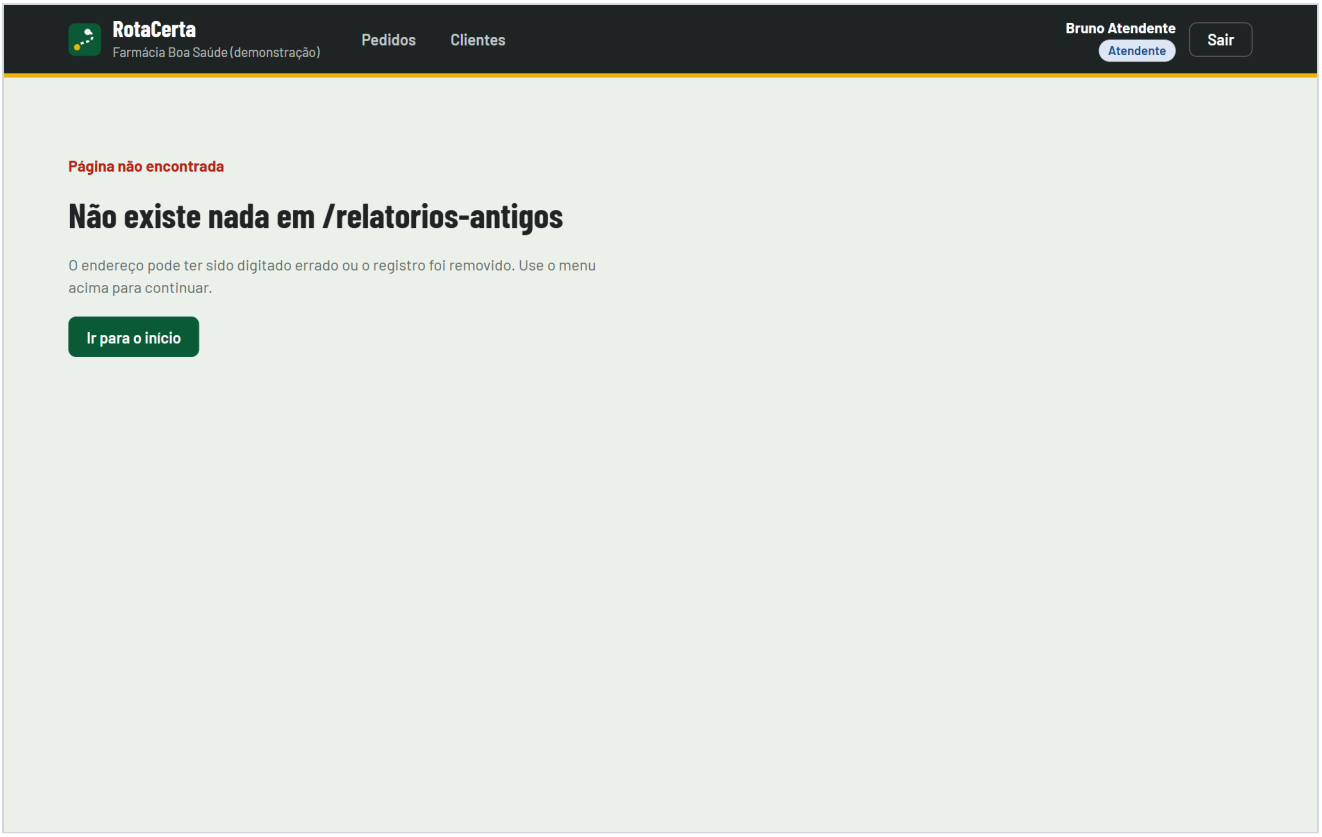


Figura 40 Endereço que não existe no sistema

30 Navegação entre as telas

O frontend passou a usar o React Router. Cada tela tem um endereço próprio, o botão Voltar do navegador funciona, a busca e o filtro da lista ficam no endereço e podem ser compartilhados, e as páginas internas mostram o caminho de navegação (Pedidos / Pedido #7). As rotas são protegidas: sem login o sistema leva para a tela de entrada e, depois do login, abre a página pedida originalmente.

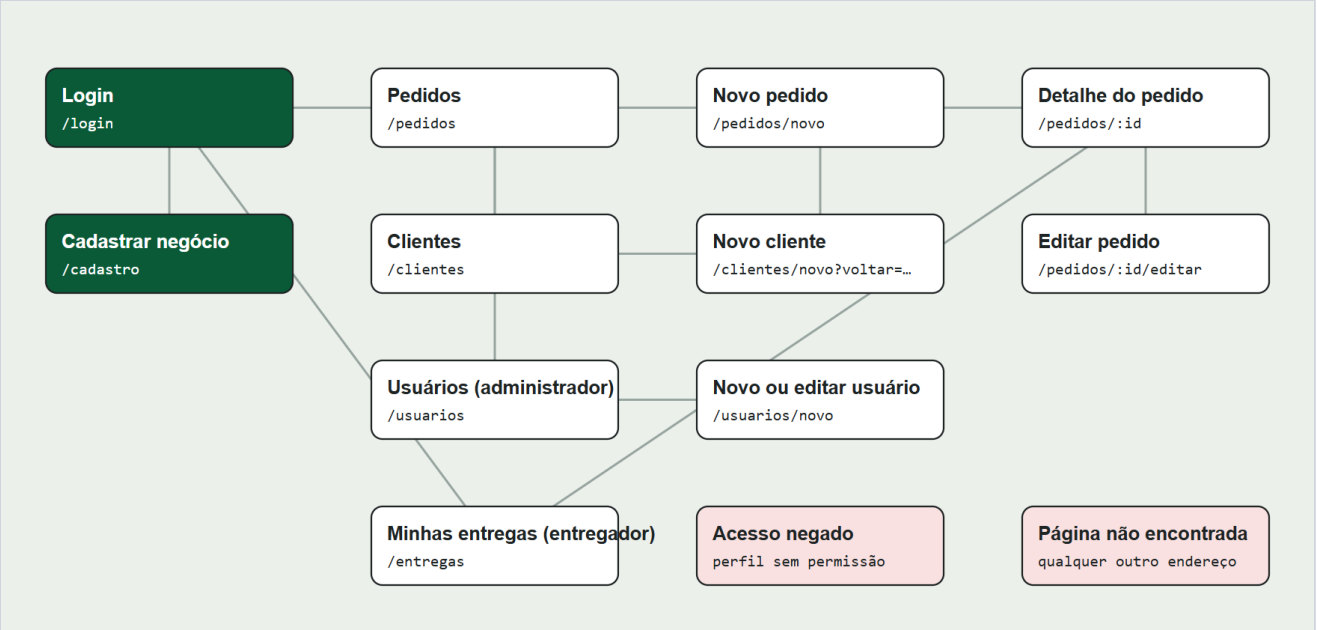


Figura 41 Mapa de navegação entre as telas do módulo

Endereço	Tela	Perfis
/login e /cadastro	Entrada e cadastro de novo negócio	Público
/pedidos	Lista de pedidos com busca e filtros	Administrador e atendente
/pedidos/novo	Cadastro de pedido	Administrador e atendente
/pedidos/:id	Detalhe e histórico do pedido	Todos; o entregador só abre os seus
/pedidos/:id/editar	Edição do pedido	Administrador e atendente
/clientes, /clientes/novo e /clientes/:id/editar	Clientes	Administrador e atendente
/entregas	Entregas do entregador	Entregador
/usuarios, /usuarios/novo e /usuarios/:id/editar	Usuários	Administrador

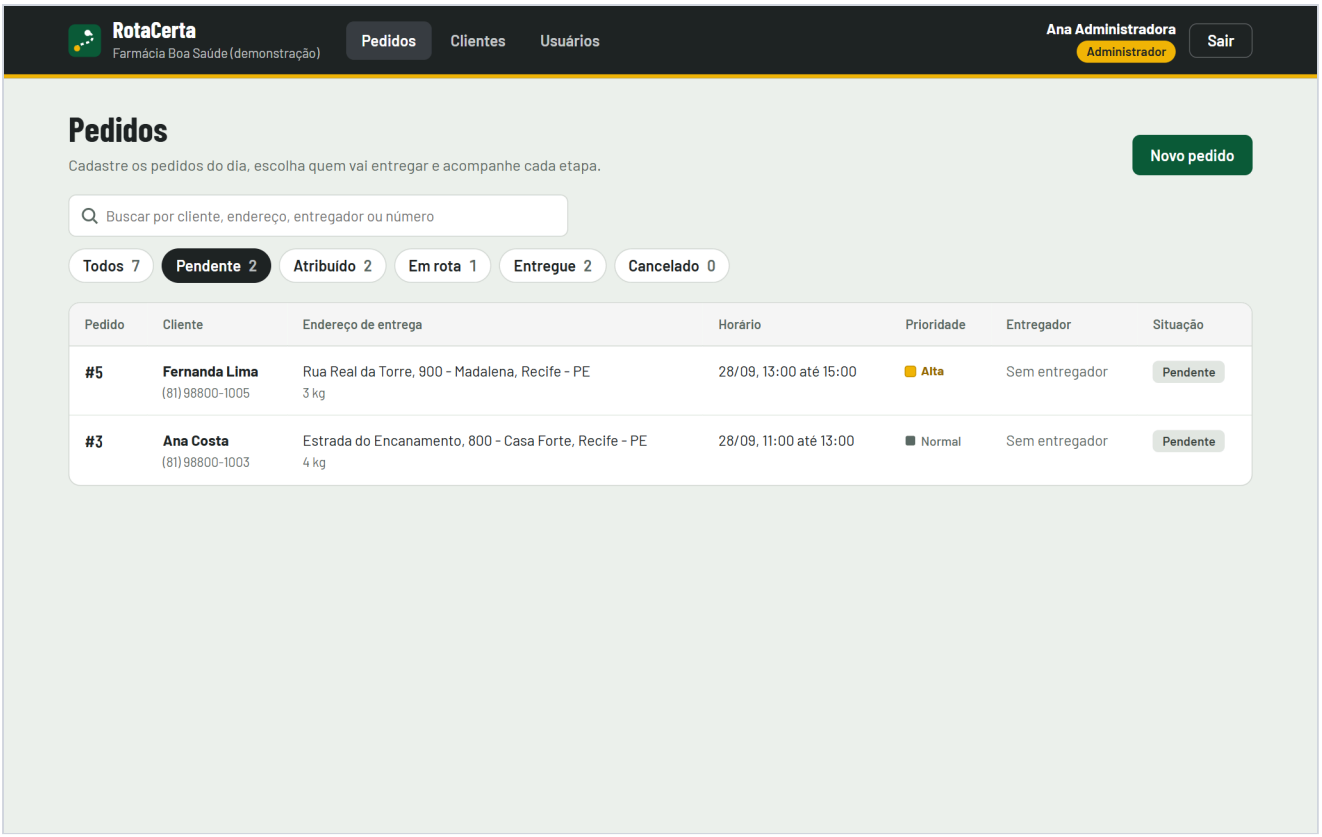


Figura 42 Filtro por situação guardado no endereço /pedidos?situacao=PENDING

```
Sem sessão, o endereço http://localhost:5173/pedidos/7 levou para http://localhost:5173/login.
Depois do login, o sistema abriu http://localhost:5173/pedidos/7, a página pedida originalmente.
```

Retorno à página pedida depois do login

31 Commits organizados e repositório

Repositório oficial. https://github.com/alvccpj/rota-certa

A Sprint 04 foi desenvolvida na branch sprint/04-modulo-pedidos-entregas, criada a partir da master com a Sprint 03 já integrada pelo Pull Request número 2. Cada etapa foi registrada em um commit próprio no momento em que ficou pronta e testada, seguindo o padrão do CONTRIBUTING.md (feat, fix, docs).

```
$ git log --reverse master..HEAD
5368fad 28/09 12:17 feat: adiciona cadastro de clientes com validação de nome e telefone
3bcaf0c 28/09 12:19 feat: valida atribuição por disponibilidade, capacidade e raio de entrega
b16b967 28/09 12:22 feat: registra histórico de situação dos pedidos
43ec835 28/09 12:23 feat: adiciona busca de endereço pelo OpenStreetMap
d9b0753 28/09 12:26 feat: adiciona validação de campos e componentes de mensagens de erro
a1fbf62 28/09 12:33 feat: organiza pedidos e entregas em páginas com navegação por URL
2a83d43 28/09 12:35 feat: adiciona telas de clientes integradas ao cadastro de pedidos
c221bdc 28/09 12:37 fix: registra cadastro e atribuição como etapas separadas no histórico
```

Commits da Sprint 04 em ordem cronológica

32 Ajustes no planejamento, na arquitetura e na modelagem

Modelagem

Foi criada a tabela order_status_history (id, order_id, status, note, changed_by, changed_at), ligada a orders com exclusão em cascata e a users pelo autor da mudança. Ela atende ao RF08, manter o histórico das entregas, que o modelo da Sprint 02 cobria apenas pela situação atual do pedido. Bancos criados antes desta versão recebem a tabela automaticamente quando a API inicia. As demais tabelas não mudaram.

Arquitetura e API

- O pedido passou a referenciar um cliente cadastrado (customer_id), em vez de criar o cliente pelo nome digitado, o que evita cadastros duplicados.
- Novas rotas: /customers, /geocode, /couriers/me/availability e o histórico em GET /orders/{id}.
- As regras de atribuição ficaram concentradas no módulo backend/app/rules.py, usado pelas rotas de pedidos e de entregadores.
- A busca de endereço usa o serviço Nominatim do OpenStreetMap por meio da API, que identifica o sistema como exige a política do serviço. Se ele estiver indisponível, o usuário marca o ponto no mapa.
- No frontend, o React Router substituiu a troca de telas por abas, e os formulários laterais viraram páginas próprias.

Planejamento

O módulo escolhido reúne o que o cronograma previa para as Sprints 04 e 05 (clientes, busca de endereço, entregadores e disponibilidade) e antecipa o histórico de entregas. As etapas seguintes continuam priorizando o componente de otimização.

Sprint	Período	Entrega principal	Situação
1	03/09 a 05/09	Planejamento, requisitos e backlog	Concluída
2	06/09 a 19/09	Arquitetura, modelos, protótipos, banco e estrutura	Concluída
3	20/09 a 26/09	Banco conectado, login, perfis, usuários e CRUD de pedidos	Concluída
4	27/09 a 03/10	Módulo Pedidos e entregas completo	Concluída
5	04/10 a 10/10	Rotas do dia no mapa e atribuição automática de pedidos	Planejada
6	11/10 a 17/10	Vizinho mais próximo integrado aos pedidos do dia	Planejada
7	18/10 a 24/10	2-opt, gravação das rotas e linha de base	Planejada
8	25/10 a 31/10	Paralelização para múltiplos entregadores	Planejada
9	01/11 a 07/11	Experimentos sequencial versus paralelo	Planejada
10	08/11 a 14/11	Relatórios e painel do dia	Planejada
11	15/11 a 21/11	Testes, usabilidade e segurança	Planejada
12	22/11 a 28/11	Documentação e ajustes	Planejada
Final	29/11 a 05/12	Correções, vídeos e preparação para a banca	Planejada

33 Testes automatizados

A suíte passou de 21 para 48 testes. Os novos testes cobrem clientes, validações de telefone, nome e senha, regras de capacidade, disponibilidade, raio e horário, o histórico de situação e a busca de endereço, esta sem acessar a internet.

```
$ python -m unittest discover backend/tests -v
test_login_rejects_wrong_password (test_api.AuthTestCase.test_login_rejects_wrong_password) ... ok
test_login_returns_token_and_profile (test_api.AuthTestCase.test_login_returns_token_and_profile) ... ok
test_password_is_stored_as_hash (test_api.AuthTestCase.test_password_is_stored_as_hash) ... ok
test_protected_routes_require_token (test_api.AuthTestCase.test_protected_routes_require_token) ... ok
test_register_creates_isolated_establishment (test_api.AuthTestCase.test_register_creates_isolated_establishment) ...
ok
test_delivered_order_cannot_be_edited (test_api.OrdersTestCase.test_delivered_order_cannot_be_edited) ... ok
test_order_crud (test_api.OrdersTestCase.test_order_crud) ... ok
test_order_requires_customer_from_establishment
(test_api.OrdersTestCase.test_order_requires_customer_from_establishment) ... ok
test_order_window_must_be_valid (test_api.OrdersTestCase.test_order_window_must_be_valid) ... ok
test_courier_advances_status_in_order (test_api.ProfilesTestCase.test_courier_advances_status_in_order) ... ok
test_courier_cannot_create_orders (test_api.ProfilesTestCase.test_courier_cannot_create_orders) ... ok
test_courier_sees_only_own_orders (test_api.ProfilesTestCase.test_courier_sees_only_own_orders) ... ok
test_only_admin_manages_users (test_api.ProfilesTestCase.test_only_admin_manages_users) ... ok
test_admin_creates_user_who_can_log_in (test_api.UsersTestCase.test_admin_creates_user_who_can_log_in) ... ok
test_courier_requires_load_capacity (test_api.UsersTestCase.test_courier_requires_load_capacity) ... ok
test_deactivated_user_cannot_log_in (test_api.UsersTestCase.test_deactivated_user_cannot_log_in) ... ok
test_duplicate_email_is_rejected (test_api.UsersTestCase.test_duplicate_email_is_rejected) ... ok
test_password_needs_letters_and_numbers (test_api.UsersTestCase.test_password_needs_letters_and_numbers) ... ok
test_courier_cannot_access_customers (test_customers.CustomersTestCase.test_courier_cannot_access_customers) ... ok
test_customer_crud (test_customers.CustomersTestCase.test_customer_crud) ... ok
test_customer_with_orders_cannot_be_deleted
(test_customers.CustomersTestCase.test_customer_with_orders_cannot_be_deleted) ... ok
test_duplicate_customer_is_rejected (test_customers.CustomersTestCase.test_duplicate_customer_is_rejected) ... ok
test_name_needs_letters (test_customers.CustomersTestCase.test_name_needs_letters) ... ok
test_phone_must_have_area_code (test_customers.CustomersTestCase.test_phone_must_have_area_code) ... ok
test_courier_cannot_search (test_geocoding.GeocodingTestCase.test_courier_cannot_search) ... ok
test_label_falls_back_to_display_name (test_geocoding.GeocodingTestCase.test_label_falls_back_to_display_name) ... ok
test_label_follows_brazilian_format (test_geocoding.GeocodingTestCase.test_label_follows_brazilian_format) ... ok
test_query_needs_minimum_length (test_geocoding.GeocodingTestCase.test_query_needs_minimum_length) ... ok
test_search_uses_depot_as_reference (test_geocoding.GeocodingTestCase.test_search_uses_depot_as_reference) ... ok
test_unavailable_service_returns_guidance (test_geocoding.GeocodingTestCase.test_unavailable_service_returns_guidance)
... ok
test_cancellation_keeps_the_reason (test_history.HistoryTestCase.test_cancellation_keeps_the_reason) ... ok
test_creation_with_courier_records_two_steps
(test_history.HistoryTestCase.test_creation_with_courier_records_two_steps) ... ok
test_deleting_order_removes_its_history (test_history.HistoryTestCase.test_deleting_order_removes_its_history) ... ok
test_demo_orders_have_consistent_history (test_history.HistoryTestCase.test_demo_orders_have_consistent_history) ... ok
test_full_lifecycle_is_recorded (test_history.HistoryTestCase.test_full_lifecycle_is_recorded) ... ok
test_same_status_is_rejected (test_history.HistoryTestCase.test_same_status_is_rejected) ... ok
test_empty_route_has_zero_distance (test_optimizer.RoutingTestCase.test_empty_route_has_zero_distance) ... ok
test_nearest_neighbor_keeps_every_stop_once
(test_optimizer.RoutingTestCase.test_nearest_neighbor_keeps_every_stop_once) ... ok
test_optimize_route_returns_order_and_distance
(test_optimizer.RoutingTestCase.test_optimize_route_returns_order_and_distance) ... ok
test_parallel_and_sequential_return_same_routes
(test_optimizer.RoutingTestCase.test_parallel_and_sequential_return_same_routes) ... ok
test_two_opt_never_worsens_route (test_optimizer.RoutingTestCase.test_two_opt_never_worsens_route) ... ok
test_courier_capacity_is_respected (test_order_rules.OrderRulesTestCase.test_courier_capacity_is_respected) ... ok
test_courier_with_route_in_progress_cannot_go_offline
(test_order_rules.OrderRulesTestCase.test_courier_with_route_in_progress_cannot_go_offline) ... ok
test_couriers_list_shows_current_load (test_order_rules.OrderRulesTestCase.test_couriers_list_shows_current_load) ...
ok
test_delivery_must_be_inside_radius (test_order_rules.OrderRulesTestCase.test_delivery_must_be_inside_radius) ... ok
test_offline_courier_cannot_receive_orders
(test_order_rules.OrderRulesTestCase.test_offline_courier_cannot_receive_orders) ... ok
test_weight_has_upper_limit (test_order_rules.OrderRulesTestCase.test_weight_has_upper_limit) ... ok
test_window_cannot_end_in_the_past (test_order_rules.OrderRulesTestCase.test_window_cannot_end_in_the_past) ... ok
Ran 48 tests in 10.603s
OK
```

Execução da suíte de testes

34 Dificuldades encontradas e próximos passos

Dificuldades

Dificuldade	Como foi tratada
O mapa escapava do formulário quando o campo entrava em estado de erro	O React substituía as classes que o Leaflet adiciona ao elemento do mapa. O mapa passou a ficar em um elemento interno que o React não altera
Dependência de um serviço externo para buscar endereços	A API identifica o sistema, limita os resultados à região e, se o serviço falhar, orienta a marcar o ponto no mapa. Os testes simulam o serviço
Bancos da Sprint 03 sem a tabela de histórico	A API cria a tabela ao iniciar quando ela não existe, sem apagar dados
Manter commits organizados durante o desenvolvimento	O trabalho foi dividido em etapas testadas separadamente e cada etapa gerou um commit próprio

Próximos passos

- Mostrar no mapa as rotas do dia de cada entregador e sugerir a atribuição automática dos pedidos pendentes.
- Integrar o otimizador sequencial e paralelo aos pedidos reais e gravar as rotas em routes e route_stops.
- Registrar as execuções em optimization_runs para os experimentos de desempenho.
- Distribuir as próximas tarefas entre os integrantes, cada um com os próprios commits e pull requests revisados por outro colega.

Situação da Sprint 04

Entrega obrigatória	Situação	Evidência
Primeiro módulo totalmente funcional	Concluído	Seções 25 e 26: fluxo completo de Pedidos e entregas
Persistência de dados	Concluído	Seção 27: dados preservados depois de desligar e religar banco e API
Validações	Concluído	Seção 28: regras na interface e na API
Mensagens de erro	Concluído	Seção 29: validação, regras, permissões, servidor indisponível
Navegação entre telas	Concluído	Seção 30: rotas, mapa de navegação e caminho da página
Commits organizados	Concluído	Seção 31: um commit por etapa na branch da Sprint 04

Parte V Sprint 05 Segundo módulo funcionando

A Sprint 05 entrega o segundo módulo do RotaCerta: Roteirização e desempenho. Seguindo a orientação de avançar de forma objetiva para o núcleo de otimização, o módulo liga o otimizador aos pedidos reais gravados no banco, mostra no mapa a rota de cada entregador, permite que o entregador inicie e conclua a rota pelo celular e compara a execução sequencial, a paralela em CPU e a paralela em GPU com CUDA, registrando tempo, speedup e eficiência.

35 Módulo implementado: Roteirização e desempenho

O módulo atende aos casos de uso UC03 (gerar rotas otimizadas), UC04 (visualizar a rota do dia) e UC05 (atualizar o andamento da entrega) e aos requisitos RF04 (rota otimizada por entregador), RF06 (rota como lista ordenada e mapa), RF07 e RF08 (situação e histórico), além do RNF01 (desempenho com processamento paralelo). Ele usa a estrutura e o primeiro módulo das sprints anteriores: login, perfis, clientes, pedidos e atribuição.

Funcionalidade	Perfis	Tela ou rota da API
Geração das rotas do dia a partir dos pedidos atribuídos, no modo sequencial, paralelo em CPU ou GPU (CUDA)	Administrador e atendente	/rotas | POST /routes/generate
Mapa com o ponto de saída, o trajeto de cada entregador e as paradas numeradas; lista ordenada com horário estimado	Administrador e atendente	/rotas | GET /routes
Cadastro do ponto de saída no mapa, com busca de endereço	Administrador	/rotas | PUT /establishment/depot
Rota do entregador no celular, com mapa, próxima parada e botão Iniciar rota	Entregador	/entregas | GET /routes/me e PATCH /routes/{id}/start
Conclusão automática da parada e da rota conforme as entregas são confirmadas	Entregador	/entregas | PATCH /orders/{id}/status
Comparação de desempenho com tempo, speedup, eficiência e gráfico, em instância sintética ou nos pedidos reais	Administrador	/desempenho | POST /optimizer/benchmark
Histórico das comparações e das gerações de rotas gravado no banco	Administrador	/desempenho | GET /optimizer/runs

Fluxo completo de utilização

1. O atendente cadastra e atribui os pedidos aos entregadores, como no primeiro módulo.
2. Na tela Rotas, ele escolhe o modo de execução e clica em Gerar rotas. A API agrupa os pedidos atribuídos por entregador, executa o otimizador e grava as rotas, as paradas e a execução no banco.
3. O mapa mostra o trajeto de cada entregador, saindo e voltando ao ponto de saída, e a lista traz a ordem das paradas, a distância, a duração e o horário estimado de cada chegada.
4. No celular, a entregadora abre Minhas entregas, vê a rota no mapa e toca em Iniciar rota. Todos os pedidos da rota passam para Em rota, com registro no histórico.
5. A cada entrega confirmada a parada é concluída e a próxima fica destacada. Quando todas terminam, a rota passa para Concluída.

6. O administrador abre Desempenho, executa a comparação entre os modos e acompanha o histórico das medições.

36 Evidências das funcionalidades

As capturas foram feitas com o sistema em execução sobre um banco recém-criado com os dados de demonstração: a Farmácia Boa Saúde, dois entregadores e catorze pedidos, dez deles atribuídos e aguardando rota.

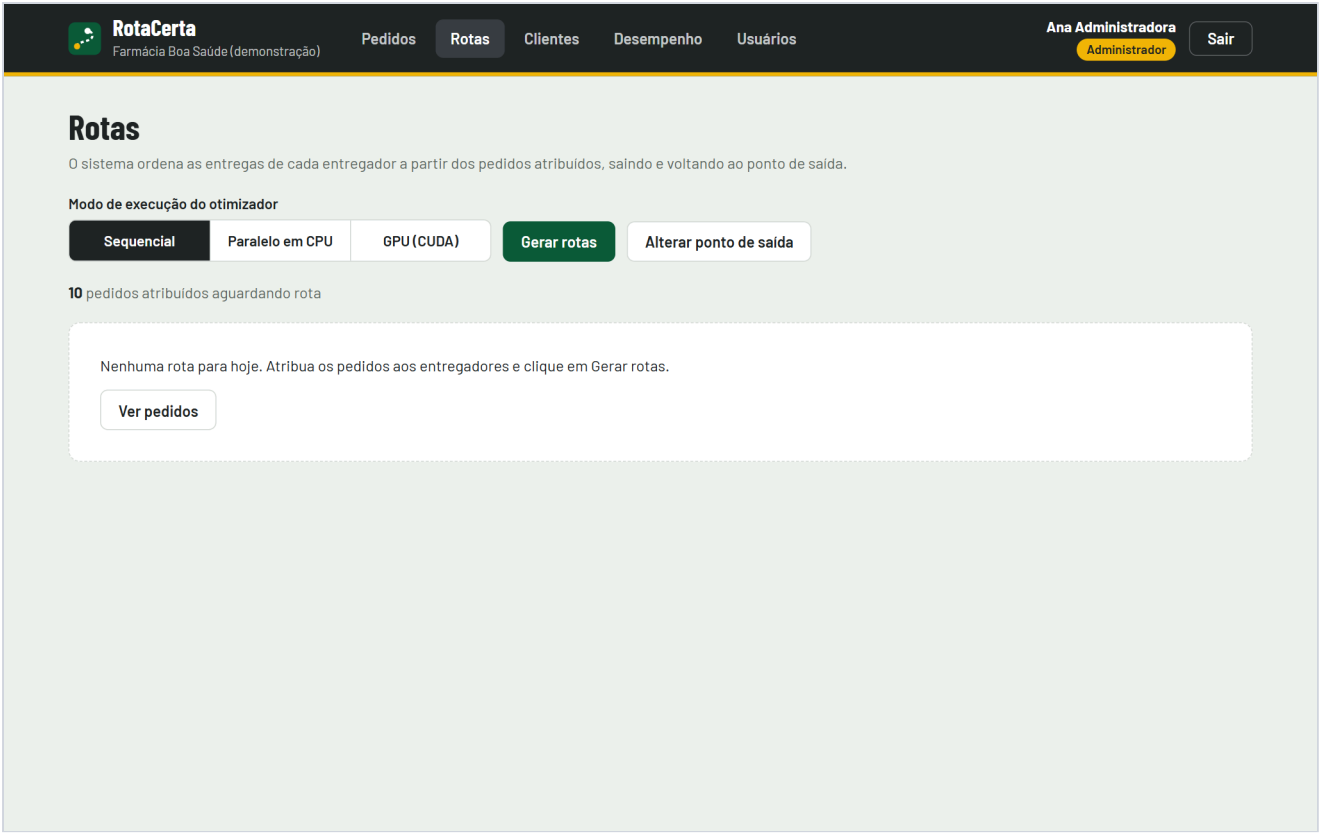


Figura 43 Tela Rotas antes da geração: dez pedidos atribuídos aguardando rota

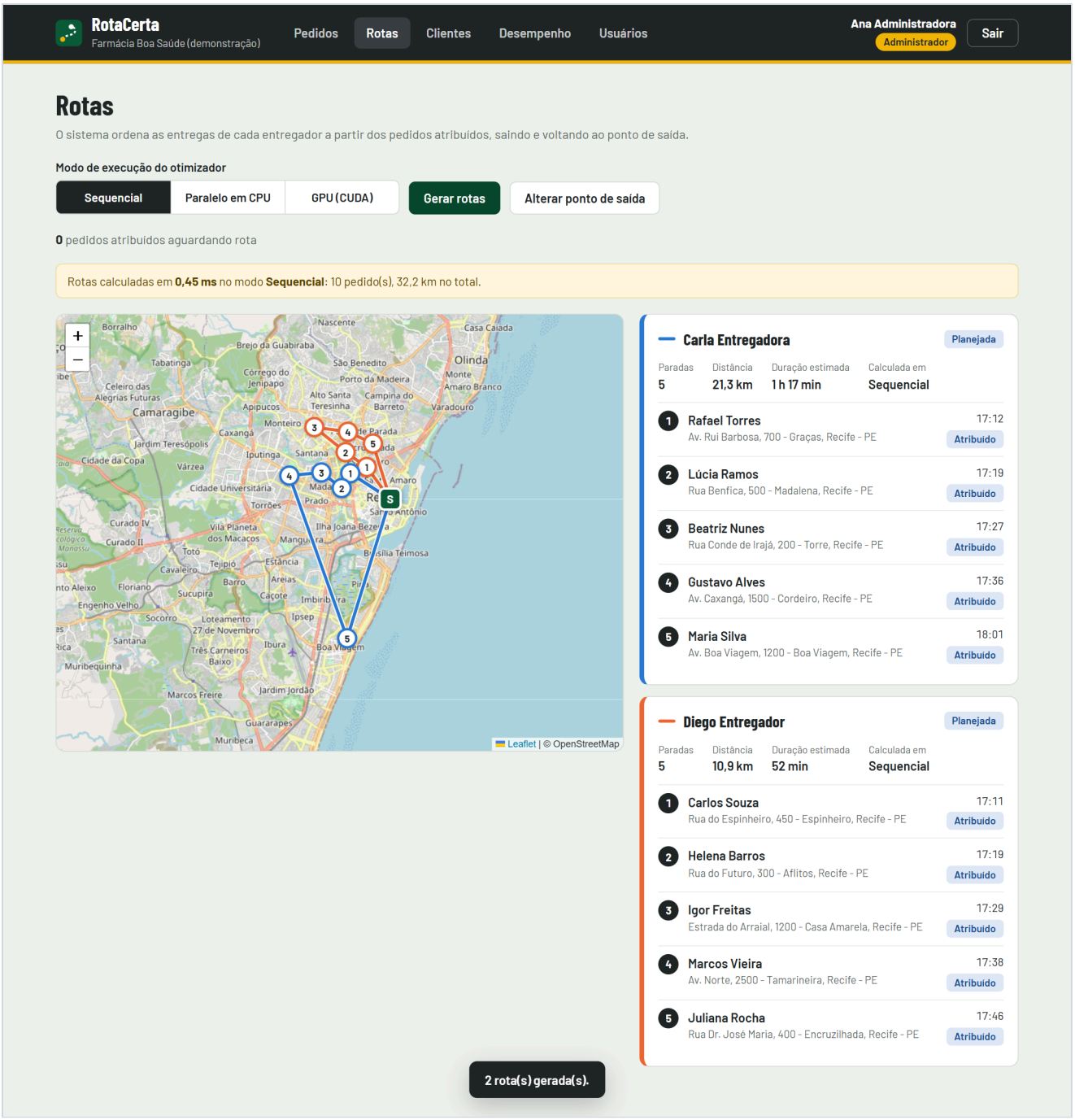


Figura 44 Rotas geradas no modo sequencial, com o trajeto de cada entregador no mapa e a lista de paradas

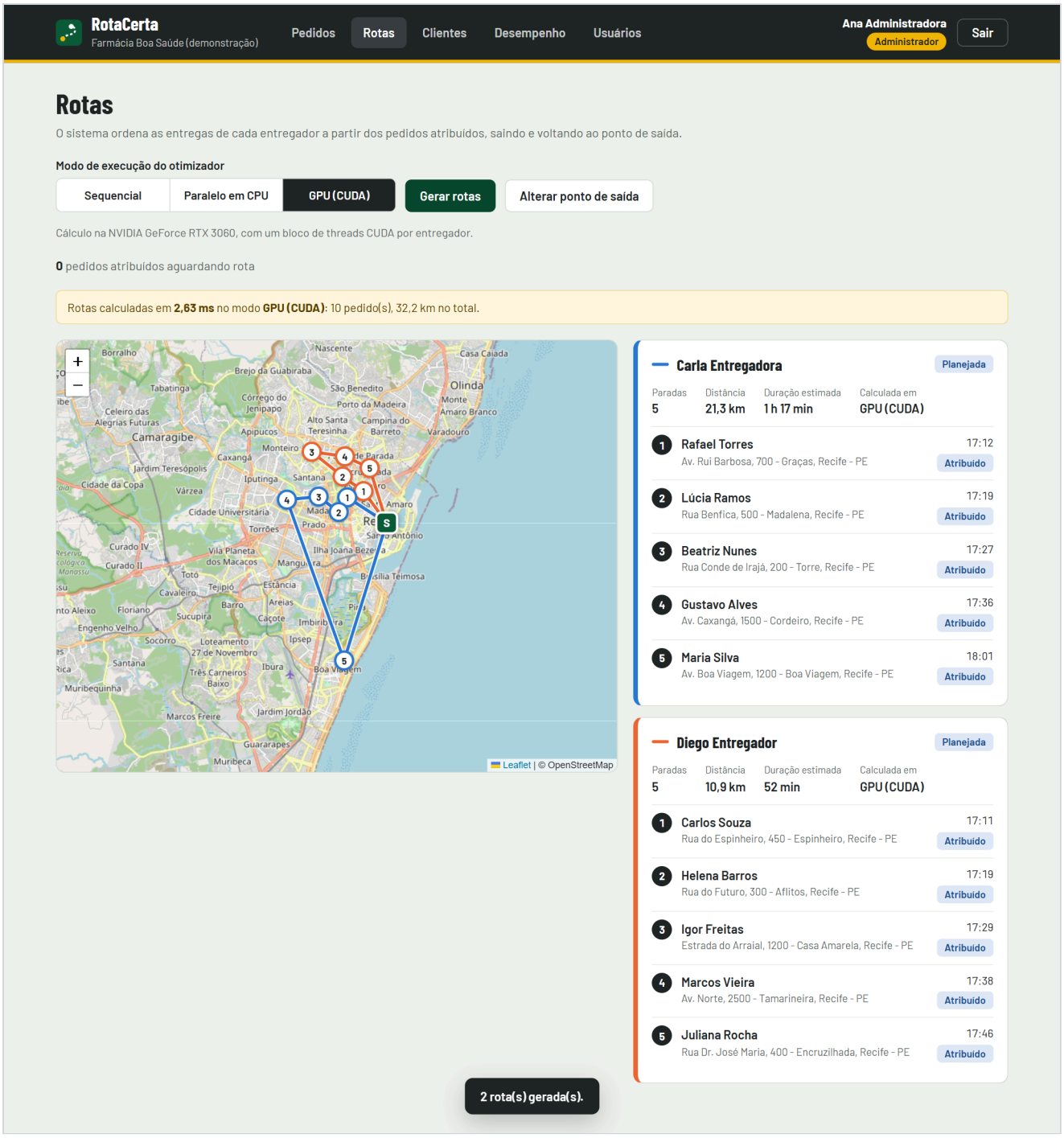


Figura 45 As mesmas rotas geradas no modo GPU (CUDA): mesma ordem e mesma distância

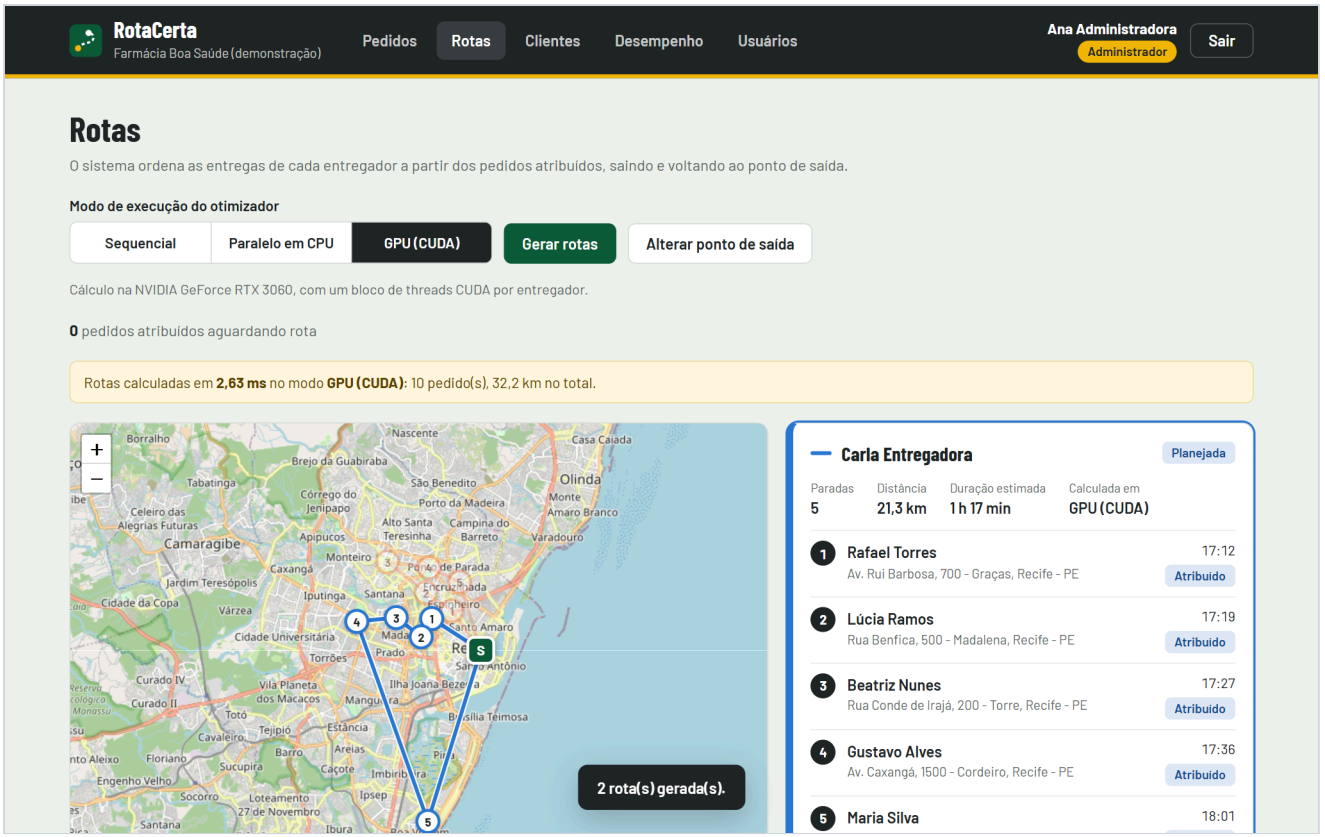


Figura 46 Rota da Carla destacada ao clicar no cartão; as demais ficam esmaecidas

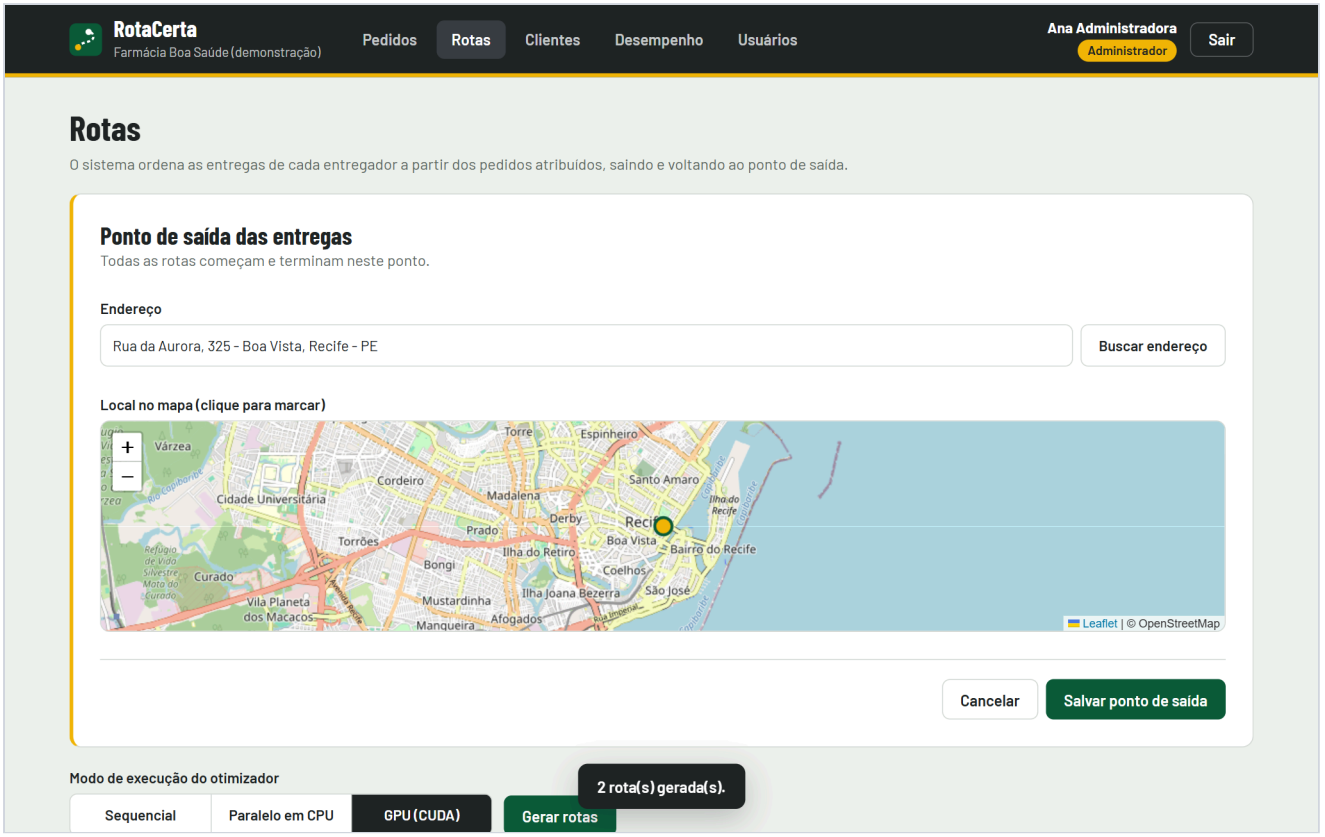


Figura 47 Cadastro do ponto de saída, de onde as rotas partem e para onde voltam

RotaCerta
Farmácia Boa Saúde (demonstração)
Entregador
Sair
Minhas entregas

Minhas entregas

Pedidos atribuídos a você. Atualize o andamento a cada etapa.

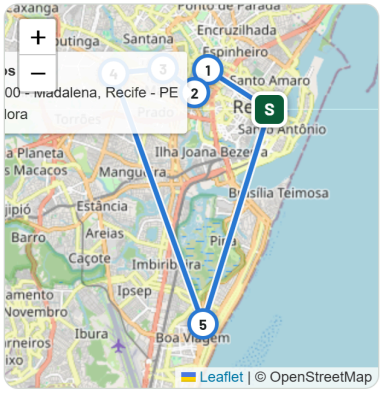
Sua disponibilidade

Disponível | Fora de serviço

Sua rota

Planejada · 5 paradas · 21,3 km · 1 h 17 min estimados

Iniciar rota



1 **Rafael Torres** — Próxima · 17:12
Av. Rui Barbosa, 700 - Graças, Recife - PE — Atribuído

2 **Lúcia Ramos** — 17:19
Rua Benfica, 500 - Madalena, Recife - PE — Atribuído

3 **Beatriz Nunes** — 17:27
Rua Conde de Irajá, 200 - Torre, Recife - PE — Atribuído

4 **Gustavo Alves** — 17:36
Av. Caxangá, 1500 - Cordeiro, Recife - PE — Atribuído

5 **Maria Silva** — 18:01
Av. Boa Viagem, 1200 - Boa Viagem, Recife - PE — Atribuído

Saída e retorno: Rua da Aurora, 325 - Boa Vista, Recife - PE

Todas 6 | Atribuído 5 | Em rota 1 | Entregue 0

Atribuído — Alta

Gustavo Alves

Av. Caxangá, 1500 - Cordeiro, Recife - PE

RotaCerta
Farmácia Boa Saúde (demonstração)
Entregador
Sair
Minhas entregas

Minhas entregas

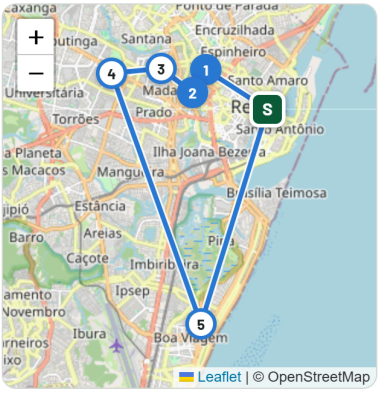
Pedidos atribuídos a você. Atualize o andamento a cada etapa.

Sua disponibilidade

Disponível | Fora de serviço

Sua rota

Em andamento · 5 paradas · 21,3 km · 1 h 17 min estimados



1 **Rafael Torres** — 17:12
Av. Rui Barbosa, 700 - Graças, Recife - PE — Entregue

2 **Lúcia Ramos** — 17:19
Rua Benfica, 500 - Madalena, Recife - PE — Entregue

3 **Beatriz Nunes** — Próxima · 17:27
Rua Conde de Irajá, 200 - Torre, Recife - PE — Em rota

4 **Gustavo Alves** — 17:36
Av. Caxangá, 1500 - Cordeiro, Recife - PE — Em rota

5 **Maria Silva** — 18:01
Av. Boa Viagem, 1200 - Boa Viagem, Recife - PE — Em rota

Saída e retorno: Rua da Aurora, 325 - Boa Vista, Recife - PE

Todas 6 | Atribuído 0 | Em rota 4 | Entregue 2

Em rota — Alta

Gustavo Alves

Av. Caxangá, 1500 - Cordeiro, Recife - PE

Horário: 01/10, 18:00 até 20:00
Peso: 0,4 kg

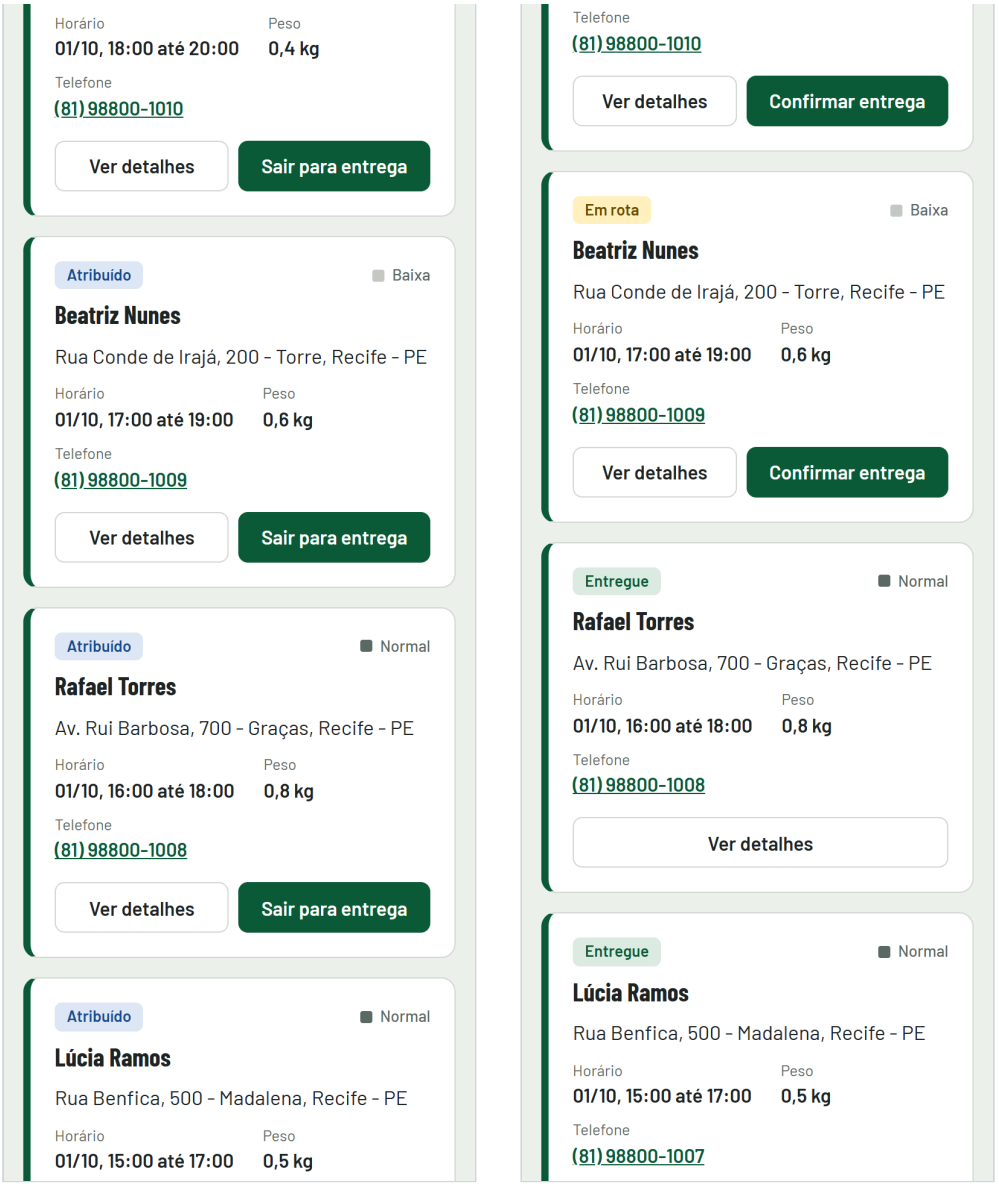


Figura 48 Entregadora no celular: rota planejada e, depois de iniciada, com duas entregas feitas e a próxima parada destacada

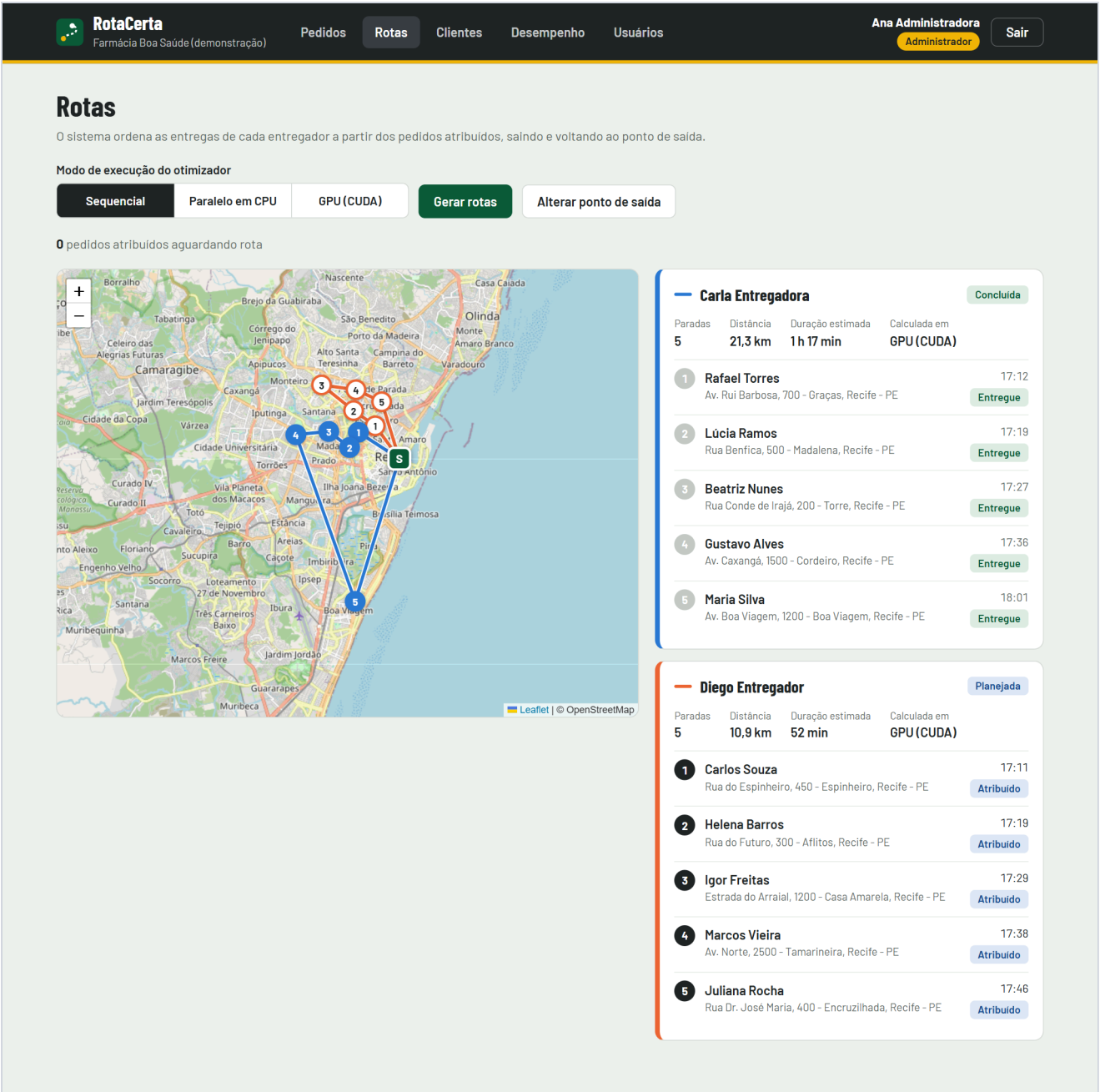


Figura 49 Visão do administrador com a rota da Carla concluída e a do Diego ainda planejada

37 Integração com o banco de dados

O módulo usa três tabelas que já constavam do modelo da Sprint 02 e ainda não tinham uso: routes guarda a rota de cada entregador (data, situação, modo de execução, distância e duração), route_stops guarda cada parada (ordem, pedido, distância desde a parada anterior, horário estimado e situação) e optimization_runs registra cada execução do otimizador com o tempo medido. As alterações no modelo estão na seção 43.

```
-- Rotas gravadas na tabela routes
rota | entregador        | status  | modo | km     | minutos | paradas
-----+-------------------+---------+------+--------+---------+--------
3    | Carla Entregadora | PLANNED | GPU  | 21.307 | 77      | 5
4    | Diego Entregador  | PLANNED | GPU  | 10.938 | 52      | 5
(2 linhas)

-- Paradas da rota 3 na tabela route_stops
rota | ordem | pedido | cliente       | km_trecho | chegada | parada  | pedido_status
-----+-------+--------+---------------+-----------+---------+---------+--------------
3    | 1     | 8      | Rafael Torres | 2.291     | 17:12   | PENDING | ASSIGNED
3    | 2     | 7      | Lúcia Ramos   | 0.846     | 17:19   | PENDING | ASSIGNED
3    | 3     | 9      | Beatriz Nunes | 1.222     | 17:27   | PENDING | ASSIGNED
3    | 4     | 10     | Gustavo Alves | 1.569     | 17:36   | PENDING | ASSIGNED
3    | 5     | 1      | Maria Silva   | 8.344     | 18:01   | PENDING | ASSIGNED
(5 linhas)

-- Execuções registradas na tabela optimization_runs
id | finalidade       | entrada | modo       | workers | rotas | paradas | tempo_ms | speedup | eficiencia |
mesmas_rotas
---+------------------+---------+------------+---------+-------+---------+----------+---------+------------+-----------
--
1  | ROUTE_GENERATION | REAL    | SEQUENTIAL | 1       | 2     | 10      | 0.453    |         |            |
2  | ROUTE_GENERATION | REAL    | GPU        | 512     | 2     | 10      | 2.625    |         |            |
(2 linhas)
```

Rotas, paradas e execuções gravadas logo depois da geração pela interface

```
-- Situação das rotas depois das entregas
rota | entregador        | status    | modo | km     | minutos | paradas
-----+-------------------+-----------+------+--------+---------+--------
3    | Carla Entregadora | COMPLETED | GPU  | 21.307 | 77      | 5
4    | Diego Entregador  | PLANNED   | GPU  | 10.938 | 52      | 5
(2 linhas)

-- Paradas da rota 3
rota | ordem | pedido | cliente       | km_trecho | chegada | parada    | pedido_status
-----+-------+--------+---------------+-----------+---------+-----------+--------------
3    | 1     | 8      | Rafael Torres | 2.291     | 17:12   | COMPLETED | DELIVERED
3    | 2     | 7      | Lúcia Ramos   | 0.846     | 17:19   | COMPLETED | DELIVERED
3    | 3     | 9      | Beatriz Nunes | 1.222     | 17:27   | COMPLETED | DELIVERED
3    | 4     | 10     | Gustavo Alves | 1.569     | 17:36   | COMPLETED | DELIVERED
3    | 5     | 1      | Maria Silva   | 8.344     | 18:01   | COMPLETED | DELIVERED
(5 linhas)

-- Histórico do pedido #8
status    | observacao                             | autor             | horario
----------+----------------------------------------+-------------------+--------------
PENDING   | Pedido cadastrado                      | Bruno Atendente   | 01/10 16:30:32
ASSIGNED  | Atribuído a Carla Entregadora          | Bruno Atendente   | 01/10 16:35:32
IN_ROUTE  | Saiu para entrega na rota 3, parada 1 | Carla Entregadora | 01/10 17:07:02
DELIVERED | Entrega confirmada                     | Carla Entregadora | 01/10 17:07:02
(4 linhas)
```

Situação da rota, das paradas e o histórico de um pedido depois das entregas

Para demonstrar que os dados são recuperados pela aplicação, a API foi reiniciada depois do fluxo completo e a tela Rotas foi aberta de novo: as rotas, as paradas e o histórico de medições continuaram iguais, lidos do PostgreSQL.

```
[01/10/2026 17:06:46] Fase 1: fluxo completo executado com o sistema no ar.
[01/10/2026 17:11:35] Fase 2: API reiniciada; rotas e execuções consultadas de novo.
GET /health -> {'status': 'ok', 'database': 'connected', 'dialect': 'postgresql', 'server_version': '18.1'}
-- Registros no banco depois do reinício
rotas | paradas | execucoes
------+---------+----------
2     | 10      | 42
(1 linha)
```

Registro do reinício da API e contagem dos registros no banco

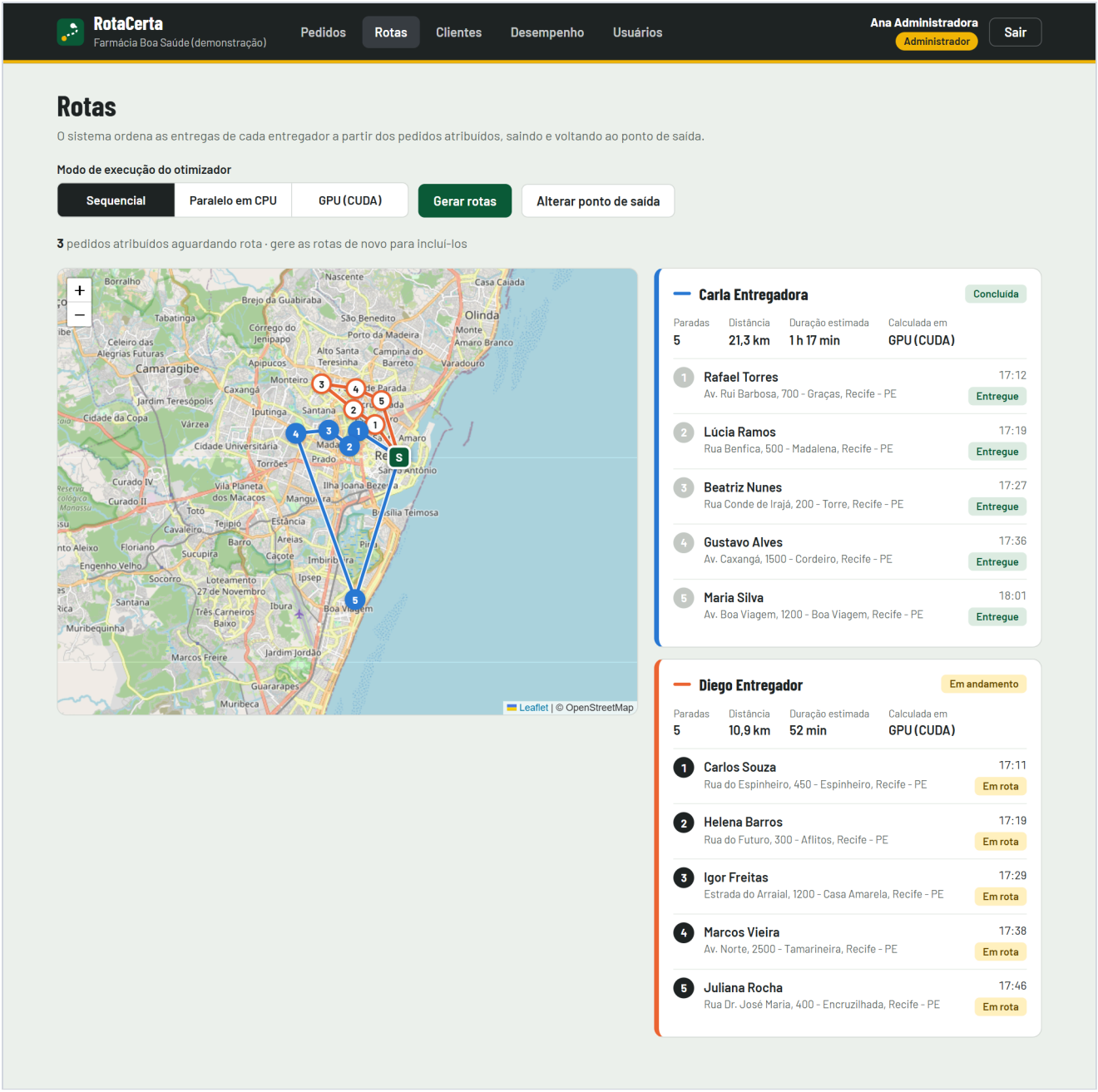


Figura 50 Tela Rotas depois de reiniciar a API, com as mesmas rotas

38 Regras de negócio

As regras abaixo foram implementadas no backend (módulo backend/app/planning.py) e valem para qualquer cliente da API, não só para a interface.

Regra	Descrição
RN01	Só entram na rota pedidos na situação Atribuído. Pedidos em rota, entregues ou cancelados ficam de fora.
RN02	Entregador fora de serviço não recebe rota; a resposta informa o motivo.
RN03	Entregador com rota em andamento não recebe outra; os pedidos novos aguardam a próxima geração.
RN04	Gerar de novo substitui as rotas ainda planejadas. Uma rota em andamento nunca é recalculada.
RN05	Toda rota sai do ponto de saída e volta a ele. Sem o ponto marcado no mapa, a geração é recusada e o administrador é orientado a marcá-lo.
RN06	A ordem das paradas minimiza a distância total (vizinho mais próximo seguido de 2-opt).
RN07	Duração e horário de chegada são estimados com velocidade média de 25 km/h e 5 minutos de atendimento por parada, valores configuráveis.
RN08	Iniciar a rota passa todos os pedidos atribuídos dela para Em rota e registra no histórico. Só o próprio entregador, o administrador ou o atendente iniciam; entregador fora de serviço não inicia.
RN09	Entrega confirmada conclui a parada; pedido cancelado marca a parada como não realizada. Quando todas terminam, a rota fica Concluída.
RN10	Trocar o entregador, mudar o endereço ou cancelar um pedido de rota planejada descarta essa rota, que precisa ser gerada de novo.
RN11	Pedido de rota iniciada ou concluída não pode ser excluído, só cancelado. Pedido de rota planejada pode ser excluído e a rota é descartada.
RN12	O modo GPU só fica disponível com GPU NVIDIA e CuPy instalados; sem eles, a interface desabilita a opção e a API explica o motivo.
RN13	A comparação de desempenho é exclusiva do administrador, aceita até 12.000 paradas e usa a mediana de 1 a 5 repetições.
RN14	Só o administrador altera o ponto de saída.

O quadro a seguir mostra as respostas da API a cada regra quando chamada diretamente, em sequência, sobre os mesmos dados.

```
POST /routes/generate pelo entregador
  -> 403 Seu perfil não tem permissão para esta ação.

PATCH /routes/{id}/start em rota já concluída
  -> 409 Esta rota já foi encerrada e não pode ser iniciada.

PATCH /routes/{id}/start da rota de outro entregador
  -> 404 Rota não encontrada.

PATCH /routes/{id}/start pelo próprio entregador
  -> 200 rota 4 IN_PROGRESS, pedidos: ['IN_ROUTE']

POST /orders atribuído (Rafaela Moura)
  -> 201 pedido #15 ASSIGNED

POST /orders atribuído (Sérgio Lins)
  -> 201 pedido #16 ASSIGNED

POST /routes/generate com rota em andamento
  -> 200 rotas: ['Carla Entregadora (COMPLETED, 5 paradas)', 'Diego Entregador (IN_PROGRESS, 5 paradas)', 'Carla
Entregadora (PLANNED, 1 parada)']; aguardando: 1; fora: ['Diego Entregador: Já está com uma rota em andamento. Os novos
pedidos entram na próxima rota.']

DELETE /orders de pedido em rota iniciada
  -> 409 O pedido já faz parte de uma rota iniciada ou concluída e não pode ser excluído. Cancele-o.

PUT /orders trocando o entregador de pedido em rota planejada
  -> 200 pedido #16 agora com Diego Entregador

GET /routes depois da troca
  -> 200 rotas: ['Carla Entregadora (COMPLETED, 5 paradas)', 'Diego Entregador (IN_PROGRESS, 5 paradas)']; aguardando:
2

POST /orders atribuído (Tânia Prado)
  -> 201 pedido #17 ASSIGNED

PATCH /couriers/me/availability OFFLINE (Carla)
  -> 200 OFFLINE

POST /routes/generate com entregadora fora de serviço
  -> 409 Nenhuma rota foi gerada. Carla Entregadora: Está fora de serviço, então os pedidos atribuídos não entraram em
nenhuma rota. Diego Entregador: Já está com uma rota em andamento. Os novos pedidos entram na próxima rota.

POST /optimizer/benchmark pela atendente
  -> 403 Seu perfil não tem permissão para esta ação.

POST /optimizer/benchmark com 64 x 500 paradas
  -> 422 [{'type': 'value_error', 'loc': ['body'], 'msg': 'Value error, Use no máximo 12.000 paradas no total
(entregadores × paradas por entregador).', 'input': {'couriers': 64, 'stops_per_courier': 500}, 'ctx': {'error': {}}}]

PUT /establishment/depot pela atendente
  -> 403 Seu perfil não tem permissão para esta ação.
```

Regras de negócio verificadas pela API

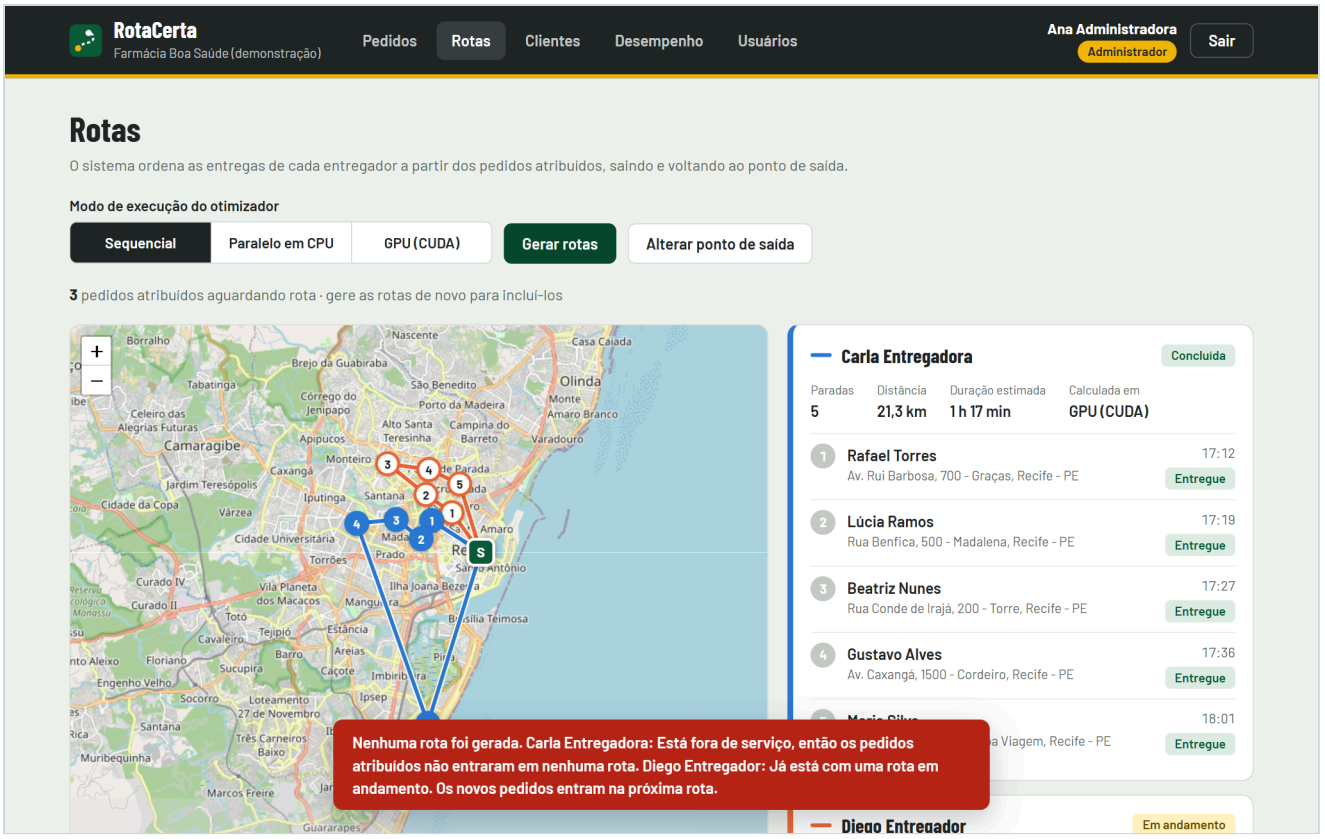


Figura 51 A mesma recusa vista na interface: nenhum entregador pode receber rota e o motivo de cada um é informado

Regras alteradas em relação ao planejamento

Alteração	Justificativa
A roteirização, prevista para as Sprints 06 a 10, foi antecipada para a Sprint 05	Orientação da professora para avançar ao núcleo de otimização, já que a parte operacional estava madura
Prioridade e janela de horário não alteram a ordem das paradas nesta versão	Os três modos precisam otimizar exatamente o mesmo objetivo para a comparação de desempenho ser justa. A prioridade continua visível em cada parada; as restrições entram na próxima sprint
A atribuição dos pedidos continua manual	A atribuição automática, listada nos próximos passos da Sprint 04, foi adiada para concentrar a sprint no cálculo das rotas e nas medições
O ponto de saída passou a ser marcado no mapa pelo administrador	Estabelecimentos cadastrados pela tela de cadastro só tinham o endereço, sem coordenadas, e não conseguiriam gerar rotas
A rota é desenhada com linhas retas entre as paradas	O cálculo usa a distância geodésica (haversine). O traçado pelas ruas depende de um serviço externo e ficou como evolução

39 Núcleo de otimização e paralelização com CUDA

O problema de cada entregador é encontrar a ordem de visita das paradas que reduz a distância total, saindo e voltando ao ponto de saída. A solução usa duas heurísticas clássicas do problema do caixeiro-viajante: o vizinho mais próximo monta uma rota inicial indo sempre para a parada mais próxima ainda não visitada, e o 2-opt melhora essa rota invertendo trechos enquanto houver ganho.

Nesta sprint o núcleo foi reescrito para ficar igual nos três modos. Ele calcula uma vez a matriz de distâncias haversine entre todos os pontos e usa o 2-opt por melhor melhoria: a cada rodada avalia todas as inversões possíveis pela diferença de distância e aplica a melhor. Os três modos usam a mesma

matriz, o mesmo critério de desempate (menor índice) e o mesmo limite de melhoria, por isso chegam exatamente às mesmas rotas. A comparação mede só a forma de execução, não diferenças de algoritmo.

Modo	Como executa	Paralelismo
Sequencial	Um processo calcula as rotas uma depois da outra com NumPy. É a linha de base	Nenhum
Paralelo em CPU	As rotas são distribuídas em fatias entre processos de um pool que fica aberto entre as chamadas	Um processo por núcleo, escolhido pelo usuário (1, 2, 4, 8 ou 16)
Paralelo em GPU (CUDA)	Um kernel CUDA próprio resolve todas as rotas num único lançamento	Um bloco de 256 threads por rota; milhares de threads no total

Como o kernel CUDA funciona

O kernel foi escrito em CUDA C e é compilado em tempo de execução pelo NVRTC por meio do CuPy (RawKernel). Cada bloco de threads resolve a rota de um entregador, em três etapas dentro do mesmo lançamento:

1. Matriz de distâncias: as 256 threads do bloco dividem entre si os pares de pontos da rota e calculam a distância haversine de cada par.
2. Vizinho mais próximo: a cada passo, cada thread procura a parada mais próxima entre as que lhe cabem, e uma redução em memória compartilhada escolhe a melhor do bloco.
3. 2-opt: a cada rodada, as threads avaliam em paralelo todas as inversões possíveis, a redução escolhe a de maior ganho e as threads invertem o trecho juntas. O laço termina quando nenhuma inversão encurta a rota.

Para que a GPU chegue aos mesmos resultados da CPU, o kernel usa precisão dupla, o mesmo desempate por menor índice e a opção --fmad=false, que impede o compilador de fundir multiplicações e somas numa única operação com arredondamento diferente do NumPy.

40 Comparação sequencial, paralela e GPU

A comparação segue a mesma metodologia em todas as medições: todos os modos recebem a mesma entrada; antes de medir, os processos da CPU são iniciados e o kernel CUDA é compilado, para que o tempo inclua só o cálculo e a troca de dados; cada modo é executado várias vezes e vale a mediana. O speedup é o tempo sequencial dividido pelo tempo do modo, e a eficiência é o speedup dividido pelo número de processos. Na GPU a eficiência não se aplica, porque o paralelismo é de milhares de threads e não de processos.

```
GET /optimizer/capabilities
{'cpu_count': 16, 'gpu_available': True, 'gpu_name': 'NVIDIA GeForce RTX 3060', 'gpu_reason': None}

CPU: 16 núcleos lógicos | GPU: NVIDIA GeForce RTX 3060
```

Máquina usada nas medições

RotaCerta
Farmácia Boa Saúde (demonstração)

Pedidos | Rotas | Clientes | Desempenho | Usuários

Ana Administradora
Administrador
Sair

Desempenho do otimizador

Executa o mesmo algoritmo (vizinho mais próximo + 2-opt) em modo sequencial, paralelo em CPU e na GPU, sobre a mesma entrada, e registra no banco o tempo, o speedup e a eficiência.

16 núcleos lógicos de CPU GPU **NVIDIA GeForce RTX 3060** disponível

Entrada

Instância sintética | Pedidos atribuídos

Paradas sorteadas num raio de 8 km do ponto de saída, sempre as mesmas para a mesma semente.

Entregadores (rotas): 16

Paradas por entregador: 200

Semente: 42

Processos na CPU: 1 2 4 8 16

Repetições por modo: 3

Vale a mediana dos tempos.

☑ Incluir a GPU (CUDA)

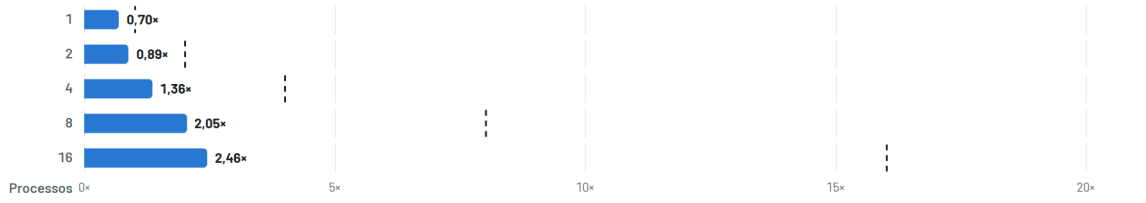
Executar comparação

Resultado

01/10, 17:07 · instância sintética · 16 rotas, 3200 paradas

Sequencial (linha de base)	Melhor resultado em CPU	GPU (CUDA)	Qualidade das rotas
185 ms	2,46×	8,88×	Idênticas
	com 16 processos · eficiência 15%	20,8 ms · 4.096 threads	2.534,0 km no total em todos os modos

Speedup da CPU por número de processos



Configuração	Tempo (mediana)	Speedup	Eficiência	Distância total	Mesmas rotas do sequencial
Sequencial	185 ms	1,00×	100%	2.534,0 km	Sim
CPU · 1 processo	266 ms	0,70×	70%	2.534,0 km	Sim
CPU · 2 processos	208 ms	0,89×	44%	2.534,0 km	Sim
CPU · 4 processos	136 ms	1,36×	34%	2.534,0 km	Sim
CPU · 8 processos	90,3 ms	2,05×	26%	2.534,0 km	Sim
CPU · 16 processos	75,2 ms	2,46×	15%	2.534,0 km	Sim
GPU (CUDA)	20,8 ms	8,88×	–	2.534,0 km	Sim

Speedup = tempo sequencial ÷ tempo do modo. Eficiência = speedup ÷ número de processos (100% seria o ganho ideal). Na GPU a eficiência não se aplica: o paralelismo é de milhares de threads, não de processos.

Comparações registradas

Data	Entrada	Rotas × paradas	Sequencial	Melhor CPU	GPU
01/10, 17:07	Sintética	16 × 200	185 ms	2,46× com 16	8,88×

Últimas gerações de rotas

Figura 52 Tela Desempenho com uma comparação de 16 rotas de 200 paradas

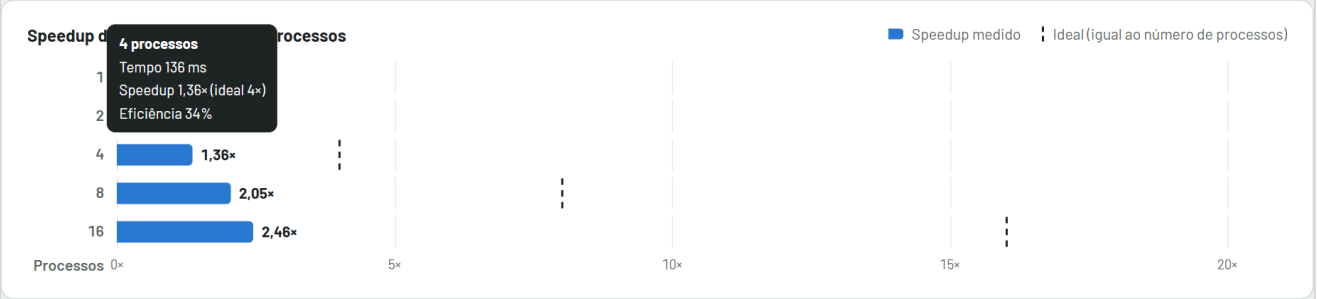


Figura 53 Gráfico de speedup da CPU por número de processos, com a marca do speedup ideal e o detalhe de uma barra

Para observar como o ganho depende do tamanho do problema, a comparação foi repetida com os pedidos reais e com instâncias sintéticas crescentes, sempre com a mesma semente.

```
[01/10/2026 17:11:45] Máquina: 16 núcleos lógicos, GPU NVIDIA GeForce RTX 3060

Entrada: pedidos atribuídos | 2 rotas | 3 paradas | mediana de 5 execuções
Configuração        Tempo (ms)  Speedup  Eficiência  Distância (km)  Mesmas rotas
Sequencial                0.11    1.00x        100%           9.924           sim
CPU 1 processo(s)         0.56    0.20x         20%           9.924           sim
CPU 2 processo(s)         0.65    0.17x          9%           9.924           sim
GPU (CUDA)                1.24    0.09x           -           9.924           sim

Entrada: instância sintética | 8 rotas | 400 paradas | mediana de 3 execuções
Configuração        Tempo (ms)  Speedup  Eficiência  Distância (km)  Mesmas rotas
Sequencial                4.85    1.00x        100%         646.193           sim
CPU 1 processo(s)         6.07    0.80x         80%         646.193           sim
CPU 2 processo(s)         3.48    1.39x         70%         646.193           sim
CPU 4 processo(s)         2.25    2.16x         54%         646.193           sim
CPU 8 processo(s)         2.55    1.90x         24%         646.193           sim
GPU (CUDA)                2.66    1.82x           -         646.193           sim

Entrada: instância sintética | 16 rotas | 1600 paradas | mediana de 3 execuções
Configuração        Tempo (ms)  Speedup  Eficiência  Distância (km)  Mesmas rotas
Sequencial               35.67    1.00x        100%        1826.327           sim
CPU 1 processo(s)        37.71    0.95x         95%        1826.327           sim
CPU 2 processo(s)        19.87    1.79x         90%        1826.327           sim
CPU 4 processo(s)        11.98    2.98x         74%        1826.327           sim
CPU 8 processo(s)         9.16    3.90x         49%        1826.327           sim
CPU 16 processo(s)        8.53    4.18x         26%        1826.327           sim
GPU (CUDA)                4.25    8.39x           -        1826.327           sim

Entrada: instância sintética | 16 rotas | 3200 paradas | mediana de 3 execuções
Configuração        Tempo (ms)  Speedup  Eficiência  Distância (km)  Mesmas rotas
Sequencial              201.43    1.00x        100%        2533.956           sim
CPU 1 processo(s)       299.11    0.67x         67%        2533.956           sim
CPU 2 processo(s)       219.76    0.92x         46%        2533.956           sim
CPU 4 processo(s)       154.08    1.31x         33%        2533.956           sim
CPU 8 processo(s)        93.50    2.15x         27%        2533.956           sim
CPU 16 processo(s)       84.19    2.39x         15%        2533.956           sim
GPU (CUDA)               18.70   10.77x           -        2533.956           sim

Entrada: instância sintética | 32 rotas | 9600 paradas | mediana de 3 execuções
Configuração        Tempo (ms)  Speedup  Eficiência  Distância (km)  Mesmas rotas
Sequencial             1395.37    1.00x        100%        6141.539           sim
CPU 1 processo(s)      1476.26    0.94x         95%        6141.539           sim
CPU 2 processo(s)       870.04    1.60x         80%        6141.539           sim
CPU 4 processo(s)       741.31    1.88x         47%        6141.539           sim
CPU 8 processo(s)       621.06    2.25x         28%        6141.539           sim
CPU 16 processo(s)      680.43    2.05x         13%        6141.539           sim
GPU (CUDA)               57.06   24.45x           -        6141.539           sim
```

Experimentos com tamanhos crescentes (mediana de 3 execuções; pedidos reais com 5)

Três comportamentos aparecem nos experimentos. Primeiro, com os pedidos reais da demonstração (três paradas), todos os modos paralelos ficam mais lentos que o sequencial: o cálculo leva cerca de 0,1 ms, menos que o custo fixo de enviar os dados a outro processo (cerca de 0,5 ms) ou de copiá-los para a GPU e lançar o kernel (cerca de 1 ms). Para a operação diária de um pequeno negócio o modo sequencial já basta; o paralelismo compensa quando o problema cresce, como em várias filiais, muitos entregadores ou simulações. Segundo, na CPU a melhor eficiência ficou entre 2 e 4 processos (90% e 74% na instância de 16 rotas com 100 paradas) e o speedup parou de crescer acima de 8 processos, porque o processador tem 8 núcleos físicos e os 16 núcleos lógicos dividem os mesmos recursos. Nas duas instâncias maiores o ganho da CPU foi menor e até o pool com 1 processo ficou mais lento que o sequencial; num script isolado, fora da API, os dois tempos são iguais (cerca de 200 ms), por isso a hipótese é a disputa de CPU com outros programas abertos durante a medição, registrada como pendência na seção 42. Terceiro, o ganho da GPU cresce com o tamanho do problema, de 1,8 vezes com 8 rotas de 50 paradas para 24,5 vezes com 32 rotas de 300 paradas: mais rotas ocupam mais dos 28 multiprocessadores da RTX 3060 e mais paradas significam mais trocas do 2-opt avaliadas ao mesmo

tempo. Em todas as medições os três modos chegaram exatamente às mesmas rotas e à mesma distância total.

```
-- Execuções registradas na tabela optimization_runs
id | finalidade       | entrada   | modo       | workers | rotas | paradas | tempo_ms | speedup | eficiencia |
mesmas_rotas
---+------------------+-----------+------------+---------+-------+---------+----------+---------+------------+---------
---
1  | ROUTE_GENERATION | REAL      | SEQUENTIAL | 1       | 2     | 10      | 0.453    |         |            |
2  | ROUTE_GENERATION | REAL      | GPU        | 512     | 2     | 10      | 2.625    |         |            |
3  | ROUTE_GENERATION | REAL      | PARALLEL   | 1       | 1     | 1       | 0.103    |         |            |
4  | BENCHMARK        | SYNTHETIC | SEQUENTIAL | 1       | 16    | 3200    | 184.758  | 1.000   | 1.0000     | True
5  | BENCHMARK        | SYNTHETIC | PARALLEL   | 1       | 16    | 3200    | 265.795  | 0.695   | 0.6951     | True
6  | BENCHMARK        | SYNTHETIC | PARALLEL   | 2       | 16    | 3200    | 208.104  | 0.888   | 0.4439     | True
7  | BENCHMARK        | SYNTHETIC | PARALLEL   | 4       | 16    | 3200    | 136.317  | 1.355   | 0.3388     | True
8  | BENCHMARK        | SYNTHETIC | PARALLEL   | 8       | 16    | 3200    | 90.284   | 2.046   | 0.2558     | True
9  | BENCHMARK        | SYNTHETIC | PARALLEL   | 16      | 16    | 3200    | 75.172   | 2.458   | 0.1536     | True
10 | BENCHMARK        | SYNTHETIC | GPU        | 4096    | 16    | 3200    | 20.798   | 8.884   |            | True
11 | BENCHMARK        | REAL      | SEQUENTIAL | 1       | 2     | 3       | 0.087    | 1.000   | 1.0000     | True
12 | BENCHMARK        | REAL      | PARALLEL   | 1       | 2     | 3       | 0.524    | 0.166   | 0.1659     | True
13 | BENCHMARK        | REAL      | PARALLEL   | 2       | 2     | 3       | 0.586    | 0.148   | 0.0742     | True
14 | BENCHMARK        | REAL      | GPU        | 512     | 2     | 3       | 0.668    | 0.130   |            | True
15 | BENCHMARK        | SYNTHETIC | SEQUENTIAL | 1       | 8     | 400     | 4.948    | 1.000   | 1.0000     | True
16 | BENCHMARK        | SYNTHETIC | PARALLEL   | 1       | 8     | 400     | 6.208    | 0.797   | 0.7971     | True
17 | BENCHMARK        | SYNTHETIC | PARALLEL   | 2       | 8     | 400     | 3.724    | 1.329   | 0.6643     | True
18 | BENCHMARK        | SYNTHETIC | PARALLEL   | 4       | 8     | 400     | 2.129    | 2.324   | 0.5811     | True
19 | BENCHMARK        | SYNTHETIC | PARALLEL   | 8       | 8     | 400     | 2.066    | 2.395   | 0.2994     | True
20 | BENCHMARK        | SYNTHETIC | PARALLEL   | 8       | 8     | 400     | 2.030    | 2.438   | 0.3047     | True
21 | BENCHMARK        | SYNTHETIC | GPU        | 2048    | 8     | 400     | 1.266    | 3.909   |            | True
22 | BENCHMARK        | SYNTHETIC | SEQUENTIAL | 1       | 16    | 1600    | 35.660   | 1.000   | 1.0000     | True
23 | BENCHMARK        | SYNTHETIC | PARALLEL   | 1       | 16    | 1600    | 36.573   | 0.975   | 0.9750     | True
24 | BENCHMARK        | SYNTHETIC | PARALLEL   | 2       | 16    | 1600    | 20.155   | 1.769   | 0.8846     | True
25 | BENCHMARK        | SYNTHETIC | PARALLEL   | 4       | 16    | 1600    | 11.642   | 3.063   | 0.7658     | True
26 | BENCHMARK        | SYNTHETIC | PARALLEL   | 8       | 16    | 1600    | 8.793    | 4.056   | 0.5069     | True
27 | BENCHMARK        | SYNTHETIC | PARALLEL   | 16      | 16    | 1600    | 7.926    | 4.499   | 0.2812     | True
28 | BENCHMARK        | SYNTHETIC | GPU        | 4096    | 16    | 1600    | 2.822    | 12.634  |            | True
29 | BENCHMARK        | SYNTHETIC | SEQUENTIAL | 1       | 16    | 3200    | 184.893  | 1.000   | 1.0000     | True
30 | BENCHMARK        | SYNTHETIC | PARALLEL   | 1       | 16    | 3200    | 288.417  | 0.641   | 0.6411     | True
31 | BENCHMARK        | SYNTHETIC | PARALLEL   | 2       | 16    | 3200    | 208.505  | 0.887   | 0.4434     | True
32 | BENCHMARK        | SYNTHETIC | PARALLEL   | 4       | 16    | 3200    | 136.215  | 1.357   | 0.3393     | True
33 | BENCHMARK        | SYNTHETIC | PARALLEL   | 8       | 16    | 3200    | 84.067   | 2.199   | 0.2749     | True
34 | BENCHMARK        | SYNTHETIC | PARALLEL   | 16      | 16    | 3200    | 76.568   | 2.415   | 0.1509     | True
35 | BENCHMARK        | SYNTHETIC | GPU        | 4096    | 16    | 3200    | 17.212   | 10.742  |            | True
36 | BENCHMARK        | SYNTHETIC | SEQUENTIAL | 1       | 32    | 9600    | 1049.949 | 1.000   | 1.0000     | True
37 | BENCHMARK        | SYNTHETIC | PARALLEL   | 1       | 32    | 9600    | 1982.567 | 0.530   | 0.5296     | True
38 | BENCHMARK        | SYNTHETIC | PARALLEL   | 2       | 32    | 9600    | 897.313  | 1.170   | 0.5851     | True
39 | BENCHMARK        | SYNTHETIC | PARALLEL   | 4       | 32    | 9600    | 741.115  | 1.417   | 0.3542     | True
40 | BENCHMARK        | SYNTHETIC | PARALLEL   | 8       | 32    | 9600    | 662.509  | 1.585   | 0.1981     | True
41 | BENCHMARK        | SYNTHETIC | PARALLEL   | 16      | 32    | 9600    | 775.115  | 1.355   | 0.0847     | True
42 | BENCHMARK        | SYNTHETIC | GPU        | 8192    | 32    | 9600    | 90.265   | 11.632  |            | True
43 | BENCHMARK        | REAL      | SEQUENTIAL | 1       | 2     | 3       | 0.114    | 1.000   | 1.0000     | True
44 | BENCHMARK        | REAL      | PARALLEL   | 1       | 2     | 3       | 0.564    | 0.201   | 0.2013     | True
45 | BENCHMARK        | REAL      | PARALLEL   | 2       | 2     | 3       | 0.654    | 0.174   | 0.0868     | True
46 | BENCHMARK        | REAL      | GPU        | 512     | 2     | 3       | 1.244    | 0.091   |            | True
47 | BENCHMARK        | SYNTHETIC | SEQUENTIAL | 1       | 8     | 400     | 4.846    | 1.000   | 1.0000     | True
48 | BENCHMARK        | SYNTHETIC | PARALLEL   | 1       | 8     | 400     | 6.069    | 0.799   | 0.7986     | True
49 | BENCHMARK        | SYNTHETIC | PARALLEL   | 2       | 8     | 400     | 3.485    | 1.391   | 0.6953     | True
50 | BENCHMARK        | SYNTHETIC | PARALLEL   | 4       | 8     | 400     | 2.247    | 2.157   | 0.5393     | True
51 | BENCHMARK        | SYNTHETIC | PARALLEL   | 8       | 8     | 400     | 2.553    | 1.899   | 0.2373     | True
52 | BENCHMARK        | SYNTHETIC | GPU        | 2048    | 8     | 400     | 2.663    | 1.820   |            | True
53 | BENCHMARK        | SYNTHETIC | SEQUENTIAL | 1       | 16    | 1600    | 35.667   | 1.000   | 1.0000     | True
54 | BENCHMARK        | SYNTHETIC | PARALLEL   | 1       | 16    | 1600    | 37.712   | 0.946   | 0.9458     | True
55 | BENCHMARK        | SYNTHETIC | PARALLEL   | 2       | 16    | 1600    | 19.871   | 1.795   | 0.8975     | True
56 | BENCHMARK        | SYNTHETIC | PARALLEL   | 4       | 16    | 1600    | 11.984   | 2.976   | 0.7440     | True
57 | BENCHMARK        | SYNTHETIC | PARALLEL   | 8       | 16    | 1600    | 9.156    | 3.895   | 0.4869     | True
58 | BENCHMARK        | SYNTHETIC | PARALLEL   | 16      | 16    | 1600    | 8.526    | 4.183   | 0.2615     | True
59 | BENCHMARK        | SYNTHETIC | GPU        | 4096    | 16    | 1600    | 4.251    | 8.389   |            | True
60 | BENCHMARK        | SYNTHETIC | SEQUENTIAL | 1       | 16    | 3200    | 201.429  | 1.000   | 1.0000     | True
61 | BENCHMARK        | SYNTHETIC | PARALLEL   | 1       | 16    | 3200    | 299.110  | 0.673   | 0.6734     | True
62 | BENCHMARK        | SYNTHETIC | PARALLEL   | 2       | 16    | 3200    | 219.760  | 0.917   | 0.4583     | True
63 | BENCHMARK        | SYNTHETIC | PARALLEL   | 4       | 16    | 3200    | 154.082  | 1.307   | 0.3268     | True
64 | BENCHMARK        | SYNTHETIC | PARALLEL   | 8       | 16    | 3200    | 93.502   | 2.154   | 0.2693     | True
65 | BENCHMARK        | SYNTHETIC | PARALLEL   | 16      | 16    | 3200    | 84.188   | 2.393   | 0.1495     | True
66 | BENCHMARK        | SYNTHETIC | GPU        | 4096    | 16    | 3200    | 18.700   | 10.771  |            | True
67 | BENCHMARK        | SYNTHETIC | SEQUENTIAL | 1       | 32    | 9600    | 1395.366 | 1.000   | 1.0000     | True
68 | BENCHMARK        | SYNTHETIC | PARALLEL   | 1       | 32    | 9600    | 1476.257 | 0.945   | 0.9452     | True
69 | BENCHMARK        | SYNTHETIC | PARALLEL   | 2       | 32    | 9600    | 870.038  | 1.604   | 0.8019     | True
```

```
 70 | BENCHMARK        | SYNTHETIC | PARALLEL   | 4       | 32    | 9600    | 741.310  | 1.882   | 0.4706     | True
 71 | BENCHMARK        | SYNTHETIC | PARALLEL   | 8       | 32    | 9600    | 621.059  | 2.247   | 0.2808     | True
 72 | BENCHMARK        | SYNTHETIC | PARALLEL   | 16      | 32    | 9600    | 680.428  | 2.051   | 0.1282     | True
 73 | BENCHMARK        | SYNTHETIC | GPU        | 8192    | 32    | 9600    | 57.059   | 24.455  |            | True
(73 linhas)
```

Todas as execuções registradas na tabela optimization_runs

41 Testes das funcionalidades

A suíte automatizada passou de 48 para 76 testes. Os novos testes cobrem a geração das rotas e todas as regras da seção 38, a consistência entre pedidos e rotas, as permissões por perfil, a comparação de desempenho e o núcleo de otimização, incluindo a equivalência das rotas entre os modos sequencial, paralelo e GPU. O teste da GPU é executado só em máquinas com GPU NVIDIA; nas demais aparece como ignorado. Os testes da API usam um banco SQLite em memória e não dependem do PostgreSQL.

Grupo	O que verifica
Geração de rotas	Uma rota por entregador com todos os pedidos atribuídos, ordem das paradas, distância, duração, gravação e listagem
Regras de geração	Entregador fora de serviço, rota em andamento, ponto de saída ausente, nenhum pedido aguardando, modo GPU indisponível
Acompanhamento	Entregador vê e inicia só a própria rota, pedidos passam para Em rota, rota concluída ao entregar todas as paradas
Consistência	Troca de entregador descarta a rota planejada; exclusão de pedido em rota iniciada é recusada
Perfis	Entregador não gera rotas, atendente não executa comparações nem altera o ponto de saída
Comparação	Modos e workers medidos, speedup e eficiência coerentes, mesmas rotas, limite de tamanho, pedidos reais
Otimizador	2-opt nunca piora a rota, distância informada confere com a rota, rotas vazias e de uma parada, CPU e GPU iguais, um único pool aberto

```
$ python -m unittest discover backend/tests -v
test_login_rejects_wrong_password (test_api.AuthTestCase.test_login_rejects_wrong_password) ... ok
test_login_returns_token_and_profile (test_api.AuthTestCase.test_login_returns_token_and_profile) ... ok
test_password_is_stored_as_hash (test_api.AuthTestCase.test_password_is_stored_as_hash) ... ok
test_protected_routes_require_token (test_api.AuthTestCase.test_protected_routes_require_token) ... ok
test_register_creates_isolated_establishment (test_api.AuthTestCase.test_register_creates_isolated_establishment) ...
ok
test_delivered_order_cannot_be_edited (test_api.OrdersTestCase.test_delivered_order_cannot_be_edited) ... ok
test_order_crud (test_api.OrdersTestCase.test_order_crud) ... ok
test_order_requires_customer_from_establishment
(test_api.OrdersTestCase.test_order_requires_customer_from_establishment) ... ok
test_order_window_must_be_valid (test_api.OrdersTestCase.test_order_window_must_be_valid) ... ok
test_courier_advances_status_in_order (test_api.ProfilesTestCase.test_courier_advances_status_in_order) ... ok
test_courier_cannot_create_orders (test_api.ProfilesTestCase.test_courier_cannot_create_orders) ... ok
test_courier_sees_only_own_orders (test_api.ProfilesTestCase.test_courier_sees_only_own_orders) ... ok
test_only_admin_manages_users (test_api.ProfilesTestCase.test_only_admin_manages_users) ... ok
test_admin_creates_user_who_can_log_in (test_api.UsersTestCase.test_admin_creates_user_who_can_log_in) ... ok
test_courier_requires_load_capacity (test_api.UsersTestCase.test_courier_requires_load_capacity) ... ok
test_deactivated_user_cannot_log_in (test_api.UsersTestCase.test_deactivated_user_cannot_log_in) ... ok
test_duplicate_email_is_rejected (test_api.UsersTestCase.test_duplicate_email_is_rejected) ... ok
test_password_needs_letters_and_numbers (test_api.UsersTestCase.test_password_needs_letters_and_numbers) ... ok
test_courier_cannot_access_customers (test_customers.CustomersTestCase.test_courier_cannot_access_customers) ... ok
test_customer_crud (test_customers.CustomersTestCase.test_customer_crud) ... ok
test_customer_with_orders_cannot_be_deleted
(test_customers.CustomersTestCase.test_customer_with_orders_cannot_be_deleted) ... ok
test_duplicate_customer_is_rejected (test_customers.CustomersTestCase.test_duplicate_customer_is_rejected) ... ok
test_name_needs_letters (test_customers.CustomersTestCase.test_name_needs_letters) ... ok
test_phone_must_have_area_code (test_customers.CustomersTestCase.test_phone_must_have_area_code) ... ok
test_courier_cannot_search (test_geocoding.GeocodingTestCase.test_courier_cannot_search) ... ok
test_label_falls_back_to_display_name (test_geocoding.GeocodingTestCase.test_label_falls_back_to_display_name) ... ok
test_label_follows_brazilian_format (test_geocoding.GeocodingTestCase.test_label_follows_brazilian_format) ... ok
test_query_needs_minimum_length (test_geocoding.GeocodingTestCase.test_query_needs_minimum_length) ... ok
test_search_uses_depot_as_reference (test_geocoding.GeocodingTestCase.test_search_uses_depot_as_reference) ... ok
test_unavailable_service_returns_guidance (test_geocoding.GeocodingTestCase.test_unavailable_service_returns_guidance)
... ok
test_cancellation_keeps_the_reason (test_history.HistoryTestCase.test_cancellation_keeps_the_reason) ... ok
test_creation_with_courier_records_two_steps
(test_history.HistoryTestCase.test_creation_with_courier_records_two_steps) ... ok
test_deleting_order_removes_its_history (test_history.HistoryTestCase.test_deleting_order_removes_its_history) ... ok
test_demo_orders_have_consistent_history (test_history.HistoryTestCase.test_demo_orders_have_consistent_history) ... ok
test_full_lifecycle_is_recorded (test_history.HistoryTestCase.test_full_lifecycle_is_recorded) ... ok
test_same_status_is_rejected (test_history.HistoryTestCase.test_same_status_is_rejected) ... ok
test_gpu_matches_sequential (test_optimizer.GpuRoutingTestCase.test_gpu_matches_sequential) ... ok
test_empty_and_single_stop_routes (test_optimizer.RoutingTestCase.test_empty_and_single_stop_routes) ... ok
test_empty_route_has_zero_distance (test_optimizer.RoutingTestCase.test_empty_route_has_zero_distance) ... ok
test_nearest_neighbor_keeps_every_stop_once
(test_optimizer.RoutingTestCase.test_nearest_neighbor_keeps_every_stop_once) ... ok
test_only_one_process_pool_stays_open (test_optimizer.RoutingTestCase.test_only_one_process_pool_stays_open) ... ok
test_optimize_route_returns_order_and_distance
(test_optimizer.RoutingTestCase.test_optimize_route_returns_order_and_distance) ... ok
test_parallel_and_sequential_return_same_routes
(test_optimizer.RoutingTestCase.test_parallel_and_sequential_return_same_routes) ... ok
test_parallel_mode_matches_sequential_on_many_routes
(test_optimizer.RoutingTestCase.test_parallel_mode_matches_sequential_on_many_routes) ... ok
test_reported_distance_matches_the_route (test_optimizer.RoutingTestCase.test_reported_distance_matches_the_route) ...
ok
test_two_opt_improves_nearest_neighbor_on_larger_routes
(test_optimizer.RoutingTestCase.test_two_opt_improves_nearest_neighbor_on_larger_routes) ... ok
test_two_opt_never_worsens_route (test_optimizer.RoutingTestCase.test_two_opt_never_worsens_route) ... ok
test_worker_processes_use_one_blas_thread (test_optimizer.RoutingTestCase.test_worker_processes_use_one_blas_thread)
... ok
test_courier_capacity_is_respected (test_order_rules.OrderRulesTestCase.test_courier_capacity_is_respected) ... ok
test_courier_with_route_in_progress_cannot_go_offline
(test_order_rules.OrderRulesTestCase.test_courier_with_route_in_progress_cannot_go_offline) ... ok
test_couriers_list_shows_current_load (test_order_rules.OrderRulesTestCase.test_couriers_list_shows_current_load) ...
ok
test_delivery_must_be_inside_radius (test_order_rules.OrderRulesTestCase.test_delivery_must_be_inside_radius) ... ok
test_offline_courier_cannot_receive_orders
(test_order_rules.OrderRulesTestCase.test_offline_courier_cannot_receive_orders) ... ok
test_weight_has_upper_limit (test_order_rules.OrderRulesTestCase.test_weight_has_upper_limit) ... ok
test_window_cannot_end_in_the_past (test_order_rules.OrderRulesTestCase.test_window_cannot_end_in_the_past) ... ok
test_benchmark_compares_modes_and_records_metrics
(test_routes.BenchmarkTestCase.test_benchmark_compares_modes_and_records_metrics) ... ok
test_benchmark_on_real_orders (test_routes.BenchmarkTestCase.test_benchmark_on_real_orders) ... ok
```

```
test_benchmark_size_is_limited (test_routes.BenchmarkTestCase.test_benchmark_size_is_limited) ... ok
test_capabilities (test_routes.BenchmarkTestCase.test_capabilities) ... ok
test_worker_counts_above_route_count_are_not_repeated
(test_routes.BenchmarkTestCase.test_worker_counts_above_route_count_are_not_repeated) ... ok
test_admin_sets_depot_location (test_routes.RoutesTestCase.test_admin_sets_depot_location) ... ok
test_changing_courier_discards_planned_route (test_routes.RoutesTestCase.test_changing_courier_discards_planned_route)
... ok
test_courier_sees_and_starts_own_route (test_routes.RoutesTestCase.test_courier_sees_and_starts_own_route) ... ok
test_generate_creates_one_route_per_courier_with_every_assigned_order
(test_routes.RoutesTestCase.test_generate_creates_one_route_per_courier_with_every_assigned_order) ... ok
test_generating_again_replaces_planned_routes
(test_routes.RoutesTestCase.test_generating_again_replaces_planned_routes) ... ok
test_generation_requires_depot_location (test_routes.RoutesTestCase.test_generation_requires_depot_location) ... ok
test_generation_without_assigned_orders_is_rejected
(test_routes.RoutesTestCase.test_generation_without_assigned_orders_is_rejected) ... ok
test_gpu_mode_reports_when_unavailable (test_routes.RoutesTestCase.test_gpu_mode_reports_when_unavailable) ... ok
test_offline_courier_is_left_out_with_reason (test_routes.RoutesTestCase.test_offline_courier_is_left_out_with_reason)
... ok
test_order_in_planned_route_can_be_deleted (test_routes.RoutesTestCase.test_order_in_planned_route_can_be_deleted) ...
ok
test_order_in_started_route_cannot_be_deleted
(test_routes.RoutesTestCase.test_order_in_started_route_cannot_be_deleted) ... ok
test_profiles_are_enforced (test_routes.RoutesTestCase.test_profiles_are_enforced) ... ok
test_route_completes_when_every_stop_is_delivered
(test_routes.RoutesTestCase.test_route_completes_when_every_stop_is_delivered) ... ok
test_route_distance_matches_the_sum_of_its_legs
(test_routes.RoutesTestCase.test_route_distance_matches_the_sum_of_its_legs) ... ok
test_route_in_progress_is_kept_when_generating_again
(test_routes.RoutesTestCase.test_route_in_progress_is_kept_when_generating_again) ... ok
test_routes_are_persisted_and_listed (test_routes.RoutesTestCase.test_routes_are_persisted_and_listed) ... ok
Ran 76 tests in 21.191s
OK
```

Execução da suíte de testes

Além da suíte automatizada, o roteiro de captura das evidências (tools/documentation/capture_sprint05_evidence.py) funciona como teste de ponta a ponta: ele executa pela interface e pela API o fluxo completo da seção 35, as regras da seção 38 e as comparações da seção 40, e falha se alguma tela ou resposta não for a esperada. Foi nessa execução que apareceram os dois bugs mais importantes da sprint.

42 Bugs encontrados e correções

Bug	Como foi encontrado	Correção	Situação
A comparação com 16 processos derrubava o pool de processos (BrokenProcessPool) e a API respondia erro 500	Roteiro de ponta a ponta	Cada processo carregava o NumPy com 16 threads do OpenBLAS e os pools de 1, 2, 4, 8 e 16 processos ficavam abertos juntos, esgotando a memória. O OpenBLAS passou a usar uma thread por processo e só um pool fica aberto. Novos testes cobrem os dois pontos	Corrigido
Com menos rotas que processos, a comparação repetia a mesma configuração (duas linhas "CPU 8 processos" com 8 rotas)	Roteiro de ponta a ponta (experimentos)	A quantidade efetiva de processos é limitada ao número de rotas, sem repetir, com teste automatizado	Corrigido
A mensagem de entregador fora de serviço dizia "os pedidos dele" também para entregadoras	Registro das regras (seção 38)	Texto neutro e teste que verifica a mensagem	Corrigido
No celular, o mapa da rota abria sem mostrar as paradas	Teste manual pela interface	O enquadramento era calculado antes de o mapa ter o tamanho final; agora é refeito quando o tamanho é conhecido	Corrigido

Bug	Como foi encontrado	Correção	Situação
Com as abas novas, o menu superior ultrapassava a largura da tela no celular	Teste manual com 390 px de largura	O menu passou a rolar na horizontal; a página voltou a ter exatamente a largura da tela	Corrigido
O rótulo do speedup ficava em cima da linha do ideal no gráfico	Revisão visual	O rótulo ganhou fundo próprio	Corrigido
Rota de uma parada aparecia como "1 paradas"	Teste manual	Plural ajustado	Corrigido
Um pedido reaberto depois de uma parada não realizada não poderia voltar para uma rota (order_id é único em route_stops)	Revisão do código	A nova parada substitui o registro antigo da rota encerrada	Corrigido
A troca de entregador não era detectada antes de gravar, porque o SQLAlchemy só atualiza o campo assigned_courier_id ao enviar ao banco	Revisão do código	A verificação passou a usar o relacionamento com o entregador	Corrigido
Um teste de regeração de rotas falhava só no SQLite, que reaproveita números de registros apagados	Suíte automatizada	O teste passou a verificar o conteúdo das rotas, não os números	Corrigido
As operações prontas do CuPy não compilavam na pasta do projeto, que tem acento (FÁBRICA)	Instalação do CuPy	O kernel foi escrito sem depender dos headers do CuPy, o que dispensa o caminho com acento	Contornado

Pendências e ações previstas

Pendência	Ação prevista
As medições variam quando outros programas usam a CPU ou a GPU ao mesmo tempo	Repetir os experimentos finais com a máquina dedicada, mais repetições e o registro da carga do sistema (Sprint 07)
Dentro da API, nas instâncias maiores, o pool com 1 processo ficou mais lento que o sequencial, embora num script isolado os tempos sejam iguais	Medir de novo com a máquina dedicada e, se a diferença continuar, medir separadamente o tempo de cálculo dentro de cada processo
A primeira geração no modo GPU depois de iniciar a API inclui a compilação do kernel, cerca de um segundo	Compilar o kernel quando a API inicia, em segundo plano
O contêiner Docker da API não acessa a GPU	Documentar a configuração com NVIDIA Container Toolkit ou manter o modo GPU só na execução sem Docker

43 Ajustes no planejamento, na arquitetura e na modelagem

Modelagem

- As tabelas routes, route_stops e optimization_runs, já previstas desde a Sprint 02, passaram a ser usadas e ganharam mapeamento no SQLAlchemy.
- A coluna execution_mode de routes e de optimization_runs passou a aceitar GPU, além de SEQUENTIAL e PARALLEL.

- optimization_runs ganhou as colunas purpose (geração de rotas ou comparação), input_source (pedidos reais ou instância sintética), run_group (agrupa as execuções de uma comparação), speedup, efficiency e same_routes.
- Bancos criados nas sprints anteriores recebem essas alterações automaticamente quando a API inicia, sem perda de dados. A atualização foi verificada num banco da Sprint 04.

Arquitetura e API

- Novos módulos no backend: planning.py (regras das rotas), benchmark.py (metodologia da comparação), optimizer/gpu.py (kernel CUDA) e os routers routes.py e optimizer.py.
- Novas rotas da API: POST /routes/generate, GET /routes, GET /routes/me, PATCH /routes/{id}/start, GET e PUT /establishment/depot, GET /optimizer/capabilities, POST /optimizer/benchmark e GET /optimizer/runs.
- As rotas de pedidos chamam planning.py a cada mudança, para manter as rotas coerentes com os pedidos.
- Dependências: NumPy no backend e, opcionalmente, CuPy (backend/requirements-gpu.txt), que instala pelo pip o compilador e as bibliotecas do CUDA.
- Frontend: novas telas Rotas e Desempenho, o componente RouteMap e a rota do entregador dentro de Minhas entregas.
- A escolha entre C++ com OpenMP e CUDA, deixada em aberto no backlog da Sprint 01, foi resolvida a favor de CUDA: a equipe tem GPU NVIDIA disponível, o modelo de milhares de threads se encaixa na avaliação das trocas do 2-opt e o CuPy permite manter a API em Python.

Planejamento

Com a roteirização antecipada, o cronograma das próximas sprints foi reorganizado em torno da qualidade das rotas, dos experimentos e dos relatórios.

Sprint	Período	Entrega principal	Situação
1	03/09 a 05/09	Planejamento, requisitos e backlog	Concluída
2	06/09 a 19/09	Arquitetura, modelos, protótipos, banco e estrutura	Concluída
3	20/09 a 26/09	Banco conectado, login, perfis, usuários e CRUD de pedidos	Concluída
4	27/09 a 28/09	Módulo Pedidos e entregas completo	Concluída
5	29/09 a 03/10	Módulo Roteirização e desempenho com CPU paralela e GPU	Concluída
6	04/10 a 10/10	Atribuição automática dos pedidos e prioridade na ordem das paradas	Planejada
7	11/10 a 17/10	Experimentos controlados e relatório de desempenho (RF09)	Planejada
8	18/10 a 24/10	Painel administrativo do dia (RF10)	Planejada
9	25/10 a 31/10	Janelas de horário nas rotas e traçado pelas ruas	Planejada
10	01/11 a 07/11	Testes, usabilidade e segurança	Planejada
11	08/11 a 21/11	Documentação e ajustes	Planejada
Final	22/11 a 05/12	Correções, vídeos e preparação para a banca	Planejada

44 Repositório GitHub

Repositório oficial. https://github.com/alvccpj/rota-certa

A Sprint 05 foi desenvolvida na branch sprint/05-modulo-otimizacao-rotas, criada a partir da master com a Sprint 04 integrada pelo Pull Request número 4. O passo a passo para executar o projeto foi separado no arquivo COMO-RODAR.md, com os comandos para PowerShell e Git Bash comentados um a um, e o README passou a descrever os módulos e a paralelização com CUDA.

```
$ git log --reverse 1ef871a..HEAD~1
1c19228 01/10 14:09 docs: update README and add COMO-RODAR guide for local setup instructions
d8fe522 01/10 16:51 feat: reescreve o otimizador com matriz de distâncias e adiciona o modo GPU com CUDA
9c03ef8 01/10 16:51 feat: gera, persiste e acompanha as rotas dos entregadores
54d8b2b 01/10 16:58 feat: adiciona as telas de rotas, rota do entregador e desempenho do otimizador
72b5261 01/10 17:02 fix: remove o gênero fixo da mensagem de entregador fora de serviço
cf670ab 01/10 17:06 fix: evita a quebra do pool de processos na comparação de desempenho
04c8ee1 01/10 17:11 fix: não repete configurações com mais processos do que rotas na comparação
```

Commits da Sprint 05 em ordem cronológica

45 Dificuldades encontradas e próximos passos

Dificuldades

Dificuldade	Como foi tratada
O CuPy não compilava as próprias operações porque o caminho do projeto tem acento	O kernel foi escrito em CUDA C sem depender dos headers do CuPy; as cópias de memória e o kernel funcionam em qualquer pasta
Fazer a GPU chegar exatamente às mesmas rotas da CPU	Precisão dupla, desempate por menor índice, mesma ordem nas somas e compilação sem fusão de multiplicação e soma
Medir desempenho no Windows, onde criar processos é caro	O pool de processos fica aberto e é aquecido antes de medir; as rotas são enviadas em fatias
Variação das medições com outros programas abertos e com 8 núcleos físicos para 16 lógicos	Mediana de várias execuções; a análise considera o limite físico de núcleos; os experimentos finais serão repetidos com a máquina dedicada
Capturar as evidências num banco recém-criado sem apagar os dados locais de desenvolvimento	As evidências foram capturadas num schema separado do PostgreSQL, com uma segunda instância da API e do frontend

Próximos passos

- Atribuição automática dos pedidos pendentes respeitando disponibilidade, capacidade e raio, com a rota calculada logo em seguida.
- Considerar a prioridade e as janelas de horário na ordem das paradas, mantendo a equivalência entre os modos.
- Repetir os experimentos com a máquina dedicada e produzir o relatório de desempenho (RF09).
- Compilar o kernel CUDA quando a API inicia e documentar o uso da GPU com Docker.
- Distribuir as tarefas entre os integrantes, com pull requests revisados por outro colega.

Situação da Sprint 05

Entrega obrigatória	Situação	Evidência
Segundo módulo completo	Concluído	Seções 35 e 36: fluxo completo de roteirização, do pedido atribuído à rota concluída
Integração com o banco de dados	Concluído	Seção 37: routes, route_stops e optimization_runs, com dados recuperados depois de reiniciar a API
Atualização das regras de negócio	Concluído	Seção 38: 14 regras e as alterações justificadas
Testes das funcionalidades	Concluído	Seção 41: 76 testes automatizados e o roteiro de ponta a ponta
Correção dos bugs encontrados	Concluído	Seção 42: bugs, correções e pendências com ação prevista
Repositório GitHub atualizado	Concluído	Seção 44: branch, commits, README e COMO-RODAR.md
Dificuldades e próximos passos	Concluído	Seção 45

Parte VI Sprint 06 Aprimoramento do sistema

A Sprint 06 entrega o terceiro módulo do RotaCerta: Painel operacional e relatórios. O módulo transforma os dados já produzidos por pedidos, entregadores, rotas e execuções do otimizador em uma visão única da operação, com indicadores por período, alertas de pendências, atividades recentes, atalhos para os fluxos principais e relatório exportável em CSV.

46 Terceiro módulo: Painel operacional e relatórios

O painel é a nova página inicial dos perfis Administrador e Atendente. Os valores vêm do PostgreSQL pela API; não há números fixos na interface. O perfil Entregador continua entrando diretamente em Minhas entregas e não acessa os dados consolidados do estabelecimento.

Funcionalidade	Integração utilizada	Perfis e acesso
Indicadores de pedidos por situação e taxa de conclusão	orders e customers	Administrador e atendente | /painel
Entregadores disponíveis, ocupados e fora de serviço	users e couriers	Administrador e atendente | /painel
Rotas por situação e distância total planejada	routes e route_stops	Administrador e atendente | /painel
Execuções, tempo médio, melhor speedup e último modo	optimization_runs	Administrador e atendente | /painel
Filtro por período, pedidos recentes e alertas operacionais	consultas integradas da API	Administrador e atendente | GET /dashboard/summary
Relatório operacional em CSV	pedidos ligados ao cliente, entregador e rota	Administrador e atendente | GET /dashboard/report

Fluxo de utilização

1. O atendente cadastra o cliente e o pedido e atribui um entregador, usando o módulo Pedidos e entregas.
2. Na tela Rotas, os pedidos atribuídos são agrupados por entregador, otimizados e persistidos com as paradas e a distância total.
3. O entregador inicia a rota e confirma as entregas pelo celular; as mudanças atualizam pedidos, paradas e a situação da rota.
4. Ao abrir a Visão geral, o painel consulta os mesmos registros e atualiza os indicadores, o fluxo dos pedidos, os alertas e as atividades recentes.
5. O administrador ou atendente seleciona um período e exporta o relatório CSV com pedido, cliente, entregador, situação e vínculo com a rota.

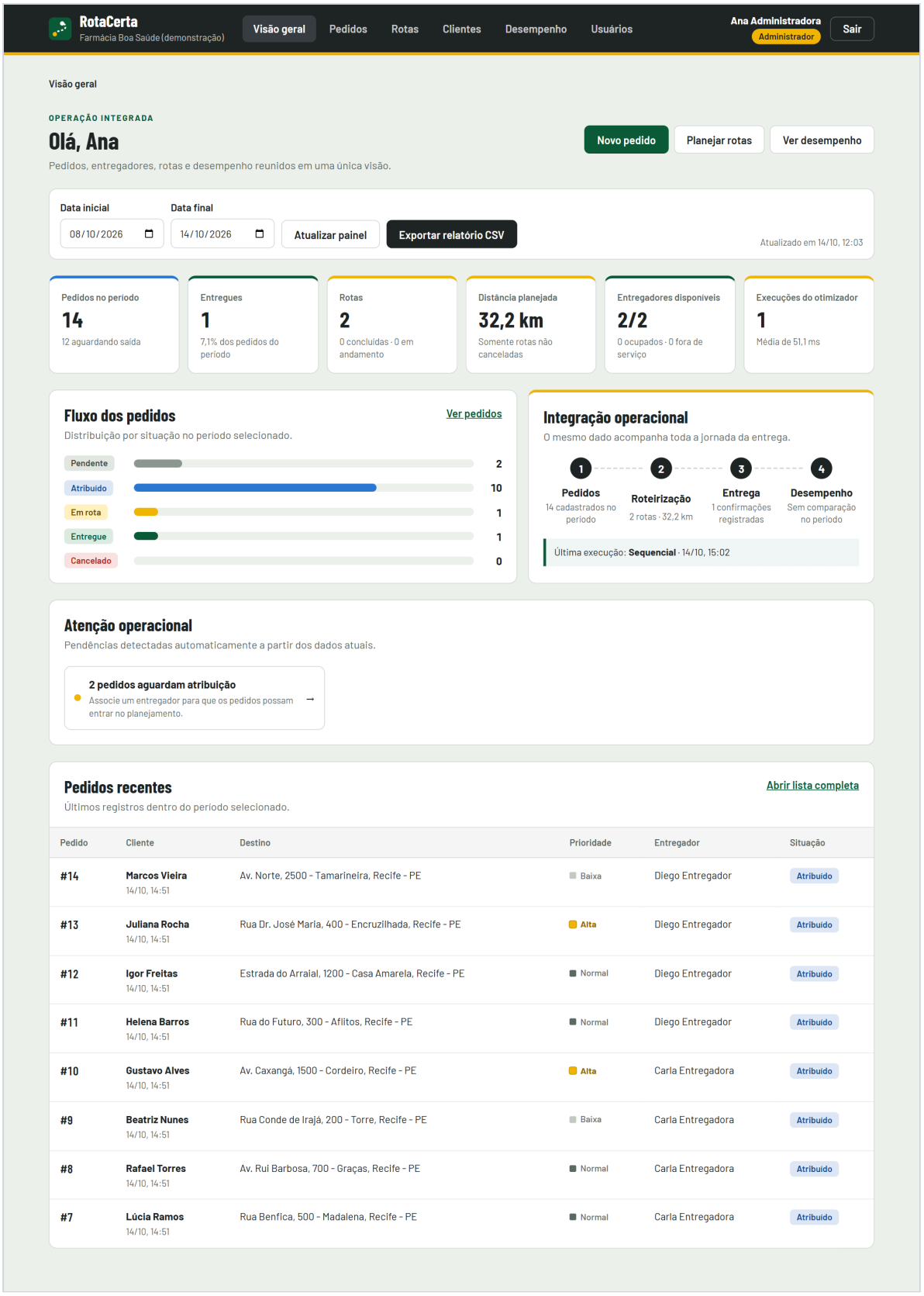


Figura 54 Painel operacional com indicadores, alertas e pedidos recentes calculados a partir dos dados persistidos

47 Integração entre os módulos

A integração foi implementada no backend/app/reporting.py. O serviço consulta as entidades dos dois módulos anteriores no mesmo estabelecimento e devolve um contrato único para a interface. O router

/dashboard aplica autenticação e autoriza somente administradores e atendentes. O período padrão cobre os últimos sete dias, aceita até 366 dias e é validado tanto na interface quanto na API.

Origem	Informação reutilizada	Resultado no painel
Pedidos e clientes	situação, prioridade, peso, destino, cliente e data	totais por situação, taxa de conclusão e pedidos recentes
Entregadores	perfil ativo e disponibilidade	quantidade total, disponíveis, ocupados e fora de serviço
Rotas e paradas	situação, data, distância e vínculo com o pedido	rotas planejadas, em andamento, concluídas e distância total
Otimizador	modo, tempo, speedup e data da execução	quantidade de execuções, tempo médio, melhor speedup e última execução

Figura 55 Fluxo dos pedidos e sequência de integração entre pedido, roteirização, entrega e desempenho

```
Evidência da integração entre os módulos

GET /dashboard/summary -> HTTP 200
- Período consultado: 08/10/2026 a 14/10/2026
- Pedidos: 14 no total (2 pendentes, 10 atribuídos e 1 entregue)
- Entregadores: 2 cadastrados
- Rotas: 2 rotas persistidas, com 32,245 km no total
- Histórico de desempenho: 1 execução antes do novo processamento

POST /routes/generate -> HTTP 200
- Resultado: 2 rotas geradas e gravadas no banco de dados

GET /dashboard/summary após a geração -> HTTP 200
- Histórico de desempenho atualizado para 2 execuções
- Pedidos, entregadores e rotas permaneceram consistentes

GET /dashboard/report -> HTTP 200
- 14 registros integrados retornados para o relatório operacional
- Exportação do arquivo CSV concluída pelo navegador

O arquivo integracao.txt conserva as respostas completas da API utilizadas
para produzir esta síntese.
```

Síntese das respostas da API antes e depois da geração das rotas e do relatório

48 Evidências das funcionalidades

As evidências foram capturadas com o estabelecimento de demonstração, catorze pedidos, dois entregadores e duas rotas geradas no modo sequencial. Depois da geração, o painel passou a mostrar duas rotas, 32,2 km planejados e a execução registrada no histórico do otimizador.

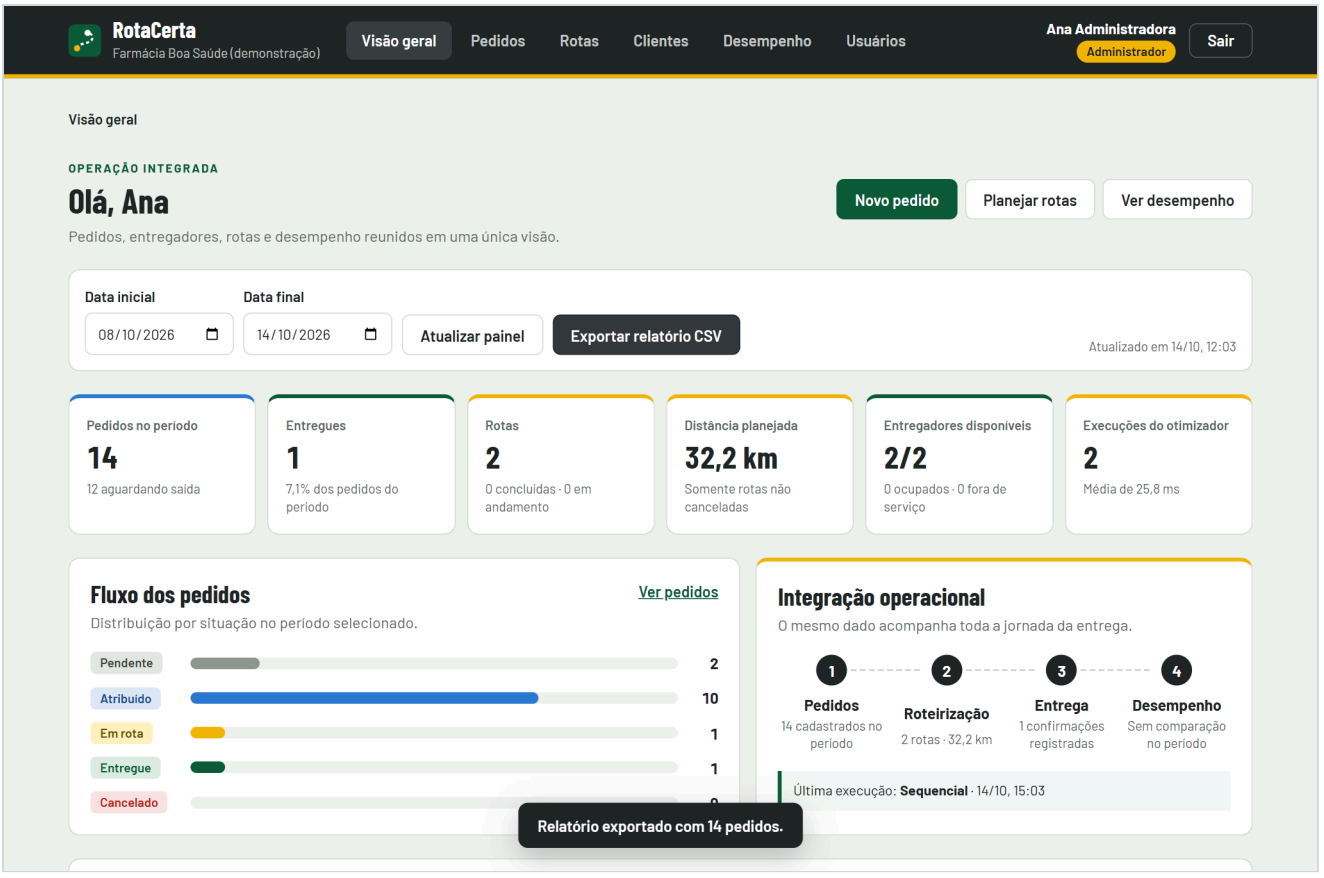


Figura 56 Relatório CSV exportado com os catorze pedidos do período e confirmação exibida pela interface

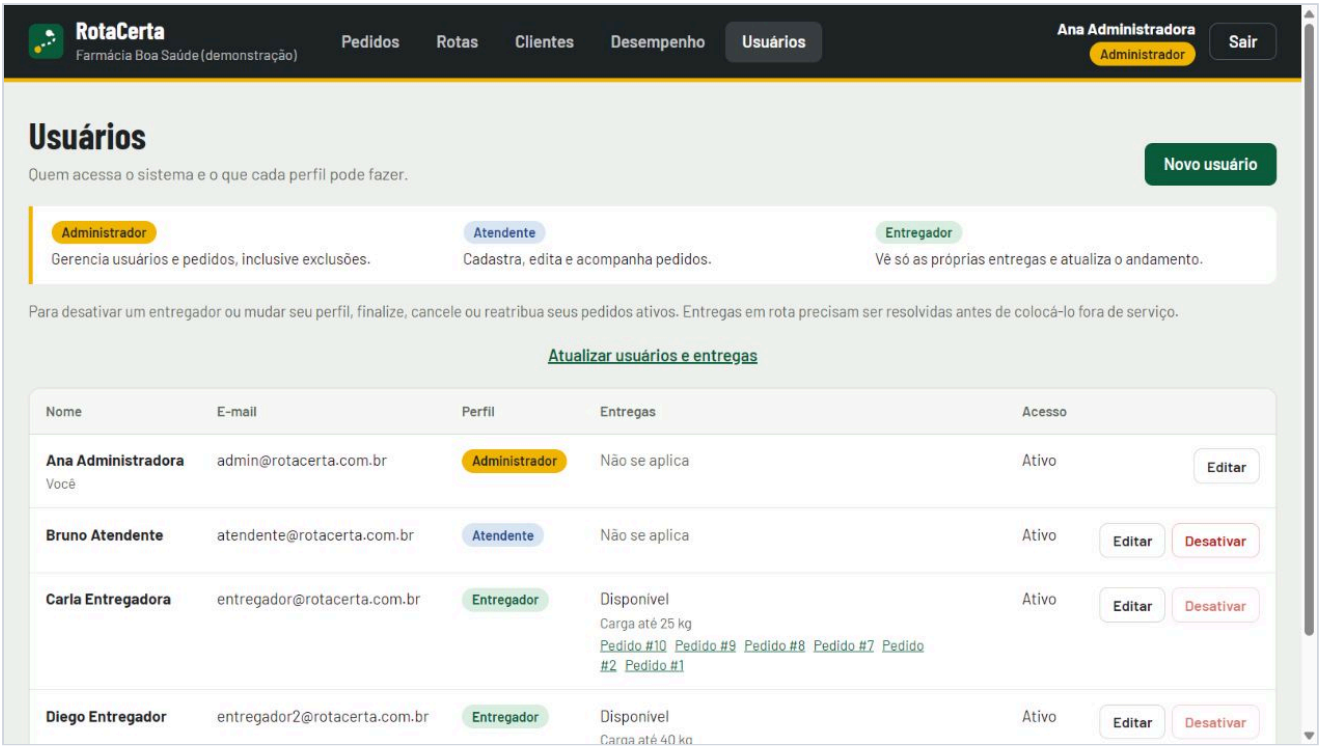


Figura 57 Gestão de usuários com acesso aos pedidos ativos e proteção contra a desativação indevida de entregadores

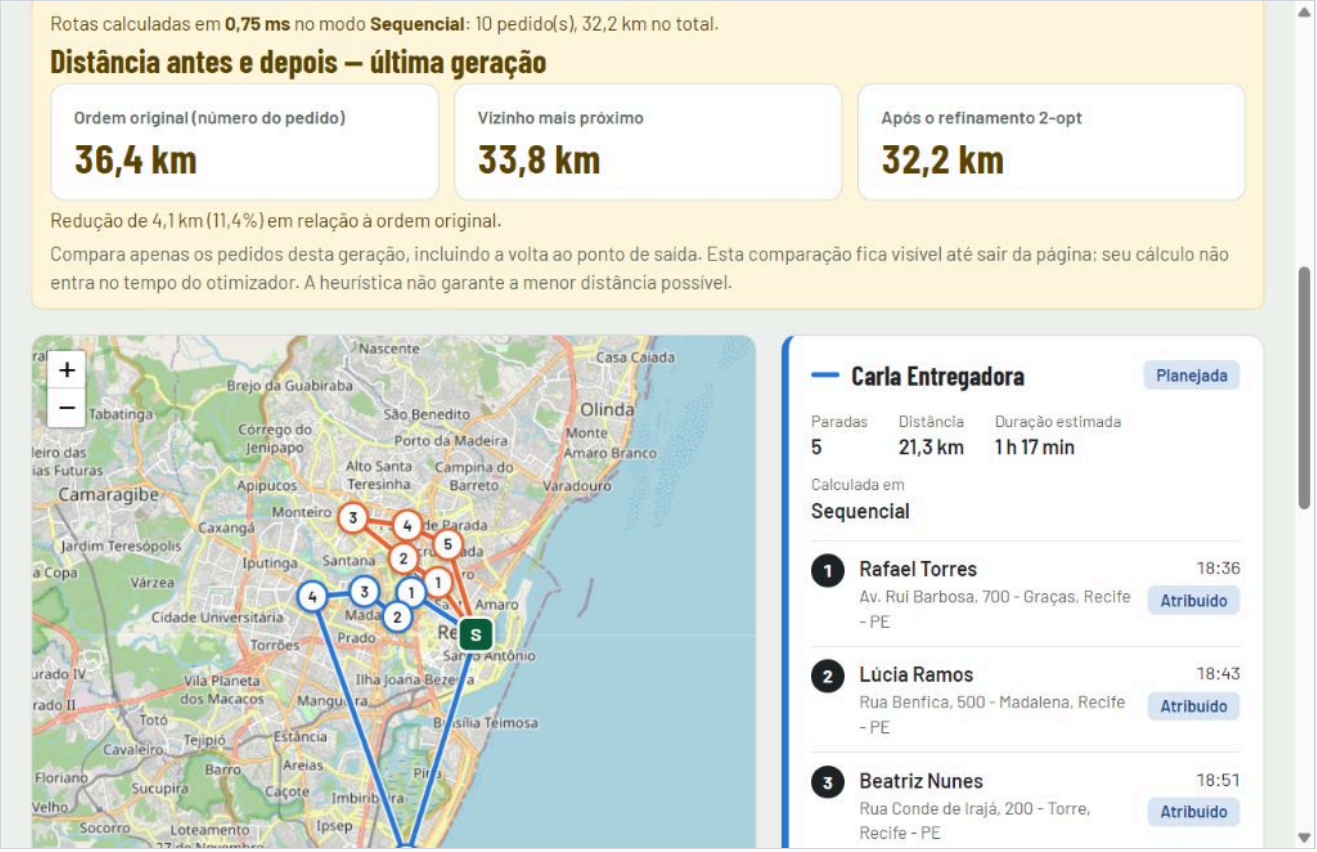


Figura 58 Comparação entre a ordem original, vizinho mais próximo e refinamento 2-opt na geração das rotas

Relatório exportável

O botão Exportar relatório CSV consulta GET /dashboard/report usando o período selecionado e gera um arquivo compatível com planilhas. Cada linha contém o pedido, cliente, entregador, endereço, situação, prioridade, peso, data de cadastro e os dados da rota quando existem. O arquivo usa ponto e vírgula e codificação UTF-8 com marcador para preservar acentos no Excel.

Controle de acesso

As duas rotas da API exigem os perfis Administrador ou Atendente. O Entregador recebe HTTP 403 mesmo que tente chamar a API diretamente. Na interface, a rota /painel também usa o controle de perfil e o menu do entregador não apresenta a Visão geral.

49 Melhorias na interface e na navegação

A interface ganhou hierarquia mais clara para a operação diária: indicadores no início da página, filtros reunidos, ações frequentes próximas ao título, distribuição visual das situações, alertas que levam diretamente à ação necessária e tabela recente com acesso ao detalhe do pedido. Foram implementados estados de carregamento, erro, período sem dados e operação sem alertas.

Melhoria	Implementação
Página inicial útil	Administrador e atendente entram em /painel; o entregador continua em /entregas
Navegação desktop	Visão geral passou a ser a primeira opção; espaçamento adaptativo evita opções cortadas em larguras intermediárias
Navegação móvel	menu recolhível com botão acessível, estado expandido e página ativa destacada

Melhoria	Implementação
Atalhos	Novo pedido, Planejar rotas e Ver desempenho ficam disponíveis no cabeçalho do painel
Acompanhamento	listas de pedidos e entregas são atualizadas a cada 15 segundos quando a aba está visível, preservando os filtros
Regras visíveis	ações de situação usam as permissões devolvidas pela API; pedidos ativos aparecem vinculados ao entregador
Acessibilidade	rótulos nos filtros, foco visível, navegação semântica, títulos de tabela e mensagens com região de status

Figura 59 Menu móvel expandido com acesso organizado aos três módulos e às áreas administrativas

50 Testes e qualidade técnica

A suíte combinada passou de 76 para 90 testes aprovados. Além dos cinco testes do painel, os testes adicionados por William verificam transições de situação, motivo de cancelamento, preservação de janelas antigas, proteção de entregadores com pedidos ativos e comparação de qualidade das rotas. Um teste adicional da implementação CUDA permanece ignorado automaticamente em máquinas sem GPU NVIDIA.

```
$ python -m unittest discover -s tests -v

90 testes aprovados
1 teste ignorado: GPU NVIDIA com CUDA não disponível nesta máquina
0 falhas
0 erros

Resultado final:
Ran 90 tests in 59.508s
OK (skipped=1)

Os testes adicionados na Sprint 06 verificam:
- consolidação dos dados de demonstração no painel;
- atualização dos indicadores depois da geração das rotas;
- filtros de período e situação no relatório;
- validação de períodos inválidos ou superiores a 366 dias;
- bloqueio do painel e do relatório para o perfil Entregador;
- transições de situação de pedidos conforme o perfil;
- exigência e persistência do motivo de cancelamento;
- preservação e validação das janelas de entrega;
- proteção de entregadores com pedidos ativos;
- comparação das distâncias original, vizinho mais próximo e 2-opt.

Validação realizada após a integração de dev/william com
sprint/06-aprimoramento-sistema.
```

Resultado da suíte automatizada do backend

O frontend foi compilado para produção depois da implementação. A versão do Vite e a dependência transitiva source-map-js foram atualizadas, eliminando os avisos de segurança sem mudança de versão principal.

```
$ npm run build

vite v7.3.7 building client environment for production...
74 modules transformed.
dist/index.html                   1.21 kB | gzip:   0.64 kB
dist/assets/index-LoyevqfR.css   42.73 kB | gzip:  12.55 kB
dist/assets/index-DxF-8KgD.js   476.00 kB | gzip: 145.66 kB
build concluído com sucesso em 4.70s

$ npm audit --audit-level=high
found 0 vulnerabilities

Correções aplicadas:
- Vite atualizado de 7.1.5 para 7.3.7;
- source-map-js atualizado para 1.2.2;
- menu desktop ajustado para não cortar opções em larguras intermediárias;
- menu móvel recolhível com botão, estado expandido e indicação da página ativa.
- atualização automática das listas sem perda de filtros;
- ações de pedido orientadas pelas permissões da API;
- comparação visual da qualidade das rotas.
```

Build de produção e auditoria das dependências do frontend

51 Ajustes no planejamento e na arquitetura

Planejamento

O Painel administrativo do dia, originalmente previsto como RF10 para uma etapa posterior, foi antecipado para atender ao terceiro módulo da Sprint 06 e consolidar os dois módulos já funcionais. A validação de janelas de horário foi reforçada na edição dos pedidos; a atribuição automática e o uso da prioridade e das janelas como restrições do otimizador permanecem planejados para as próximas etapas.

Sprint	Período	Entrega principal	Situação
1	03/09 a 05/09	Planejamento, requisitos e backlog	Concluída
2	06/09 a 19/09	Arquitetura, modelos, protótipos, banco e estrutura	Concluída
3	20/09 a 26/09	Banco conectado, autenticação, perfis e CRUD	Concluída
4	27/09 a 28/09	Módulo Pedidos e entregas	Concluída
5	29/09 a 03/10	Módulo Roteirização e desempenho	Concluída
6	11/10 a 17/10	Painel operacional, relatórios e integração	Concluída
7	18/10 a 24/10	Dashboard, relatórios, pesquisa, filtros, exportação e histórico	Concluída
8	25/10 a 31/10	Janelas de horário e traçado pelas ruas	Planejada
9	01/11 a 07/11	Testes de usabilidade, segurança e acessibilidade	Planejada
10	08/11 a 14/11	Notificações e evolução dos relatórios	Planejada
11	15/11 a 21/11	Documentação, medições finais e ajustes	Planejada
Final	22/11 a 05/12	Vídeos, demonstração e preparação final	Planejada

Arquitetura

- Novo serviço backend/app/reporting.py para manter agregações e relatório separados das rotas HTTP.
- Novo router backend/app/routers/dashboard.py com GET /dashboard/summary e GET /dashboard/report.
- Novos contratos Pydantic para indicadores, período, pedidos recentes e linhas do relatório.
- Nova página frontend/src/pages/DashboardPage.tsx e contratos TypeScript equivalentes.
- Regras de transição e proteção de entregadores centralizadas no backend/app/rules.py e expostas à interface pelo contrato de detalhe do pedido.
- Componente AutoRefresh para atualizar listas visíveis sem descartar filtros, além da comparação de qualidade retornada pela geração de rotas.
- Nenhuma tabela ou migração foi necessária: o módulo usa relações e dados existentes no modelo da Sprint 02 e implementados nas Sprints 04 e 05.

52 Repositório GitHub

Repositório oficial. https://github.com/alvccpj/rota-certa

A Sprint 06 foi desenvolvida na branch sprint/06-aprimoramento-sistema, criada a partir da master com a Sprint 05 integrada. As três contribuições de William, publicadas em dev/william, foram revisadas, validadas em conjunto e integradas por merge identificado. O módulo, os testes, as evidências e o documento cumulativo permaneceram organizados em commits separados para facilitar a revisão por Pull Request.

```
$ git log --reverse --format="%h %ad %s" --date=format:"%d/%m %H:%M" origin/master..HEAD

40ceaaf 14/10 11:08 Vinicius Ribau | docs: preparar planejamento da sprint 06
56b127e 14/10 11:57 William Coelho de Morais | fix: valida pedidos e protege entregadores com entregas ativas
1a06f70 14/10 11:57 William Coelho de Morais | feat: melhora acompanhamento de entregas e comparacao de rotas
c080a68 14/10 11:57 William Coelho de Morais | docs: registra alteracoes e participacao de William na equipe
b73d2c3 14/10 12:04 Vinicius Ribau | feat: implementar painel operacional da sprint 06
e8b0c06 14/10 12:16 Vinicius Ribau | docs: concluir evidências e documento da sprint 06
16cefdb 14/10 12:49 Vinicius Ribau | docs: adicionar Guilherme à equipe
17bc4fb 14/10 12:52 Vinicius Ribau | merge: integrar melhorias de William à Sprint 06
```

Commits da Sprint 06 em ordem cronológica

53 Dificuldades encontradas e próximos passos

Dificuldades

Dificuldade	Como foi tratada
Consolidar quatro grupos de dados sem duplicar regras dos módulos anteriores	As consultas ficaram em reporting.py e reutilizam as entidades e vínculos já persistidos
Manter o painel útil em períodos sem rotas ou sem execuções	Indicadores aceitam zero, textos explicam a ausência e alertas sugerem a próxima ação
Adicionar uma opção ao menu sem cortar a navegação	Espaçamento adaptativo no desktop e menu recolhível no celular
Dependências do frontend com alertas de segurança	Atualização compatível do Vite e de source-map-js, seguida de novo build e auditoria
Comprovar a integração sem depender apenas das telas	Testes automatizados e evidência das respostas da API antes e depois da geração das rotas
Reunir contribuições produzidas em branches paralelas	A árvore combinada foi verificada sem conflitos, executou 90 testes e recebeu novo build antes da integração

Próximos passos

- Adicionar filtros do relatório por situação e entregador também na interface.
- Incluir tendências semanais de entregas, distância e taxa de conclusão.
- Implementar atribuição automática respeitando disponibilidade, capacidade e raio.
- Considerar prioridade e janela de horário durante a otimização das rotas.
- Repetir as medições finais de CPU e GPU com a máquina dedicada.

54 Situação da Sprint 06

Entrega obrigatória	Situação	Evidência
Terceiro módulo implementado	Concluído	Seção 46: painel, filtros, indicadores, alertas, atividades e relatório CSV
Integração entre módulos	Concluída	Seção 47: dados de pedidos, entregadores, rotas e otimizador na mesma visão
Funcionalidades demonstráveis	Concluídas	Seção 48: telas, exportação e controle de acesso

Entrega obrigatória	Situação	Evidência
Melhorias na interface	Concluídas	Seção 49: hierarquia, estados, alertas, atalhos e responsividade
Ajustes de navegação	Concluídos	Seção 49: nova página inicial, menu desktop e menu móvel
Testes e segurança	Concluídos	Seção 50: 90 testes, build aprovado e zero vulnerabilidades
Repositório atualizado	Concluído	Seção 52: branch da Sprint 06 e histórico de commits
Dificuldades e próximos passos	Concluído	Seção 53

Parte VII Sprint 07 Sistema quase completo

A Sprint 07 aproxima o RotaCerta de sua conclusão funcional. O painel da Sprint anterior foi preservado e conectado a duas novas áreas: Relatórios, para pesquisar, filtrar e exportar os dados operacionais, e Histórico, para acompanhar alterações de pedidos e execuções do otimizador em ordem cronológica.

55 Dashboard e acompanhamento operacional

O dashboard em /painel utiliza dados reais do estabelecimento autenticado. O período selecionado controla os totais de pedidos, a taxa de conclusão, as rotas, a distância planejada e as métricas do otimizador. A disponibilidade dos entregadores, os alertas e os pedidos recentes complementam a visão para apoiar decisões diárias.

Indicador	Fonte persistida	Uso operacional
Pedidos e distribuição por situação	orders	Identificar pendências e acompanhar a conclusão das entregas
Rotas e distância planejada	routes e route_stops	Acompanhar o planejamento e a execução logística
Disponibilidade dos entregadores	users e couriers	Verificar a capacidade operacional antes de distribuir pedidos
Execuções e tempo do otimizador	optimization_runs	Acompanhar o componente avançado e suas métricas

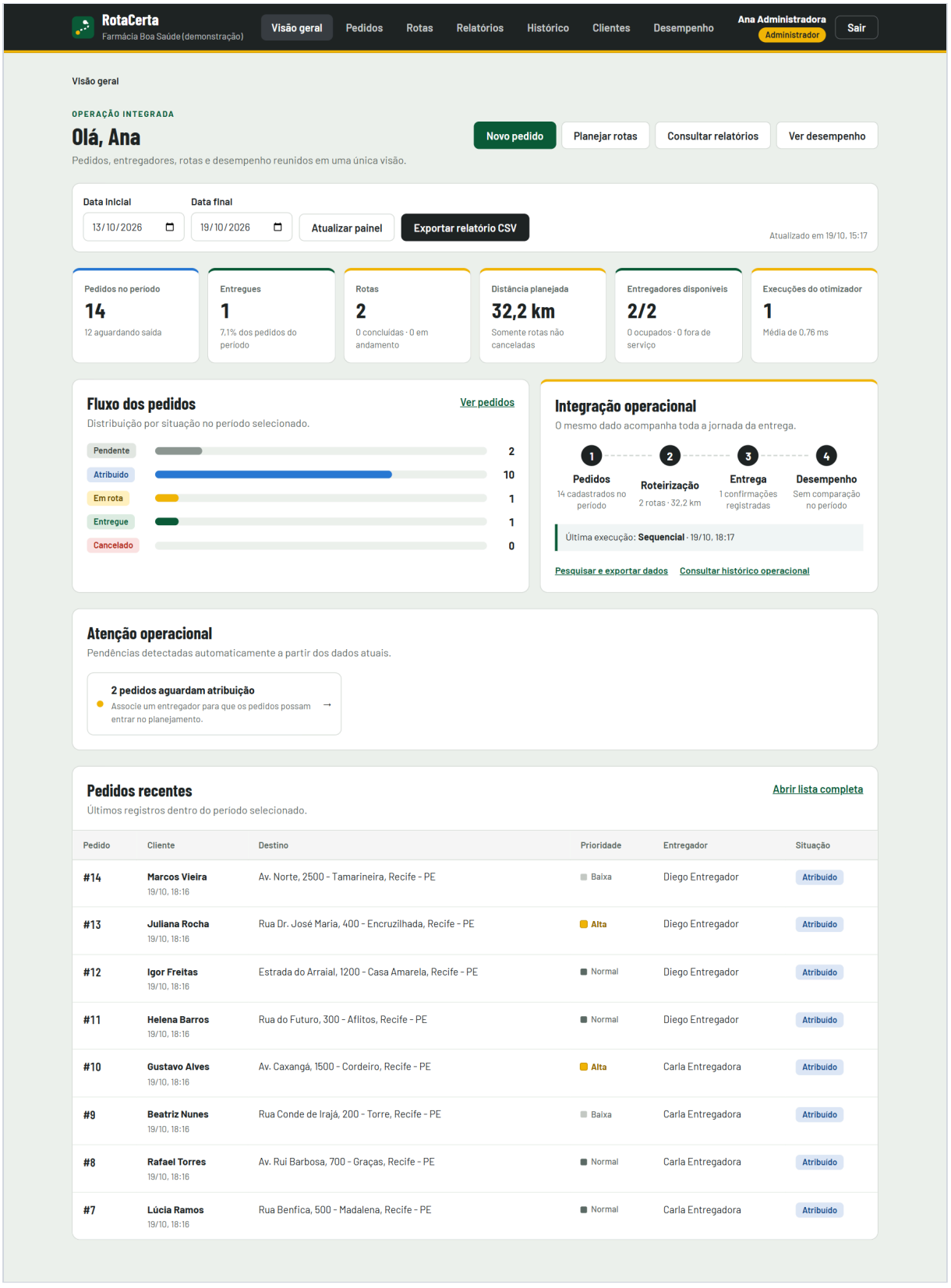


Figura 60 Dashboard funcional com catorze pedidos, duas rotas, 32,2 km planejados e métricas do otimizador

56 Relatórios desenvolvidos

A página /relatorios cruza pedidos, clientes, entregadores e rotas na mesma consulta. Além da listagem detalhada, os resultados produzem quatro resumos: quantidade de pedidos, peso total, pedidos com rota e entregas concluídas. Cada pedido mantém acesso ao detalhe e ao histórico individual.

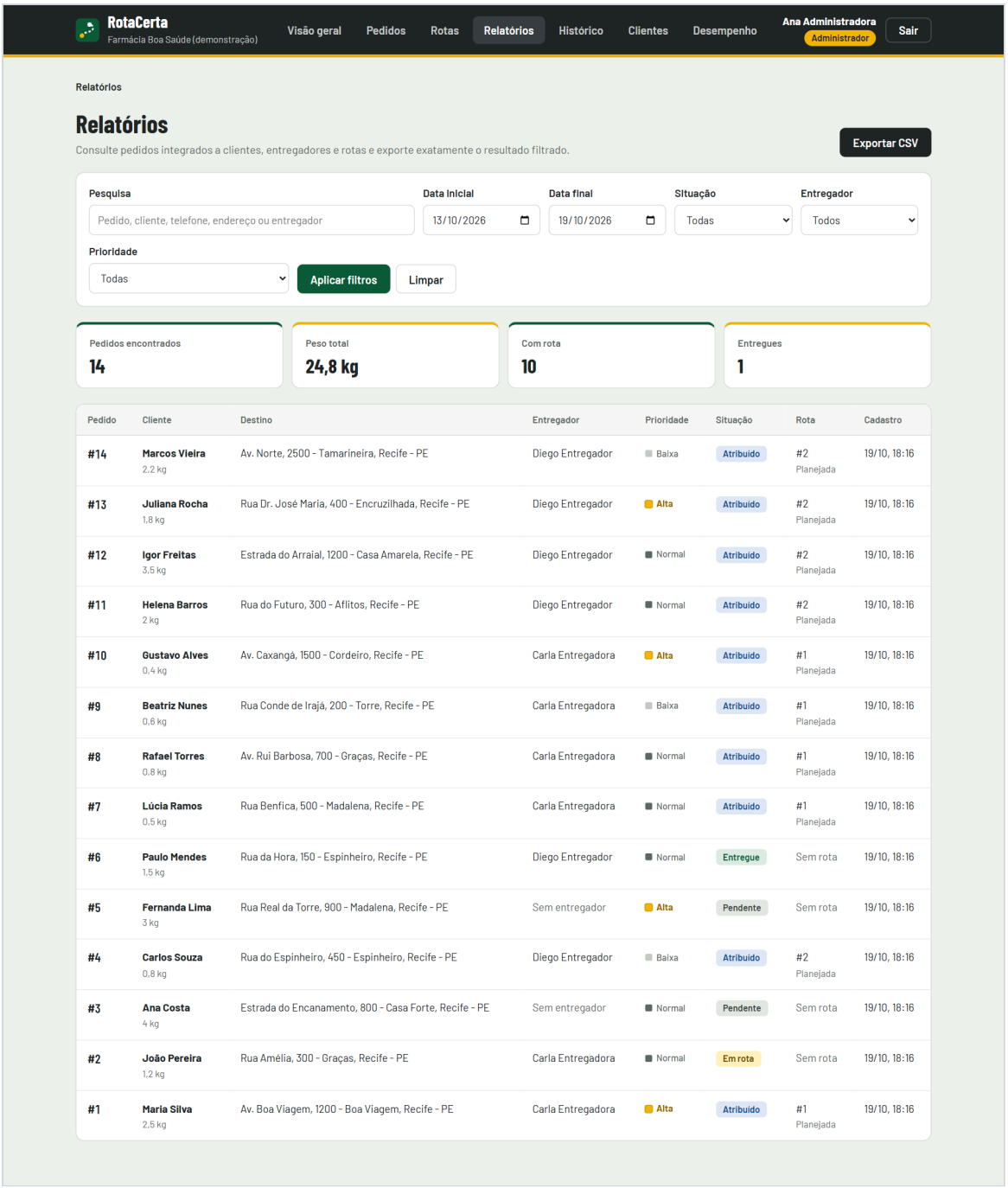


Figura 61 Relatório operacional com dados dos módulos anteriores, resumos consolidados e acesso aos pedidos

57 Pesquisas e filtros

A pesquisa livre localiza registros por número do pedido, nome ou telefone do cliente, endereço e nome do entregador. Ela pode ser combinada com período, situação, entregador e prioridade. No histórico, os critérios são período, texto, tipo de operação e situação do pedido. A API aplica todos os critérios antes de devolver os registros, mantendo a interface e a exportação consistentes.

Área	Critérios implementados	Resultado demonstrado
Relatório	texto, período, situação, entregador e prioridade	Boa Viagem + Atribuído + prioridade Alta retornou somente o pedido #1
Histórico	texto, período, tipo de operação e situação	Otimização retornou somente as execuções registradas do otimizador
Dashboard	período inicial e final	Indicadores e pedidos recentes recalculados para o intervalo

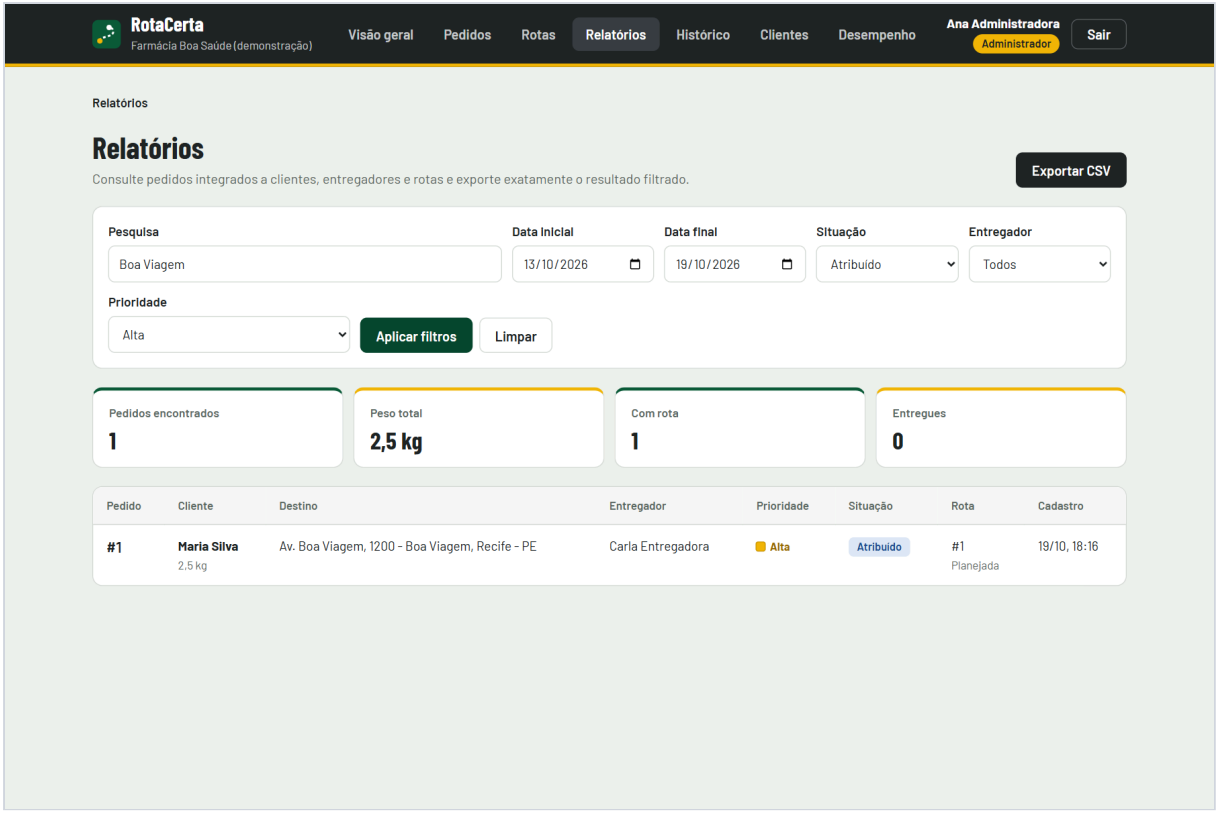


Figura 62 Pesquisa e filtros combinados reduzindo o relatório ao pedido de Maria Silva em Boa Viagem

58 Exportação CSV

A exportação é aplicável ao projeto porque os pequenos negócios podem arquivar, compartilhar ou analisar a operação em uma planilha. O arquivo CSV contém exatamente o resultado filtrado e inclui pedido, cliente, entregador, endereço, situação, prioridade, peso, cadastro e dados da rota. O separador é ponto e vírgula e o conteúdo inclui marcador UTF-8 para preservar acentos em planilhas configuradas em português.

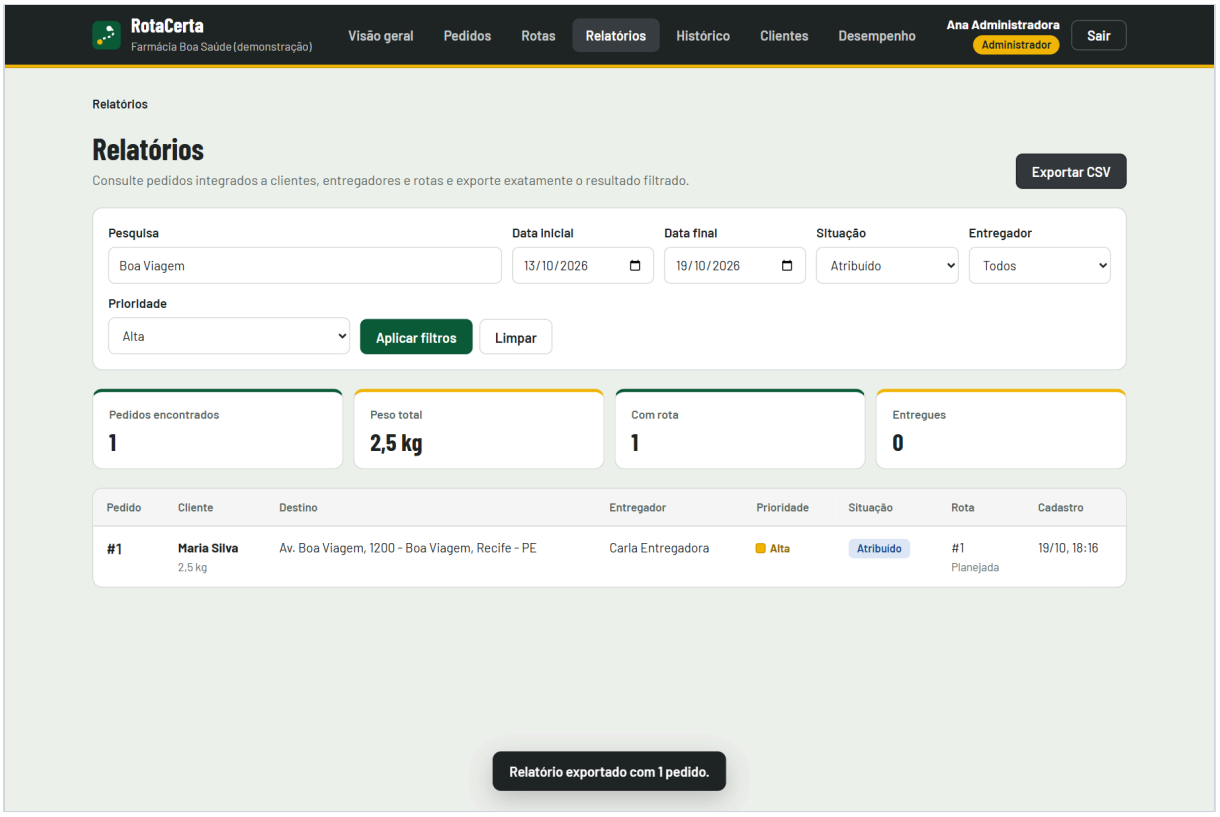


Figura 63 Confirmação da exportação CSV do único pedido retornado pela pesquisa e pelos filtros

59 Logs e histórico de operações

A rota /historico reúne registros persistidos de duas origens. As mudanças de situação vêm de order_status_history e informam pedido, cliente, situação, observação, responsável e data. As execuções do otimizador vêm de optimization_runs e informam modo, quantidade de pedidos, tempo e data. A resposta é ordenada do evento mais recente para o mais antigo e limitada aos quinhentos eventos mais recentes do estabelecimento.

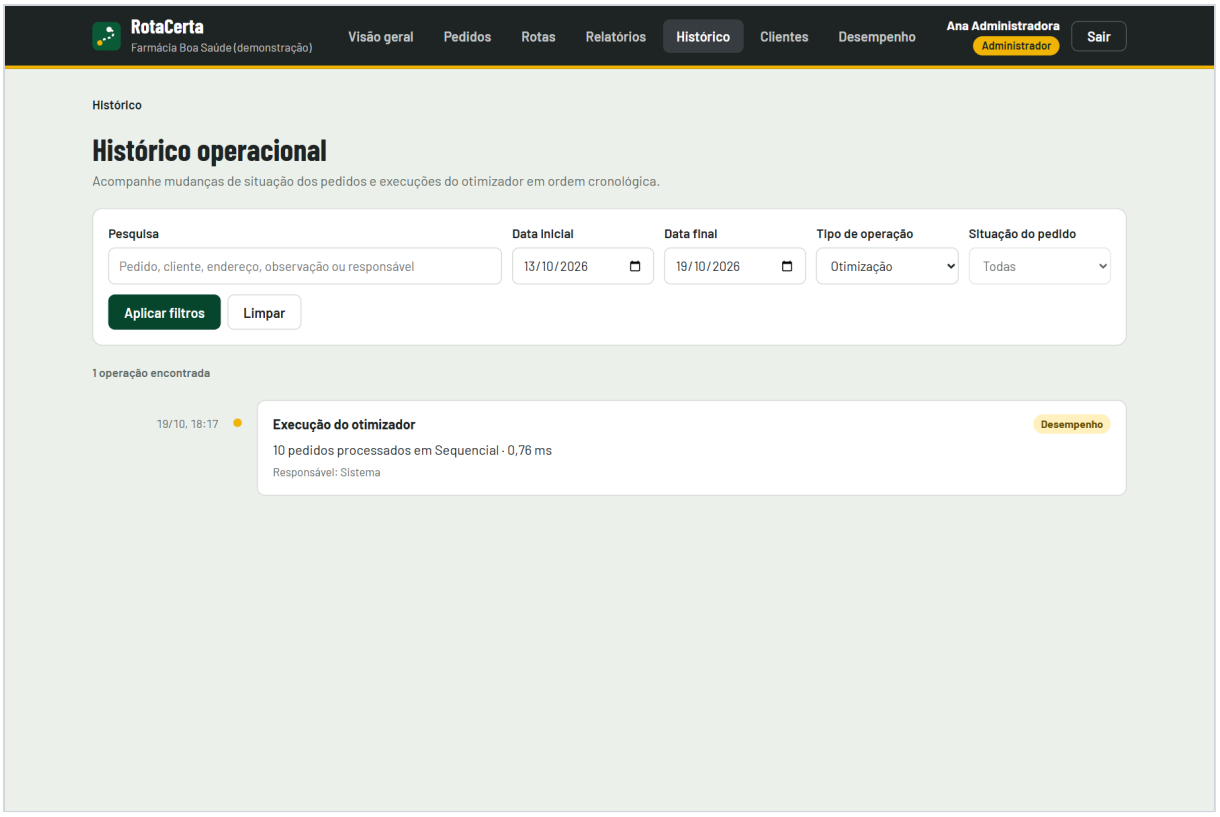


Figura 64 Histórico filtrado por Otimização, comprovando a rastreabilidade das execuções do componente avançado

```
Base local persistida — 19/10/2026

GET /dashboard/summary
14 pedidos | 2 rotas | 32,245 km | 2 entregadores | 1 execução do otimizador

GET /dashboard/report
14 pedidos integrados a clientes, entregadores e rotas

GET /dashboard/report?q=Boa Viagem&status=ASSIGNED&priority=1
1 resultado: Pedido #1 | Maria Silva | Carla Entregadora | Rota #1

GET /dashboard/history
29 alterações de pedidos | 1 execução do otimizador

POST /routes/generate
2 rotas persistidas; execução registrada no histórico

Exportação CSV pelo navegador
Concluída com o único registro retornado pelos filtros aplicados
```

Resumo das consultas executadas sobre a base persistida durante a validação manual

60 Integração com os módulos anteriores

Clientes e pedidos fornecem os dados pesquisáveis. A atribuição relaciona o pedido ao entregador. A geração de rotas associa as paradas e registra a execução do otimizador. As atualizações realizadas durante a entrega produzem eventos de situação. Dashboard, relatórios, CSV e histórico usam esses mesmos registros, sem manter cópias paralelas.

1. O Administrador ou Atendente cadastra o cliente e o pedido, aplicando as regras de capacidade, disponibilidade e raio já existentes.
2. O pedido atribuído entra na roteirização e recebe o vínculo com uma rota persistida.

3. A execução sequencial, paralela ou em GPU é salva em optimization_runs e passa a aparecer no dashboard e no histórico.
4. O Entregador inicia a rota e atualiza a entrega; cada transição é salva em order_status_history.
5. Os perfis internos pesquisam e filtram os dados, exportam a listagem e auditam a sequência dos eventos.

Segurança e limites

- Administrador e Atendente acessam dashboard, relatórios e histórico; Entregador recebe HTTP 403.
- Todas as consultas são isoladas por estabelecimento.
- O período máximo de consulta é 366 dias.
- O relatório retorna até 2.000 linhas e o histórico até 500 eventos por consulta.

61 Testes e qualidade técnica

A suíte do backend terminou com 93 testes aprovados e um teste ignorado por exigir GPU NVIDIA com CUDA. Os novos testes verificam a pesquisa por endereço, a combinação de entregador, prioridade e situação, a união do histórico de pedidos com execuções do otimizador, os filtros do histórico e a restrição do perfil Entregador.

```
Validação da Sprint 07 — backend
Data: 19/10/2026

Comando:
backend\.venv\Scripts\python.exe -m unittest discover -s tests -v

Resultado:
Ran 93 tests in 80.310s
OK (skipped=1)

O teste ignorado valida o modo GPU e depende de uma GPU NVIDIA com CUDA, indisponível na máquina de validação. Os
testes sequencial e paralelo em CPU foram executados normalmente.

Testes adicionados nesta Sprint:
- pesquisa do relatório por endereço;
- combinação de entregador, prioridade e situação;
- histórico com eventos de pedidos e execuções do otimizador;
- pesquisa e filtros do histórico;
- restrição do histórico ao perfil Entregador.
```

Resultado da suíte automatizada do backend

O frontend foi compilado para produção com TypeScript e Vite. A auditoria das dependências não encontrou vulnerabilidades conhecidas no nível configurado.

```
Validação da Sprint 07 — frontend
Data: 19/10/2026

Build de produção:
npm run build

Resultado:
- TypeScript compilado sem erros;
- 77 módulos transformados;
- dist/index.html: 1,21 kB;
- CSS: 45,46 kB (12,98 kB gzip);
- JavaScript: 486,07 kB (147,98 kB gzip);
- build concluído com Vite 7.3.7.

Auditoria de dependências:
npm audit --audit-level=high

Resultado:
found 0 vulnerabilities
```

Build de produção e auditoria das dependências do frontend

62 Ajustes no planejamento e na arquitetura

O cronograma anterior reservava a Sprint 07 para atribuição automática e prioridade nas rotas. O enunciado oficial priorizou dashboard, relatórios, pesquisas, filtros, exportação e histórico. O planejamento foi ajustado para cumprir essa sequência sem remover as regras existentes; atribuição automática e uso da prioridade como restrição matemática permanecem no backlog.

Ajuste	Justificativa	Impacto
Relatórios ganharam página própria	Os filtros e a tabela ultrapassaram o escopo de um botão no dashboard	Navegação mais clara e consulta detalhada sem sobrecarregar o painel
Novo endpoint /dashboard/history	Era necessário combinar rastreabilidade de pedidos e do otimizador	Contrato único para a linha do tempo, preservando as tabelas existentes
Exportação reutiliza o resultado filtrado	Evitar divergência entre a tela e o arquivo	CSV consistente com todos os critérios aplicados
Novos limites de consulta	Evitar respostas excessivas durante a demonstração e evolução	Período, relatório e histórico possuem limites explícitos

63 Repositório GitHub

Repositório oficial. https://github.com/alvccpj/rota-certa

A implementação foi realizada na branch sprint/07-sistema-quase-completo, criada a partir da master com a Sprint 06 integrada. Código, testes, evidências e documentação foram separados em commits rastreáveis antes da revisão por Pull Request.

```
Branch: sprint/07-sistema-quase-completo
Base: master com a Sprint 06 integrada

b09fd52  feat: implementar consultas e rastreabilidade da sprint 07

O commit reúne backend, frontend, testes automatizados, script de captura e evidências funcionais. O documento
cumulativo é registrado em commit próprio após sua renderização e revisão visual.
```

Registro da implementação da Sprint 07

64 Dificuldades encontradas e próximos passos

Dificuldades

Dificuldade	Tratamento adotado
Combinar filtros de entidades relacionadas sem duplicar pedidos	Consulta SQLAlchemy mantém um registro por pedido e aplica os critérios antes da ordenação e do limite
Unificar eventos de estruturas diferentes	Pedidos e otimizações são convertidos para um contrato comum e ordenados por data
Exportar exatamente o que a interface mostra	O CSV reutiliza as linhas devolvidas pela consulta filtrada
Manter o menu legível com novas áreas	Navegação horizontal adaptativa no desktop e menu recolhível em telas pequenas

Próximos passos

- Adicionar paginação no servidor quando o volume real ultrapassar os limites atuais.
- Ampliar os testes de interface de ponta a ponta para todas as combinações de filtros.
- Avaliar exportação em PDF se o usuário precisar de um formato fechado para impressão.
- Executar medições com maior volume de dados e em máquina equipada com GPU CUDA.
- Realizar revisão final de acessibilidade, segurança e apresentação do produto.

65 Situação da Sprint 07

Entrega obrigatória	Situação	Evidência principal
Dashboard funcional	Concluído	Seção 55: indicadores reais, período, alertas e acompanhamento
Relatórios	Concluídos	Seção 56: consulta consolidada e resumos
Pesquisas	Concluídas	Seção 57: número, cliente, telefone, endereço e entregador
Filtros	Concluídos	Seção 57: período, situação, entregador, prioridade e tipo
Exportação	Concluída	Seção 58: CSV correspondente ao resultado filtrado
Logs ou histórico	Concluídos	Seção 59: pedidos e execuções do otimizador
Integração com módulos anteriores	Concluída	Seção 60: fluxo completo sobre dados persistidos
Repositório atualizado	Concluído na branch	Seção 63: código, testes, evidências e documento

Parte VIII Sprint 08 Consolidação funcional

Validação final: 138 testes de backend aprovados, um ignorado por ausência de GPU CUDA; 15 testes de frontend aprovados. Build de produção concluído e auditoria npm sem vulnerabilidades. As evidências usam uma base SQLite isolada e persistida, com a aplicação real em execução.

Os registros anteriores documentam a evolução do projeto. As regras e decisões desta parte prevalecem onde houver alteração em relação às sprints anteriores.

Projeto RotaCerta, Grupo 04, turma 8NA. Implementação e validação em 01/11/2026.

Repositório: https://github.com/alvccpj/rota-certa

Branch: sprint/08-consolidacao-funcional, baseada em master após o merge da Sprint 07 (PR #10).

Funcionalidades concluídas e rastreabilidade

Requisito original	Implementação na versão consolidada
RF01 Autenticação e perfis	Cadastro do negócio, login, senha Argon2, JWT e autorização consultada no banco a cada requisição.
RF02 Pedidos	Cadastro, consulta, edição e exclusão autorizada; cliente, endereço, posição, peso, prioridade e janela.
RF03 Entregadores	Cadastro, disponibilidade e carga ativa; bloqueio da redução de capacidade abaixo da carga.
RF04 Rotas otimizadas	Vizinho mais próximo e 2-opt; execução sequencial, paralela em CPU e opcional em GPU.
RF05 Atribuição	Manual e automática. A automática prioriza urgência e prazo, respeita capacidade, raio e disponibilidade.
RF06 Lista e mapa	Sequência de paradas, mapa, previsão de chegada, espera, atraso previsto e link para navegação externa.
RF07 Situação	Etapas autorizadas, início de rota, confirmação de entrega e cancelamento com justificativa.
RF08 Histórico	Eventos de pedidos com autor e horário, rotas e execuções do otimizador persistidas.
RF09 Relatórios	Consultas e CSV filtrados; comparação de tempo, distância, speedup e eficiência.
RF10 Painel	Indicadores e alertas calculados com os dados do estabelecimento.

Atribuição automática

POST /orders/auto-assign exige Administrador ou Atendente. Considera somente pedidos pendentes sem entregador do estabelecimento autenticado. Ordena prioridade (1 antes de 2 e 3), prazo final (ausente por último) e ID. Entre entregadores elegíveis, escolhe a menor carga ativa em kg, com desempate por ID. A carga do lote é somada antes da escolha seguinte.

O entregador precisa estar ativo, com perfil correto e disponível, sem rota em andamento ou pedido em rota. Pedidos vencidos, fora do raio ou sem capacidade disponível permanecem pendentes com motivo explícito. O histórico registra a atribuição e seu autor. Repetir a operação não redistribui pedidos já atribuídos. O botão informa que a ação abrange todos os pendentes, inclusive os ocultos pelos filtros, e exige confirmação.

No PostgreSQL a operação bloqueia a linha do estabelecimento e as linhas selecionadas dentro da transação para serializar chamadas automáticas. A demonstração SQLite não comprova concorrência PostgreSQL; os testes desta entrega validam as regras e persistência. Ensaios concorrentes entre atribuições manuais e

automáticas ainda são recomendados antes de uso em produção.

Horários e modelo de dados

O agendamento acrescenta espera quando a chegada calculada antecede o início da janela. Essa espera se propaga para as paradas seguintes e para a duração da rota. Quando a previsão ultrapassa o fim da janela, a interface mostra o atraso em minutos. As previsões são recalculadas ao iniciar a rota.

route_stops.waiting_minutes foi adicionado com valor padrão zero. A atualização é incremental, preserva registros anteriores e pode ser executada novamente. O modelo SQL foi atualizado para instalações novas. desired_start, desired_end, delay_minutes e timing_status da resposta são obtidos dos campos já existentes, sem duplicar os pedidos.

Permissões

Ação	Administrador	Atendente	Entregador
Gerir usuários e ponto de saída	Sim	Não	Não
Clientes e criação/edição de pedidos	Sim	Sim	Não
Excluir pedidos	Sim, com regras da rota	Não	Não
Atribuição automática e geração de rotas	Sim	Sim	Não
Painel, relatórios e histórico gerencial	Sim	Sim	Não
Benchmark de desempenho	Sim	Não	Não
Ver/atualizar entrega	Do estabelecimento	Do estabelecimento	Somente a própria
Dados de outro estabelecimento	Não	Não	Não

As restrições são aplicadas na API e na interface. Sem autenticação a API retorna 401; perfil insuficiente retorna 403; IDs de recursos de outro estabelecimento ou entregador retornam 404. A desativação e a alteração de perfil têm efeito nas próximas requisições, inclusive com token emitido anteriormente.

Regras corrigidas

- Entregador sem cadastro operacional associado não pode visualizar pedidos pendentes sem responsável.

- Não é permitido reduzir a capacidade de carga abaixo dos pedidos ativos.

- Após a saída, cliente, entregador, endereço, coordenadas e peso são protegidos para não mudar uma entrega em execução.

- Pedidos vinculados a rotas iniciadas ou encerradas não são reabertos sobrescrevendo a parada antiga. Para nova entrega, cadastra-se outro pedido; o histórico original é preservado.

- Alterar a janela de um pedido planejado invalida a rota para recalcular os horários.

- Datas recebidas com fusos diferentes são normalizadas para UTC antes de persistir. Representações equivalentes mantêm a mesma previsão e não invalidam o plano.

- Alterar o ponto de saída é bloqueado durante rotas em andamento; em rotas planejadas, exige nova geração.

- A geração e o início verificam acesso ativo do entregador e capacidade. A geração revalida o raio.

- Regenerar rotas preserva os planos dos entregadores que ficaram fora da operação.

- CSV neutraliza texto que poderia ser interpretado como fórmula em planilhas.

Usabilidade

Confirmação de atribuição em lote com foco de teclado, Escape e retorno ao botão; prevenção de envio duplicado; resumo com pedidos atribuídos e motivos dos recusados; instruções da sequência de planejamento; janelas e atrasos legíveis nas paradas; navegação externa por coordenadas; limpeza de filtros nos estados sem resultados. Os recursos continuam adaptados à tela de celular.

Ajustes de escopo

Os RF01 a RF10 estão cobertos dentro do escopo operacional acima. Para tornar explícitos os limites das ideias registradas nas sprints anteriores:

- O mapa representa distâncias geográficas por Haversine. A integração OSRM para traçado e matriz viária foi retirada do escopo desta entrega, mantendo navegação externa opcional. Essa escolha evita depender de um servidor externo durante a demonstração e mantém a mesma entrada na comparação entre modos. O sistema não calcula trânsito nem promete o menor percurso viário.

- Prioridade e prazo ordenam a atribuição; janelas geram espera e alerta de atraso. Não há garantia matemática de todas as janelas viáveis nem otimização combinatória com prioridade como restrição rígida. Isso preserva o objetivo comum de distância entre os modos sequencial, CPU e GPU.

- Notificações por WhatsApp, SMS ou push, sugeridas como evolução no backlog inicial, ficam fora do escopo consolidado. O acompanhamento usa painel, histórico e atualização periódica das listas.

- Exportação CSV cobre a necessidade operacional; XLSX e PDF de relatórios não fazem parte dos requisitos finais. O PDF acadêmico cumulativo é gerado separadamente.

- Atribuição é uma heurística determinística por carga; não garante a melhor distribuição global.

Execução e demonstração

A instalação padrão continua usando PostgreSQL conforme COMO-RODAR.md. Para esta máquina, que não possui PostgreSQL nem Docker, há demonstração local persistida:

cd "C:\Users\v.ribau.fernandes\Documents\Nassau\rotacerta"

.\backend\.venv\Scripts\python.exe tools/run_local_demo.py

Abra http://localhost:5173. Conta administrativa: admin@rotacerta.com.br; senha demonstrativa rotacerta123. O banco isolado fica em backend/rotacerta-demo.db. Ctrl+C encerra os servidores sem apagar os dados. Requer as dependências Python de backend/requirements.txt e npm ci em frontend; não requer pgAdmin. Esse modo não deve ser apresentado como evidência de execução em PostgreSQL.

Evidências e testes

As capturas reais ficam em assets/; os registros reproduzíveis em evidencias/. O script tools/documentation/capture_sprint08_evidence.py exercita fluxos HTTP e interface em uma instância local de demonstração. Os testes adicionais cobrem atribuição, perfis, isolamento de estabelecimentos, regras de carga, integridade de rotas, janelas e migração. Os resultados finais estão em evidencias/testes.txt e evidencias/frontend.txt.

Validação em 01/11/2026: 139 testes de backend executados, com 138 aprovados e um ignorado por ausência de CUDA; 15 testes de frontend aprovados; build de produção concluído; auditoria npm com zero

vulnerabilidades reportadas. Os registros do build e da auditoria estão em evidencias/build.txt e evidencias/auditoria.json. Isso não substitui teste de aceitação da equipe nem valida concorrência em PostgreSQL.

Dificuldades e próximos passos

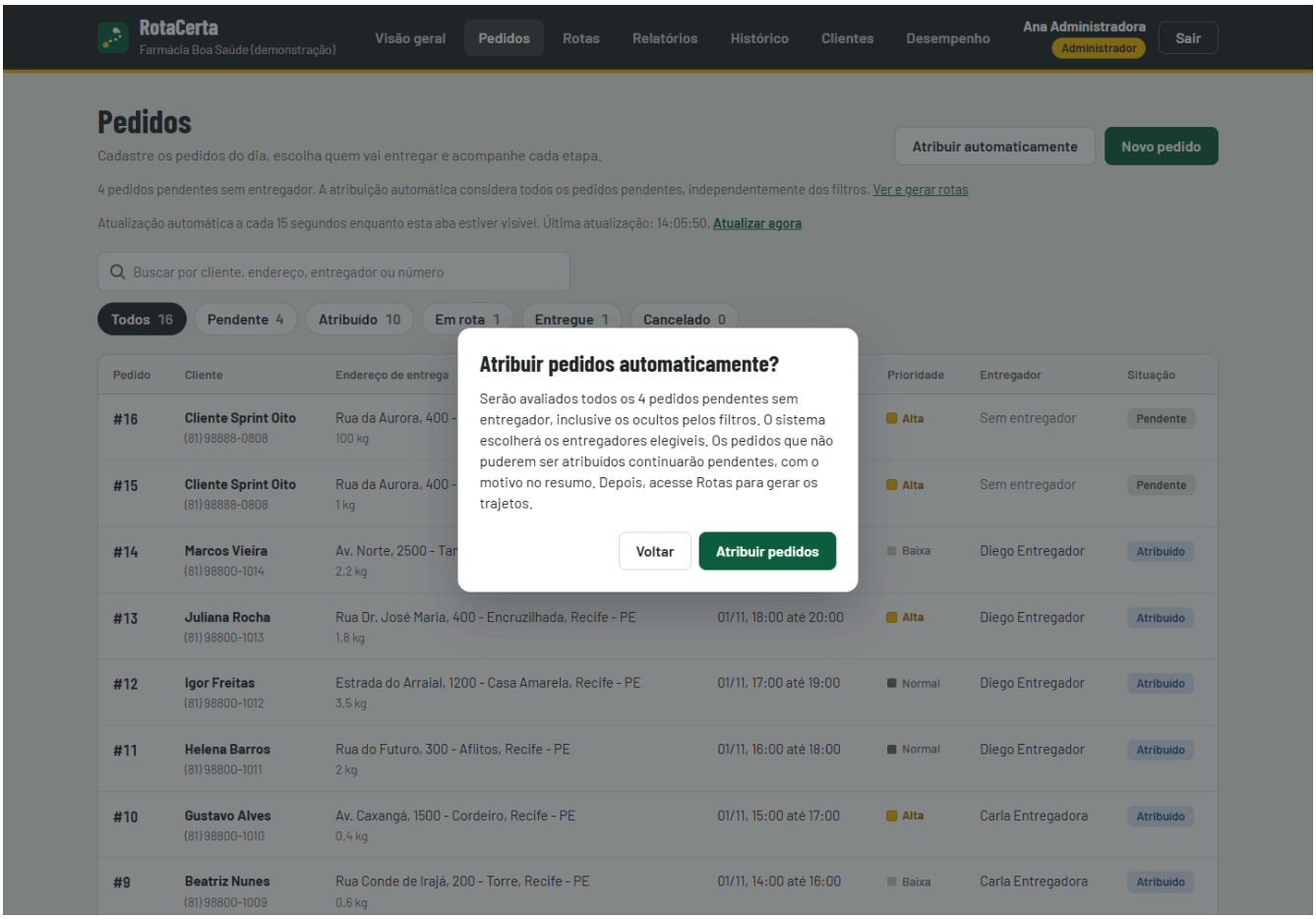
A revisão exigiu conciliar dados atuais do pedido com paradas já registradas e evitar que alterações retroativas mudassem a história da rota. O tratamento foi proteger dados após a saída e invalidar planos afetados antes da execução. Também foi necessário separar tempo de espera operacional do tempo medido pelo algoritmo.

As próximas etapas são testes de aceitação com a equipe, ensaios de concorrência no PostgreSQL, repetição das medições em máquina com CUDA, revisão final de acessibilidade e preparação da apresentação. A GPU não está disponível nesta máquina e seu teste é registrado como ignorado, sem simular resultado.

O Scrum Master deverá revisar a entrega e anexar somente o PDF cumulativo das Sprints 01 a 08 no Teams. Nenhum envio ao Teams é feito pelo projeto ou pelos scripts.

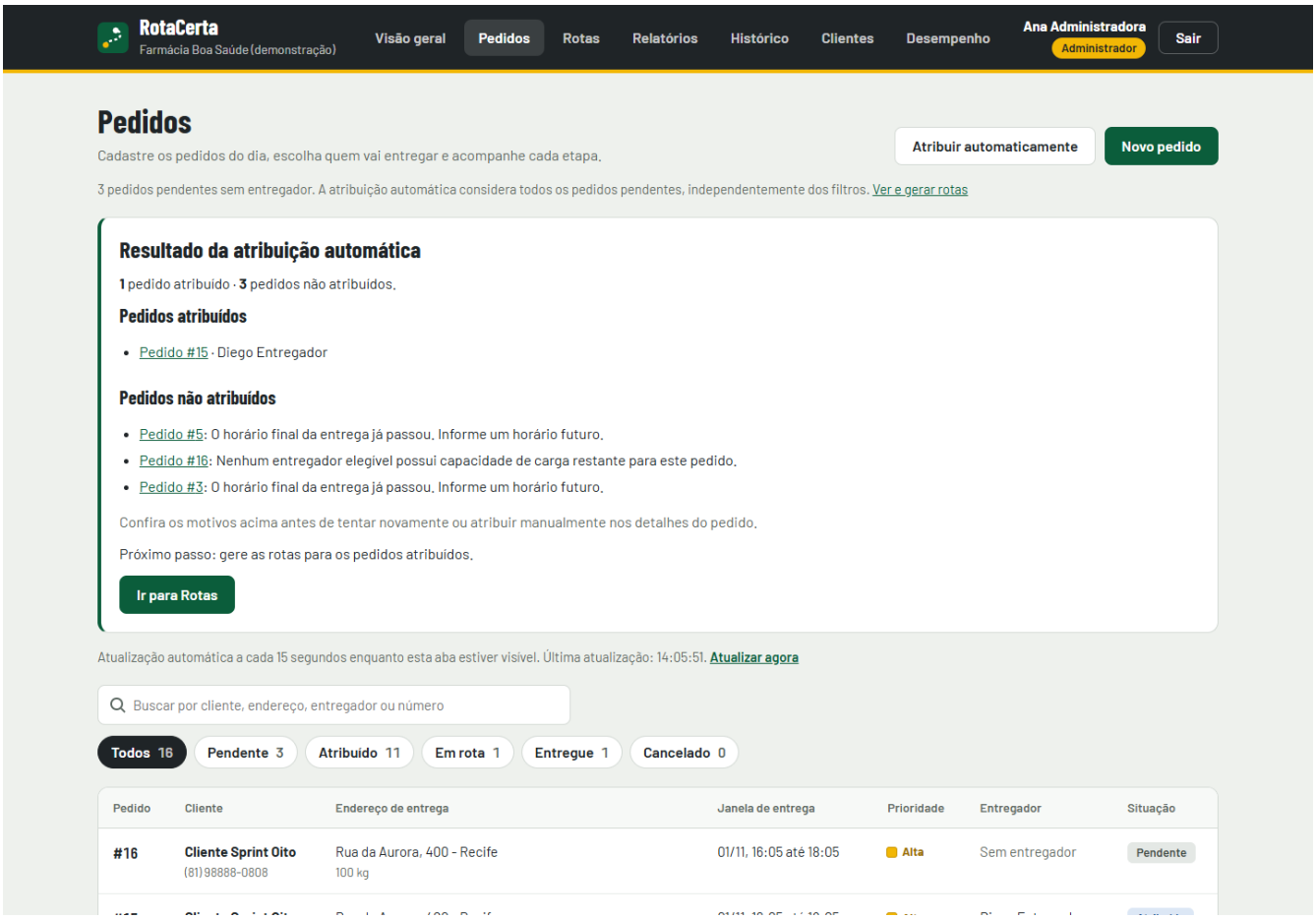
Confirmação da atribuição em lote

A interface informa o alcance da operação antes de atribuir os pedidos.



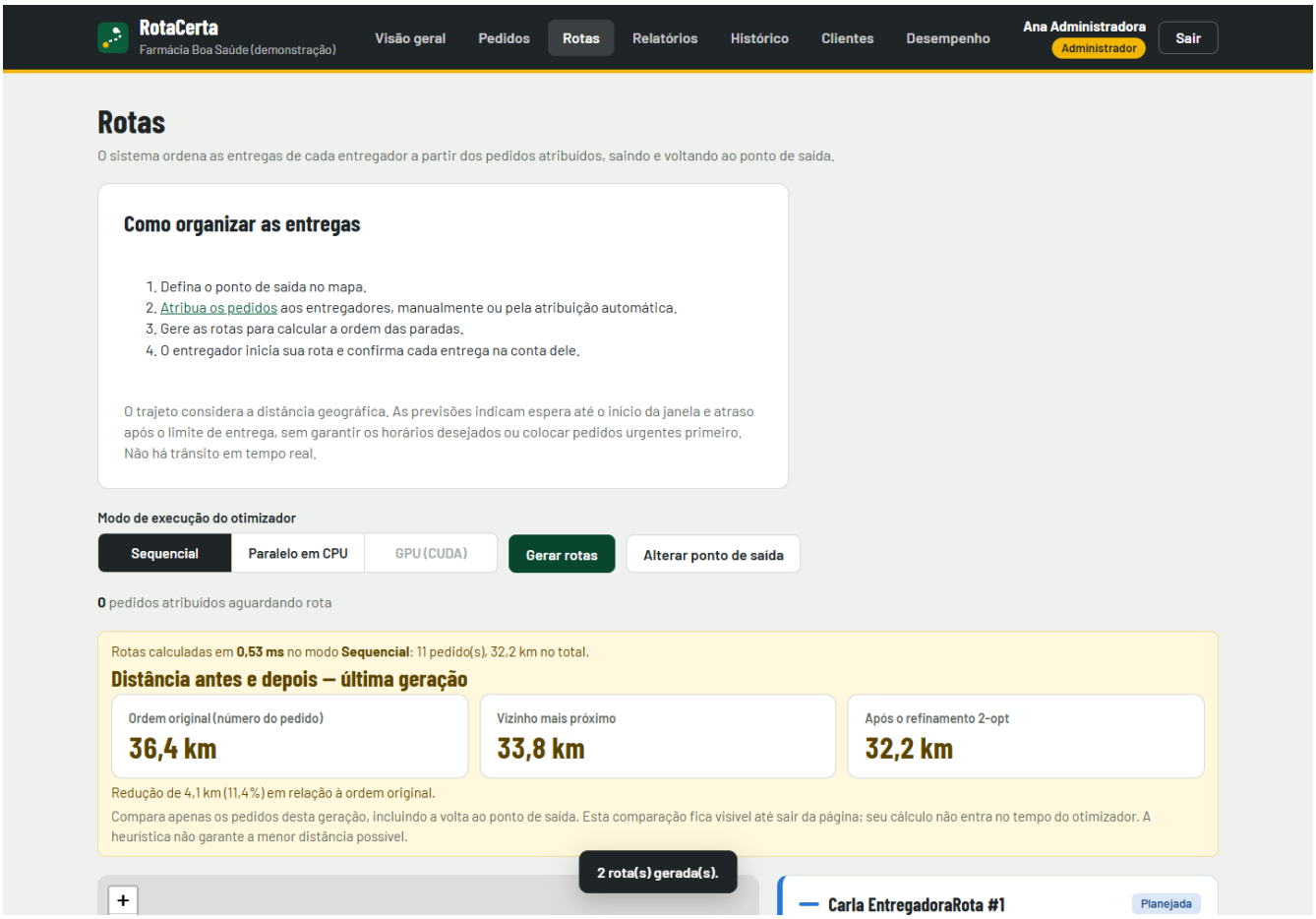
Resultado da atribuição automática

Pedido elegível atribuído e pedido de 100 kg mantido pendente por falta de capacidade.



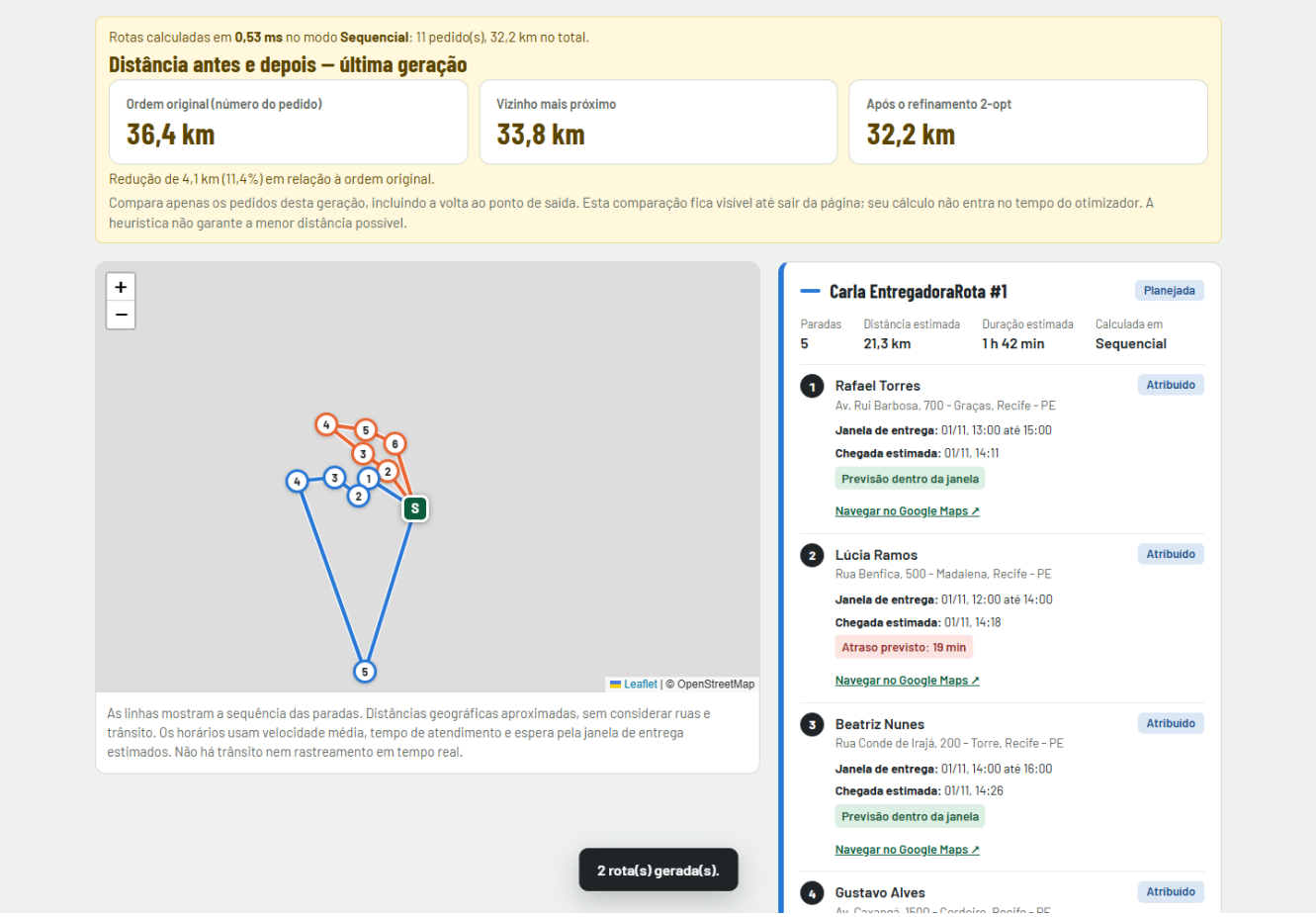
Planejamento de rotas integrado

Rotas geradas a partir dos pedidos atribuídos e persistidas no banco.



Janelas e previsões das entregas

As paradas mostram previsão de chegada, espera e atraso. As distâncias são geográficas.

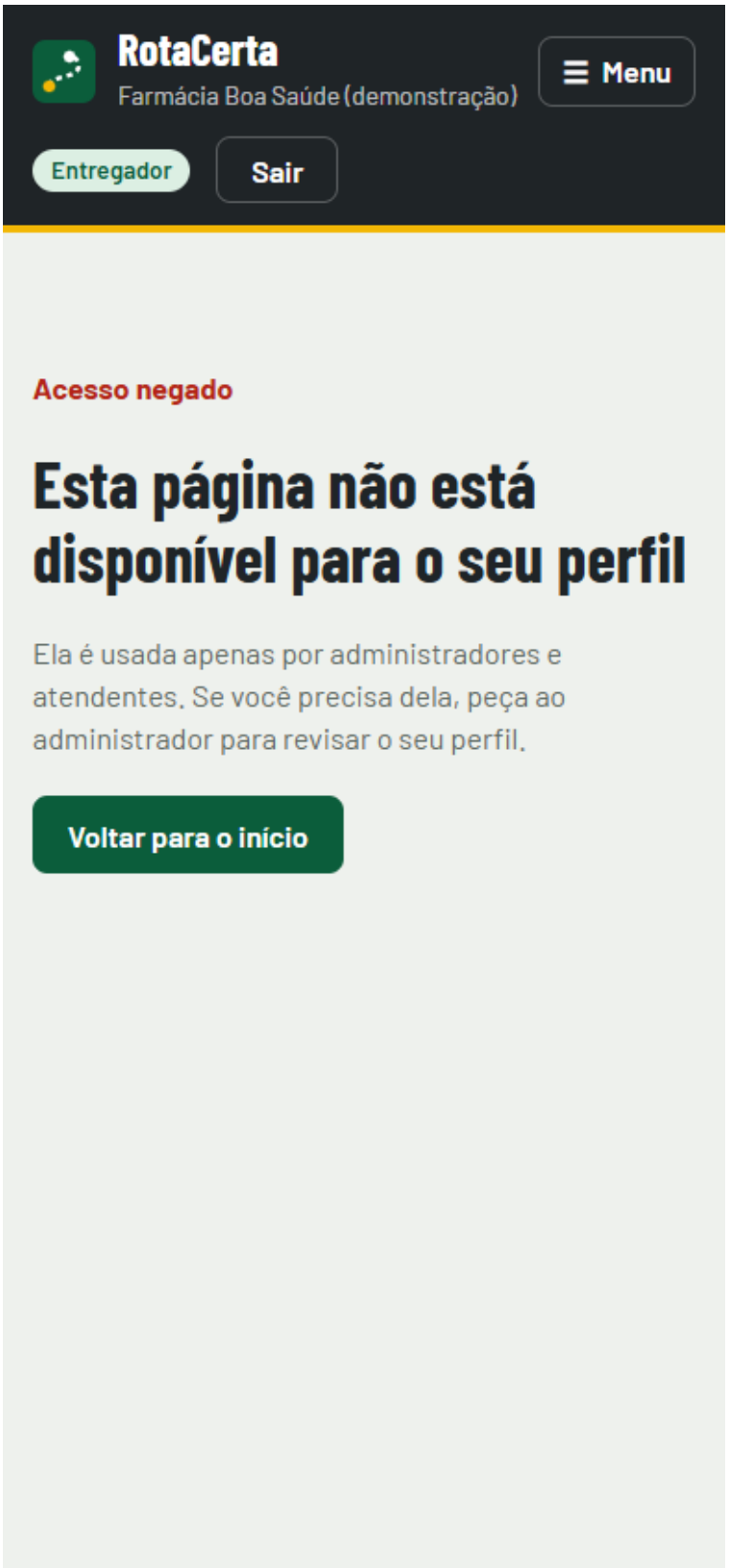


Entregador no celular

Rota iniciada com a conta do entregador. A confirmação de entrega foi verificada pela API.

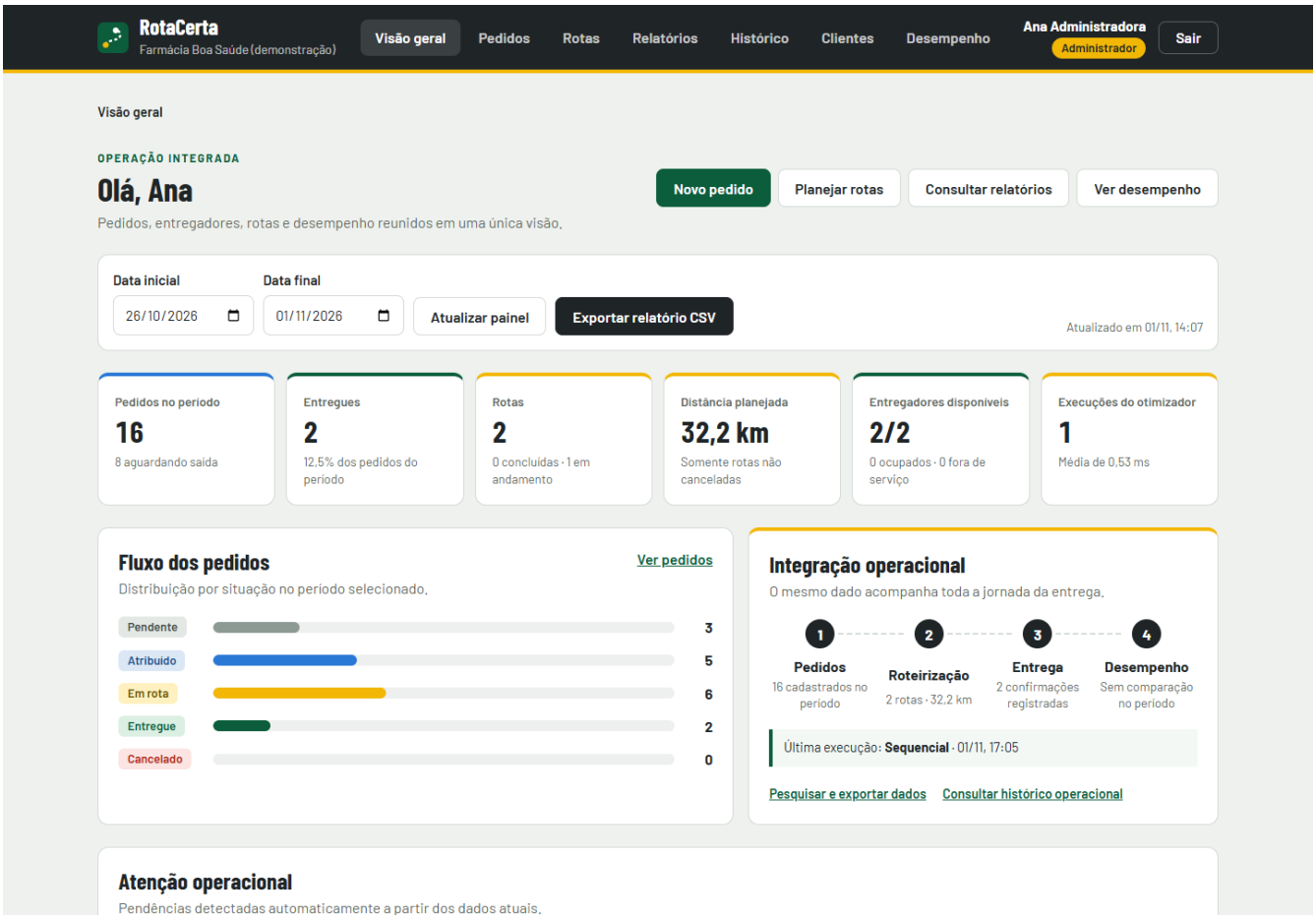
Controle de permissões na interface

O entregador não acessa relatórios gerenciais. A API também bloqueia a consulta com HTTP 403.



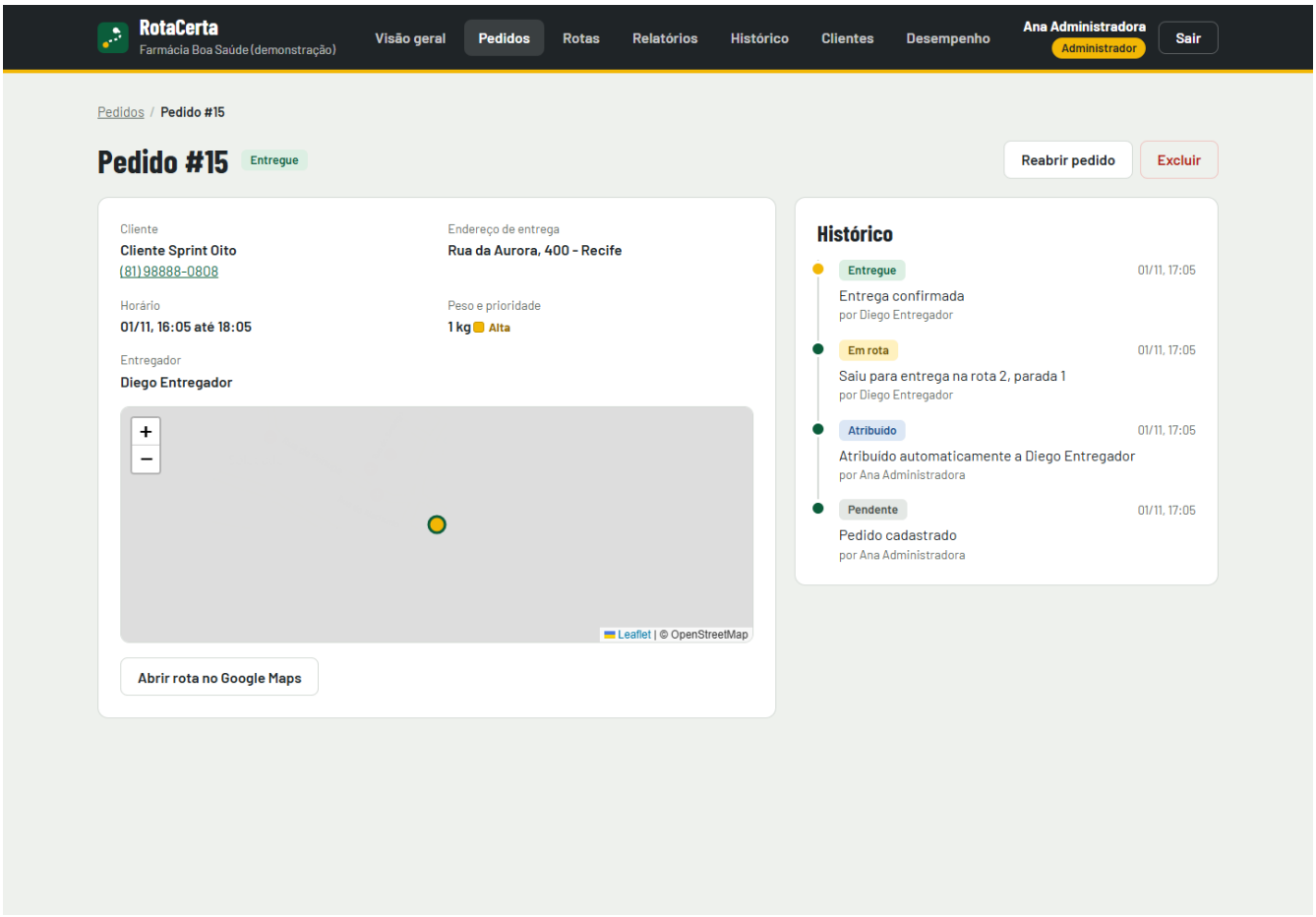
Painel depois da execução

Indicadores consultados após atribuição, roteirização e confirmação de uma entrega.



Histórico da entrega

Consulta do pedido depois de confirmar a entrega. Eventos mantêm autor e horário.



Parte IX - Sprint 09: Testes, correções e documentação

Grupo 04 - RotaCerta | UNINASSAU, Ciência da Computação, turma 8NA, 2026.2
Álvaro Jordão - 01748200
Arthur Sales - 01593811
Vinícius Trigueiro - 01794959
William Coelho de Morais - 01263977
Guilherme de Luna Barros - 01615925
Repositório: https://github.com/alvccpj/rota-certa | branch `sprint/09-testes-validacao`

Objetivo e resultado

A Sprint 09 verificou o sistema completo antes da versão final: testes funcionais, de validação e de navegação executados na aplicação real, correção dos problemas encontrados e atualização da documentação técnica para a versão atual.

Verificação	Casos	Aprovados	Onde está o registro
Testes funcionais (interface e API)	22	22	evidencias/casos_de_teste.json
Testes de validação	20	20	evidencias/casos_de_teste.json
Testes de navegação	19	19	evidencias/casos_de_teste.json
Testes automatizados do backend em SQLite	150	150	evidencias/testes_backend_sqlite.txt
Testes automatizados do backend em PostgreSQL 18.1	150	150	evidencias/testes_backend_postgresql.txt
Testes automatizados do frontend	50	50	evidencias/testes_frontend.txt
Build de produção e auditoria npm	2	2	evidencias/build.txt e evidencias/auditoria_npm.json

Foram encontrados e corrigidos 9 bugs (BUG-01 a BUG-09). Cada correção tem teste de regressão que falha no código anterior e passa no código atual. Não há bug conhecido em aberto; as pendências de escopo estão listadas no fim deste documento.

Como os testes foram realizados

Os 61 casos de teste são executados pelo script `tools/documentation/capture_sprint09_evidence.py`. Ele abre o sistema no Google Chrome (Playwright), age como o usuário (preenche campos, clica em botões e menus, usa a tela de celular) e confere o resultado na tela e, quando faz sentido, também na API. Cada caso registra o procedimento, o resultado esperado, o resultado obtido e a situação. Quando um caso falha, o motivo é registrado e os demais continuam. Os casos que dependem só do servidor (por exemplo, valores que a interface já bloqueia) foram executados direto na API, como faria um cliente mal-intencionado ou com defeito.

Ambiente da execução oficial:

Item	Valor
Data	07/11/2026, 11h34 (horário de Recife)

Item	Valor
Banco	PostgreSQL 18.1, schema isolado `sprint09_testes`, recriado antes da execução com os dados de demonstração
API e sistema	FastAPI em 127.0.0.1:8019; frontend (Vite) em 127.0.0.1:5189
Navegador	Google Chrome via Playwright, telas de 1366 px (computador) e 390 px (celular)
Máquina	Ryzen 7 5700X, 16 threads; GPU NVIDIA GeForce RTX 3060 (CUDA)

O schema `public`, usado no desenvolvimento, não foi alterado. Antes da execução oficial o roteiro foi rodado algumas vezes durante a investigação; a execução registrada é a última, numa base limpa, com 61 de 61 casos aprovados.

Como reproduzir

Na raiz do repositório, com o PostgreSQL ligado e as dependências instaladas (`COMO-RODAR.md`):

```
.\backend\.venv\Scripts\python.exe tools/run_test_environment.py
```

O comando recria o schema `sprint09_testes`, sobe a API (porta 8019) e o sistema (porta 5189). Em outro terminal:

```
.\backend\.venv\Scripts\python.exe tools/documentation/capture_sprint09_evidence.py
```

O resultado é gravado em `docs/sprint-09/evidencias/casos_de_teste.json` e as capturas em `docs/sprint-09/assets/`. As suítes automatizadas rodam com os comandos da seção "Testes automatizados" de `COMO-RODAR.md`.

Testes funcionais

Cobrem os módulos e operações do sistema: autenticação (login, saída e cadastro de negócio), clientes (cadastro, pesquisa, edição e exclusão), pedidos (cadastro, consulta com filtro, edição, exclusão e cancelamento), usuários (cadastro e desativação), atribuição automática, geração de rotas nos modos sequencial e GPU, execução da entrega pelo entregador no celular, painel, relatório com exportação CSV, histórico operacional e comparação de desempenho. A tabela completa, com procedimento, resultado esperado e resultado obtido de cada caso, está em `evidencias/casos_de_teste.json` e no PDF cumulativo.

Casos de teste (22 de 22 aprovados na execução de 2026-11-07 11:34):

ID	Caso	Procedimento	Resultado esperado	Resultado obtido	Situação
F01	**Login com credenciais válidas** Autenticação	1. Abrir /login 2. Informar admin@rotacerta.com.br e a senha de demonstração 3. Clicar em Entrar	Usuário autenticado e levado ao painel com saudação pelo nome.	Redirecionado para /painel; saudação 'Olá, Ana'; /auth/me confirma admin@rotacerta.com.br (ADMIN).	**Aprovado**
F02	**Encerrar sessão** Autenticação	1. Com a sessão do F01, clicar em Sair 2. Tentar abrir /painel pelo endereço	Sessão removida; /painel volta a exigir login.	Após Sair: /login; sessão local apagada (None); /painel redirecionou ao login.	**Aprovado**

ID	Caso	Procedimento	Resultado esperado	Resultado obtido	Situação
F03	**Cadastro de novo negócio** Autenticação	1. Abrir /cadastro 2. Preencher negócio, endereço de saída, nome, e-mail e senha válidos 3. Clicar em Criar conta	Conta de administrador criada para um estabelecimento isolado, já autenticada no painel.	Conta criada e autenticada em /painel; topo mostra 'Padaria Sprint Nove'; o novo negócio começa com 0 pedidos.	**Aprovado**
F04	**Cadastro de cliente** Clientes	1. Abrir Clientes > Novo cliente 2. Informar nome e telefone sem formatação 3. Clicar em Cadastrar cliente	Cliente salvo e listado com telefone no formato (81) 99999-0909.	Cliente #15 listado com telefone (81) 99999-0909.	**Aprovado**
F05	**Pesquisa de clientes** Clientes	1. Em Clientes, digitar 'Helena' na busca	Somente clientes cujo nome ou telefone contém o termo são exibidos.	2 linha(s) exibida(s), todas contendo 'Helena'.	**Aprovado**
F06	**Edição de cliente** Clientes	1. Na linha do cliente, clicar em Editar 2. Trocar o telefone 3. Clicar em Salvar alterações	Alteração persistida no banco e refletida na lista.	API retorna telefone (81) 3333-0909 para o cliente #15.	**Aprovado**
F07	**Cadastro de pedido** Pedidos	1. Abrir Pedidos > Novo pedido 2. Escolher o cliente, endereço, latitude/longitude, peso '3,5', prioridade Alta e janela futura 3. Clicar em Cadastrar pedido	Pedido salvo como Pendente com os dados informados; tela do pedido aberta.	Pedido #15: situação PENDING, peso 3.5 kg, prioridade 1.	**Aprovado**
F08	**Consulta de pedidos com filtro** Pedidos	1. Em Pedidos, clicar no filtro Pendente	A lista mostra apenas pedidos pendentes e o filtro fica no endereço da página.	3 linhas exibidas, todas Pendente; API confirma 3 pendentes; endereço /pedidos?situacao=PENDING.	**Aprovado**
F09	**Edição de pedido** Pedidos	1. Abrir o pedido do F07 > Editar pedido 2. Alterar o peso para 4 3. Clicar em Salvar alterações	Novo peso persistido e exibido no pedido.	API retorna peso 4.0 kg para o pedido #15.	**Aprovado**
F10	**Exclusão de pedido com confirmação** Pedidos	1. Criar pedido pendente 2. Abrir o pedido e clicar em Excluir 3. Confirmar em Excluir pedido	Diálogo de confirmação exibido; após confirmar, o pedido deixa de existir (HTTP 404).	Pedido #16 excluído; consulta posterior retorna HTTP 404.	**Aprovado**
F11	**Cadastro de usuário atendente** Usuários	1. Abrir Usuários > Novo usuário 2. Informar nome, e-mail, senha e perfil Atendente 3. Clicar em Cadastrar usuário	Usuário criado e capaz de entrar com o perfil Atendente.	Login do novo usuário: HTTP 200, perfil ATTENDANT.	**Aprovado**
F12	**Desativação de usuário** Usuários	1. Na lista de usuários, clicar em Desativar na linha do F11 2. Confirmar	Usuário marcado como inativo e impedido de entrar.	Linha marcada como inativa; novo login retorna HTTP 403: Usuário desativado. Procure o administrador.	**Aprovado**

ID	Caso	Procedimento	Resultado esperado	Resultado obtido	Situação
F13	**Exclusão de cliente sem pedidos** Clientes	1. Criar cliente sem pedidos 2. Clicar em Excluir na linha 3. Confirmar em Excluir cliente	Cliente removido (HTTP 404 na consulta seguinte).	Cliente #16 removido da lista; consulta retorna HTTP 404.	**Aprovado**
F14	**Atribuição automática** Atribuição	1. Em Pedidos, clicar em Atribuir automaticamente 2. Confirmar em Atribuir pedidos	Pedidos pendentes elegíveis recebem entregador; resultado e motivos exibidos.	Pedido #15 atribuído a Diego Entregador; histórico: 'Atribuído automaticamente a Diego Entregador'.	**Aprovado**
F15	**Geração de rotas no modo sequencial** Rotas	1. Abrir Rotas 2. Escolher Sequencial e clicar em Gerar rotas	Uma rota planejada por entregador, com paradas ordenadas, distância e previsão.	2 rotas SEQUENTIAL. Carla Entregadora: 5 paradas, 21.307 km, Diego Entregador: 8 paradas, 15.934 km.	**Aprovado**
F16	**Geração de rotas na GPU (CUDA)** Rotas	1. Em Rotas, escolher GPU (CUDA) e clicar em Gerar rotas	Rotas geradas pelo kernel CUDA com a mesma distância total do modo sequencial.	2 rotas GPU em NVIDIA GeForce RTX 3060; distâncias iguais ao sequencial: {'Carla Entregadora': 21.307, 'Diego Entregador': 15.934}.	**Aprovado**
F17	**Entregador inicia a rota e confirma a entrega** Entregas	1. Entrar como o entregador do pedido no celular 2. Clicar em Iniciar rota 3. Abrir o pedido e clicar em Confirmar entrega	Pedido passa a Entregue com autor e horário no histórico.	Pedido #15 DELIVERED; último evento por Diego Entregador em 2026-11-07T11:34:06.467112-03:00.	**Aprovado**
F18	**Cancelamento com justificativa** Pedidos	1. Criar pedido pendente 2. Abrir o pedido > Cancelar pedido 3. Informar o motivo e Confirmar cancelamento	Pedido Cancelado e motivo gravado no histórico.	Pedido #17 CANCELLED; motivo no histórico: 'Cliente desistiu da compra'.	**Aprovado**
F19	**Painel com indicadores atualizados** Painel	1. Abrir Visão geral depois dos fluxos anteriores	Indicadores calculados com os dados atuais do estabelecimento.	Pedidos: 16 no período, 2 entregue(s), 1 cancelado(s), conclusão 12.5%; rotas: 1 planejada(s), 1 em andamento.	**Aprovado**
F20	**Relatório filtrado e exportação CSV** Relatórios	1. Abrir Relatórios 2. Buscar 'Helena' e Aplicar filtros 3. Clicar em Exportar CSV	Somente pedidos da cliente; o CSV baixado contém as mesmas linhas.	3 linha(s) na tela e 3 no CSV 'rota-certa-relatorio-2026-11-01-a-2026-11-07.csv', todas da cliente Helena.	**Aprovado**
F21	**Histórico operacional** Histórico	1. Abrir Histórico 2. Buscar o número do pedido do F07	Eventos do pedido listados com autor e horário.	5 eventos do pedido #15; responsáveis: ['Responsável: Ana Administradora', 'Responsável: Diego Entregador'].	**Aprovado**
F22	**Comparação de desempenho sequencial, CPU e GPU** Desempenho	1. Chamar POST /optimizer/benchmark com 8 entregadores x 60 paradas, 1, 2 e 4 processos e GPU	Tempos, speedup e eficiência calculados; todos os modos produzem as mesmas rotas.	480 pedidos sintéticos; SEQUENTIAL 1p: 6.4 ms, speedup 1.0, eficiência 1.0; PARALLEL 1p: 7.2 ms, speedup 0.888, eficiência 0.8882; PARALLEL 2p: 4.7 ms, speedup 1.377, eficiência 0.6887; PARALLEL 4p: 3.6 ms, speedup 1.769, eficiência 0.4424; GPU 2048p: 6.9 ms, speedup 0.923, eficiência None	**Aprovado**

Testes de validação

Verificam se o sistema recusa dados inválidos e explica o motivo: campos obrigatórios vazios, formatos inválidos (e-mail, telefone, senha, nome), limites (peso de 0 a 500 kg, raio de 30 km, capacidade abaixo da carga ativa), regras de negócio (janela invertida ou no passado, cancelamento sem motivo, cliente e e-mail duplicados) e entradas que chegam direto à API (endereço só com espaços, fusos misturados, identificadores gigantes, caracteres de controle e token adulterado). Na interface, cada erro aparece junto ao campo, com resumo no topo do formulário e foco no primeiro campo inválido.

Casos de teste (20 de 20 aprovados na execução de 2026-11-07 11:34):

ID	Caso	Procedimento	Resultado esperado	Resultado obtido	Situação
V01	**Login com campos obrigatórios vazios** Autenticação	1. Abrir /login 2. Clicar em Entrar sem preencher	Mensagens junto aos campos, sem chamada ao servidor.	Mensagens 'Informe o e-mail.' e 'Informe a senha.'; foco no campo email.	**Aprovado**
V02	**Login com senha incorreta** Autenticação	1. Informar e-mail válido e senha errada 2. Clicar em Entrar	Mensagem genérica, sem revelar qual dado está errado.	Exibido 'E-mail ou senha inválidos.'; permaneceu em /login.	**Aprovado**
V03	**Cadastro de negócio com formatos inválidos** Autenticação	1. Em /cadastro, informar e-mail 'paula@', senha 'abc' e endereço curto 2. Clicar em Criar conta	Cada campo inválido mostra a orientação de correção.	Informe o endereço de onde saem as entregas, com rua e número. | Use pelo menos duas letras. | Informe um e-mail válido, como nome@empresa.com.br. | Use pelo menos 8 caracteres.	**Aprovado**
V04	**Cliente com nome e telefone inválidos** Clientes	1. Em Novo cliente, informar nome '123' e telefone '123' 2. Clicar em Cadastrar cliente	Mensagens para nome sem letras e telefone sem DDD.	Exibidos 'Use pelo menos duas letras.' e 'Informe o telefone com DDD, por exemplo (81) 98800-1001.'	**Aprovado**
V05	**Cliente duplicado** Clientes	1. Enviar POST /customers com o mesmo nome e telefone do cliente do F06	HTTP 409 com mensagem de duplicidade.	HTTP 409: Já existe um cliente com este nome e telefone.	**Aprovado**
V06	**Pedido com campos obrigatórios vazios** Pedidos	1. Em Novo pedido, informar só espaços no endereço 2. Clicar em Cadastrar pedido	Resumo de erros, mensagem por campo e foco no primeiro campo inválido.	Escolha o cliente do pedido. | Informe o endereço com rua, número e bairro. | Marque o local de entrega no mapa ou use Buscar endereço. | Informe o peso.	**Aprovado**
V07	**Peso fora dos limites** Pedidos	1. Em Novo pedido, informar peso 0 e depois 600 2. Clicar em Cadastrar pedido a cada valor	Mensagens 'maior que 0' e 'máximo 500'.	0 kg: 'Informe um valor maior que 0.'; 600 kg: 'O valor máximo é 500.'; API com 501 kg: HTTP 422	**Aprovado**
V08	**Janela de entrega invertida** Pedidos	1. Em Novo pedido, informar fim antes do início 2. Clicar em Cadastrar pedido	Mensagem 'O fim precisa ser depois do início.'	Exibido 'O fim precisa ser depois do início.' junto ao campo Entregar até.	**Aprovado**
V09	**Janela de entrega no passado (API)** Pedidos	1. POST /orders com desired_end uma hora atrás	HTTP 422 com orientação para informar horário futuro.	HTTP 422: O horário final da entrega já passou. Informe um horário futuro.	**Aprovado**

ID	Caso	Procedimento	Resultado esperado	Resultado obtido	Situação
V10	**Entrega fora do raio de atendimento** Pedidos	1. POST /orders com coordenadas de São Paulo	HTTP 422 informando a distância e o limite de 30 km.	HTTP 422: O local de entrega fica a 2132 km do ponto de saída. O limite de entrega é 30 km.	**Aprovado**
V11	**Cancelamento sem motivo suficiente** Pedidos	1. Abrir pedido > Cancelar pedido 2. Informar 'abc' 3. Clicar em Confirmar cancelamento	Mensagem pedindo ao menos 5 caracteres; pedido continua ativo.	Mensagem exibida; pedido #18 continua PENDING.	**Aprovado**
V12	**E-mail já cadastrado (sem diferenciar maiúsculas)** Usuários	1. POST /users com ADMIN@ROTACERTA.COM.BR	HTTP 409 informando e-mail em uso.	HTTP 409: Já existe um usuário com este e-mail.	**Aprovado**
V13	**Entregador sem capacidade e senha fraca** Usuários	1. POST /users perfil Entregador sem capacidade 2. POST /users com senha 'abcdefgh'	HTTP 422 com as duas regras explicadas.	Sem capacidade: : Value error, Informe a capacidade de carga do entregador. | Senha fraca: password: Value error, A senha precisa ter letras e números.	**Aprovado**
V14	**Endereço só com espaços (BUG-01 corrigido)** Pedidos	1. POST /orders com delivery_address de 8 espaços	HTTP 422; nada é gravado.	HTTP 422: delivery_address: Value error, Informe o endereço com rua, número e bairro.; total de pedidos 17 -> 17.	**Aprovado**
V15	**Janela com e sem fuso horário (BUG-02 corrigido)** Pedidos	1. POST /orders com início sem fuso e fim com -03:00	Pedido aceito e datas normalizadas em UTC, sem erro 500.	HTTP 201; início 2026-11-07T17:34:16Z, fim 2026-11-07T19:34:16Z.	**Aprovado**
V16	**Identificador acima do limite do banco (BUG-03 corrigido)** API	1. GET /orders/99999999999999999999 2. PUT /customers/9223372036854775808	HTTP 422 com mensagem, sem erro 500.	HTTP 422 e 422: Algum valor informado está fora do formato aceito. Confira os dados e tente de novo.	**Aprovado**
V17	**Caractere de controle no nome (BUG-04 corrigido)** Clientes	1. POST /customers com nome contendo o byte nulo (\u0000)	Cliente salvo sem o caractere inválido, sem erro 500.	HTTP 201; nome salvo: 'Joana Controle 113353'.	**Aprovado**
V18	**Período de relatório invertido** Relatórios	1. GET /dashboard/report com data final anterior à inicial	HTTP 422 com mensagem sobre o período.	HTTP 422: A data final deve ser igual ou posterior à inicial.	**Aprovado**
V19	**Capacidade abaixo da carga ativa** Usuários	1. PUT /users do entregador Diego com capacidade 0,1 kg	HTTP 422 explicando a carga ativa.	HTTP 409: A capacidade não pode ser menor que os 17,3 kg em entregas ativas.	**Aprovado**

ID	Caso	Procedimento	Resultado esperado	Resultado obtido	Situação
V20	**Requisição sem token e com token adulterado** Autenticação	1. GET /orders sem Authorization 2. GET /orders com token alterado	HTTP 401 nos dois casos.	Sem token: HTTP 401 (Sessão inválida ou expirada. Faça login novamente.); adulterado: HTTP 401.	**Aprovado**

Testes de navegação

Verificam redirecionamentos (página protegida sem login, volta à página pedida, usuário logado abrindo login), menus de cada perfil (cada item abre a tela certa e fica destacado), permissões (acesso negado na interface e 403 na API), isolamento entre estabelecimentos e entre entregadores, páginas inexistentes, identificadores inválidos no endereço, menu no celular, continuidade entre lista, detalhe e edição, sessão expirada e botão Voltar do navegador depois de sair.

Casos de teste (19 de 19 aprovados na execução de 2026-11-07 11:34):

ID	Caso	Procedimento	Resultado esperado	Resultado obtido	Situação
N01	**Página protegida sem login** Acesso	1. Sem sessão, abrir /relatorios	Redirecionamento para /login guardando a página pedida.	Redirecionado para /login; destino guardado: /relatorios.	**Aprovado**
N02	**Volta à página pedida depois do login (BUG-06 corrigido)** Acesso	1. Na tela do N01, entrar como administrador	Usuário levado a /relatorios, não ao painel.	Após o login: /relatorios.	**Aprovado**
N03	**Página pedida não permitida ao perfil que entrou** Acesso	1. Sem sessão, abrir /relatorios 2. Entrar como entregador	Entregador levado a Minhas entregas, sem tela de acesso negado.	Após o login: /entregas.	**Aprovado**
N04	**Menu e navegação do perfil Administrador** Menu	1. Entrar como Administrador 2. Clicar em cada item do menu	Menu com 8 itens (Visão geral, Pedidos, Rotas, Relatórios, Histórico, Clientes, Desempenho, Usuários); cada um abre a tela correspondente e fica destacado.	Visão geral->/painel; Pedidos->/pedidos; Rotas->/rotas; Relatórios->/relatorios; Histórico->/historico; Clientes->/clientes; Desempenho->/desempenho; Usuários->/usuarios	**Aprovado**
N05	**Menu e navegação do perfil Atendente** Menu	1. Entrar como Atendente 2. Clicar em cada item do menu	Menu com 6 itens (Visão geral, Pedidos, Rotas, Relatórios, Histórico, Clientes); cada um abre a tela correspondente e fica destacado.	Visão geral->/painel; Pedidos->/pedidos; Rotas->/rotas; Relatórios->/relatorios; Histórico->/historico; Clientes->/clientes	**Aprovado**
N06	**Menu e navegação do perfil Entregador** Menu	1. Entrar como Entregador 2. Clicar em cada item do menu	Menu com 1 itens (Minhas entregas); cada um abre a tela correspondente e fica destacado.	Minhas entregas->/entregas	**Aprovado**

ID	Caso	Procedimento	Resultado esperado	Resultado obtido	Situação
N07	**Atendente abre tela exclusiva do administrador** Permissões	1. Entrar como atendente 2. Abrir /usuarios pelo endereço 3. Clicar em Voltar para o início	Tela de acesso negado explicando quem usa a página; botão leva ao painel; API responde 403.	Acesso negado exibido; botão levou a /painel; GET /users como atendente: HTTP 403.	**Aprovado**
N08	**Entregador abre cadastro de pedido** Permissões	1. Como entregador, abrir /pedidos/novo	Acesso negado; API bloqueia criação com 403.	Tela de acesso negado; POST /orders como entregador: HTTP 403.	**Aprovado**
N09	**Entregador abre pedido de outro entregador** Permissões	1. Como entregador, abrir o endereço de um pedido atribuído a outro entregador	Mensagem de pedido não encontrado; API responde 404 sem expor dados.	Pedido #10 de Carla Entregadora: tela 'não foi encontrado'; API HTTP 404.	**Aprovado**
N10	**Endereço inexistente** Rotas da aplicação	1. Logado, abrir /pagina-que-nao-existe 2. Clicar em Ir para o início	Página não encontrada dentro do layout; botão leva ao início.	Página não encontrada exibida; botão levou a /painel.	**Aprovado**
N11	**Identificador inválido no endereço (BUG-05 corrigido)** Rotas da aplicação	1. Abrir /pedidos/abc, /pedidos/abc/editar, /clientes/abc/editar e /usuarios/abc/editar	Mensagem de registro não encontrado, e não 'Corrija o campo destacado'.	/pedidos/abc: 'O pedido #abc não foi encontrado'; /pedidos/abc/editar: 'O pedido #abc não foi encontrado.'; /clientes/abc/editar: 'Cliente não encontrado.'; /usuarios/abc/editar: 'Usuário não encontrado.'	**Aprovado**
N12	**Usuário logado abre login ou cadastro** Acesso	1. Logado como administrador, abrir /login e /cadastro	Redirecionado ao painel.	/login->/painel; /cadastro->/painel	**Aprovado**
N13	**Menu no celular** Menu	1. Em 390 px de largura, clicar em Menu 2. Escolher Clientes	Menu abre, navega e fecha sozinho após a escolha.	Fechado no início; aberto após Menu; navegou para /clientes e fechou; rolagem horizontal 0px.	**Aprovado**
N14	**Continuidade entre lista, detalhe e edição** Fluxos	1. Abrir um pedido pela lista 2. Ir para Editar pedido e clicar em Cancelar 3. Clicar no item Pedidos da trilha	Cada ação leva à tela esperada sem perder o contexto.	/pedidos/20 -> /pedidos/20/editar -> /pedidos/20 -> /pedidos	**Aprovado**
N15	**Sessão expirada no meio do uso** Acesso	1. Logado em Clientes, invalidar o token salvo no navegador 2. Recarregar a página 3. Entrar novamente	Aviso de sessão expirada no login e retorno à mesma página após entrar.	Aviso exibido no login; após entrar, voltou para /clientes?busca=Helena.	**Aprovado**
N16	**Isolamento entre estabelecimentos** Permissões	1. Como administrador da Padaria (F03), abrir um pedido da RotaCerta pelo endereço	Pedido de outro negócio não é exibido; API responde 404.	Tela 'não foi encontrado'; API HTTP 404; clientes visíveis para a Padaria: 0.	**Aprovado**
N17	**Cadastrar cliente a partir do pedido e voltar** Fluxos	1. Em Novo pedido, clicar em Cadastrar cliente 2. Salvar o cliente	Retorno automático ao Novo pedido com o fluxo preservado.	/clientes/novo?voltar=%2Fpedidos%2Fnovo -> /pedidos/novo?cliente=18	**Aprovado**
N18	**Voltar do navegador depois de sair** Acesso	1. Logado no painel, clicar em Sair 2. Usar o botão Voltar do navegador	A área protegida não é reaberta.	Após Voltar: /login.	**Aprovado**

ID	Caso	Procedimento	Resultado esperado	Resultado obtido	Situação
N19	**Menu completo visível no desktop (BUG-09 corrigido)** Menu	1. Entrar como administrador em telas de 1024, 1280, 1366 e 1920 px 2. Medir a posição de cada item do menu	Os 8 itens ficam inteiros na tela, sem rolagem escondida.	1024px: 8 itens visíveis; 1280px: 8 itens visíveis; 1366px: 8 itens visíveis; 1920px: 8 itens visíveis	**Aprovado**

Registro de bugs

ID	Severidade	Problema	Como foi encontrado	Causa	Correção	Teste de regressão	Situação
BUG-01	Média	Endereço do pedido, nome do negócio e ponto de saída formados só por espaços eram aceitos e gravados vazios.	Validação direta na API (V14).	O tamanho mínimo era conferido antes de remover os espaços.	Validadores `address_text` e `business_name` conferem o texto já normalizado (`backend/app/schemas.py`). A interface passou a usar a mesma regra.	`test_validation_s09.BlankTextTestCase` (5 testes) e V14.	Corrigido
BUG-02	Alta	Janela de entrega com uma data sem fuso e outra com fuso derrubava a API com erro 500.	Validação direta na API (V15).	Comparação entre data com e sem fuso no validador da janela.	As datas são normalizadas para UTC antes da comparação (`WindowTime` em `schemas.py`).	`WindowTimezoneTestCase` (2 testes) e V15.	Corrigido
BUG-03	Média	Identificador acima de 64 bits no endereço da API (`/orders/99999999999999999999`) gerava erro 500.	Validação direta na API, confirmada no PostgreSQL (V16).	O banco rejeita o número e o erro não era tratado.	Tratador global converte erros de valor do banco (`DataError` , `OverflowError`) em HTTP 422 com mensagem em português (`backend/app/main.py`).	`test_huge_ids_return_validation_error` e V16.	Corrigido
BUG-04	Média	Texto com o caractere nulo (`\u0000`) gerava erro 500 no PostgreSQL.	Validação direta na API no PostgreSQL (V17).	O PostgreSQL não aceita esse caractere em campos de texto.	A normalização de texto remove caracteres de controle; o tratador do BUG-03 cobre os demais campos.	`test_control_characters_are_removed_from_names` , `test_cancellation_reason_made_of_control_characters_is_rejected` e V17.	Corrigido

ID	Severidade	Problema	Como foi encontrado	Causa	Correção	Teste de regressão	Situação
BUG-05	Baixa	Endereços como `/pedidos/abc` e `/clientes/abc/editar` mostravam "Corrija o campo destacado." numa tela sem formulário.	Teste de navegação (N11).	O erro de validação do parâmetro do endereço era tratado como erro de formulário.	`frontend/src/api.ts` trata erro de parâmetro do endereço como registro não encontrado (404); a edição de pedido mostra a mensagem de não encontrado.	`sprint09.test.mjs` e N11.	Corrigido
BUG-06	Média	Depois do login, o usuário nunca voltava à página que tinha pedido; sempre ia para o início.	Teste de navegação (N02).	Ao criar a sessão, o redirecionamento de "já está logado" da tela de login acontecia antes da navegação para a página pedida.	`frontend/src/access.ts` decide o destino conforme o perfil e é usado pelo login e pelo redirecionamento; página não permitida ao perfil leva ao início dele.	`sprint09.test.mjs` (18 testes) e N01 a N03.	Corrigido
BUG-07	Média	No modo de demonstração SQLite (Sprint 08), as janelas dos pedidos de exemplo ficavam 3 horas antes do previsto. Um teste da Sprint 08 passava só por causa disso e falhava no PostgreSQL.	Execução da suíte de backend no PostgreSQL, feita pela primeira vez nesta Sprint.	Os dados de exemplo eram gravados no fuso de Recife; o SQLite descarta o fuso e a aplicação lê a data como UTC.	Os dados de exemplo são gravados em UTC (`backend/app/seed.py`); o teste da Sprint 08 usa um entregador próprio.	`SeedWindowTestCase` e `test_generated_wait_survives_a_new_database_session` , nos dois bancos.	Corrigido
BUG-08	Baixa	Com a sessão expirada, o sistema voltava ao login sem o aviso "Sua sessão expirou".	Teste de navegação (N15).	O aviso ia no estado da navegação e era substituído pelo redirecionamento das páginas protegidas.	O aviso fica no contexto da sessão; a página atual é guardada para o usuário voltar a ela (`frontend/src/session.tsx`).	N15, conferido também contra o código anterior.	Corrigido
BUG-09	Média	No computador, o menu do administrador cortava o item Usuários em todas as larguras, sem barra de rolagem visível.	Revisão das capturas dos testes de navegação; medição em 1280 a 1920 px.	Os 8 itens precisavam de 815 px e dividiam a linha com a marca e a conta, restando 689 px.	O menu passou para uma linha própria abaixo da marca e da conta, com quebra se necessário (`frontend/src/styles.css`). No celular continua o botão Menu.	N19 (1024, 1280, 1366 e 1920 px) e N13.	Corrigido

Além dos bugs do sistema, a primeira execução do roteiro teve dois casos reprovados por falhas do próprio roteiro: leitura errada do formato de `/optimizer/capabilities` (F16) e uma corrida no N15, que adulterava o token antes de o sistema terminar de confirmar a sessão e regravá-lo. O roteiro foi corrigido e a execução oficial foi refeita do zero.

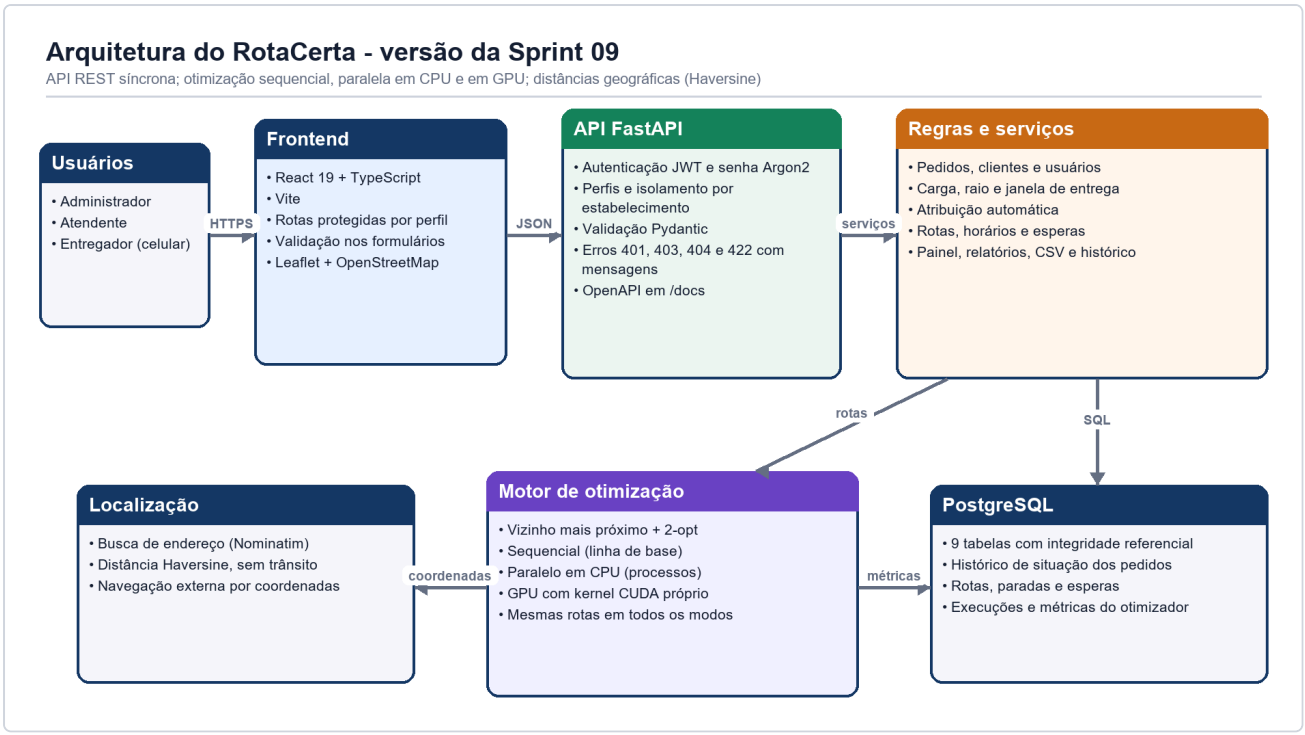
Testes automatizados

A suíte do backend passou a rodar também no PostgreSQL real com a variável `TEST_DATABASE_URL`. Cada teste recria um schema descartável aplicando `database/schema.sql`, o mesmo DDL usado na instalação. Essa execução encontrou o BUG-07, que o SQLite escondia, e fecha a pendência da Sprint 08 de validar o comportamento em PostgreSQL.

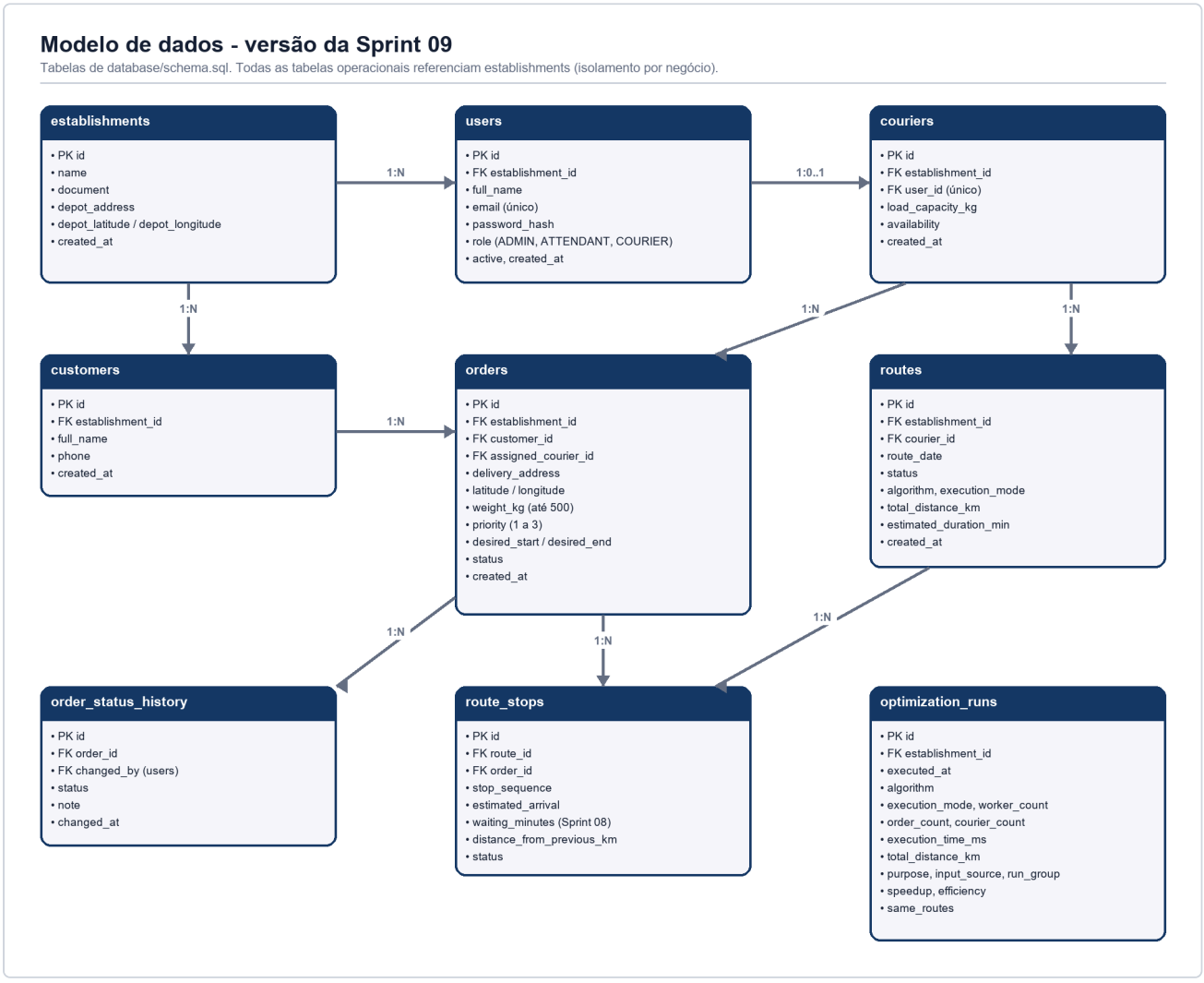
Suíte	Resultado	Observação
Backend em SQLite em memória	150 de 150 aprovados	Inclui 11 testes novos da Sprint 09; o teste de GPU executou na RTX 3060.
Backend em PostgreSQL 18.1	150 de 150 aprovados	Mesma suíte, no schema `rotacerta_testes_automatizados`.
Frontend (`node --test`)	50 de 50 aprovados	15 da Sprint 08 e 35 novos: destino após login, tratamento de erros da API e validações dos formulários.
Build de produção	Concluído	`tsc -b` e `vite build` sem erros.
Auditoria npm	0 vulnerabilidades	`npm audit` em 07/11/2026.

Atualização da documentação

Documento	Atualização
Arquitetura (`assets/arquitetura_v09.png`)	Substitui a da Sprint 02: retira o adaptador OSRM, fora do escopo desde a Sprint 08; inclui GPU com CUDA, geocodificação por Nominatim, distância Haversine, perfis, isolamento por estabelecimento e os serviços de regras, atribuição e relatórios.
Modelo de dados (`assets/mer_v09.png`)	Gerado a partir do schema atual: inclui `order_status_history` (Sprint 04), as métricas de `optimization_runs` (Sprint 05), `route_stops.waiting_minutes` (Sprint 08) e as restrições de unicidade.
Requisitos	Situação de cada requisito RF01 a RF10 na versão atual, na tabela abaixo.
`README.md`	Módulo da Sprint 09, link para o PDF 01 a 09 e para os testes.
`COMO-RODAR.md`	Testes no PostgreSQL, testes de frontend, ambiente de testes isolado e execução dos casos de teste.
`database/README.md`	Alterações das Sprints 08 e 09 e link para o modelo de dados atualizado.
Diagrama de classes da Sprint 02	Mantido como registro do planejamento. A estrutura implementada é descrita pela arquitetura e pelo modelo de dados acima e pelos módulos de `backend/app`.



Arquitetura atualizada na Sprint 09.



Modelo de dados atualizado na Sprint 09, conforme database/schema.sql.

Requisitos na versão atual

Requisito	Situação	Verificado por
RF01 Autenticação e perfis	Implementado	F01 a F03, F11, F12, V01 a V03, V20, N01 a N08, N12, N15, N18
RF02 Pedidos	Implementado	F07 a F10, F18, V06 a V11, V14, V15
RF03 Entregadores	Implementado	F11, F12, V13, V19
RF04 Rotas otimizadas	Implementado (sequencial, CPU e GPU)	F15, F16, F22
RF05 Atribuição	Implementado (manual e automática)	F14
RF06 Lista e mapa	Implementado	F15, F17, N06
RF07 Situação da entrega	Implementado	F17, F18, V11
RF08 Histórico	Implementado	F17, F18, F21
RF09 Relatórios	Implementado (CSV)	F20, V18
RF10 Painel	Implementado	F19

Os limites de escopo registrados na Sprint 08 continuam válidos: distâncias geográficas sem trânsito, janelas como previsão e não como garantia, sem notificações externas e exportação somente em CSV.

Pendências e ações previstas

- Concorrência no PostgreSQL: as regras agora são testadas no PostgreSQL real, mas ainda não houve ensaio com várias atribuições automáticas simultâneas. Ação: teste com requisições paralelas a `/orders/auto-assign` na Sprint 10.
- Teste de aceitação com usuários: os casos foram executados pela equipe com o roteiro automatizado. Ação: sessão guiada com um usuário que não participou do desenvolvimento, usando o roteiro desta Sprint, antes da apresentação final.
- Desempenho da GPU: com 480 pedidos (F22), a GPU foi mais lenta que a CPU paralela, porque o custo de enviar os dados à GPU supera o ganho nesse tamanho. Não é defeito; a comparação com instâncias maiores está na tela Desempenho e será apresentada na entrega final.
- Testes de interface no repositório: o roteiro Playwright exige o ambiente de testes ligado e ainda não roda em integração contínua. Ação: avaliar um fluxo de GitHub Actions com PostgreSQL de serviço.

Dificuldades encontradas

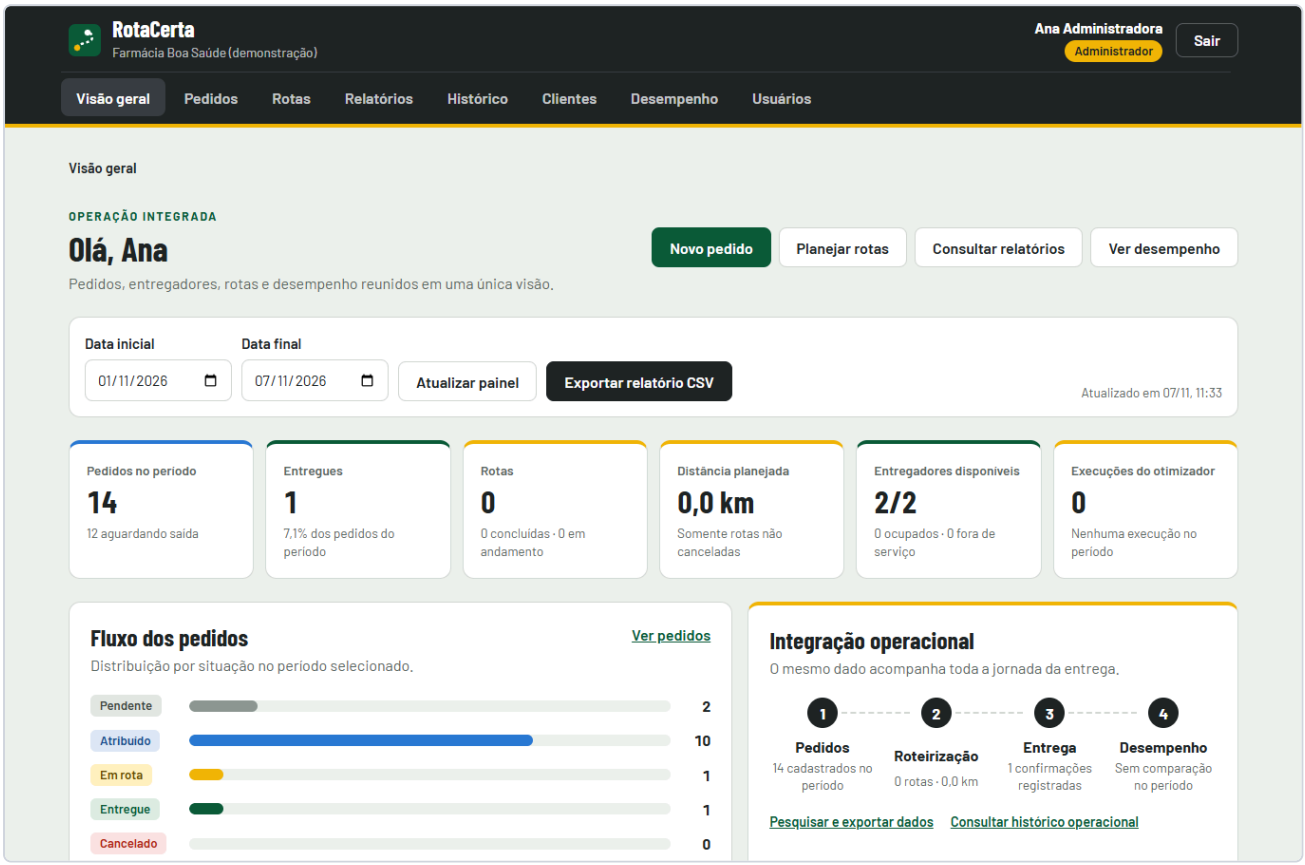
- Bugs que só aparecem num dos bancos: o BUG-07 ficava escondido no SQLite e só apareceu quando a suíte passou a rodar no PostgreSQL. A solução foi manter as duas execuções.
- Diferenciar falha do sistema de falha do teste: dois casos reprovaram na primeira execução por erro do roteiro. Cada reprovação foi investigada antes de ser registrada; o BUG-08 foi confirmado executando o mesmo cenário no código anterior.
- Evitar alterar o banco de desenvolvimento: os testes de interface criam e excluem registros. Por isso foram feitos num schema isolado, recriado a cada execução.

Próximos passos

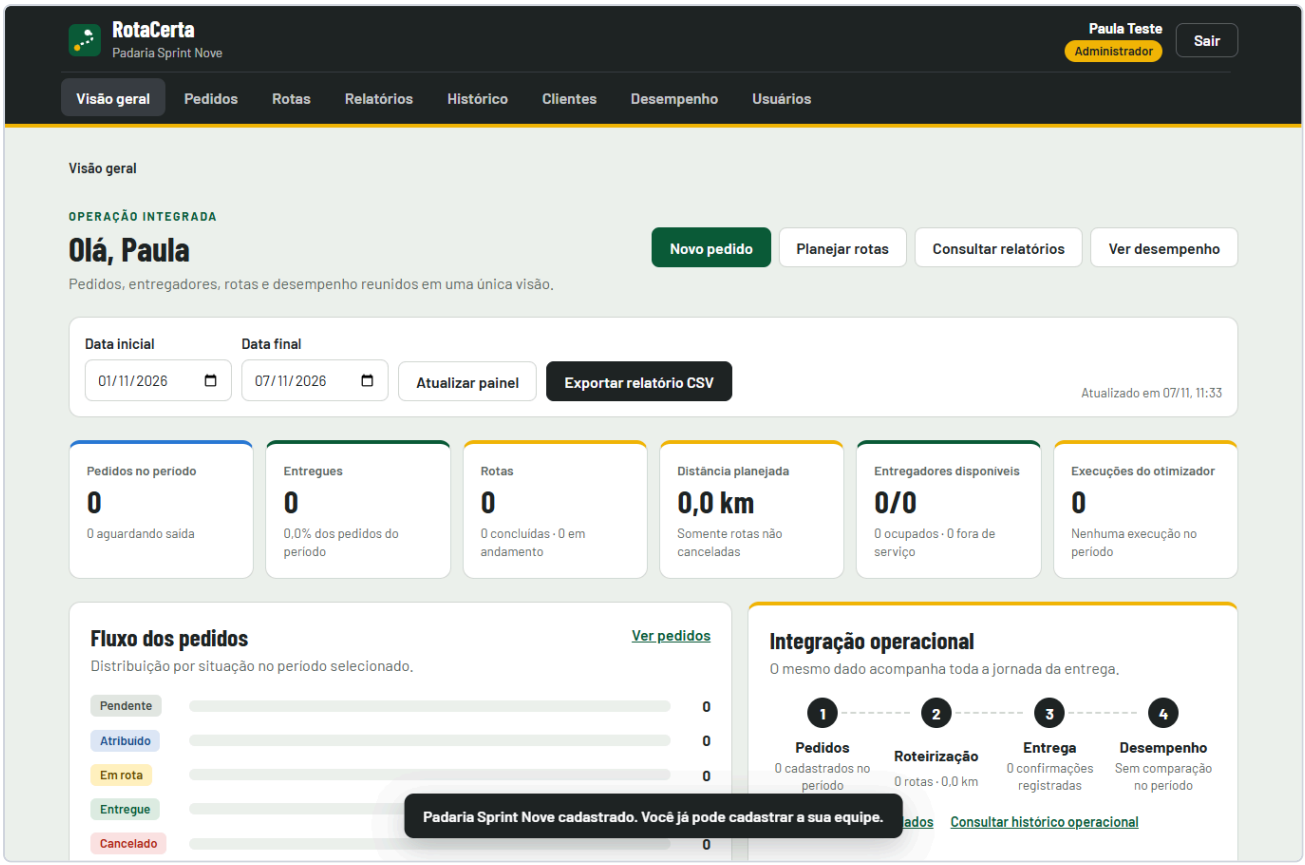
Sprint 10: ensaio de concorrência, teste de aceitação com usuário externo, revisão final da apresentação e congelamento da versão final. O Scrum Master deve revisar este documento e anexar no Teams somente o PDF cumulativo das Sprints 01 a 09 (`output/pdf/Grupo_04_RotaCerta_Sprints_01_a_09.pdf`).

Evidências da execução dos testes

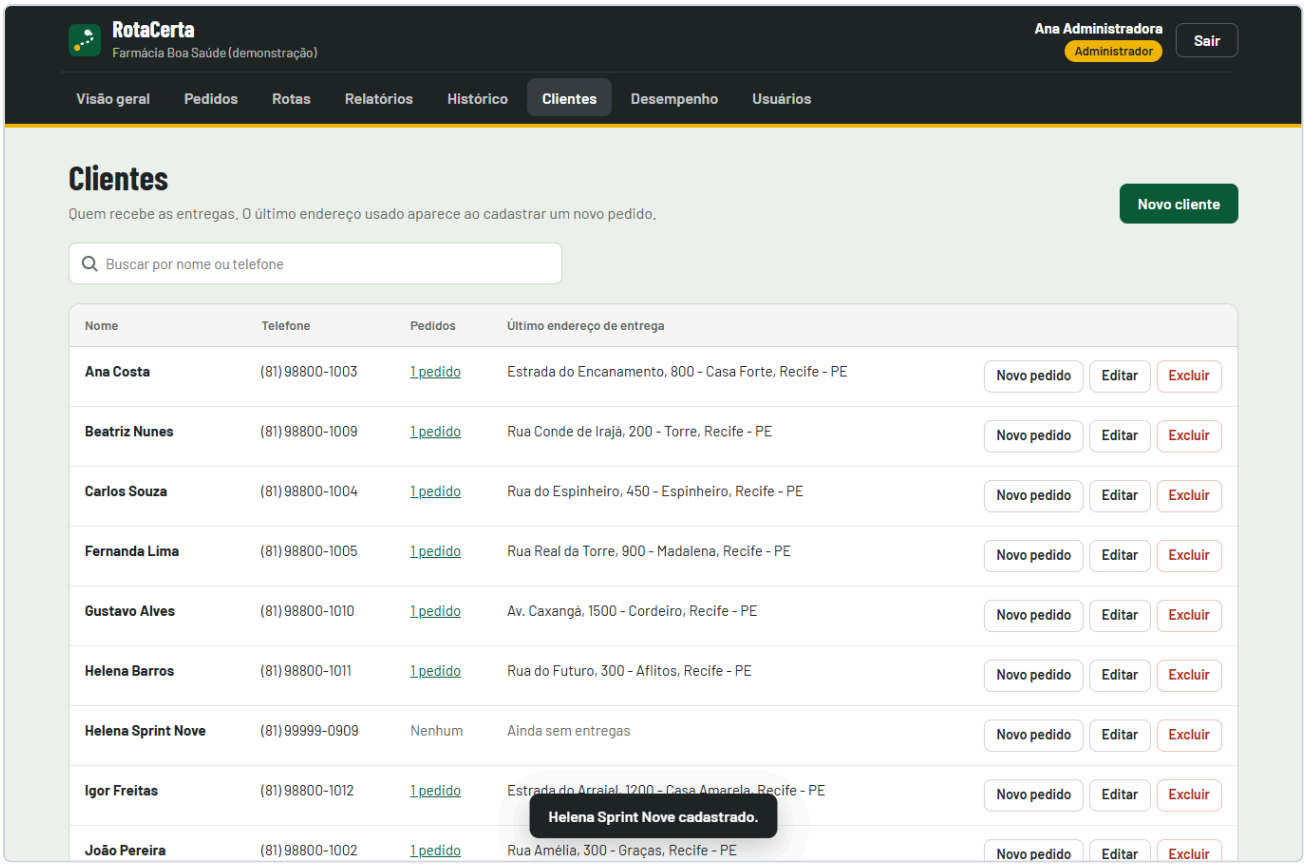
Capturas de tela salvas pelo roteiro durante a execução oficial, na aplicação real ligada ao PostgreSQL. Cada legenda informa o caso, o procedimento e o resultado obtido. As capturas de computador usam 1366 px de largura e as de celular, 390 px.



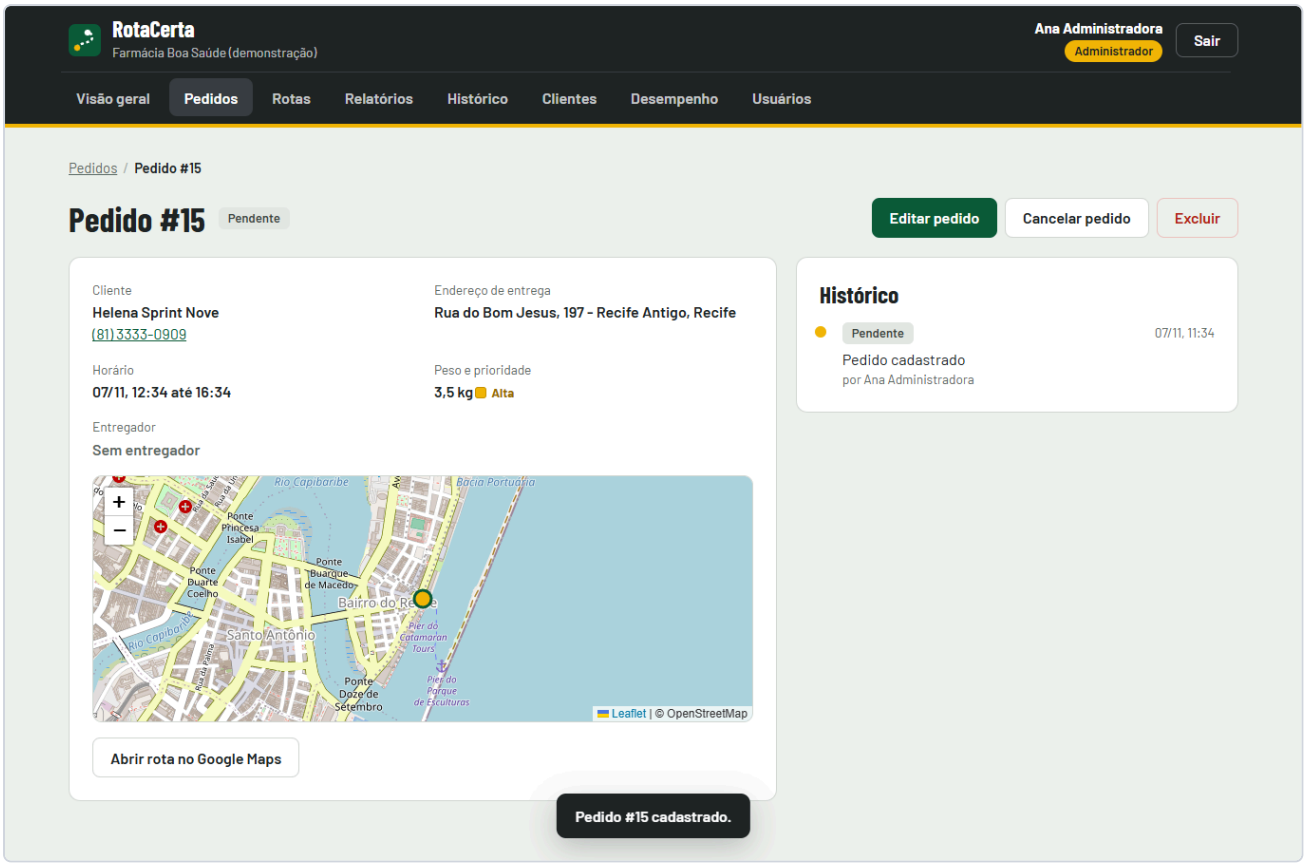
F01 - Login com credenciais válidas. Procedimento: Abrir /login; Informar admin@rotacerta.com.br e a senha de demonstração; Clicar em Entrar. Obtido: Redirecionado para /painel; saudação 'Olá, Ana'; /auth/me confirma admin@rotacerta.com.br (ADMIN).



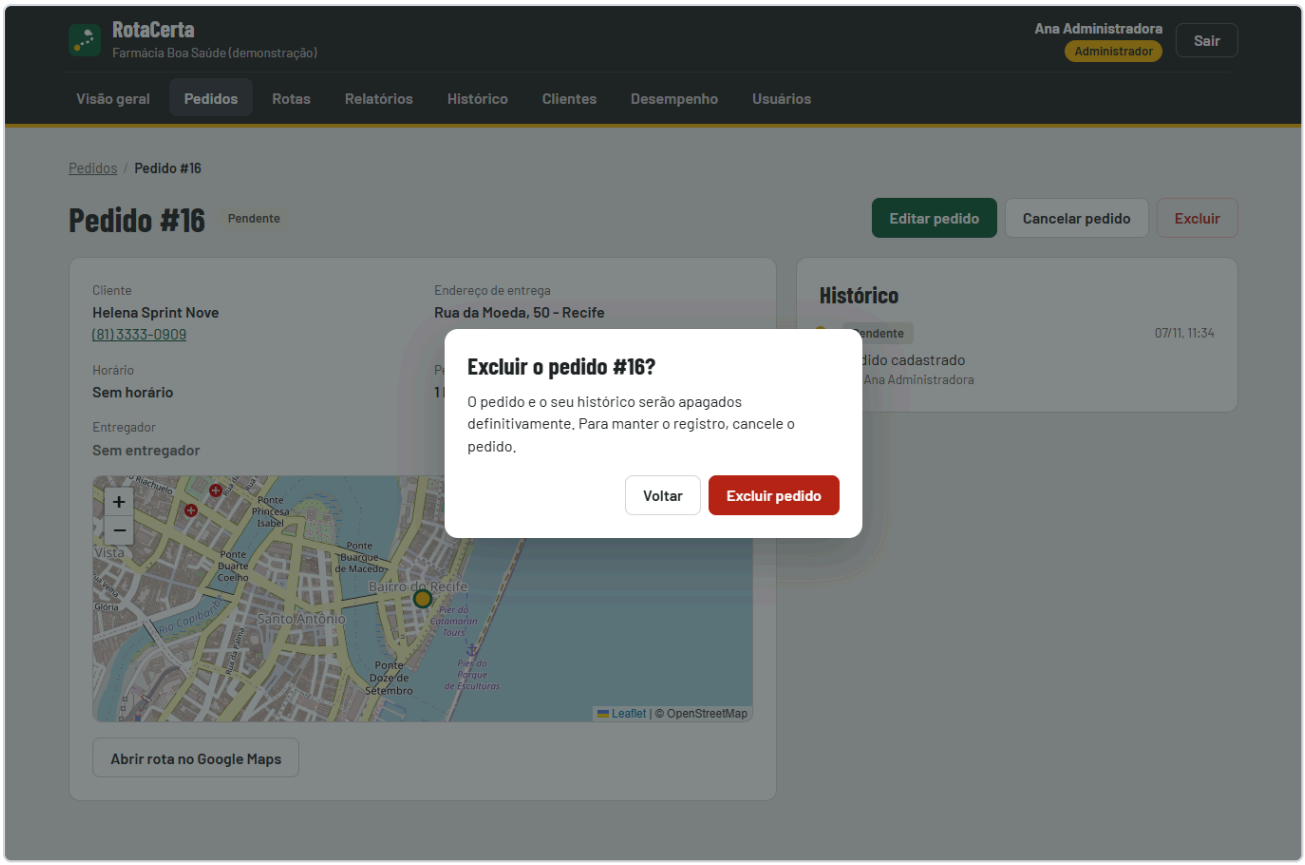
F03 - Cadastro de novo negócio. Procedimento: Abrir /cadastro; Preencher negócio, endereço de saída, nome, e-mail e senha válidos; Clicar em Criar conta. Obtido: Conta criada e autenticada em /painel; topo mostra 'Padaria Sprint Nove'; o novo negócio começa com 0 pedidos.



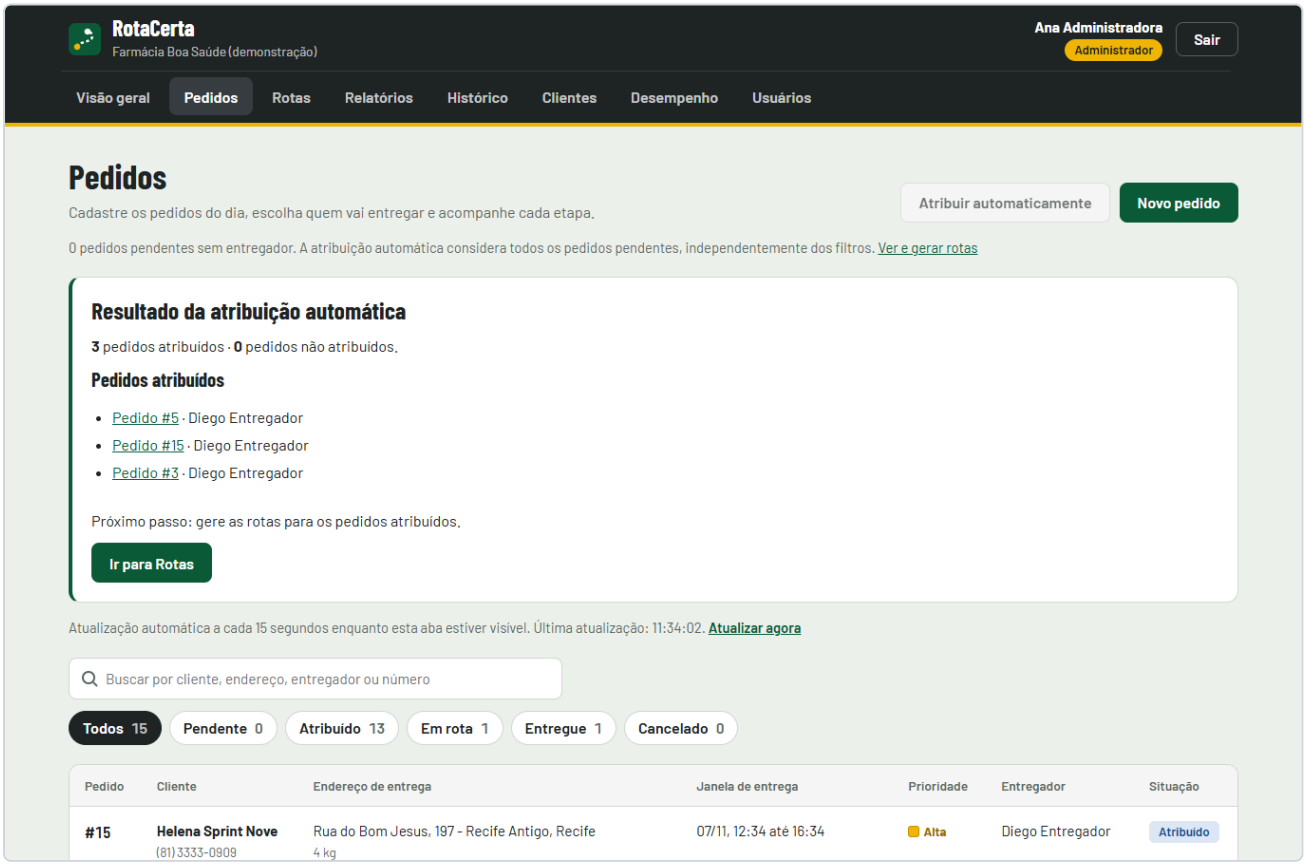
F04 - Cadastro de cliente. Procedimento: Abrir Clientes > Novo cliente; Informar nome e telefone sem formatação; Clicar em Cadastrar cliente. Obtido: Cliente #15 listado com telefone (81) 99999-0909.



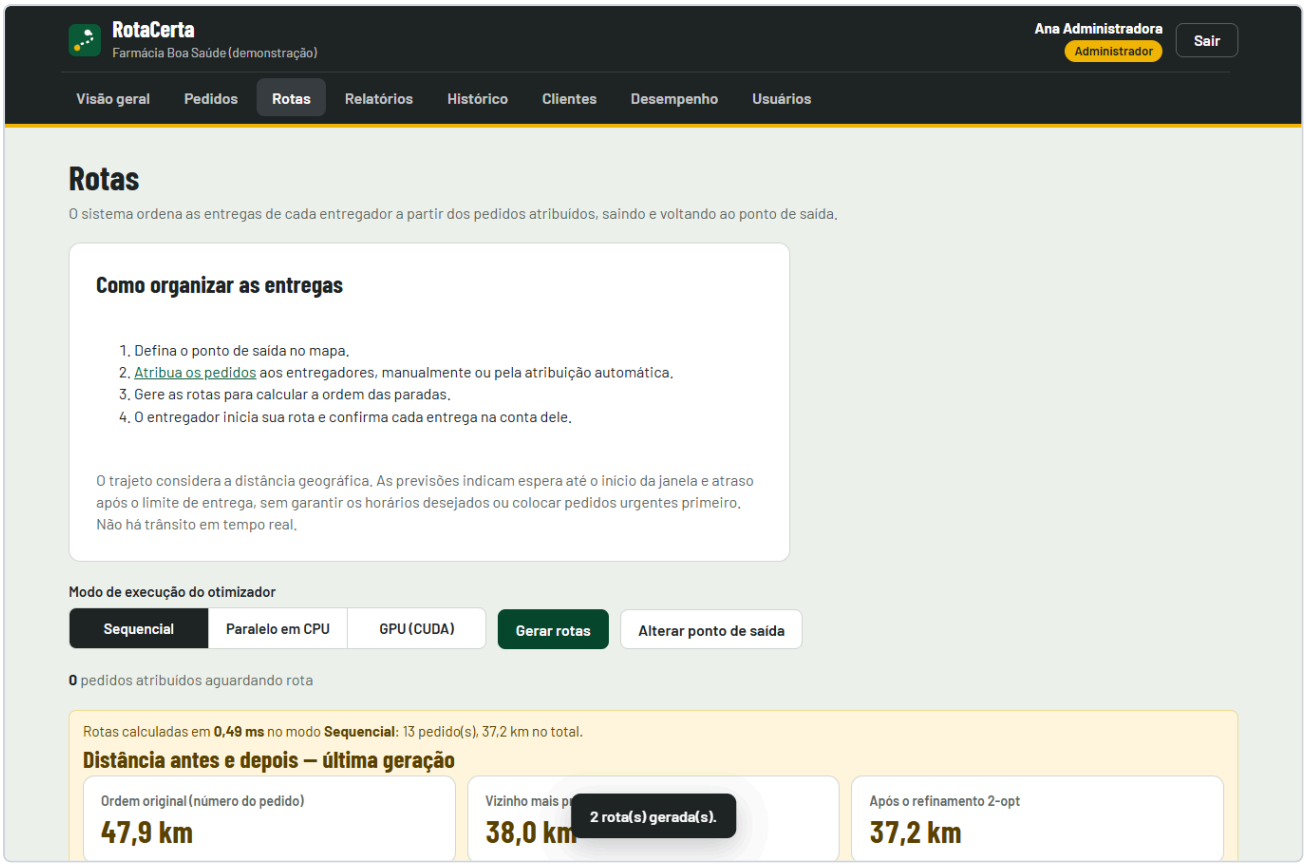
F07 - Cadastro de pedido. Procedimento: Abrir Pedidos > Novo pedido; Escolher o cliente, endereço, latitude/longitude, peso '3,5', prioridade Alta e janela futura; Clicar em Cadastrar pedido. Obtido: Pedido #15: situação PENDING, peso 3.5 kg, prioridade 1.



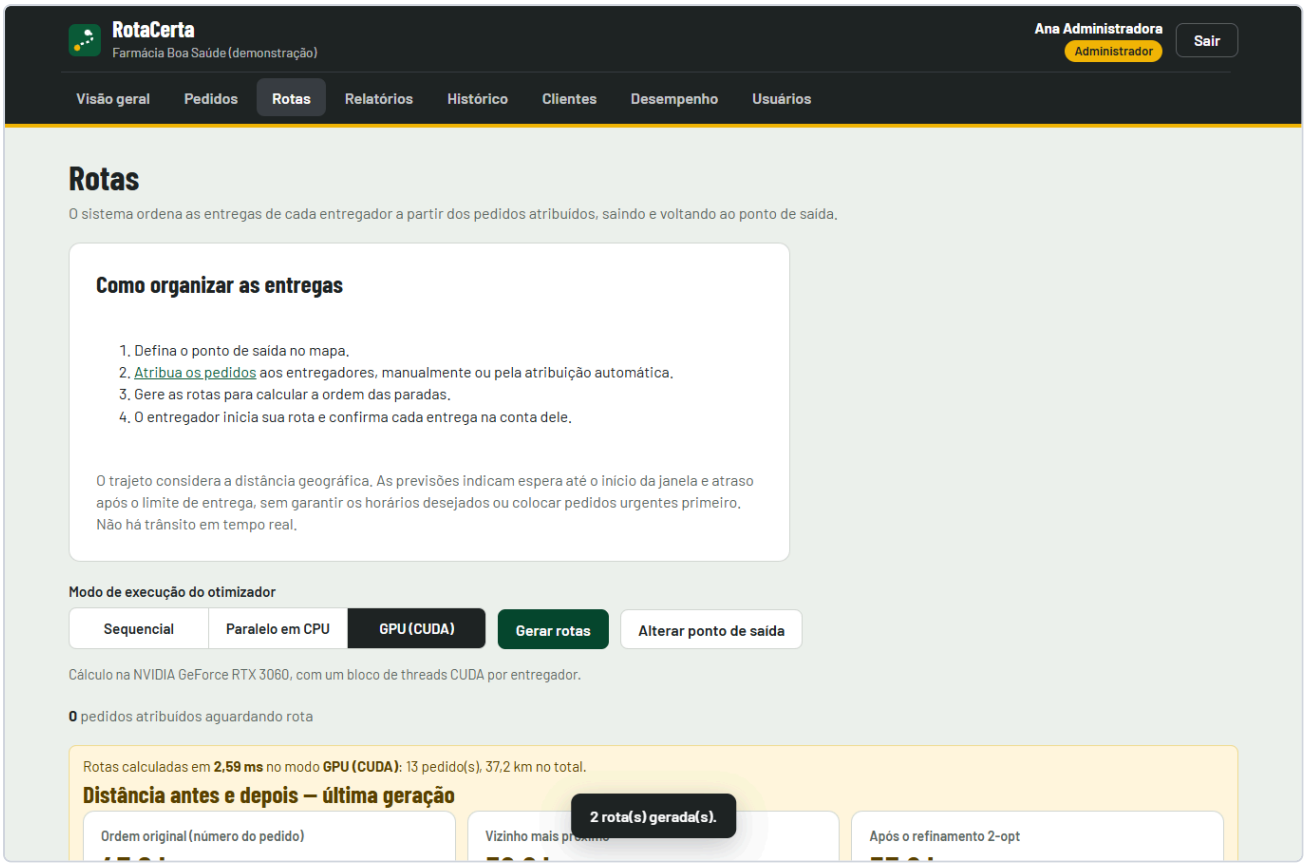
F10 - Exclusão de pedido com confirmação. Procedimento: Criar pedido pendente; Abrir o pedido e clicar em Excluir; Confirmar em Excluir pedido. Obtido: Pedido #16 excluído; consulta posterior retorna HTTP 404.



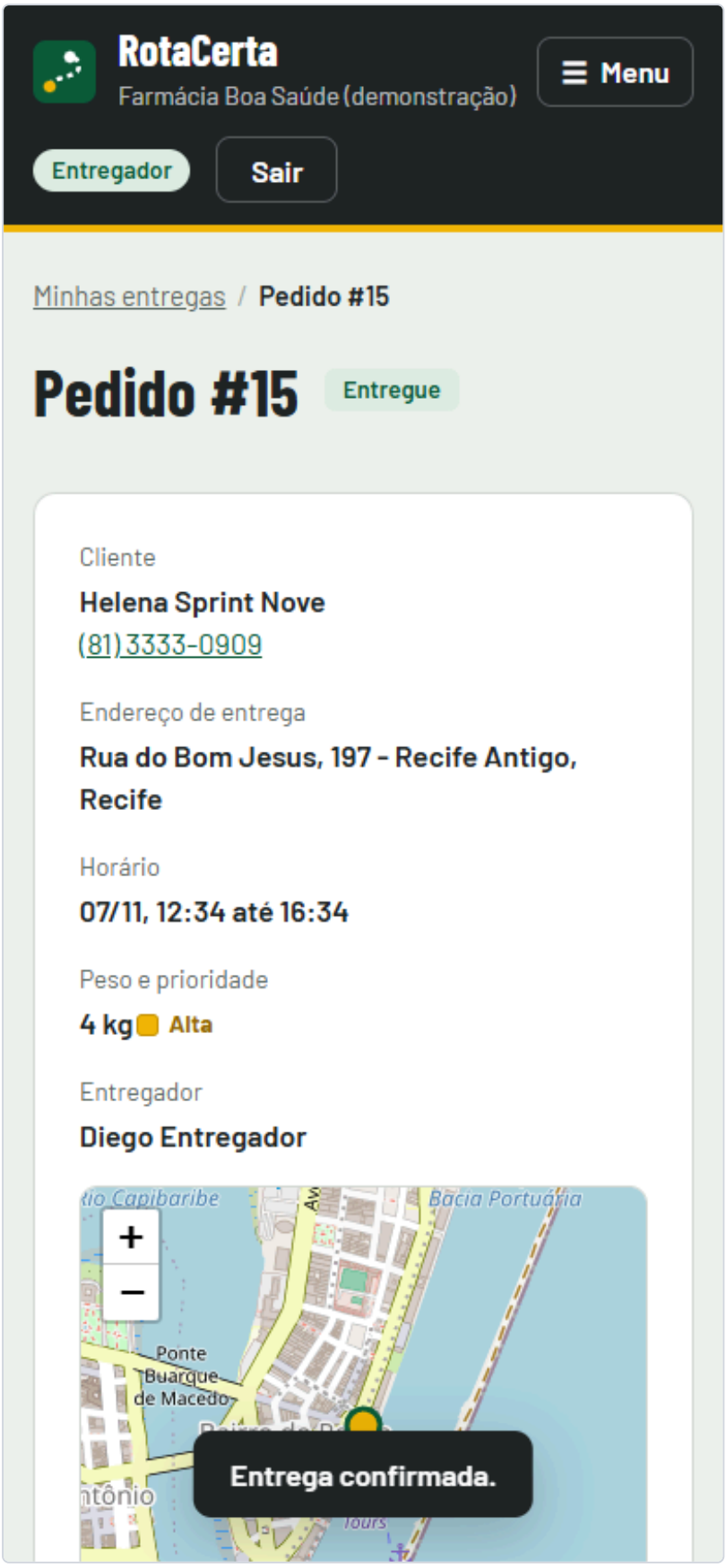
F14 - Atribuição automática. Procedimento: Em Pedidos, clicar em Atribuir automaticamente; Confirmar em Atribuir pedidos. Obtido: Pedido #15 atribuído a Diego Entregador; histórico: 'Atribuído automaticamente a Diego Entregador'.



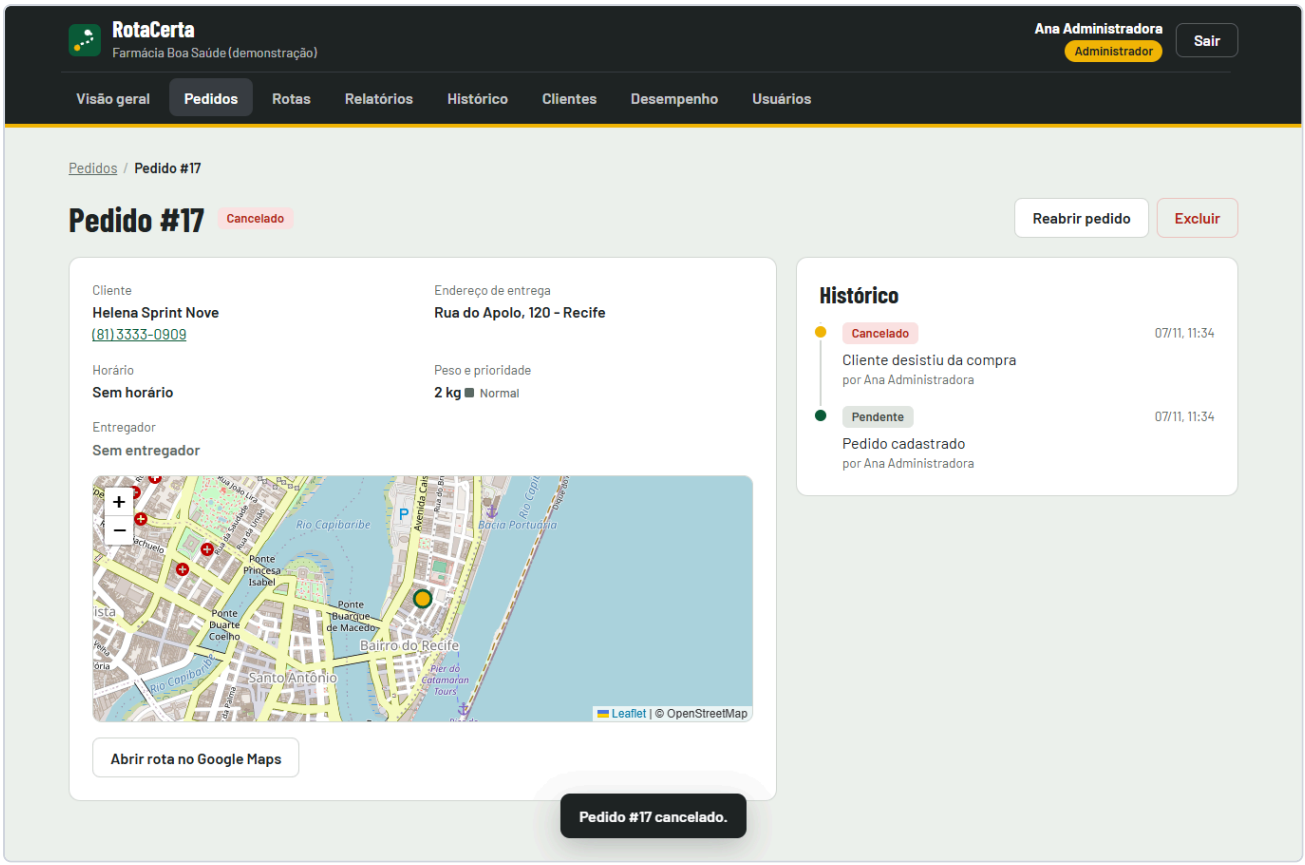
F15 - Geração de rotas no modo sequencial. Procedimento: Abrir Rotas; Escolher Sequencial e clicar em Gerar rotas. Obtido: 2 rotas SEQUENTIAL. Carla Entregadora: 5 paradas, 21.307 km, Diego Entregador: 8 paradas, 15.934 km.



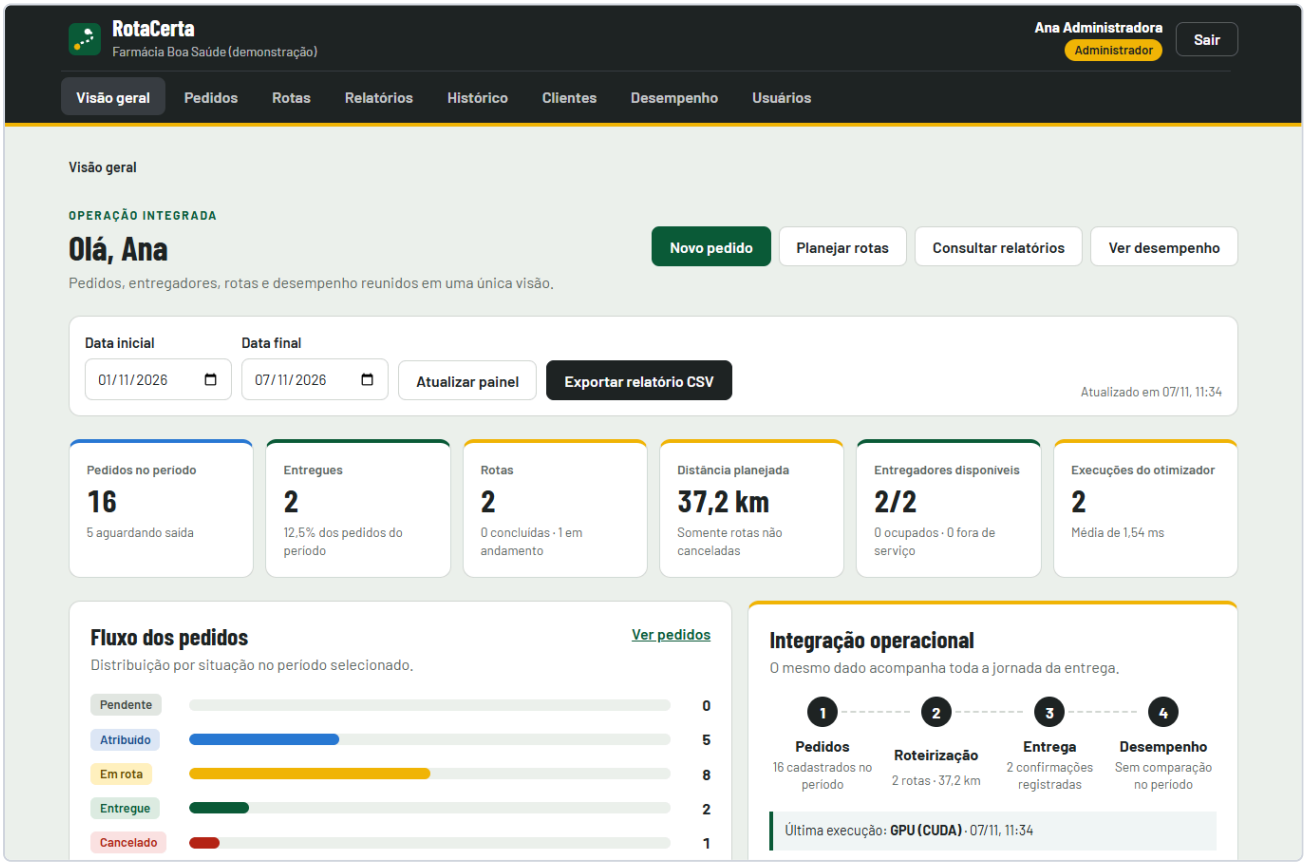
F16 - Geração de rotas na GPU (CUDA). Procedimento: Em Rotas, escolher GPU (CUDA) e clicar em Gerar rotas. Obtido: 2 rotas GPU em NVIDIA GeForce RTX 3060; distâncias iguais ao sequencial: {'Carla Entregadora': 21.307, 'Diego Entregador': 15.934}.



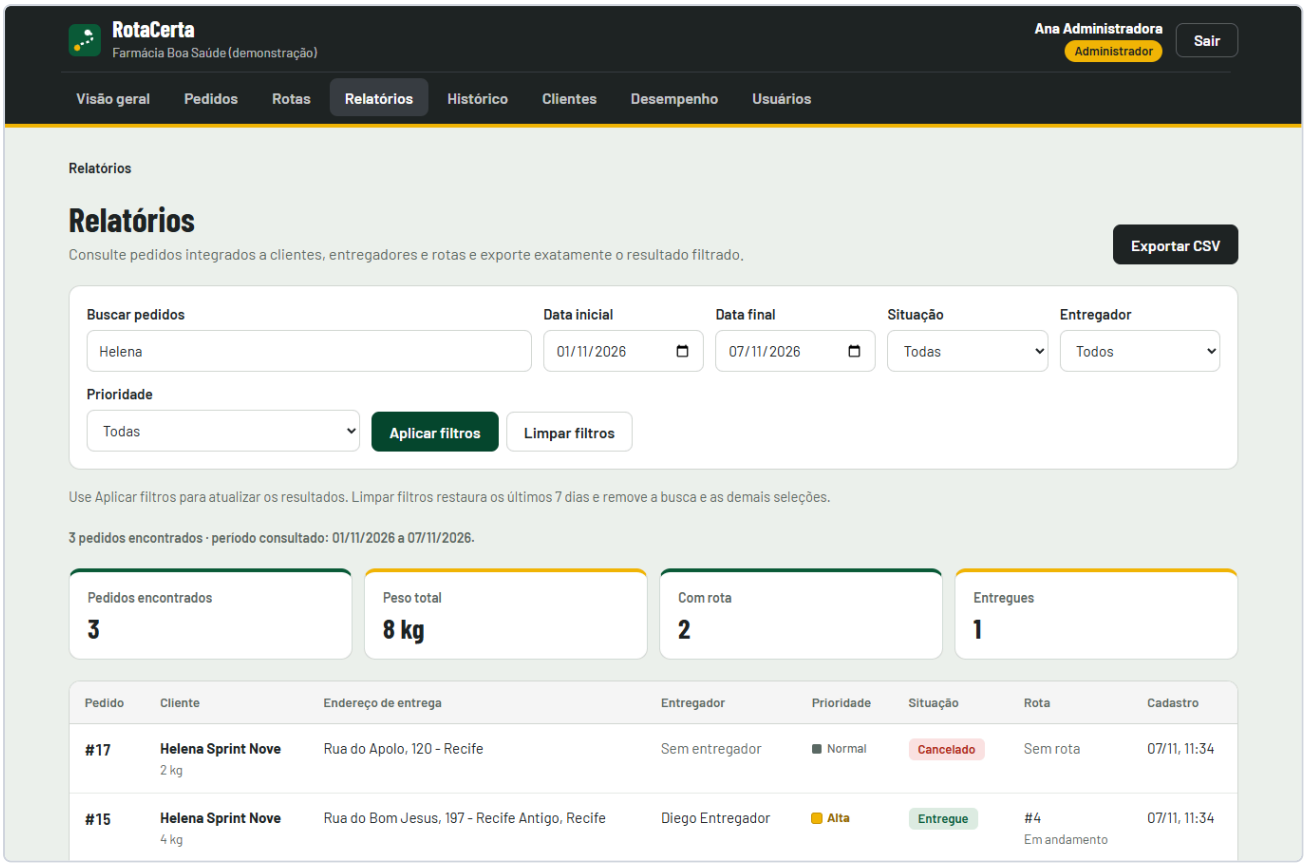
F17 - Entregador inicia a rota e confirma a entrega. Procedimento: Entrar como o entregador do pedido no celular; Clicar em Iniciar rota; Abrir o pedido e clicar em Confirmar entrega. Obtido: Pedido #15 DELIVERED; último evento por Diego Entregador em 2026-11-07T11:34:06.467112-03:00.



F18 - Cancelamento com justificativa. Procedimento: Criar pedido pendente; Abrir o pedido > Cancelar pedido; Informar o motivo e Confirmar cancelamento. Obtido: Pedido #17 CANCELLED; motivo no histórico: 'Cliente desistiu da compra'.



F19 - Painel com indicadores atualizados. Procedimento: Abrir Visão geral depois dos fluxos anteriores. Obtido: Pedidos: 16 no período, 2 entregue(s), 1 cancelado(s), conclusão 12.5%; rotas: 1 planejada(s), 1 em andamento.

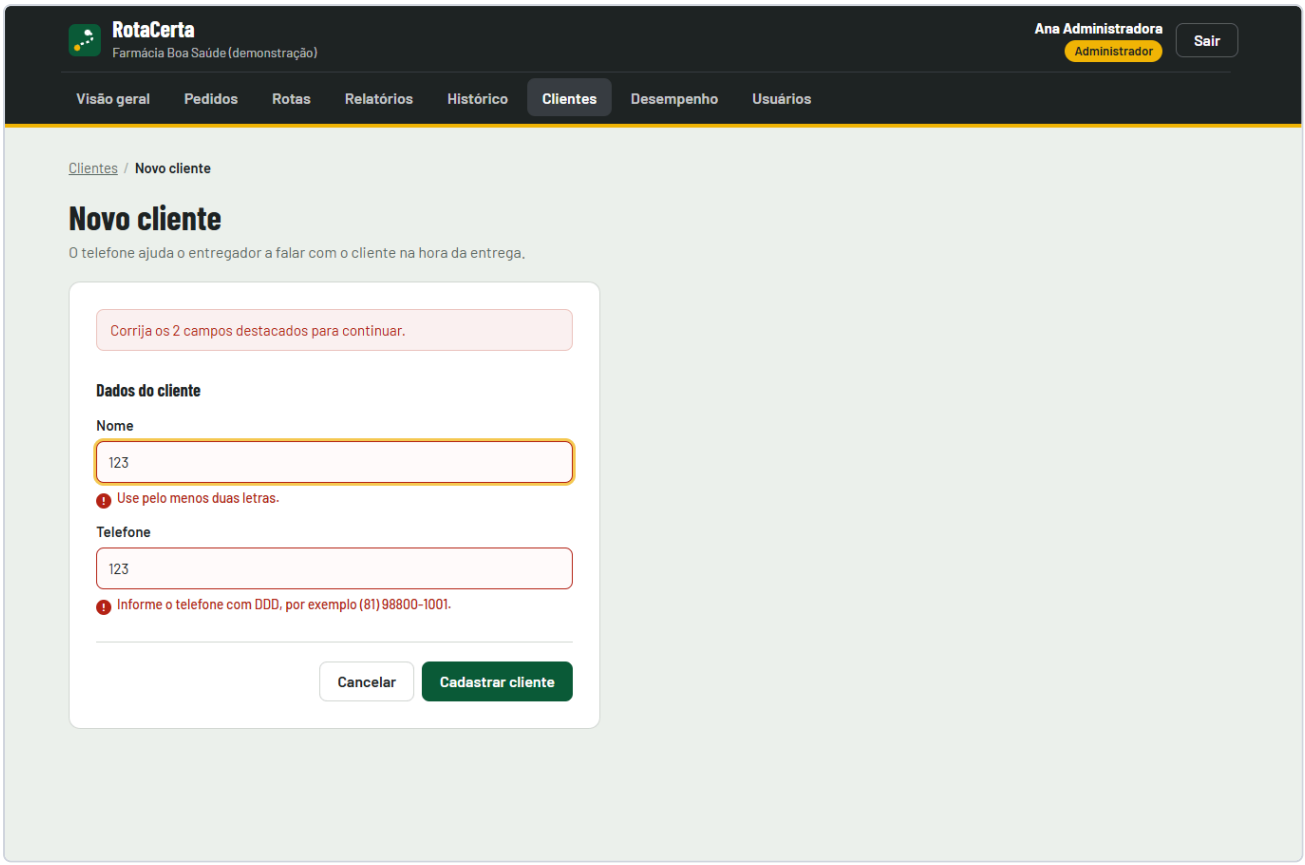


F20 - Relatório filtrado e exportação CSV. Procedimento: Abrir Relatórios; Buscar 'Helena' e Aplicar filtros; Clicar em Exportar CSV. Obtido: 3 linha(s) na tela e 3 no CSV 'rota-certa-relatorio-2026-11-01-a-2026-11-07.csv', todas da cliente Helena.

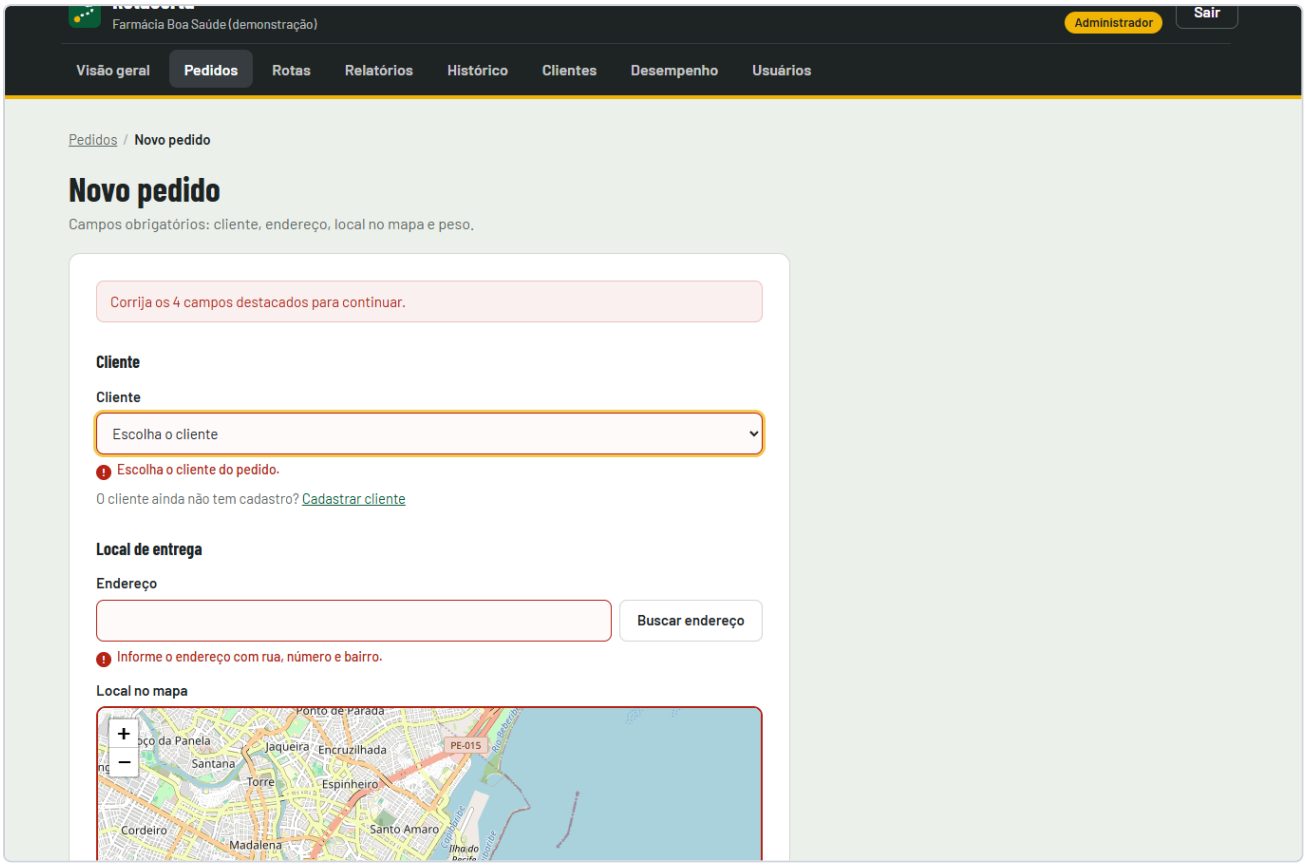
V01 - Login com campos obrigatórios vazios. Procedimento: Abrir /login; Clicar em Entrar sem preencher. Obtido: Mensagens 'Informe o e-mail.' e 'Informe a senha.'; foco no campo email.

V02 - Login com senha incorreta. Procedimento: Informar e-mail válido e senha errada; Clicar em Entrar. Obtido: Exibido 'E-mail ou senha inválidos.'; permaneceu em /login.

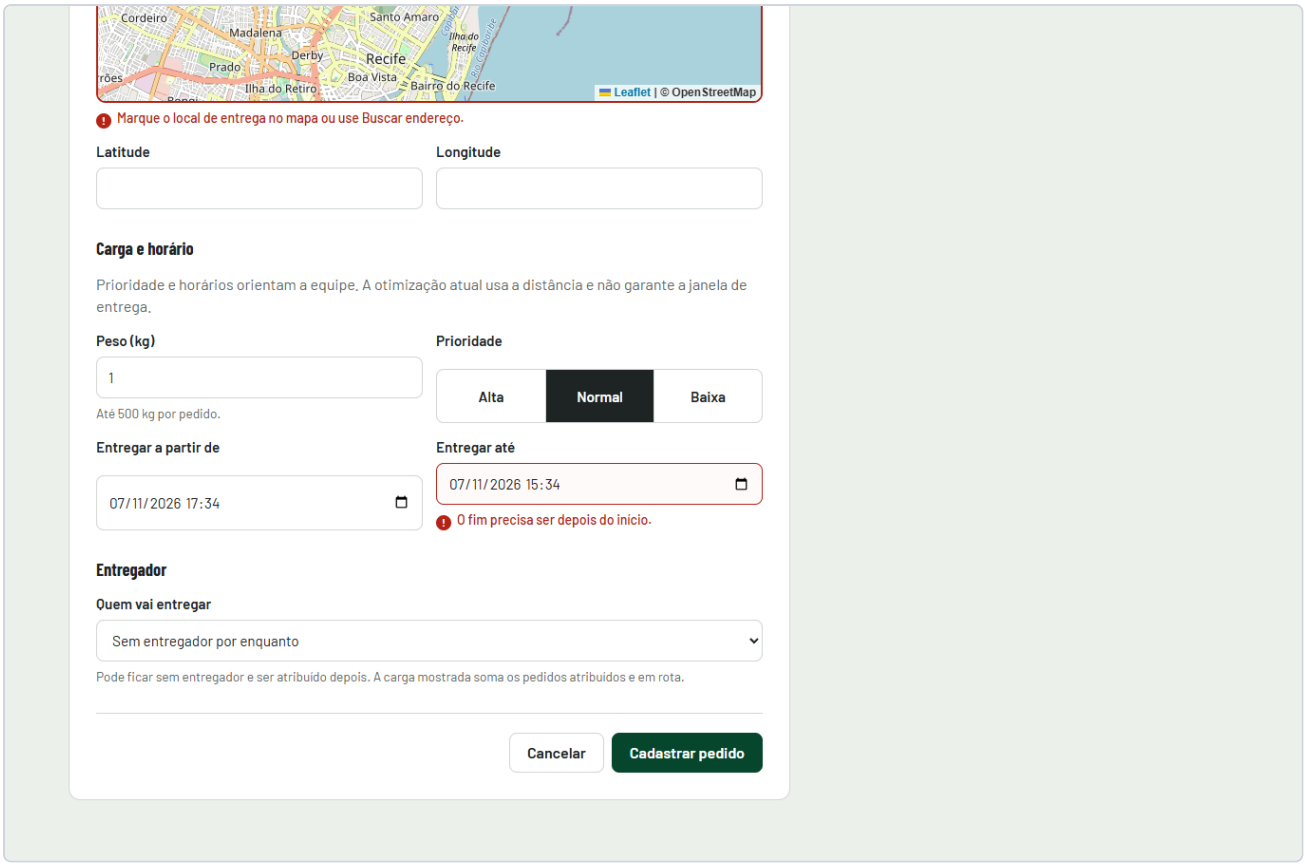
V03 - Cadastro de negócio com formatos inválidos. Procedimento: Em /cadastro, informar e-mail 'paula@', senha 'abc' e endereço curto; Clicar em Criar conta. Obtido: Informe o endereço de onde saem as entregas, com rua e número. | Use pelo menos duas letras. | Informe um e-mail válido, como nome@empresa.com.br. | Use pelo menos 8 caracteres.



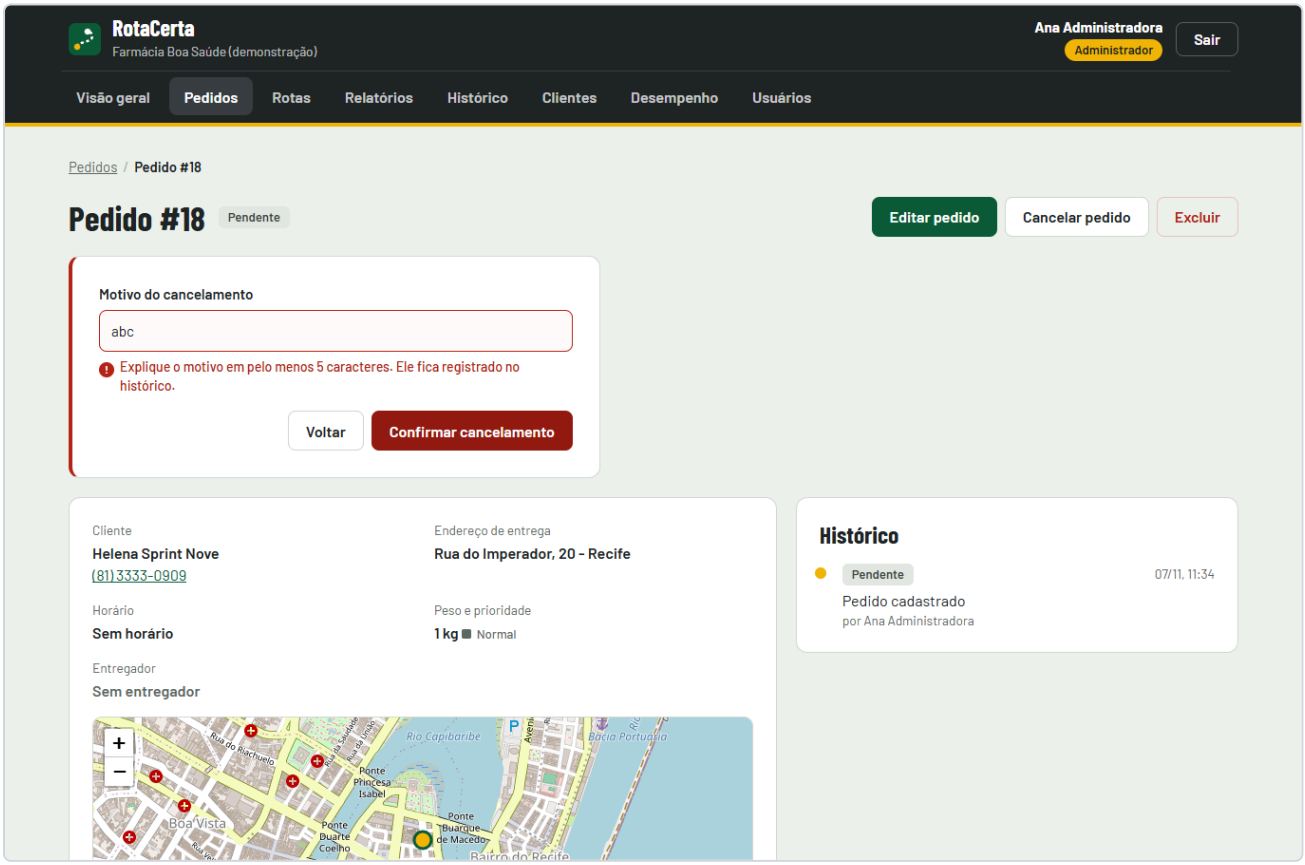
V04 - Cliente com nome e telefone inválidos. Procedimento: Em Novo cliente, informar nome '123' e telefone '123'; Clicar em Cadastrar cliente. Obtido: Exibidos 'Use pelo menos duas letras.' e 'Informe o telefone com DDD, por exemplo (81) 98800-1001.'



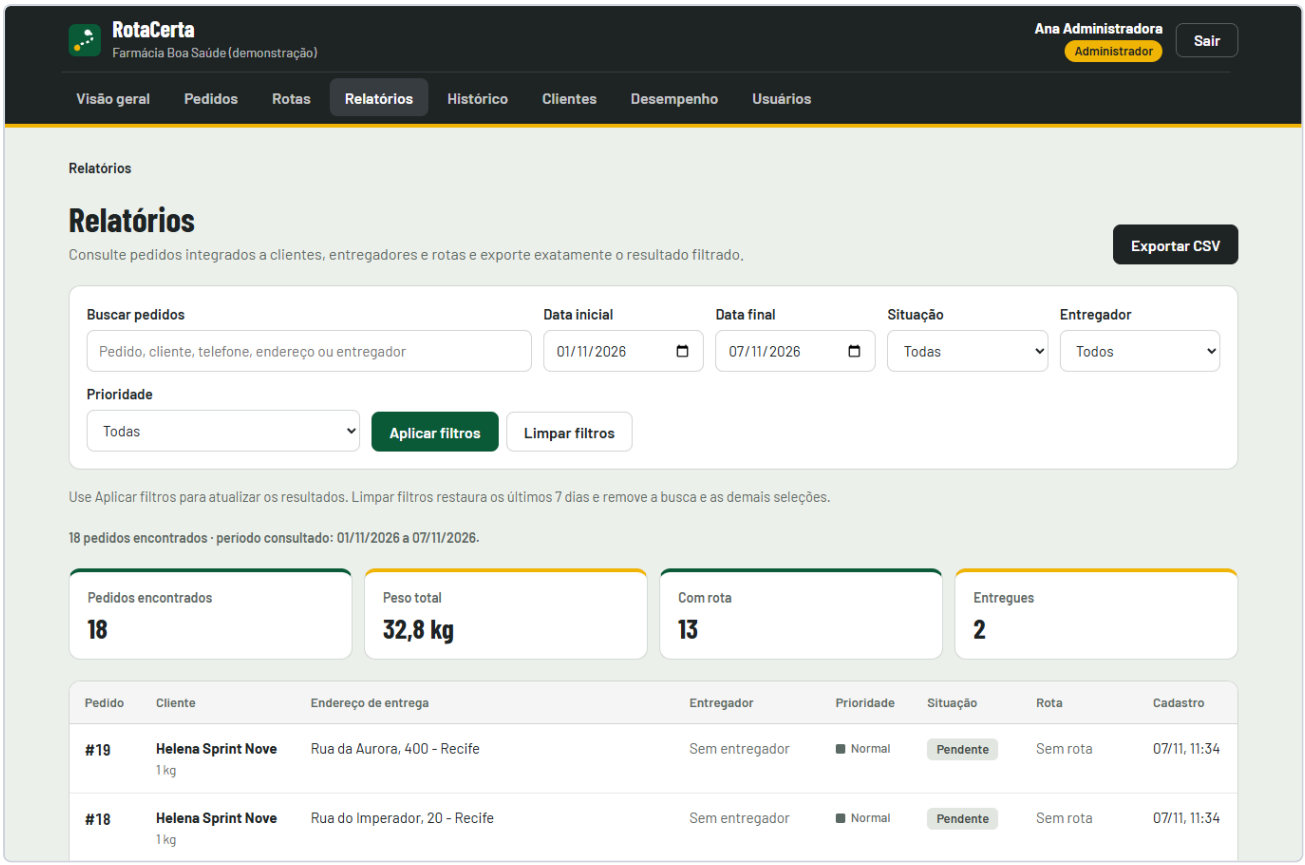
V06 - Pedido com campos obrigatórios vazios. Procedimento: Em Novo pedido, informar só espaços no endereço; Clicar em Cadastrar pedido. Obtido: Escolha o cliente do pedido. | Informe o endereço com rua, número e bairro. | Marque o local de entrega no mapa ou use Buscar endereço. | Informe o peso.



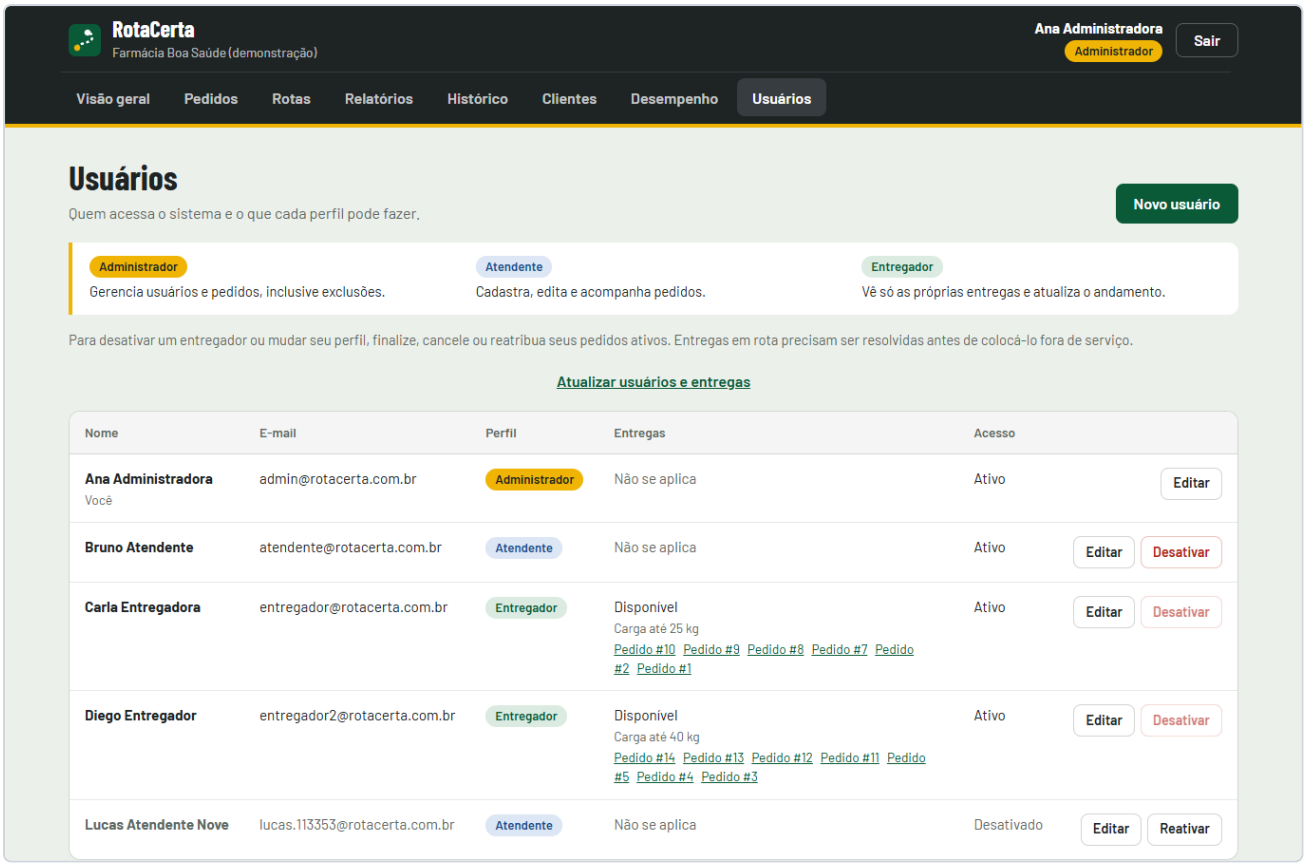
V08 - Janela de entrega invertida. Procedimento: Em Novo pedido, informar fim antes do início; Clicar em Cadastrar pedido. Obtido: Exibido 'O fim precisa ser depois do início.' junto ao campo Entregar até.



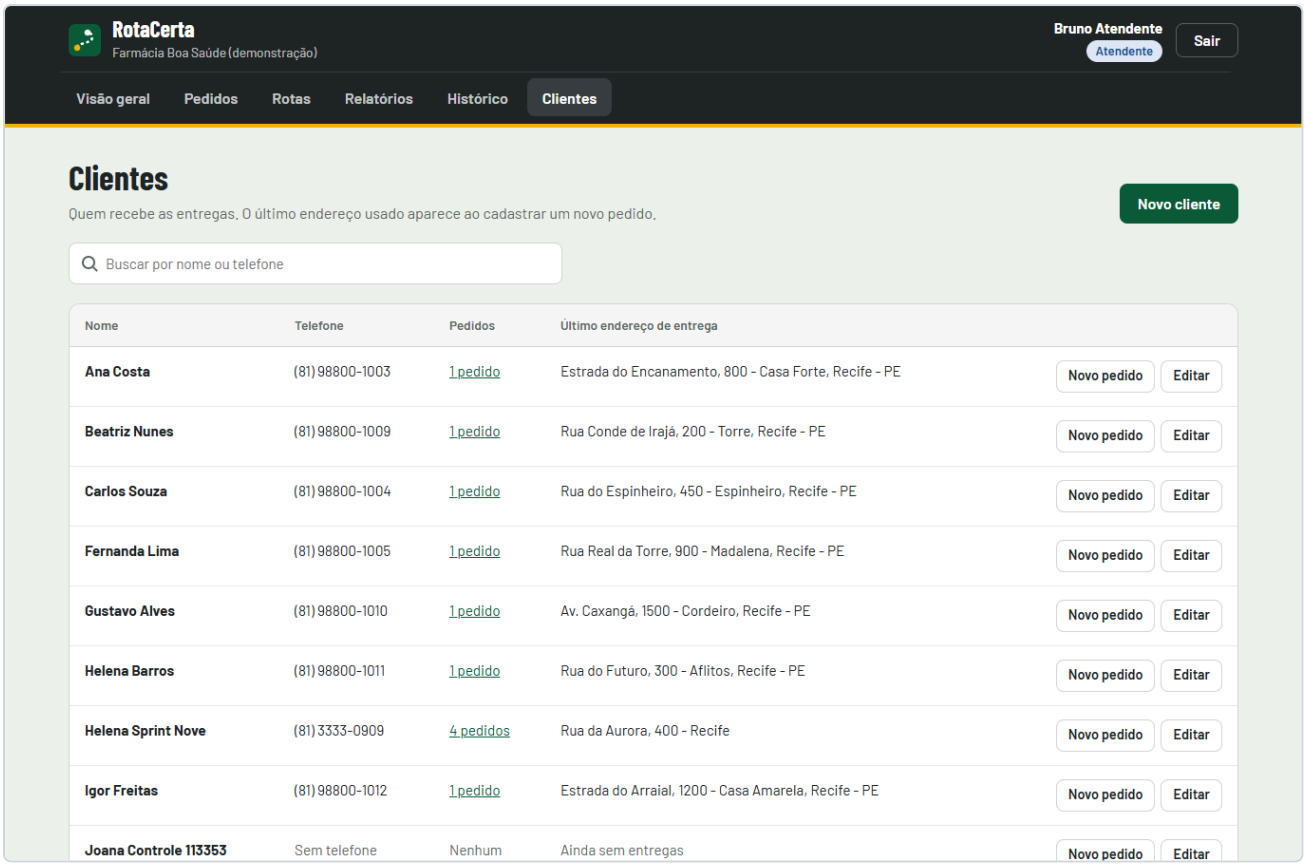
V11 - Cancelamento sem motivo suficiente. Procedimento: Abrir pedido > Cancelar pedido; Informar 'abc'; Clicar em Confirmar cancelamento. Obtido: Mensagem exibida; pedido #18 continua PENDING.



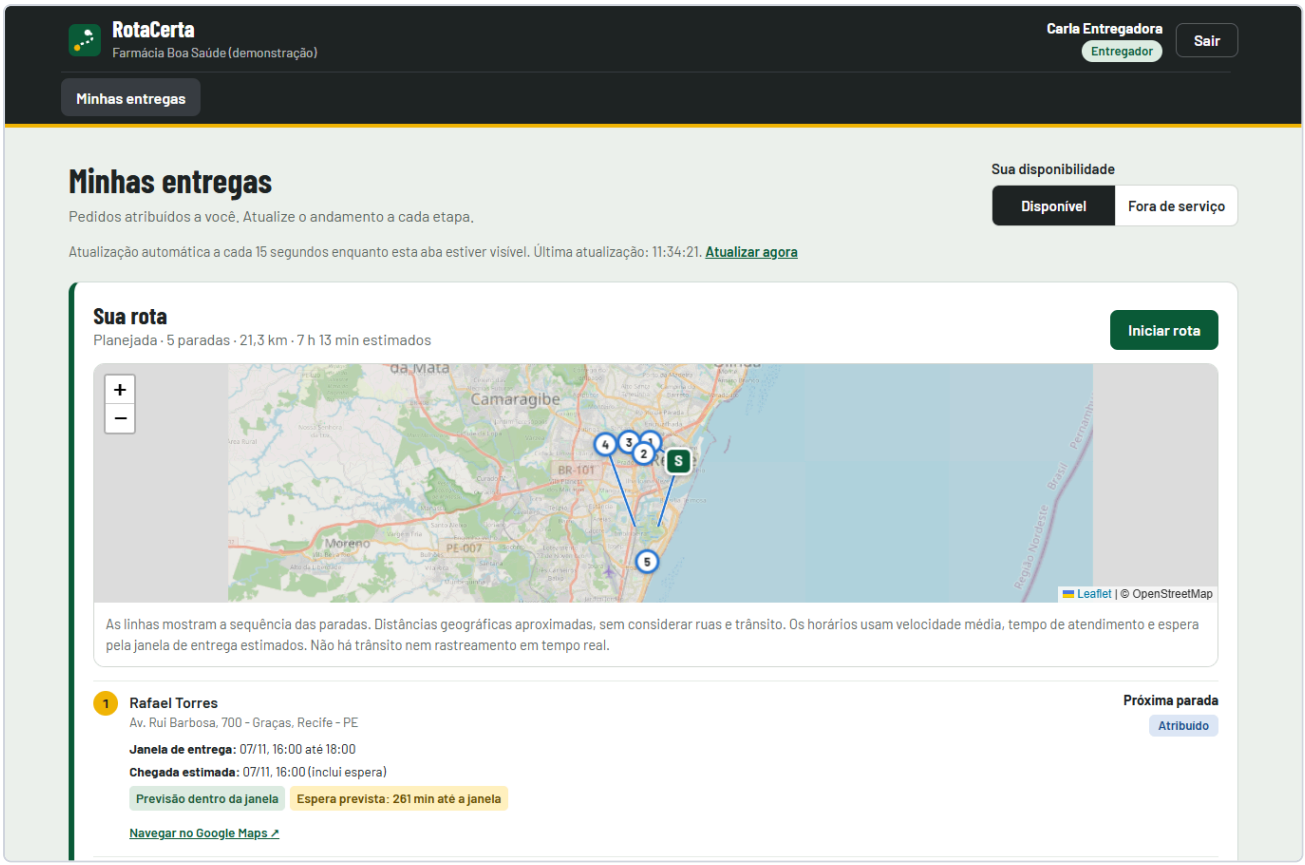
N02 - Volta à página pedida depois do login (BUG-06 corrigido). Procedimento: Na tela do N01, entrar como administrador. Obtido: Após o login: /relatorios.



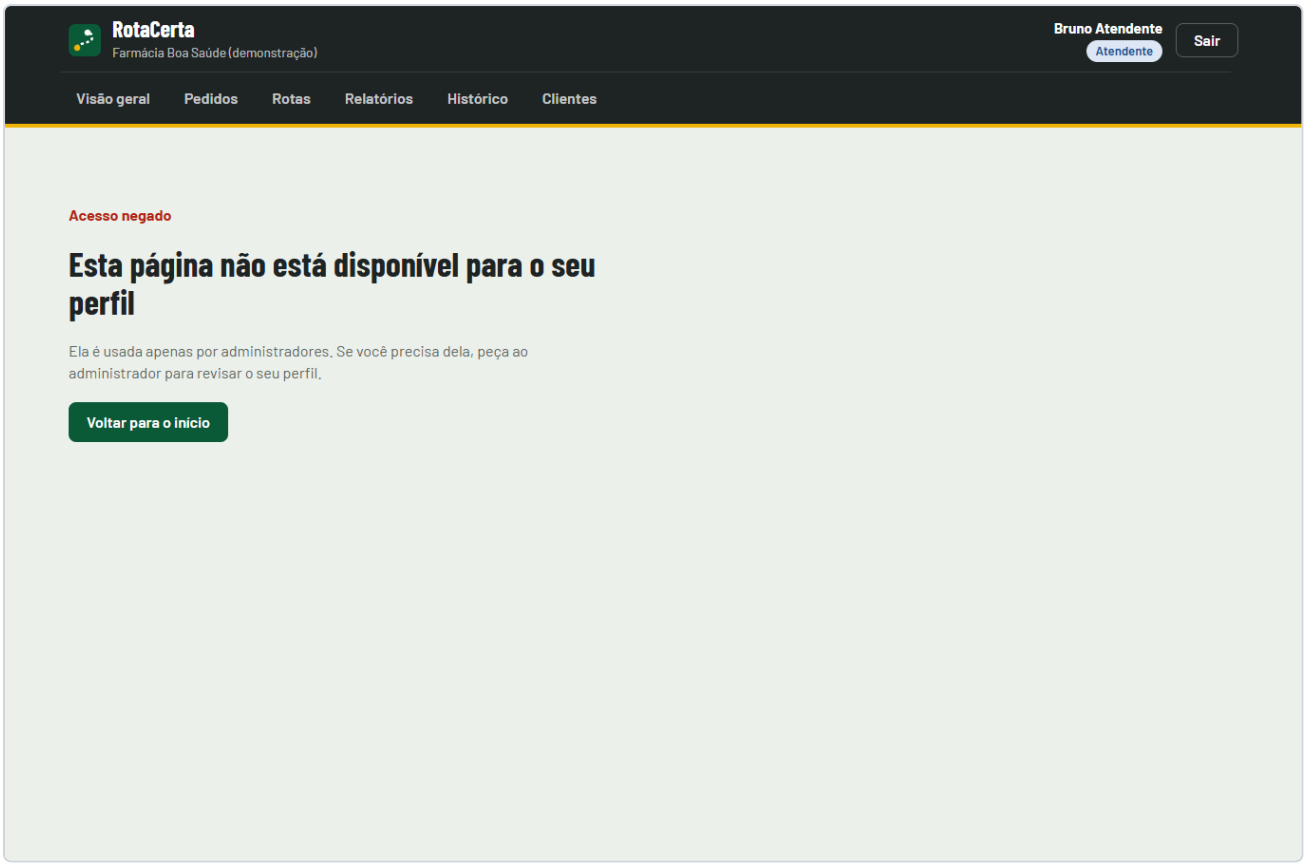
N04 - Menu e navegação do perfil Administrador. Procedimento: Entrar como Administrador; Clicar em cada item do menu. Obtido: Visão geral->/painel; Pedidos->/pedidos; Rotas->/rotas; Relatórios->/relatorios; Histórico->/historico; Clientes->/clientes; Desempenho->/desempenho; Usuários->/usuarios



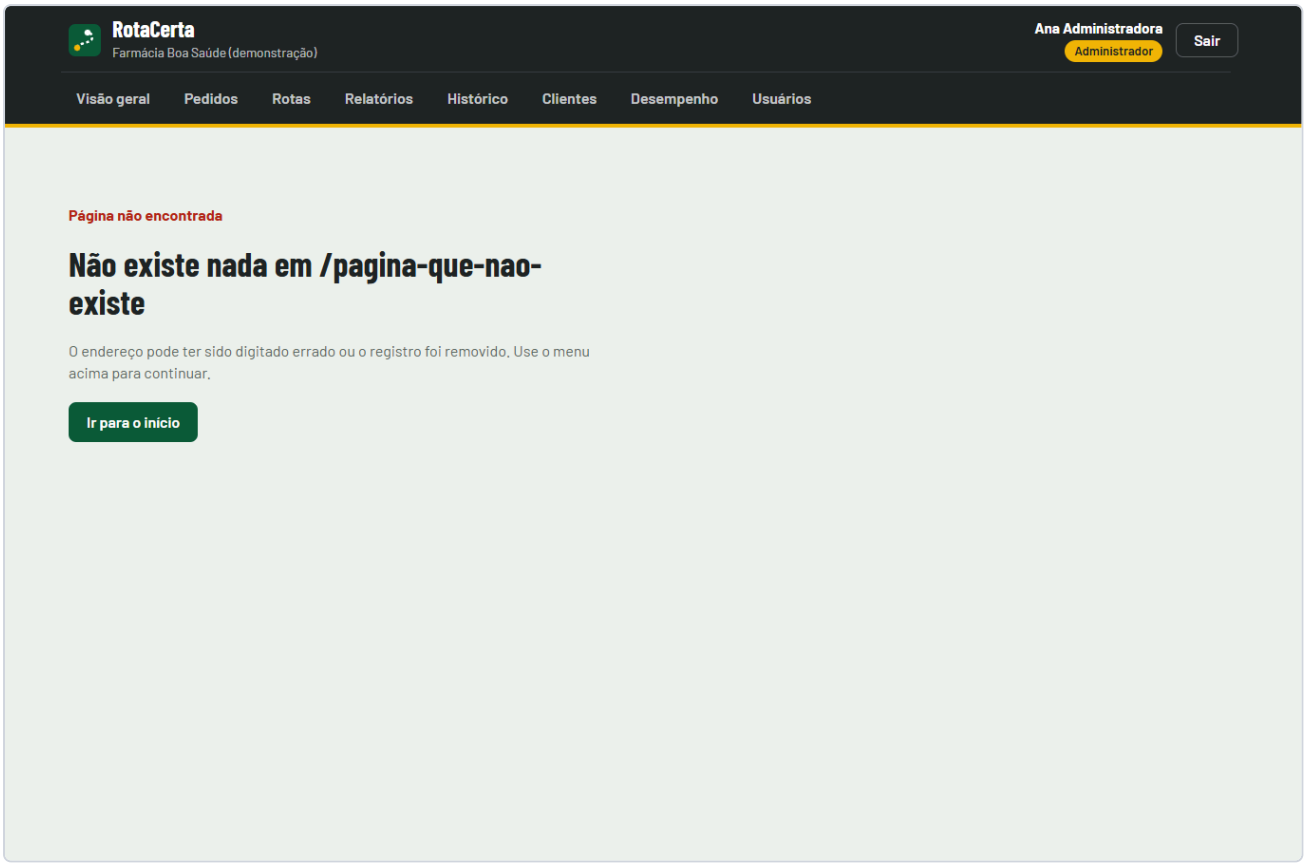
N05 - Menu e navegação do perfil Atendente. Procedimento: Entrar como Atendente; Clicar em cada item do menu. Obtido: Visão geral->/painel; Pedidos->/pedidos; Rotas->/rotas; Relatórios->/relatorios; Histórico->/historico; Clientes->/clientes



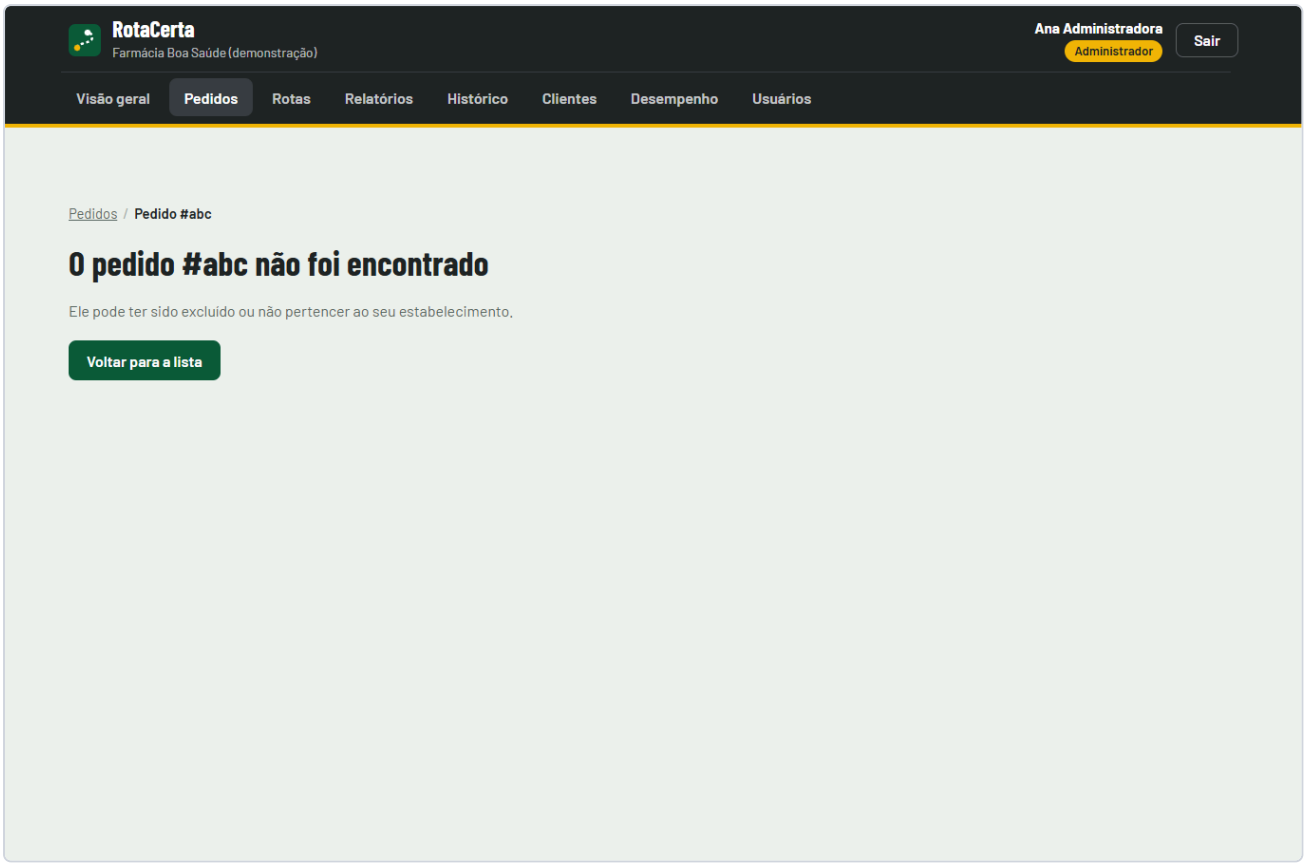
N06 - Menu e navegação do perfil Entregador. Procedimento: Entrar como Entregador; Clicar em cada item do menu. Obtido: Minhas entregas->/entregas



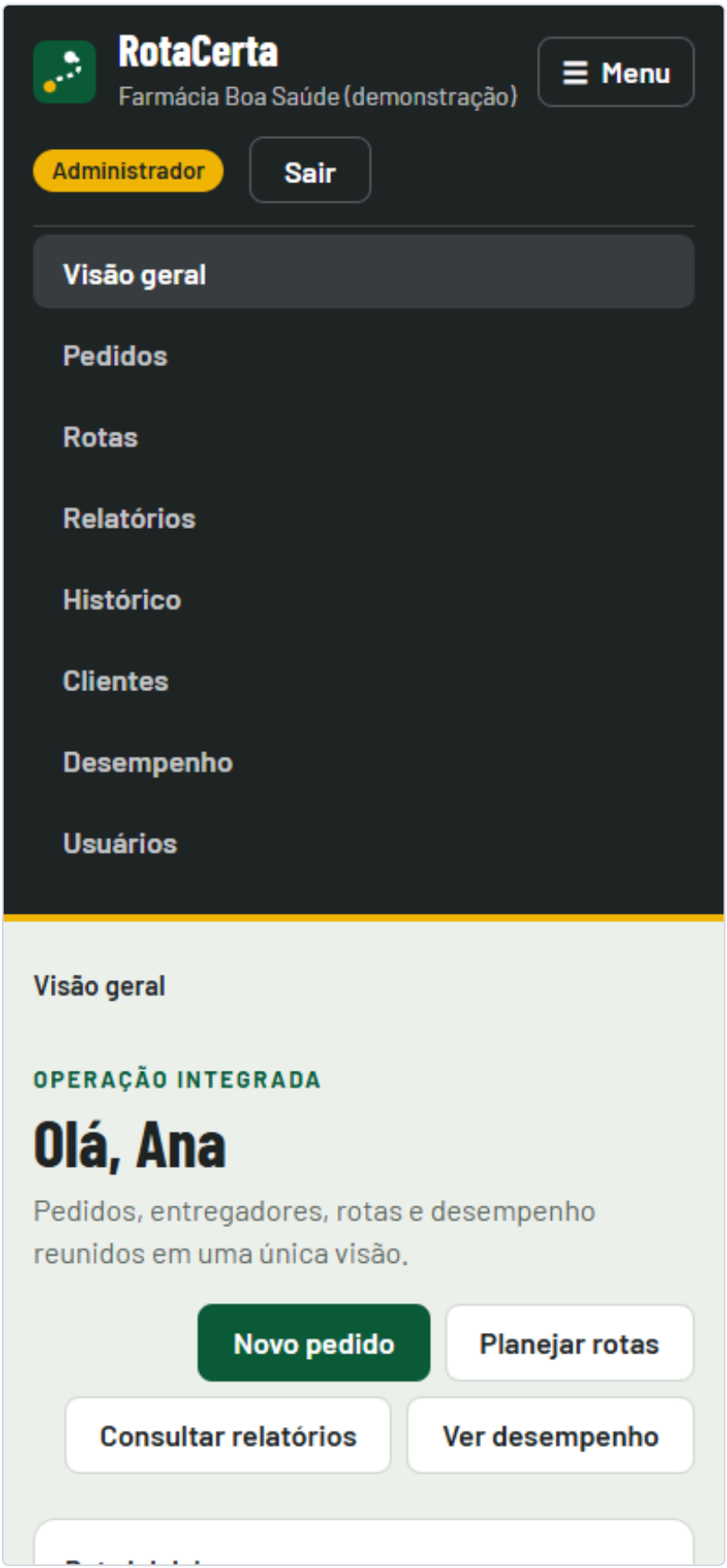
N07 - Atendente abre tela exclusiva do administrador. Procedimento: Entrar como atendente; Abrir /usuarios pelo endereço; Clicar em Voltar para o início. Obtido: Acesso negado exibido; botão levou a /painel; GET /users como atendente: HTTP 403.



N10 - Endereço inexistente. Procedimento: Logado, abrir /pagina-que-nao-existe; Clicar em Ir para o início. Obtido: Página não encontrada exibida; botão levou a /painel.

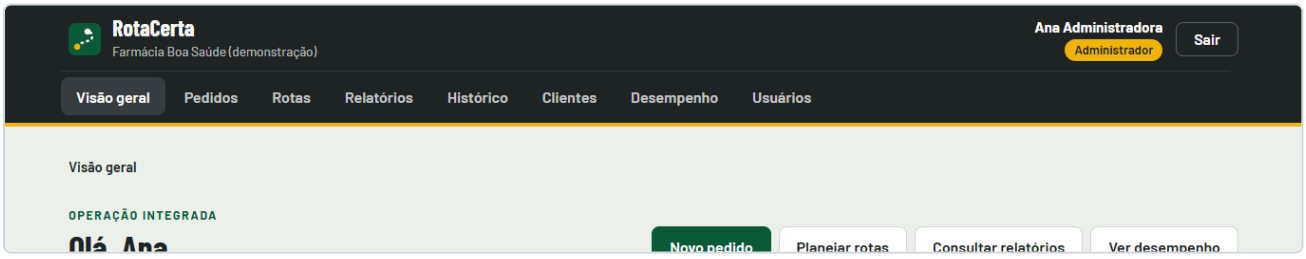


N11 - Identificador inválido no endereço (BUG-05 corrigido). Procedimento: Abrir /pedidos/abc, /pedidos/abc/editar, /clientes/abc/editar e /usuarios/abc/editar. Obtido: /pedidos/abc: 'O pedido #abc não foi encontrado'; /pedidos/abc/editar: 'O pedido #abc não foi encontrado.'; /clientes/abc/editar: 'Cliente não encontrado.'; /usuarios/abc/editar: 'Usuário não encontrado.'



N13 - Menu no celular. Procedimento: Em 390 px de largura, clicar em Menu; Escolher Clientes. Obtido: Fechado no início; aberto após Menu; navegou para /clientes e fechou; rolagem horizontal 0px.

N15 - Sessão expirada no meio do uso. Procedimento: Logado em Clientes, invalidar o token salvo no navegador; Recarregar a página; Entrar novamente. Obtido: Aviso exibido no login; após entrar, voltou para /clientes?busca=Helena.



N19 - Menu completo visível no desktop (BUG-09 corrigido). Procedimento: Entrar como administrador em telas de 1024, 1280, 1366 e 1920 px; Medir a posição de cada item do menu. Obtido: 1024px: 8 itens visíveis; 1280px: 8 itens visíveis; 1366px: 8 itens visíveis; 1920px: 8 itens visíveis

Parte X - Sprint 10

Versão Release Candidate

Grupo 04 - RotaCerta | UNINASSAU | Ciência da Computação | Turma 8NA | 2026.2

Consolidação e verificações em 10/11/2026. Entrega até 14/11/2026 às 23h59, horário de Recife. Versão candidata `0.10.0-rc.1`, na branch `codex/sprint-10-release-candidate`, baseada em `f0b9e20` (Sprint 09 integrada).

Repositório: https://github.com/alvccpj/rota-certa

Identificação da equipe

Integrante	Matrícula	Responsabilidade inicial registrada na Sprint 01
Álvaro Jordão	01748200	Scrum Master e Product Owner; organização, requisitos e priorização
Arthur Sales	01593811	Backend e núcleo de otimização e paralelização
Vinícius Trigueiro	01794959	Frontend, banco de dados e documentação
William Coelho de Morais	01263977	Desenvolvimento e testes
Guilherme de Luna Barros	01615925	Desenvolvimento e validação do sistema

As responsabilidades acima preservam a identificação inicial da equipe e não substituem o registro de contribuições no Git. O envio único no Teams cabe ao Scrum Master Álvaro Jordão, após a revisão da equipe.

Descrição da candidata e resultado

A versão candidata integra autenticação e perfis, clientes, usuários e entregadores, pedidos, atribuição manual e automática, roteirização, execução de entregas, painel, consultas, histórico e comparação de desempenho. O fluxo cadastro de cliente e pedido, atribuição, geração de rota, início pelo entregador e confirmação da entrega foi executado no sistema real, com persistência no PostgreSQL.

Os testes executáveis desta rodada terminaram sem falhas. Foi corrigido um problema de rolagem da interface no celular. A estrutura final do banco permaneceu compatível com o DDL existente; o contrato das 37 operações HTTP foi exportado e conferido contra a API em execução. O README passou a reunir os requisitos, configuração, instalação, banco, comandos, estrutura e equipe.

A candidata está preparada para a demonstração técnica dos módulos verificados. A aceitação com usuário externo continua pendente. O modo CUDA não foi revalidado nesta máquina ARM sem GPU NVIDIA; sua execução anterior está preservada na Sprint 09.

Entregas e localização das evidências

Entrega obrigatória	Resultado da Sprint 10	Evidência no repositório
Sistema estável	Fluxos reais, suítes automatizadas e concorrência sem falhas nos casos executados	`evidencias/casos_de_teste.json`, `testes_backend_postgresql.txt`, `concorrencia.txt`
Interface final	Navegação por perfil, validação de formulários, computador e celular; BUG-10 corrigido	`assets/`, `evidencias/interface_rc.json`
Banco final	Nove tabelas compatíveis com os modelos, 14 FKs sem órfãos e isolamento verificado	`evidencias/banco_e_contrato.json`, `database/schema.sql`
APIs documentadas	37 operações com métodos, parâmetros, modelos, acesso e exemplos	`docs/api/README.md`, `ENDPOINTS.md`, `openapi.json`
README atualizado	Objetivo, tecnologias, requisitos, instalação, banco, execução, estrutura e equipe	`README.md`, `COMO-RODAR.md`

Os registros citados pelo nome estão em `docs/sprint-10/evidencias/`, e as capturas, em `docs/sprint-10/assets/`. Os demais caminhos partem da raiz do repositório. O PDF de entrega é `output/pdf/Grupo_04_RotaCerta_Sprints_01_a_10.pdf`.

Ambiente e método de verificação

Execução em macOS ARM64, Python 3.14.7, dez processadores lógicos e PostgreSQL 17.10. Foi usado um cluster separado na porta 5540, banco descartável `rotacerta_rc`, sem alterar o PostgreSQL de desenvolvimento. O roteiro de navegador usou o schema `sprint10_testes`, API na porta 8020 e Vite na porta 5190. As suítes do backend recriam somente `rotacerta_testes_automatizados`.

O Chrome foi controlado por Playwright com interface em português e fuso de Recife. O roteiro reutiliza os 61 casos da Sprint 09, com capturas novas e registro de procedimento, esperado, obtido e situação. A verificação extra de interface confere as larguras de 320, 390 e 768 px e o acesso às ações por rolagem interna da tabela. O teste N19 cobre o menu em 1024, 1280, 1366 e 1920 px.

Resultados dos testes

Verificação	Resultado atual	Registro
Funcionais na aplicação real	21 aprovados, 1 não executado por ausência de CUDA	`casos_de_teste.json`, F01 a F22
Validação na aplicação real	20 aprovados, 0 falhas	`casos_de_teste.json`, V01 a V20
Navegação na aplicação real	19 aprovados, 0 falhas	`casos_de_teste.json`, N01 a N19
Backend em SQLite	152 descobertos: 149 aprovados, 3 dispensados	`testes_backend_sqlite.txt`
Backend em PostgreSQL	152 descobertos: 151 aprovados, 1 dispensado	`testes_backend_postgresql.txt`
Frontend	50 aprovados, 0 falhas	`testes_frontend.txt`
Regressão visual adicional	320, 390 e 768 px sem overflow da página; ações acessíveis	`interface_rc.json`
Build de produção	TypeScript e Vite concluídos	`build.txt`
Auditoria npm	Zero vulnerabilidades reportadas nesta execução	`auditoria_npm.json`
Banco e contrato	Nove tabelas, 14 FKs sem órfãos, 37 operações sincronizadas	`banco_e_contrato.json`

No SQLite, são dispensados o teste CUDA e os dois testes específicos de concorrência PostgreSQL. No PostgreSQL, apenas CUDA é dispensado. Os dois testes de concorrência já estão incluídos nos 152, não devem ser somados novamente. No navegador, F16 não foi executado; F22 comparou sequencial e CPU paralela com 1, 2 e 4 processos, com igualdade das rotas. Nenhum desses resultados representa homologação externa ou teste de carga de produção.

Fluxos principais demonstrados

Fluxo	Procedimento e resultado observado	Casos
Acesso e cadastro	Login, saída, novo negócio e perfis corretos; negócios isolados	F01 a F03, N01 a N08, N16
Clientes e pedidos	Cadastro, pesquisa, edição e exclusão permitida; dados persistidos	F04 a F13
Distribuição e rotas	Pedido atribuído automaticamente; duas rotas sequenciais com paradas e métricas	F14, F15
Entrega no celular	Entregador iniciou rota e confirmou entrega; autor e horário no histórico	F17
Cancelamento	Motivo válido gravado; motivo curto recusado	F18, V11
Acompanhamento	Indicadores coerentes, relatório filtrado/exportado e histórico pesquisável	F19 a F21
Otimizador	Mesmas rotas nos modos sequencial e CPU paralela na comparação	F22
Persistência	Nova conexão SQL e nova sessão HTTP leram os mesmos 19 IDs e situações do negócio de demonstração	`banco_e_contrato.json`

As capturas desta Sprint acompanham o PDF. O registro JSON contém os resultados de todos os 61 casos, inclusive o motivo da não execução de F16. Os prints são evidências complementares do sistema executado; os testes também verificam respostas da API e estados persistidos.

Concorrência na atribuição automática

A pendência técnica da Sprint 09 foi verificada em PostgreSQL real. Em cada rodada foram criados 30 pedidos de 1 kg e dois entregadores disponíveis de 10 kg. Oito clientes de teste dispararam `POST /orders/auto-assign` após uma barreira de sincronização, usando sessões de banco independentes. Foram três rodadas, totalizando 24 requisições concorrentes nesse cenário.

Rodada	Chamadas simultâneas	Atribuídos únicos	Permaneceram pendentes	Carga por entregador	Eventos de atribuição
1	8	20	10	10 kg e 10 kg	20, sem duplicação
2	8	20	10	10 kg e 10 kg	20, sem duplicação
3	8	20	10	10 kg e 10 kg	20, sem duplicação

Um segundo teste disparou oito chamadas com a fila vazia: todas retornaram `assigned: []` e `skipped: []`, sem novos eventos. Não houve erro HTTP, estouro de capacidade nem reatribuição. O bloqueio de linha do estabelecimento existente em `app/assignment.py` serializou as chamadas dessa operação até o commit.

O ensaio cobre atribuição automática contra atribuição automática no mesmo estabelecimento. Escritas simultâneas de tipos diferentes e volume elevado continuam fora dessa comprovação. Reprodução e saída estão em `backend/tests/test_release_s10.py` e `evidencias/concorrencia.txt`.

Interface final e correção realizada

A interface mantém a identidade visual verde, amarelo e grafite, com componentes compartilhados para formulários, confirmações, mensagens, indicadores e mapas. Os menus respeitam o perfil, a página atual recebe destaque e as páginas protegidas redirecionam corretamente. O celular usa menu recolhível, cartões de entrega e rolagem dentro das tabelas largas. Campos inválidos exibem mensagem junto ao campo, resumo e foco no primeiro erro.

BUG-10 - Rolagem horizontal indevida em Clientes no celular. A execução N13 detectou overflow de 251 px na página em uma rodada. A reprodução identificou o rótulo absoluto de acessibilidade do cabeçalho da tabela fora de um bloco de posicionamento local. A tabela rolável não continha sua contribuição à largura do documento no Chrome/macOS observado. Foi incluído `position: relative` em `.table-wrap` (`frontend/src/styles.css`), mantendo a rolagem da tabela e as ações acessíveis. Severidade média; situação corrigida.

Após a correção, o documento mediu exatamente 320, 390 e 768 px nas respectivas larguras; a tabela de 981 px continuou rolável dentro da página. N13 passou e o roteiro adicional confirmou a visibilidade do botão Excluir após a rolagem. O caso F08 também passou a aguardar a renderização das linhas filtradas: a URL era atualizada antes do React concluir a tabela. Esse ajuste foi no roteiro e não representa um bug adicional do produto.

Estrutura final do banco de dados

O arquivo `database/schema.sql` é o DDL final da candidata. Não houve nova migração na Sprint 10; o modelo atualizado da Sprint 09 permanece válido. As nove tabelas e todas as colunas foram comparadas com os modelos SQLAlchemy.

Tabela	Colunas	Finalidade e principais vínculos
establishments	7	Negócio e ponto de saída; origem do isolamento
users	8	Conta, perfil, ativação e hash; FK para negócio; e-mail único
couriers	6	Capacidade e disponibilidade; FKs para negócio e usuário único
customers	5	Cliente vinculado ao negócio
orders	13	Local, peso, prioridade, janela e situação; cliente e entregador opcional
order_status_history	6	Pedido, situação, observação, autor e horário
routes	10	Negócio, entregador, data, situação, algoritmo, modo e métricas
route_stops	8	Rota, pedido único, sequência, chegada, espera e situação
optimization_runs	16	Comparações/gerações por negócio, modos, tempos, distância e ganho

As PKs são `BIGSERIAL`, FKs `BIGINT`, medidas `NUMERIC` e instantes `TIMESTAMPTZ`. Há unicidade de usuário por entregador, pedido por parada e sequência por rota, além dos checks de perfis, situações, modos, pesos, capacidade, prioridade, janela, espera e métricas. Os índices atendem às consultas de pedidos por negócio/situação, entregador, histórico, rotas e execuções.

Foram inspecionadas 14 FKs, todas sem registros órfãos. As consultas de consistência não encontraram vínculo entre negócios diferentes em pedido/cliente, pedido/entregador, rota/entregador ou parada/pedido. A API complementa as FKs com as regras de isolamento, transição, raio e carga. O JSON de inspeção contém colunas, PKs, FKs, checks, índices, unicidades e contagem de registros.

Para instalação nova, o Compose aplica o DDL no primeiro volume ou a API aplica o arquivo ao encontrar um PostgreSQL vazio. Bancos anteriores recebem as atualizações incrementais de `app/database.py`. Os dados de demonstração são opcionais (`SEED_DEMO_DATA`). Detalhes em `database/README.md`.

APIs documentadas

O guia permanente está em
https://github.com/alvccpj/rota-certa/blob/codex/sprint-10-release-candidate/docs/api/README.md

O catálogo contém as 37 operações HTTP, com métodos, rotas, parâmetros, obrigatoriedade, modelos de entrada/saída e perfis. O contrato OpenAPI 3.1 inclui todos os campos, tipos e limites e foi comparado com `GET /openapi.json`, sem diferenças. O exportador `tools/documentation/export_api_docs.py` permite atualizar ambos a partir da aplicação.

Com a API ligada: Swagger em `http://localhost:8000/docs`, ReDoc em `http://localhost:8000/redoc` e contrato em `http://localhost:8000/openapi.json`. Esses endereços são locais; o guia e o JSON do repositório podem ser consultados sem um servidor publicado. O PDF inclui o catálogo resumido e uma captura do Swagger.

O login recebe `email` e `password` em JSON e devolve `access_token`, `token_type` e `user`. As chamadas protegidas usam `Authorization: Bearer TOKEN`. ADMIN administra usuários, ponto de saída e desempenho; ADMIN e ATTENDANT operam os módulos de gestão; COURIER opera suas próprias entregas. O endpoint experimental `/optimizer/compare` é público e não persiste pedidos.

Exemplo de entrada de pedido (substituir o ID do cliente por um ID do próprio negócio):

```
{"customer_id": 1, "delivery_address": "Rua do Bom Jesus, 197 - Recife",
 "latitude": -8.0617, "longitude": -34.8709, "weight_kg": 2, "priority": 1}
```

`POST /orders` retorna 201 com o pedido, cliente e situação inicial. Em seguida, `POST /orders/auto-assign` retorna 200 com `assigned` e `skipped`; `POST /routes/generate` aceita `{"mode":"SEQUENTIAL","workers":1}`. O entregador consulta `/routes/me`, inicia `/routes/{route_id}/start` com PATCH e confirma o pedido em `/orders/{order_id}/status` com `{"status":"DELIVERED"}`. Os filtros e os exemplos completos com curl estão no guia.

HTTP	Interpretação
200 / 201 / 204	Consulta/operação concluída, criação concluída ou exclusão/desativação sem corpo
401 / 403	Sessão inválida ou perfil/conta sem acesso
404	Registro inexistente ou não visível
409	Conflito de negócio, duplicidade, transição proibida ou GPU indisponível
422	Campo, janela, identificador ou vínculo inválido
503	Banco indisponível no health check ou serviço de geocodificação indisponível

Erros de negócio usam `detail` textual; validações de modelo usam uma lista `detail` com `loc`, `msg` e `type`. O relatório retorna JSON; a exportação CSV é feita pela interface. Não há endpoint de CSV.

README e execução da candidata

README atualizado:
https://github.com/alvccpj/rota-certa/blob/codex/sprint-10-release-candidate/README.md

O README identifica projeto, objetivo, tecnologias e os cinco integrantes. Inclui Git, Python 3.12+, Node.js 20.19+ ou 22.12+ e PostgreSQL 16+ como requisitos; GPU é opcional. Apresenta instalação com Docker ou manual, variáveis de ambiente, criação do banco, inicialização, testes e estrutura. O guia `COMO-RODAR.md` complementa os comandos para Windows.

Execução rápida com Docker, na raiz:

```
cp .env.example .env
docker compose up --build
```

O sistema abre em `http://localhost:5173`; a API, em `http://localhost:8000`. As contas de exemplo são `admin@rotacerta.com.br`, `atendente@rotacerta.com.br` e `entregador@rotacerta.com.br`, com senha `rotacerta123`. Em instalação manual, crie usuário e banco PostgreSQL, ajuste `backend/.env`, instale as dependências, execute `uvicorn app.main:app` em `backend` e `npm run dev` em `frontend`. A execução desta Sprint foi manual em PostgreSQL; Docker não foi revalidado nesta rodada.

A estrutura está organizada em `backend/app` (API e regras), `backend/tests` (testes), `frontend/src` (telas/componentes), `database` (DDL), `docs/api` (contrato e guia), `docs/sprint-*` (histórico), `tools` (roteiros) e `output/pdf` (documentos de entrega). Backend e frontend identificam a mesma versão RC.

Principais correções e ajustes

Registro	Ajuste	Comprovação
BUG-10	Contenção do rótulo absoluto na tabela; elimina overflow da página no celular	N13 e `interface_rc.json`
Pendência técnica S09	Dois testes concorrentes em PostgreSQL adicionados	Três lotes com oito chamadas e fila vazia; sem duplicação/carga excedida
Documentação de API	Guia, catálogo e contrato exportado	37 operações; contrato idêntico ao servido
Identificação da candidata	Versão `0.10.0-rc.1` no backend e frontend	`/openapi.json` e build
Reprodução dos testes	Ambiente permite schema/portas próprios; roteiro aceita destino de evidências e registra dispensas CUDA	Execução em `sprint10_testes` e códigos de saída de falha
README e banco	Instalação, estrutura, requisitos e estrutura final documentados	README, guia de execução e documentação do banco
Documento de entrega	Histórico preservado e Parte X acrescentada	PDF cumulativo das Sprints 01 a 10

As nove correções BUG-01 a BUG-09 da Sprint 09 permanecem cobertas pelas suítes e pelos casos repetidos. Não foi identificado outro defeito bloqueador nos cenários executados. Esta afirmação se limita à cobertura registrada.

Pendências e próximos passos

Pendência ou limite	Ação antes da apresentação final	Responsável pela coordenação
Aceitação com usuário externo	Realizar sessão guiada de cadastro, atribuição e entrega; registrar observações e corrigir bloqueios encontrados	Scrum Master com apoio da equipe
CUDA nesta candidata	Repetir F16 e benchmark na máquina NVIDIA; conferir rotas e métricas. A execução da Sprint 09 é histórica	Backend e otimização
Validação humana e integração	Revisar documento e pull request, integrar a candidata ao ramo principal e confirmar os links finais	Equipe e Scrum Master
Envio no Teams	Anexar somente o PDF cumulativo até 14/11/2026 às 23h59	Álvaro Jordão
Automação de interface	Avaliar CI com PostgreSQL e navegador; roteiro ainda é manualmente iniciado	Equipe

Continuam os limites de escopo aprovados nas Sprints 08 e 09: distância geográfica sem ruas/trânsito, previsões sem garantia de janelas, sem notificações externas e exportação em CSV. O ensaio concorrente não substitui teste de carga nem cobre todas as combinações de escritas. GPU pode ser mais lenta em instâncias pequenas; desempenho deve ser apresentado com a entrada, o hardware e o modo usados.

A candidata não deve receber ampliação de escopo antes da apresentação. Novas correções exigem repetir a verificação afetada e atualizar as evidências.

Reprodução da verificação

Com dependências instaladas, PostgreSQL disponível e Google Chrome instalado, ative o ambiente virtual e execute na raiz. Em Windows, substitua a ativação por `backend\.venv\Scripts\Activate.ps1`; os comandos Python são os mesmos.

```
source backend/.venv/bin/activate
pip install -r backend/requirements-docs.txt
python -m unittest discover backend/tests -v
export TEST_DATABASE_URL=postgresql+psycopg://rotacerta:rotacerta_dev@localhost:5432/rotacerta
python -m unittest discover backend/tests -v
unset TEST_DATABASE_URL
npm test --prefix frontend
npm run build --prefix frontend
python tools/documentation/export_api_docs.py
python tools/run_test_environment.py --schema sprint10_testes --api-port 8020 --web-port 5190
```

O último comando recria o schema descartável `sprint10_testes`. Para outro PostgreSQL, defina `POSTGRES_URL` antes de iniciá-lo. Em outro terminal, execute:

```
python tools/documentation/capture_sprint10_evidence.py
RC_DB='postgresql+psycopg://rotacerta:rotacerta_dev@localhost:5432/rotacerta'
DATABASE_URL="${RC_DB}?options=-csearch_path%3Dsprint10_testes" \
  python tools/documentation/verify_release.py
```

O exportador atualiza a documentação; o roteiro atualiza os registros e as capturas desta Sprint. A inspeção consulta o DDL e compara a API com SQL em outra conexão. Para reconstruir o PDF, execute `tools/documentation/build_sprint10_document.py` com as dependências de documento descritas no próprio script.

Roteiro de demonstração para a apresentação

• Entrar como administrador e abrir Visão geral; explicar indicadores e perfis.

• Cadastrar um cliente e um pedido com endereço e posição no mapa.

• Atribuir automaticamente; explicar capacidade, disponibilidade e pedidos não atribuídos.

• Gerar rotas em CPU; mostrar mapa, paradas, distância e previsão.

• Entrar como entregador no celular, iniciar a rota e confirmar a entrega.

• Conferir histórico, painel e relatório; exportar CSV do resultado filtrado.

• Abrir Swagger e executar uma consulta autenticada; mostrar o contrato versionado.

• Mostrar a comparação de desempenho nos modos disponíveis, sem prometer ganho para todo tamanho de entrada.

Consolidação para entrega

O documento único preserva as páginas 2 a 150 do PDF anterior, substitui a capa e acrescenta a Parte X com os resultados e evidências atuais. As partes anteriores permanecem como registros históricos; em caso de divergência, a Sprint 10 descreve o estado desta candidata. Os integrantes continuam identificados na capa e nesta parte.

Antes do envio, revisar o PDF, conferir o pull request e o README no GitHub. Somente o Scrum Master envia o documento cumulativo, sem necessidade de anexar arquivos de código ou PDFs separados na tarefa do Teams.

Catálogo resumido das APIs

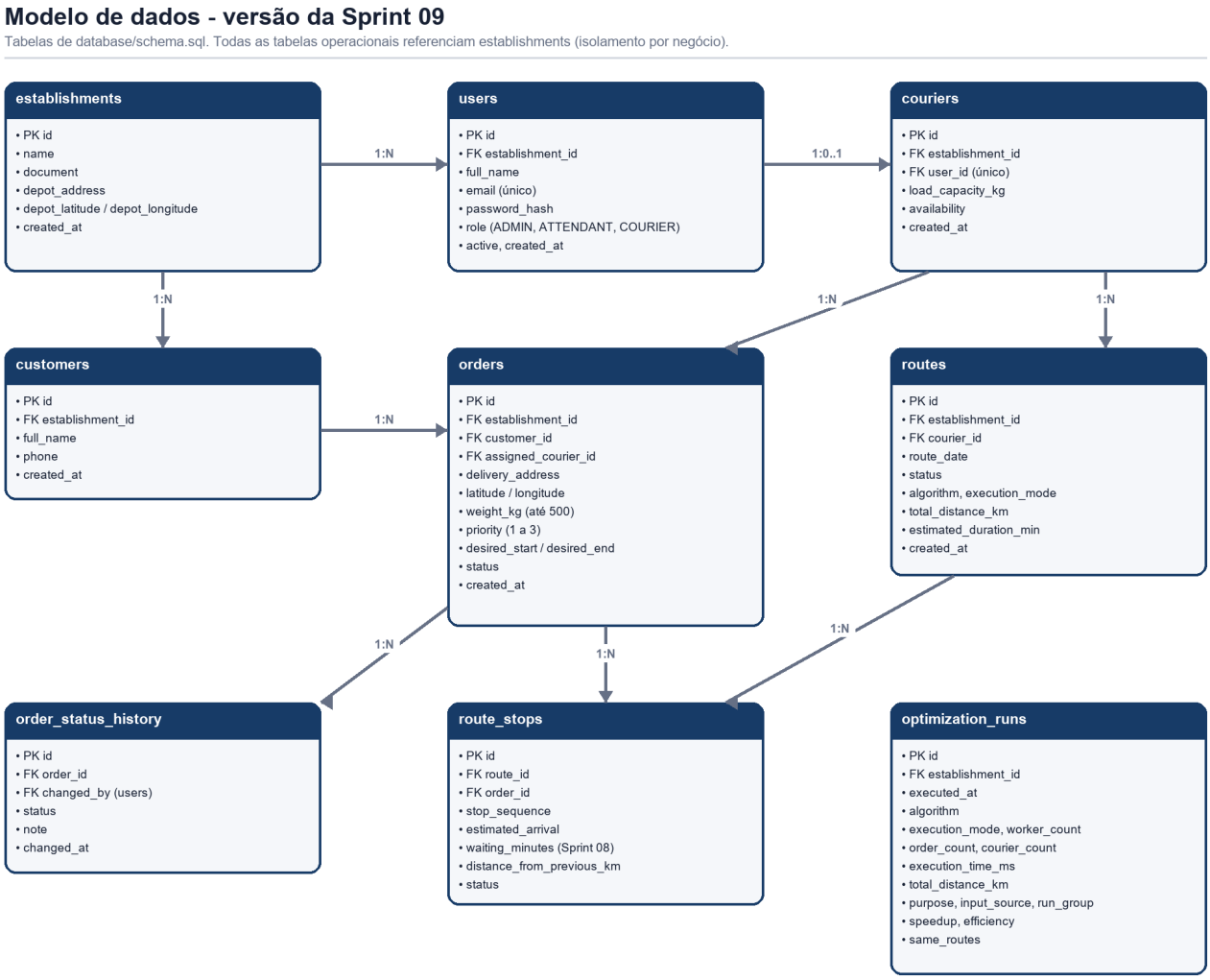
A = ADMIN; T = ATTENDANT; E = COURIER. Autenticado: qualquer perfil, respeitando propriedade e estabelecimento. As entradas são modelos JSON; IDs e filtros são detalhados no catálogo completo. As saídas indicam o modelo da resposta de sucesso.

Método	Rota	Acesso	Entrada	Resposta
POST	/auth/login	Público	LoginIn	200 TokenOut
POST	/auth/register	Público	RegisterIn	201 TokenOut
GET	/auth/me	Autenticado	-	200 UserOut
GET	/users	A	-	200 Lista UserOut
POST	/users	A	UserCreate	201 UserOut
GET	/users/{user_id}	A	-	200 UserOut
PUT	/users/{user_id}	A	UserUpdate	200 UserOut
DELETE	/users/{user_id}	A	-	204 Sem corpo
GET	/couriers	A, T	-	200 Lista CourierOption
PATCH	/couriers/me/availability	E	AvailabilityIn	200 CourierOut
GET	/customers	A, T	-	200 Lista CustomerOut
POST	/customers	A, T	CustomerIn	201 CustomerOut
GET	/customers/{customer_id}	A, T	-	200 CustomerOut
PUT	/customers/{customer_id}	A, T	CustomerIn	200 CustomerOut
DELETE	/customers/{customer_id}	A	-	204 Sem corpo
POST	/orders/auto-assign	A, T	-	200 AutoAssignOut
GET	/orders	Autenticado	-	200 Lista OrderOut
POST	/orders	A, T	OrderIn	201 OrderOut
GET	/orders/{order_id}	Autenticado	-	200 OrderDetailOut
PUT	/orders/{order_id}	A, T	OrderIn	200 OrderOut
DELETE	/orders/{order_id}	A	-	204 Sem corpo
PATCH	/orders/{order_id}/status	Autenticado	StatusIn	200 OrderOut
GET	/geocode	A, T	-	200 Lista GeocodeResult
GET	/establishment/depot	A, T	-	200 DepotOut
PUT	/establishment/depot	A	DepotIn	200 DepotOut
GET	/routes	A, T	-	200 RoutesOverviewOut
POST	/routes/generate	A, T	RouteGenerateIn	200 RouteGenerationOut

Método	Rota	Acesso	Entrada	Resposta
GET	/routes/me	E	-	200 MyRouteOut
PATCH	/routes/{route_id}/start	Autenticado	-	200 RouteOut
GET	/optimizer/capabilities	A, T	-	200 CapabilitiesOut
POST	/optimizer/benchmark	A	BenchmarkIn	200 BenchmarkOut
GET	/optimizer/runs	A	-	200 Lista OptimizationRunOut
POST	/optimizer/compare	Público	OptimizationRequest	200 Objeto
GET	/dashboard/summary	A, T	-	200 DashboardSummaryOut
GET	/dashboard/report	A, T	-	200 Lista DashboardReportRowOut
GET	/dashboard/history	A, T	-	200 Lista OperationalHistoryEventOut
GET	/health	Público	-	200 Objeto

Modelo de dados vigente na candidata

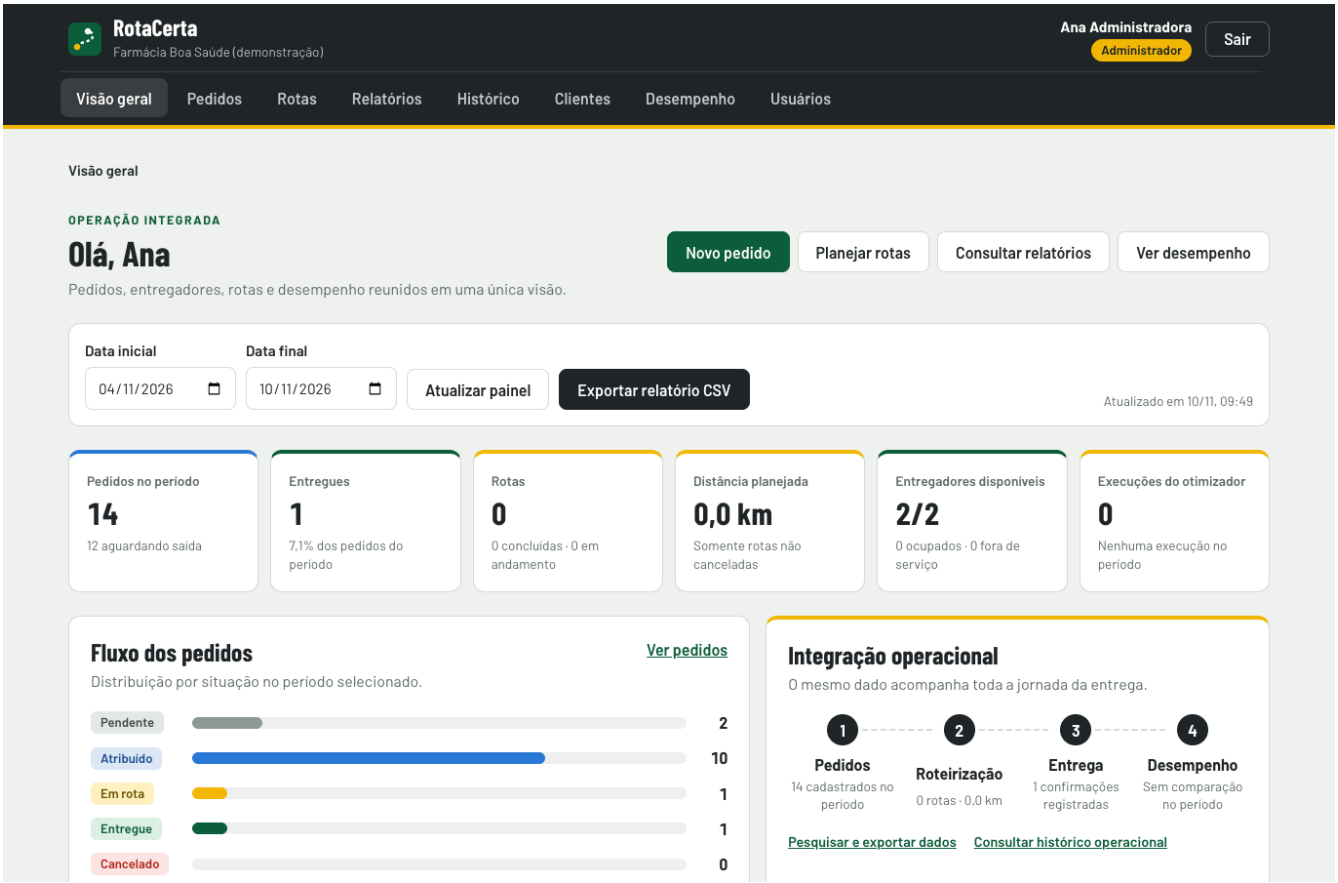
O diagrama atualizado na Sprint 09 permanece coerente com o DDL final inspecionado na Sprint 10. Não houve alteração de tabelas ou colunas nesta candidata.



Fonte técnica: database/schema.sql e modelos de backend/app/models.py. Inspeção atual: docs/sprint-10/evidencias/banco_e_contrato.json, com nove tabelas, 14 referências sem órfãos e nenhuma inconsistência nas quatro consultas de isolamento.

Acesso e painel operacional

Evidência F01 - Login com credenciais válidas.



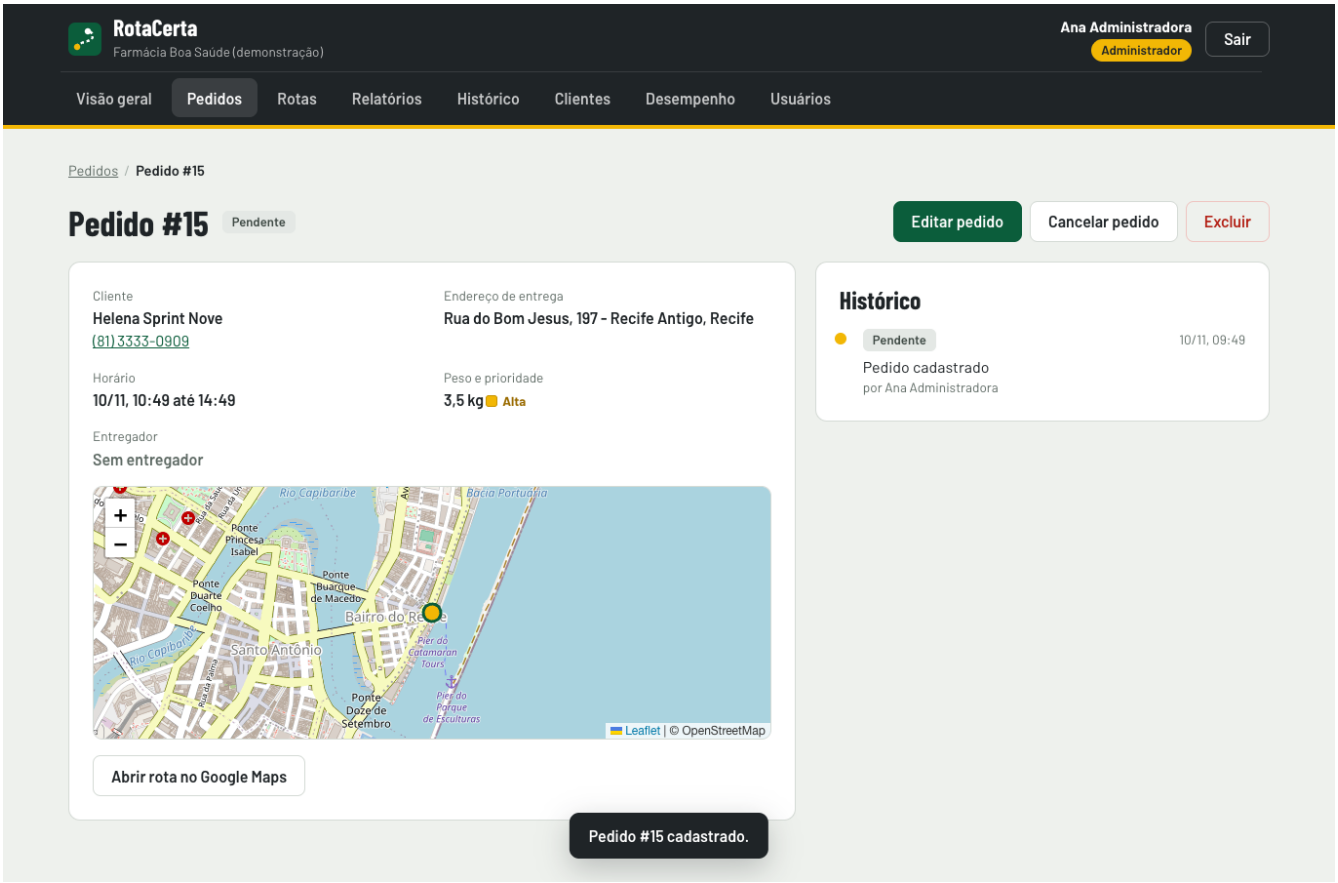
Procedimento: Abrir /login; Informar admin@rotacerta.com.br e a senha de demonstração; Clicar em Entrar.

Resultado observado: Redirecionado para /painel; saudação 'Olá, Ana'; /auth/me confirma admin@rotacerta.com.br (ADMIN).

Captura realizada no sistema executado com PostgreSQL em 10/11/2026. O resultado completo e as verificações da API estão no registro dos casos.

Cadastro de pedido integrado ao banco

Evidência F07 - Cadastro de pedido.



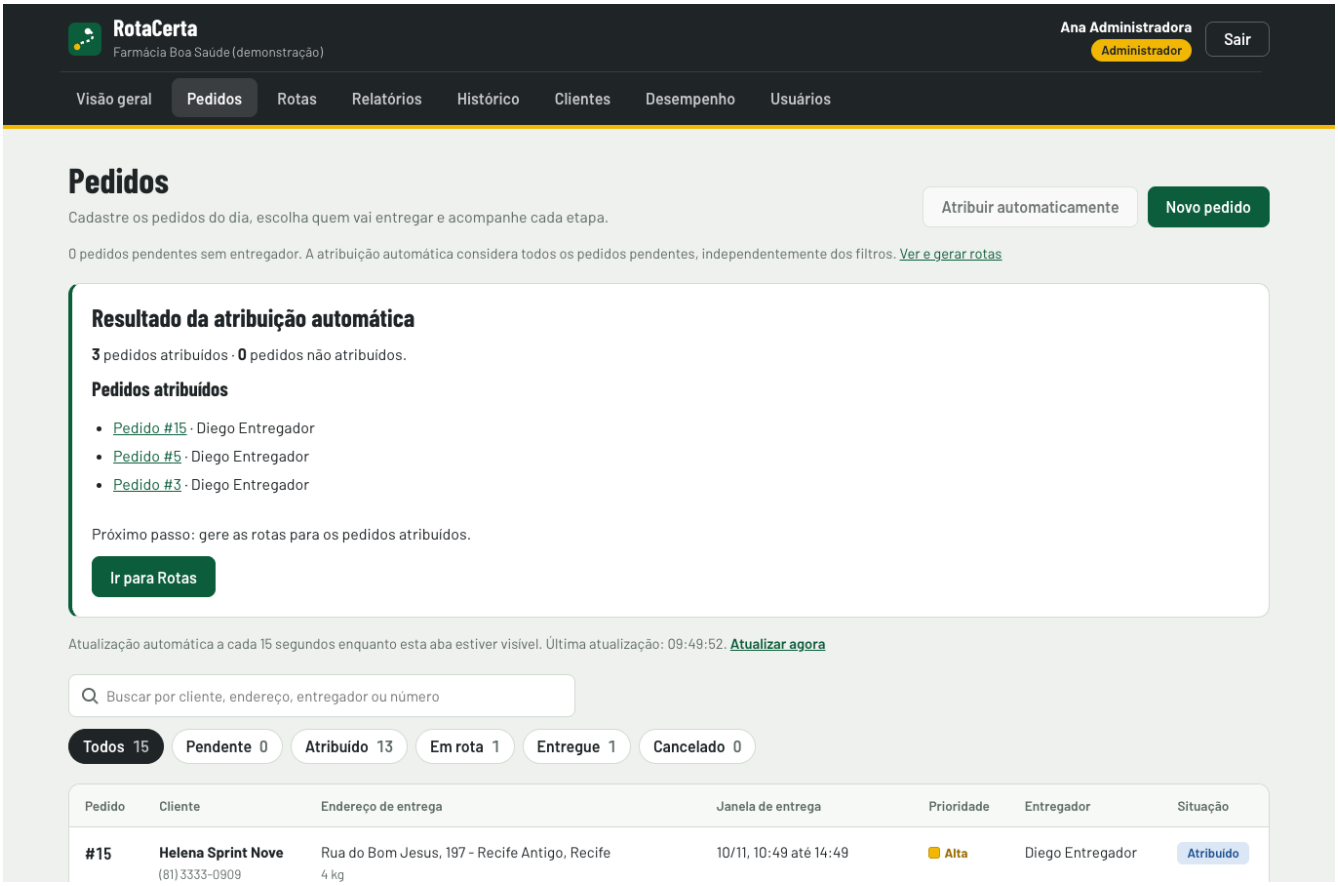
Procedimento: Abrir Pedidos > Novo pedido; Escolher o cliente, endereço, latitude/longitude, peso '3,5', prioridade Alta e janela futura; Clicar em Cadastrar pedido.

Resultado observado: Pedido #15: situação PENDING, peso 3.5 kg, prioridade 1.

Captura realizada no sistema executado com PostgreSQL em 10/11/2026. O resultado completo e as verificações da API estão no registro dos casos.

Resultado da atribuição automática

Evidência F14 - Atribuição automática.



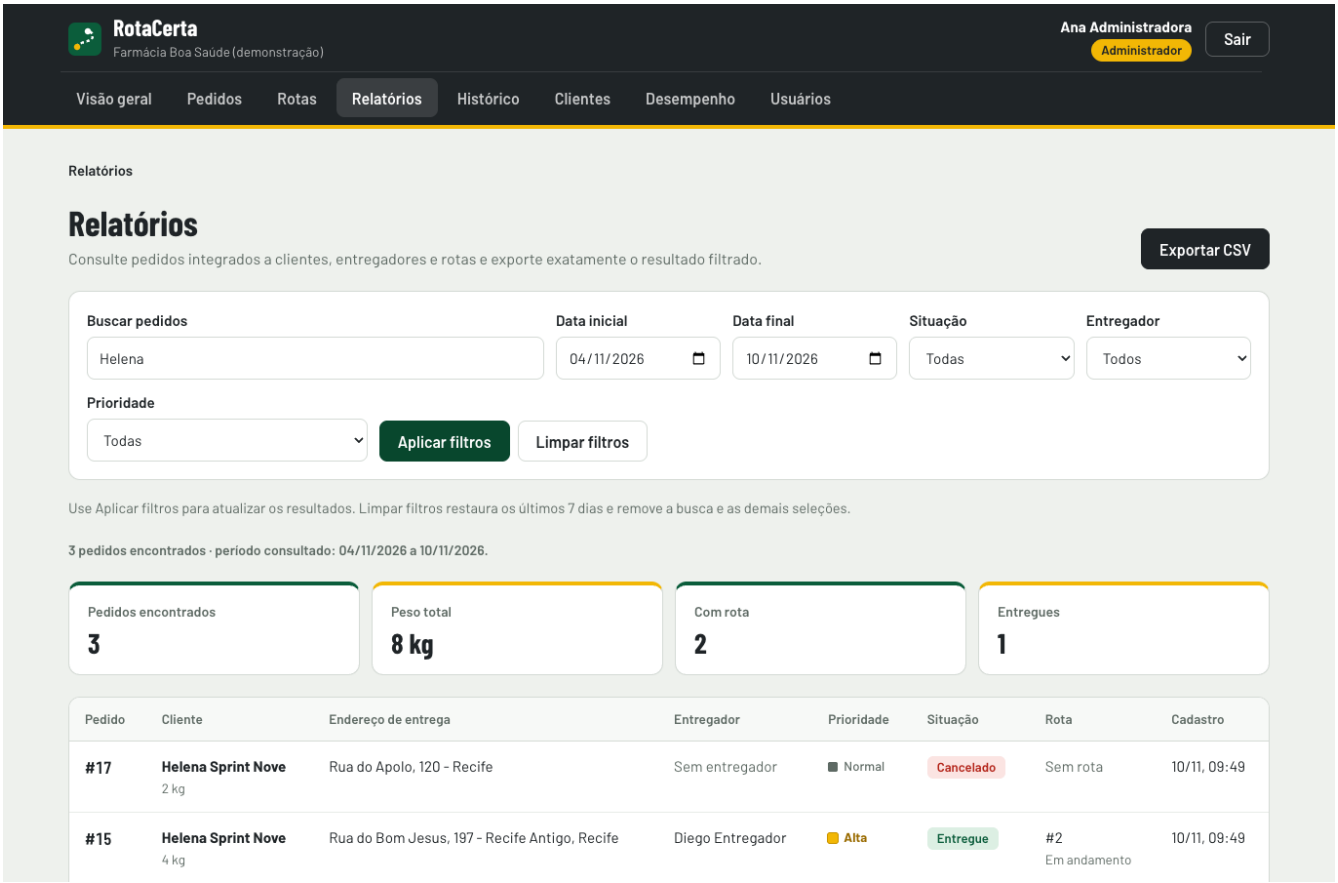
Procedimento: Em Pedidos, clicar em Atribuir automaticamente; Confirmar em Atribuir pedidos.

Resultado observado: Pedido #15 atribuído a Diego Entregador; histórico: 'Atribuído automaticamente a Diego Entregador'.

Captura realizada no sistema executado com PostgreSQL em 10/11/2026. O resultado completo e as verificações da API estão no registro dos casos.

Relatório e exportação

Evidência F20 - Relatório filtrado e exportação CSV.



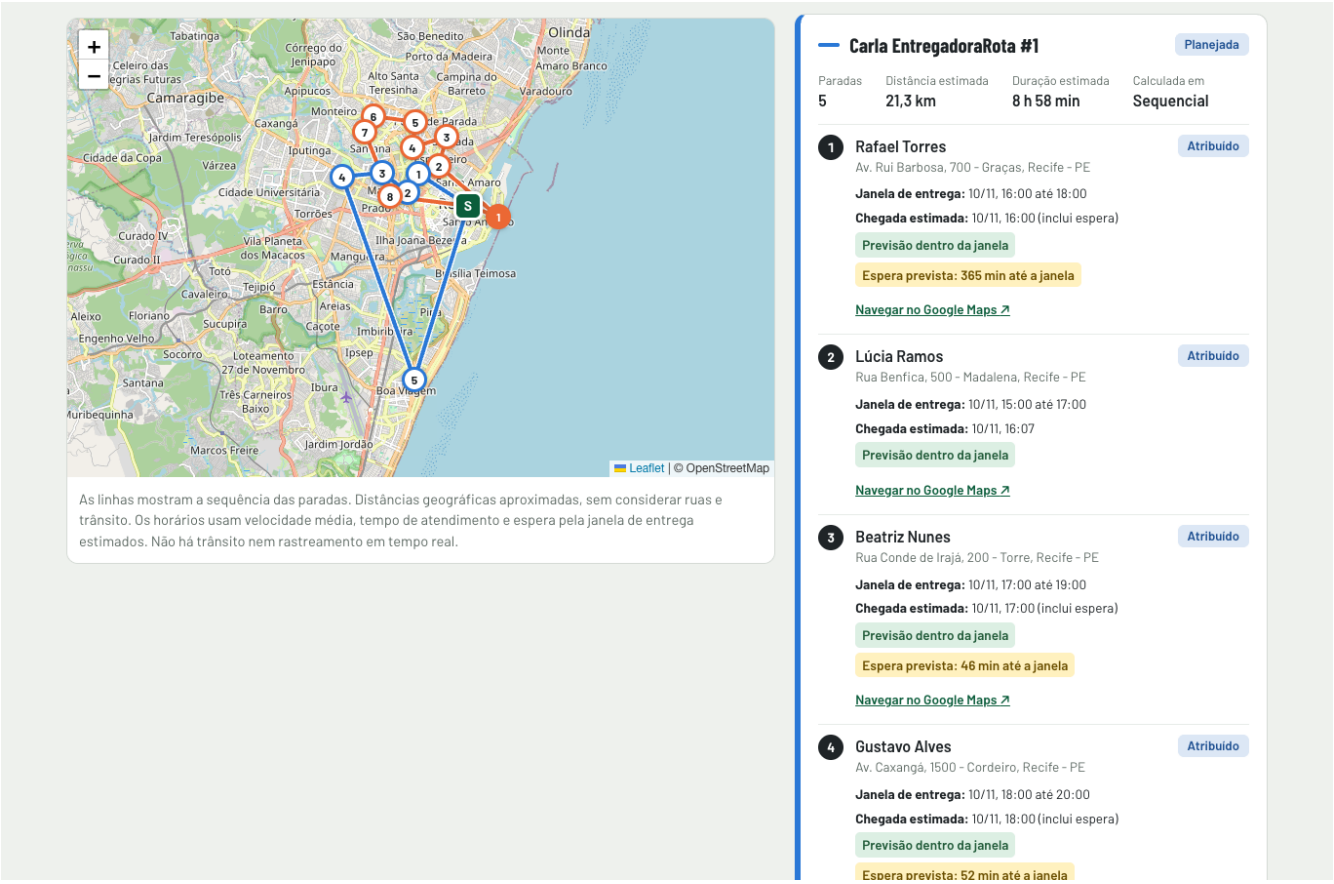
Procedimento: Abrir Relatórios; Buscar 'Helena' e Aplicar filtros; Clicar em Exportar CSV.

Resultado observado: 3 linha(s) na tela e 3 no CSV 'rota-certa-relatorio-2026-11-04-a-2026-11-10.csv', todas da cliente Helena.

Captura realizada no sistema executado com PostgreSQL em 10/11/2026. O resultado completo e as verificações da API estão no registro dos casos.

Rotas integradas ao mapa e às paradas

Consulta da candidata após o roteiro funcional, com trajetos e paradas persistidos.

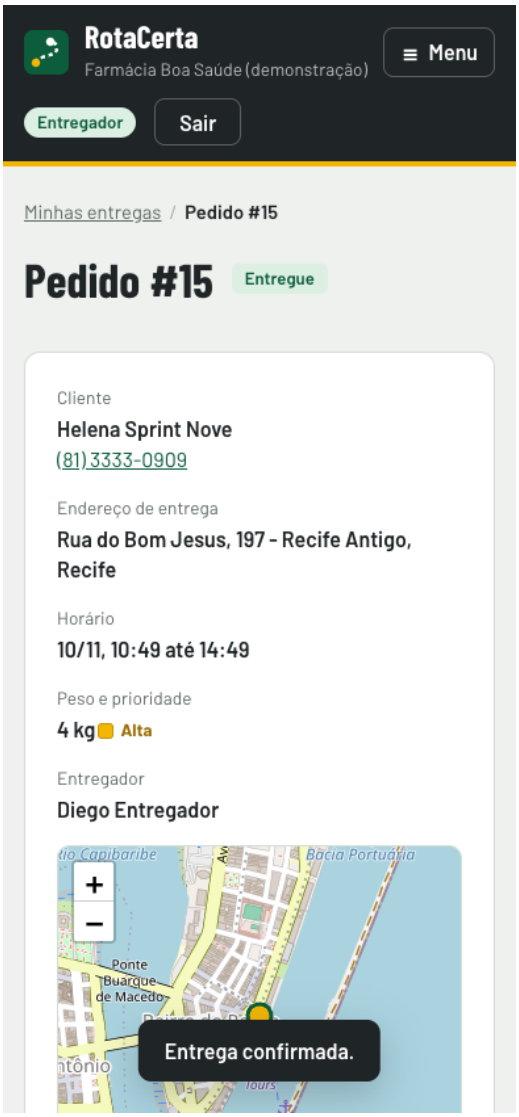


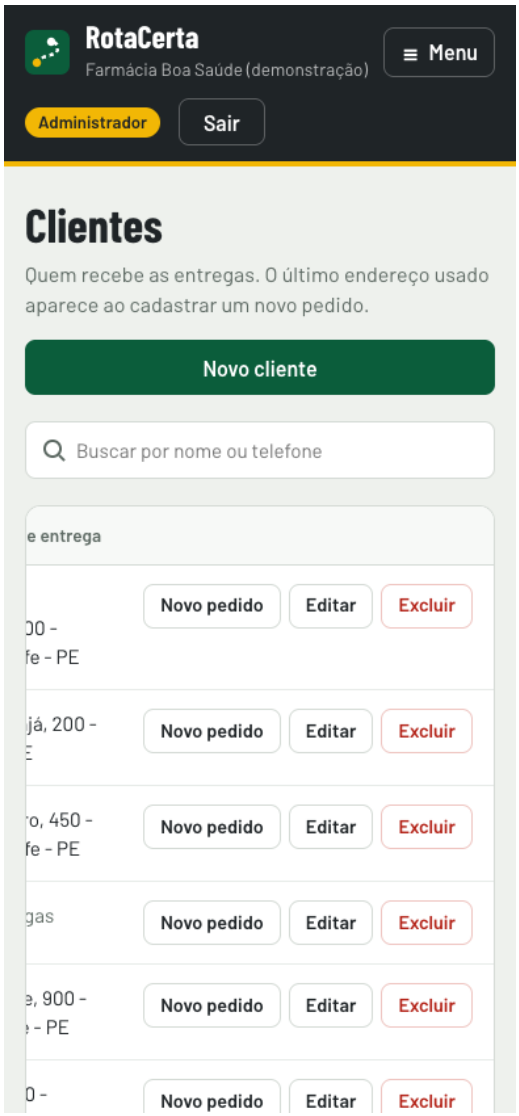
Os trajetos ligam as coordenadas e mostram a sequência de atendimento. São distâncias geográficas; não representam navegação por ruas ou trânsito em tempo real.

Resultado de F15: 2 rotas SEQUENTIAL. Carla Entregadora: 5 paradas, 21.307 km, Diego Entregador: 8 paradas, 15.934 km.

Entrega e navegação no celular

À esquerda, confirmação de entrega pelo responsável. À direita, tabela de clientes rolada internamente até as ações, após a correção do BUG-10.



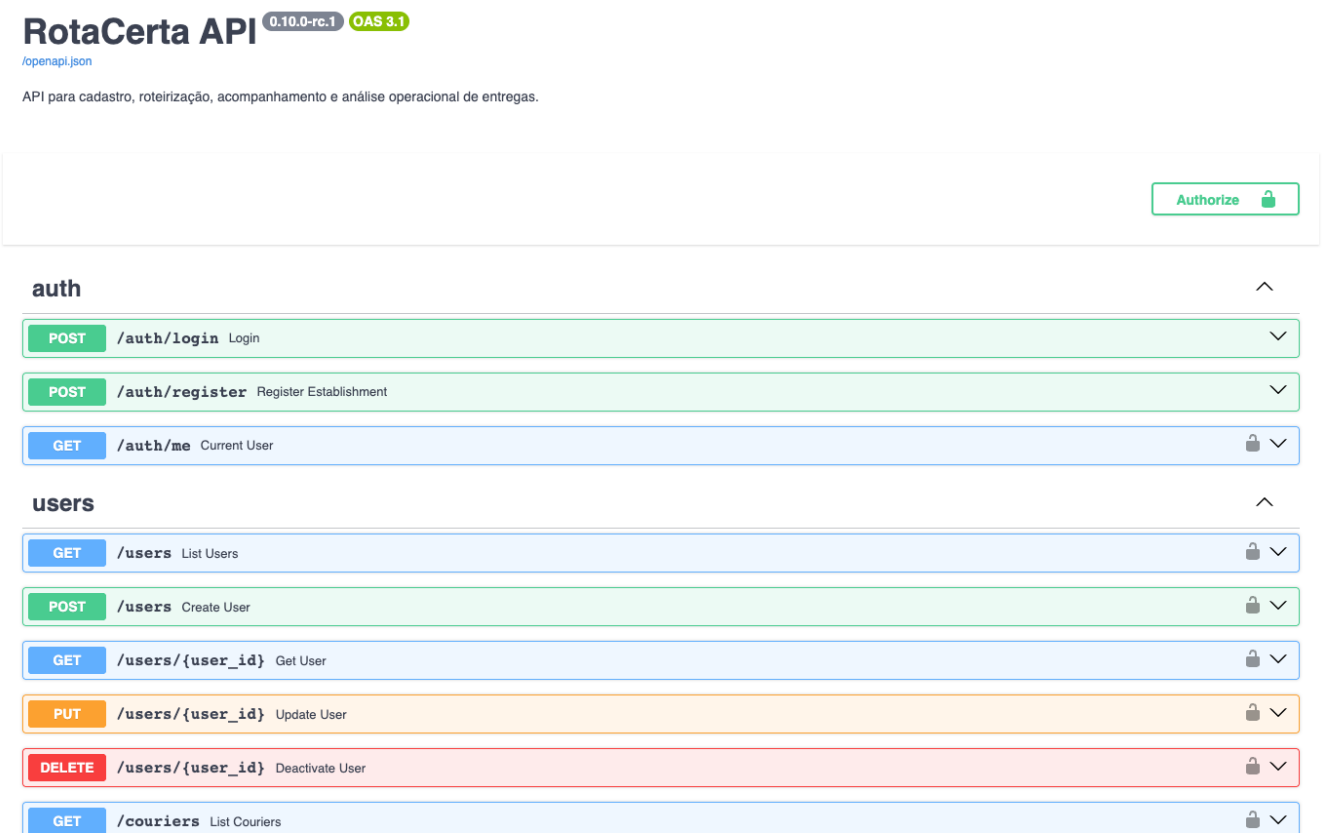


F17: Pedido #15 DELIVERED; último evento por Diego Entregador em 2026-11-10T09:49:54.603769-03:00.

A regressão adicional mediu documento e viewport iguais em 320, 390 e 768 px; a tabela de 981 px permaneceu dentro do contêiner, com botões acessíveis por rolagem.

Documentação interativa da API

Swagger carregado a partir da aplicação candidata, versão 0.10.0-rc.1.



O guia no repositório inclui autenticação Bearer, exemplos, perfis e erros. O catálogo e o contrato versionado documentam as 37 operações, seus parâmetros e modelos. A inspeção automática confirmou igualdade entre o JSON exportado e o servido.

Parte XI - Sprint 11

Preparação para entrega

Grupo 04 - RotaCerta | UNINASSAU | Ciência da Computação | Turma 8NA | 2026.2

Documentação e verificação concluídas em 10/11/2026. Prazo da atividade: 21/11/2026 às 23h59, horário de Recife. Versão 0.11.0-rc.1, preparada na branch `codex/sprint-11-preparacao-entrega`, a partir de `043b6a4`, com a Sprint 10 já integrada à master. Repositório público: https://github.com/alvccpj/rota-certa

Identificação da equipe

Integrante	Matrícula	Responsabilidade inicial registrada
Álvaro Jordão	01748200	Scrum Master e Product Owner; organização, requisitos e priorização
Arthur Sales	01593811	Backend e núcleo de otimização e paralelização
Vinícius Trigueiro	01794959	Frontend, banco de dados e documentação
William Coelho de Morais	01263977	Desenvolvimento e testes
Guilherme de Luna Barros	01615925	Desenvolvimento e validação do sistema

As responsabilidades mantêm a identificação histórica da equipe. As contribuições efetivas devem ser acompanhadas pelo histórico Git e pela revisão dos integrantes. Somente o Scrum Master Álvaro Jordão realiza o envio no Teams.

Objetivo e resultado da Sprint

A Sprint 11 prepara o projeto para outra pessoa compreender, executar e usar o sistema. Foram elaborados o Manual do Usuário ilustrado e o Manual Técnico, consolidados requisitos/casos de uso/regras e revisados os principais módulos do código, sua configuração e organização no GitHub.

A revisão resultou em duas correções de comportamento, limites para o comparador livre, retirada da senha de demonstração dos logs e ajustes nos roteiros para preservar evidências de Sprints encerradas. Os testes foram repetidos na versão atual, e os manuais descrevem somente recursos implementados.

O documento de entrega reúne todas as Sprints anteriores e a Parte XI. Os dois manuais estão incluídos integralmente no PDF e também disponíveis no repositório. Não é necessário ter a aplicação publicada para ler os materiais.

Entregas obrigatórias e documentos

Entrega	Material preparado	Localização
Manual do Usuário	Acesso, perfis, telas, cadastros, atribuição, rotas, entrega, consultas, mensagens e roteiro de treinamento, com capturas atuais	docs/manuais/MANUAL_DO_USUARIO.md
Manual Técnico	Arquitetura, tecnologias, módulos, ambiente, instalação, banco, variáveis sem valores privados, APIs, otimização, testes e manutenção	docs/manuais/MANUAL_TECNICO.md
Documentação final	RF01-RF10, RNF01-RNF07, UC01-UC07, regras, decisões e diagramas vigentes	docs/final/DOCUMENTACAO_FINAL.md
Revisão do código	Escopo por módulo, defeitos reproduzidos, correções, melhorias e validações	docs/sprint-11/REVISAO_CODIGO.md
Organização do GitHub	Índice da documentação, README atualizado, exemplo frontend, modelo de PR e política de arquivos/evidências	docs/README.md, README.md e CONTRIBUTING.md

No GitHub, todos os materiais desta entrega podem ser acessados a partir de: https://github.com/alvccpj/rota-certa/tree/codex/sprint-11-preparacao-entrega/docs

Os links permanecem vinculados à branch da entrega; após integração, os mesmos caminhos ficam disponíveis na master. O PDF único está em `output/pdf/Grupo_04_RotaCerta_Sprints_01_a_11.pdf`.

Como os materiais foram preparados

O Manual do Usuário foi escrito a partir dos nomes de campos, botões e menus da aplicação. As telas foram capturadas no sistema executado, com dados de demonstração. O texto explica o resultado esperado das ações e como resolver as mensagens comuns, sem exigir conhecimento de API ou banco para a operação diária.

O Manual Técnico foi conferido com os arquivos de configuração, dependências, roteadores, modelos e serviços. Ele distingue instalação manual, Compose e demonstração SQLite; descreve variáveis sem incluir uma senha privada real e explica que o processamento é uma heurística, sem modelo de IA treinado.

A consolidação mantém os identificadores originais dos requisitos e casos de uso. Os diagramas de arquitetura e classes foram refeitos conforme a implementação. O MER da Sprint 09 permanece válido e foi conferido contra o banco atual. Planejamentos antigos, como OSRM e possíveis notificações, são identificados como decisões históricas ou limites, sem ser apresentados como recursos disponíveis.

Ambiente e resultados da verificação

Execução em 10/11/2026, em macOS ARM64, Python 3.14.7 e PostgreSQL 17.10, num cluster separado da base de desenvolvimento. A aplicação usou o schema descartável `sprint11_testes`, API 8021 e frontend 5191. Os casos de navegador usaram Chrome/Playwright em português e fuso de Recife. Os registros abaixo estão em `docs/sprint-11/evidencias`; as capturas, em `docs/sprint-11/assets`.

Verificação	Resultado	Registro
Backend SQLite	159 descobertos, 156 aprovados e 3 dispensados	testes_backend_sqlite.txt
Backend PostgreSQL	159 descobertos, 158 aprovados e 1 dispensado	testes_backend_postgresql.txt
Frontend	55 aprovados, nenhuma falha	testes_frontend.txt

Verificação	Resultado	Registro
Aplicação no navegador	60 aprovados, nenhuma falha e 1 não executado	casos_de_teste.json e execucao_casos.txt
Interface adicional	Sem overflow em 320, 390 e 768 px; ações da tabela acessíveis	interface_rc.json
Build de produção	TypeScript e Vite concluídos	build.txt
Auditoria npm	Zero vulnerabilidades reportadas na rodada	auditoria_npm.json
Banco e contrato	Nove tabelas, 14 FKs sem órfãos e 37 operações sincronizadas	banco_e_contrato.json
Backup/restauração	Cópia restaurada em outro banco descartável; estrutura e dados comparados	backup_restauracao.json
Configuração Compose	YAML e interpolação validados; containers não reexecutados	compose.txt
Organização e links	Proteção do .gitignore, arquivos locais e padrões explícitos conferidos	organizacao_github.json

No backend, o teste CUDA é dispensado nesta máquina sem NVIDIA; no SQLite, também são dispensados os dois testes específicos de concorrência PostgreSQL. No navegador, F16 é o único não executado. Os sete testes novos de backend e os cinco casos adicionais de CSV já estão incluídos nos totais. O comportamento CUDA validado na Sprint 09 é histórico, não uma nova execução nesta candidata.

O procedimento de instalação manual foi exercitado no PostgreSQL e as instruções de backup/restauração foram testadas em dados descartáveis. As capturas mostram o produto executado, não protótipos. A execução automatizada não é substituta de um teste de aceitação com pessoa externa ao desenvolvimento.

Principais correções e melhorias

Registro	Problema ou necessidade	Ajuste e resultado
BUG-11	Resposta externa de geocodificação fora do formato podia causar erro ou coordenadas inválidas no mapa	Validação de estrutura/posição e retorno 503 com orientação para marcar manualmente
BUG-12	O utilitário CSV não neutralizava prefixos de fórmula precedidos por espaços	Regra ampliada, com cinco regressões passando
AJ11-01	Comparador público sem limite de tamanho de entrada	Até 64 rotas, 500 paradas por rota e 12.000 no total; excessos retornam 422
AJ11-02	Senha de demonstração impressa na inicialização	Mensagens passam a indicar apenas a criação do seed ou o guia de acesso
AJ11-03/04	Roteiros de documentação podiam gravar em pasta histórica por padrão	Destino explícito de catálogo e opção de destino da inspeção
AJ11-05/06	Documentos dispersos e classes antigas de planejamento	Índice vigente, dois manuais completos, consolidação e diagramas atuais

A revisão detalhada e as reproduções anteriores às correções estão em `REVISAO_CODIGO.md` e nos arquivos `regressoes_antes_*`. Esses arquivos registram falhas deliberadamente reproduzidas no código anterior. Os registros finais `testes_*` e `casos_de_teste.json` não apresentam falhas executadas.

Organização do repositório

Código permanece em backend/frontend; o DDL, em database; manuais e contratos, em docs; os geradores e executores, em tools. A raiz identifica projeto, objetivo, tecnologias, instalação, estrutura e integrantes. Foi incluído exemplo de configuração do frontend, sem segredo, e revisado o modelo de PR para registrar validação.

Arquivos .env, bases locais, caches, ambientes virtuais, node_modules, builds e materiais temporários de QA são ignorados. PDFs cumulativos, imagens usadas na documentação e registros anteriores permanecem como histórico intencional. O verificador não imprime valores de credenciais e registra seus limites: ele não examina todo o histórico Git nem todos os possíveis formatos de segredo.

O repositório foi confirmado como público. Os manuais e diagramas podem ser consultados pela avaliação no GitHub. As alterações devem ser organizadas em commits de correção e de documentação, com pull request para revisão humana. Não são atribuídas contribuições fictícias a outros integrantes.

Pendências e próximos passos

Pendência	Próximo passo	Coordenação
Aceitação com usuário externo	Usar o roteiro final do Manual do Usuário, registrar dúvidas e corrigir bloqueios reais	Scrum Master e equipe
Revalidação CUDA	Repetir F16 e comparação em máquina NVIDIA, com versões/hardware registrados	Backend e otimização
Revisão humana	Conferir os manuais, PDF e PR com outro integrante antes da integração	Equipe
Envio no Teams	Anexar somente o PDF cumulativo até 21/11/2026 às 23h59	Álvaro Jordão
Demonstração final	Ensaiar cadastro, atribuição, rota, entrega, relatório e explicação do algoritmo	Toda a equipe

Continuam os limites de escopo: trajetos geográficos sem trânsito, previsões sem garantia de janelas, exportação CSV e ausência de notificações externas/GPS ao vivo. Carga elevada, múltiplos navegadores e combinações de escritas ainda precisam de homologação específica. A infraestrutura atual não é apresentada como uma publicação de produção pronta com HTTPS, rate limit e alta disponibilidade.

Reprodução e entrega

Os comandos completos estão no Manual Técnico. A sequência de verificação é: suíte SQLite, suíte PostgreSQL com TEST_DATABASE_URL, `npm test`, build, ambiente isolado da Sprint 11, roteiro de navegador e inspeção de banco/contrato. O exportador da API e o gerador dos diagramas mantêm as fontes reproduzíveis.

O PDF é produzido por `tools/documentation/build_sprint11_document.py` usando os documentos e capturas versionados. A capa é atualizada; as páginas 2 a 169 do PDF anterior são preservadas. A Parte XI começa na página 170 e reúne este registro, os dois manuais completos, a documentação final e a revisão de código.

Após revisar o material, somente o Scrum Master envia o arquivo `Grupo_04_RotaCerta_Sprints_01_a_11.pdf` na tarefa do Teams. Não é necessário anexar manuais ou Sprints em arquivos separados.

Manual do Usuário do RotaCerta

Grupo 04 | UNINASSAU | Ciência da Computação | Turma 8NA | 2026.2. Versão 0.11.0-rc.1, revisada em 10/11/2026.

1 Apresentação e público

O RotaCerta ajuda pequenos negócios, como farmácias, mercados e restaurantes, a organizar suas entregas. Nele, a equipe cadastra clientes e pedidos, escolhe os entregadores, calcula a ordem das paradas e acompanha cada entrega.

Este manual se destina a administradores, atendentes e entregadores. O cliente que recebe o pedido não tem uma área de acesso ao sistema. As imagens foram capturadas na aplicação em funcionamento, com dados de demonstração. Nomes, números de pedidos, datas e totais podem ser diferentes na sua operação.

Para começar uma operação nova, o administrador deve cadastrar a equipe e marcar o ponto de saída das entregas. Depois, o fluxo usual é: cadastrar cliente, cadastrar pedido, atribuir entregador, gerar rota e confirmar a entrega.

2 Acesso e autenticação

Abra no navegador o endereço informado pelo responsável pelo sistema. Na instalação local padrão, use http://localhost:5173. A aplicação e o servidor precisam estar ligados; a instalação é descrita no Manual Técnico.

1. Digite seu e-mail no campo E-mail.

2. Digite sua senha no campo Senha.

3. Clique em Entrar.

4. Administradores e atendentes chegam à Visão geral; entregadores, a Minhas entregas.

Use as credenciais fornecidas pelo administrador. Os acessos de demonstração estão identificados no README do projeto e servem apenas ao ambiente de exemplo.

Tela de entrada com os campos E-mail e Senha e o botão Entrar.

Se você ainda não tem um negócio cadastrado, escolha Cadastrar meu negócio na tela de entrada. Informe nome do negócio, endereço de saída, seu nome, e-mail e senha. A senha precisa ter de 8 a 72 caracteres, com letras e números. Clique em Criar conta. Essa ação cria outro negócio e seu primeiro administrador; para entrar em uma equipe existente, peça ao administrador que cadastre você.

Para sair, clique em Sair no topo. Se aparecer Sua sessão expirou, entre novamente; o sistema tentará retornar à página que você estava consultando. Se esqueceu a senha, solicite ao administrador a alteração do seu usuário. Não há recuperação de senha por e-mail nesta versão.

3 O que cada perfil pode fazer

Recurso	Administrador	Atendente	Entregador
Visão geral, clientes, relatórios e histórico	Sim	Sim	Não
Cadastrar e editar pedidos abertos	Sim	Sim	Não
Atribuir pedidos e gerar rotas	Sim	Sim	Não
Excluir pedido ou cliente quando permitido	Sim	Não	Não
Gerenciar usuários e ponto de saída	Sim	Não	Não
Comparar desempenho do cálculo	Sim	Não	Não
Minhas entregas e própria disponibilidade	Não nesta tela	Não nesta tela	Sim
Consultar pedido e atualizar entrega própria	Gestão do negócio	Gestão do negócio	Somente pedidos atribuídos a ele

O menu mostra as opções do seu perfil. Cada negócio acessa seus próprios dados. Um entregador vê apenas os pedidos atribuídos a ele. Se uma opção não aparecer ou houver aviso de acesso negado, procure o administrador.

4 Navegação e leitura das telas

No computador, use os itens abaixo da marca RotaCerta. A seção atual fica destacada. No celular, toque em Menu, escolha a seção e o menu será recolhido. Use os links acima dos formulários para voltar à lista ou ao pedido.

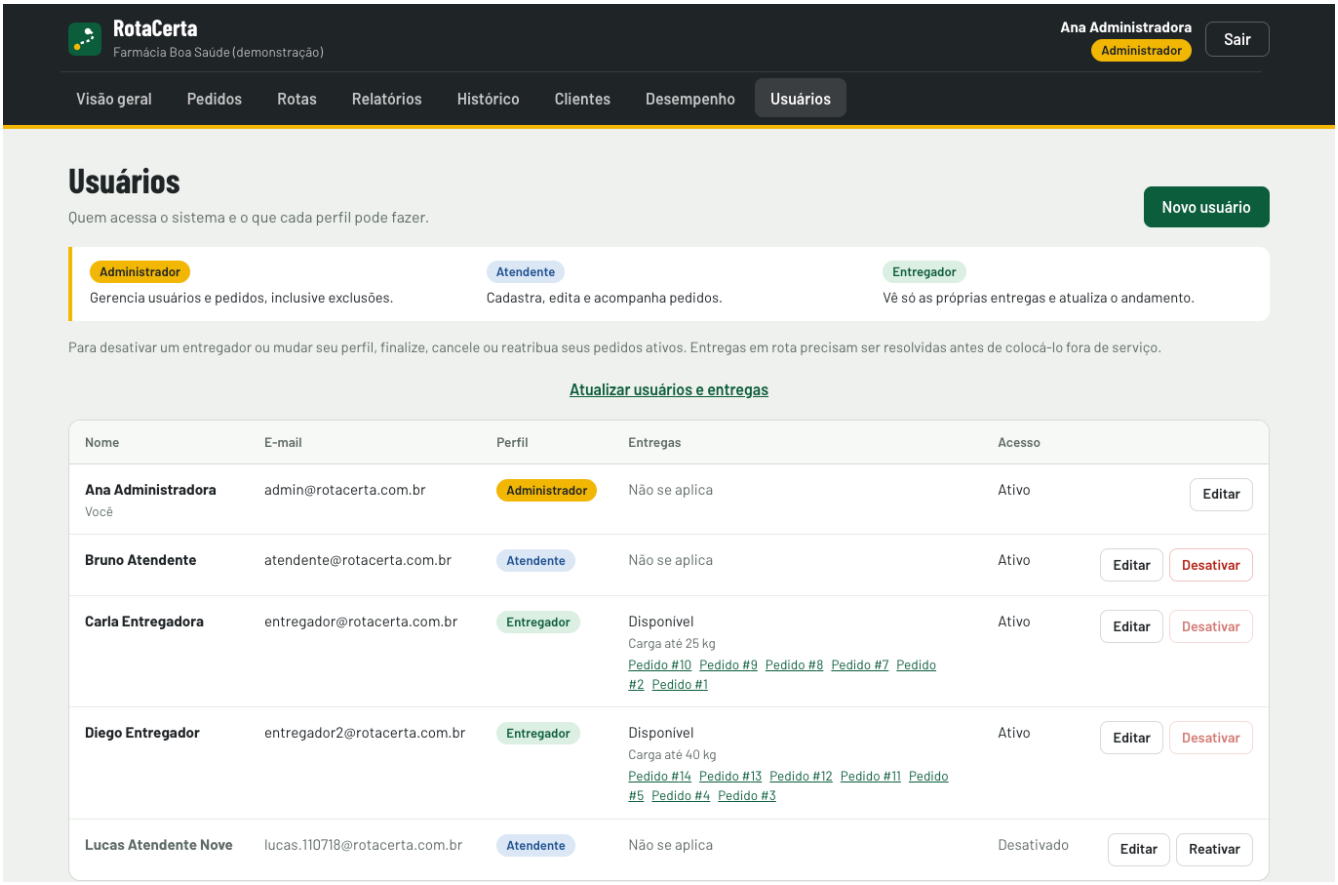
Tabelas largas podem ser deslizadas para o lado dentro de sua própria área. Em Clientes, isso permite alcançar Novo pedido, Editar e Excluir. A página continua ajustada à largura do celular. Alterações só são gravadas ao clicar no botão de salvar ou confirmar; Cancelar abandona o formulário.

Pedidos e Minhas entregas consultam atualizações a cada 15 segundos com a aba visível. A hora da última atualização aparece na tela. Clique em Atualizar agora quando precisar conferir imediatamente. Relatórios e histórico usam Aplicar filtros.

5 Preparar a equipe e o ponto de saída

Como administrador, abra Usuários e clique em Novo usuário. Preencha nome, e-mail e senha e escolha Administrador, Atendente ou Entregador. Para entregador, informe capacidade de carga maior que zero e de até 1.000 kg. Salve o cadastro. Entregadores novos começam disponíveis.

Em Usuários, Editar permite corrigir os dados, o perfil e a capacidade. Para trocar uma senha, preencha Nova senha; deixe o campo vazio para preservar a atual. Desativar retira o acesso, preservando os registros. Reativar permite acesso novamente. O administrador não pode desativar o próprio usuário.



Usuários com perfis, disponibilidade e ações de edição e ativação.

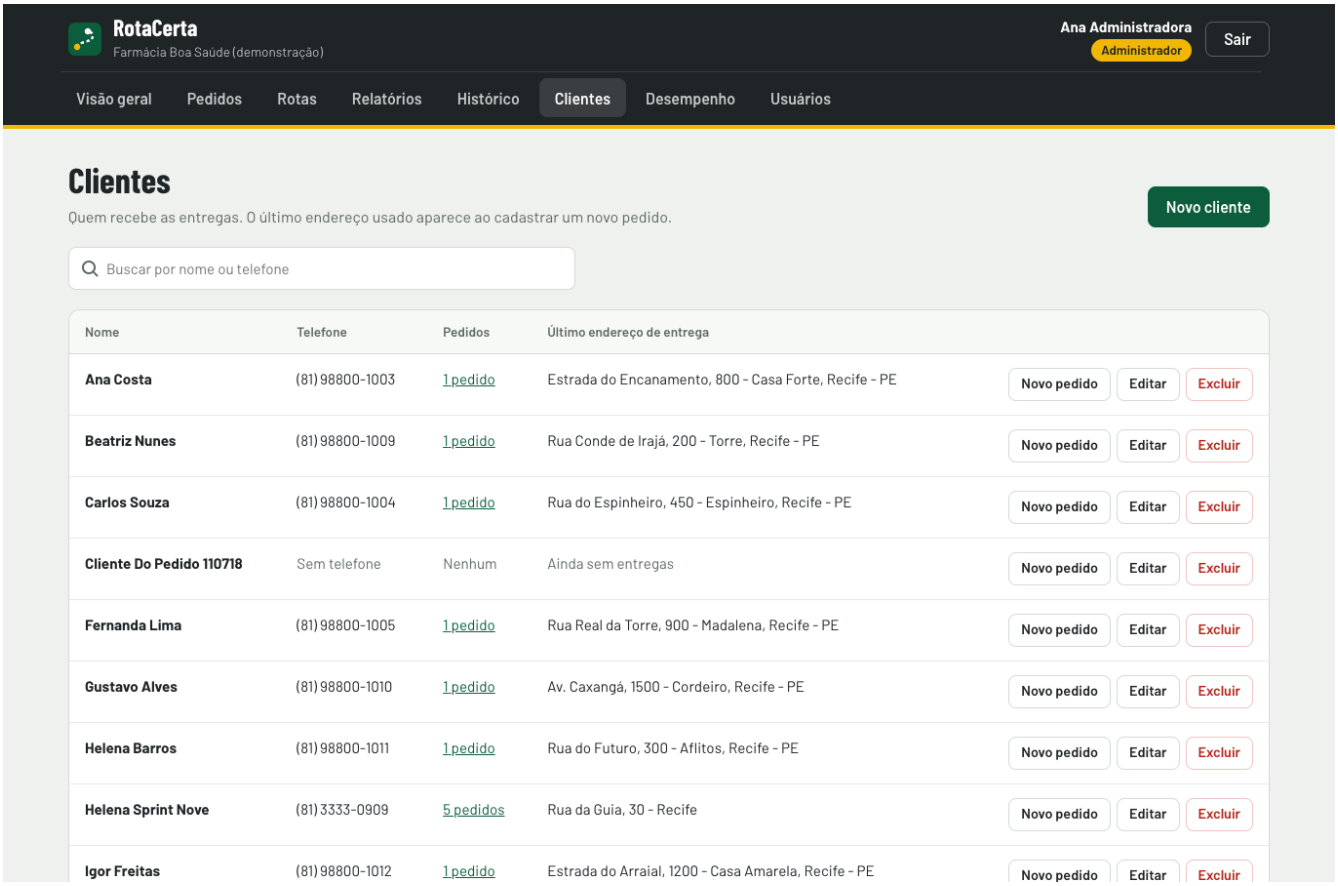
Antes de desativar ou retirar o perfil de um entregador, resolva seus pedidos atribuídos e em rota. A tela mostra links para os pedidos ativos. Para colocar fora de serviço, primeiro resolva os pedidos em rota. A capacidade não pode ficar abaixo da carga dos pedidos ativos.

Em Rotas, o administrador configura o endereço e a posição do ponto de saída no mapa. Use a busca ou clique na posição correta e salve. As rotas saem e retornam a esse ponto. Uma alteração substitui os planejamentos ainda não iniciados; enquanto houver rota em andamento, o sistema impede a troca.

6 Cadastrar e consultar clientes

1. Abra Clientes e clique em Novo cliente.
2. Informe Nome e, se desejar, Telefone com DDD.
3. Clique em Cadastrar cliente.
4. Confira a mensagem de cadastro e o cliente na lista.

A busca aceita nome ou telefone. Editar permite corrigir os dados. Novo pedido, na linha do cliente, abre o formulário com o cliente selecionado. O último endereço usado pode vir preenchido; confirme se é o endereço desta entrega.



Clientes com busca, total de pedidos, último endereço e ações.

Não é permitido cadastrar novamente a mesma combinação de nome e telefone. Somente o administrador pode excluir um cliente, e apenas quando ele não tem pedidos vinculados. Para um cliente com histórico, mantenha o cadastro.

7 Cadastrar um pedido

Abra Pedidos e clique em Novo pedido. Os campos obrigatórios são cliente, endereço, local no mapa e peso. Role o formulário para alcançar as demais seções.

1. Escolha o cliente. Se ainda não existir, clique em Cadastrar cliente; ao salvar, você volta ao pedido.

2. Informe rua, número, bairro e cidade em Endereço.

3. Clique em Buscar endereço, escolha um resultado e confirme o ponto no mapa. Você também pode marcar o local diretamente no mapa ou preencher latitude e longitude.

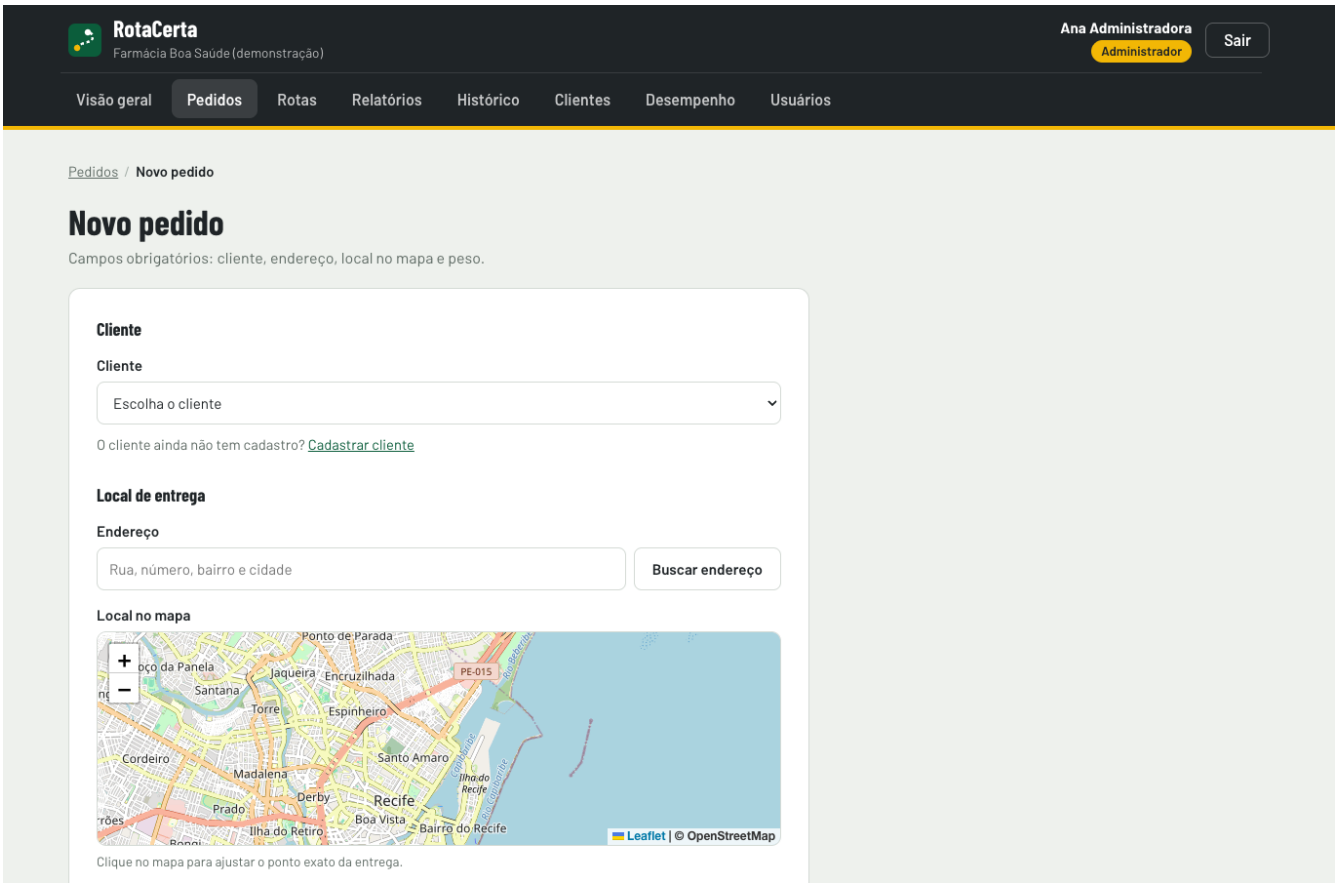
4. Informe o peso em kg, maior que zero e de até 500 kg.

5. Escolha prioridade Alta, Normal ou Baixa.

6. Se necessário, preencha Entregar a partir de e Entregar até. Um novo fim deve estar no futuro e depois do início.

7. Escolha Quem vai entregar ou deixe Sem entregador por enquanto.

8. Clique em Cadastrar pedido e confira os dados na página do pedido.



Início do formulário Novo pedido com cliente, endereço e mapa. Os campos de carga ficam abaixo.

A busca de endereço é uma ajuda: verifique número, complemento e posição antes de salvar. Se a busca estiver indisponível, a mensagem orienta a marcar o mapa. O limite padrão de entrega é 30 km a partir do ponto de saída, quando ele está definido. Um entregador fora de serviço ou sem capacidade não pode receber o pedido.

Um pedido sem entregador fica Pendente. Ao atribuir um entregador, ele passa a Atribuído. A capacidade considera os pedidos atribuídos e em rota. Prioridade e horários orientam a operação; a rota calcula distância e não garante todos os prazos.

8 Consultar editar e cancelar pedidos

Em Pedidos, use a busca por cliente, telefone, endereço, entregador ou número e os filtros de situação. Clique no número do pedido para abrir seus dados, localização e histórico. Cada mudança de situação informa o responsável e o horário.

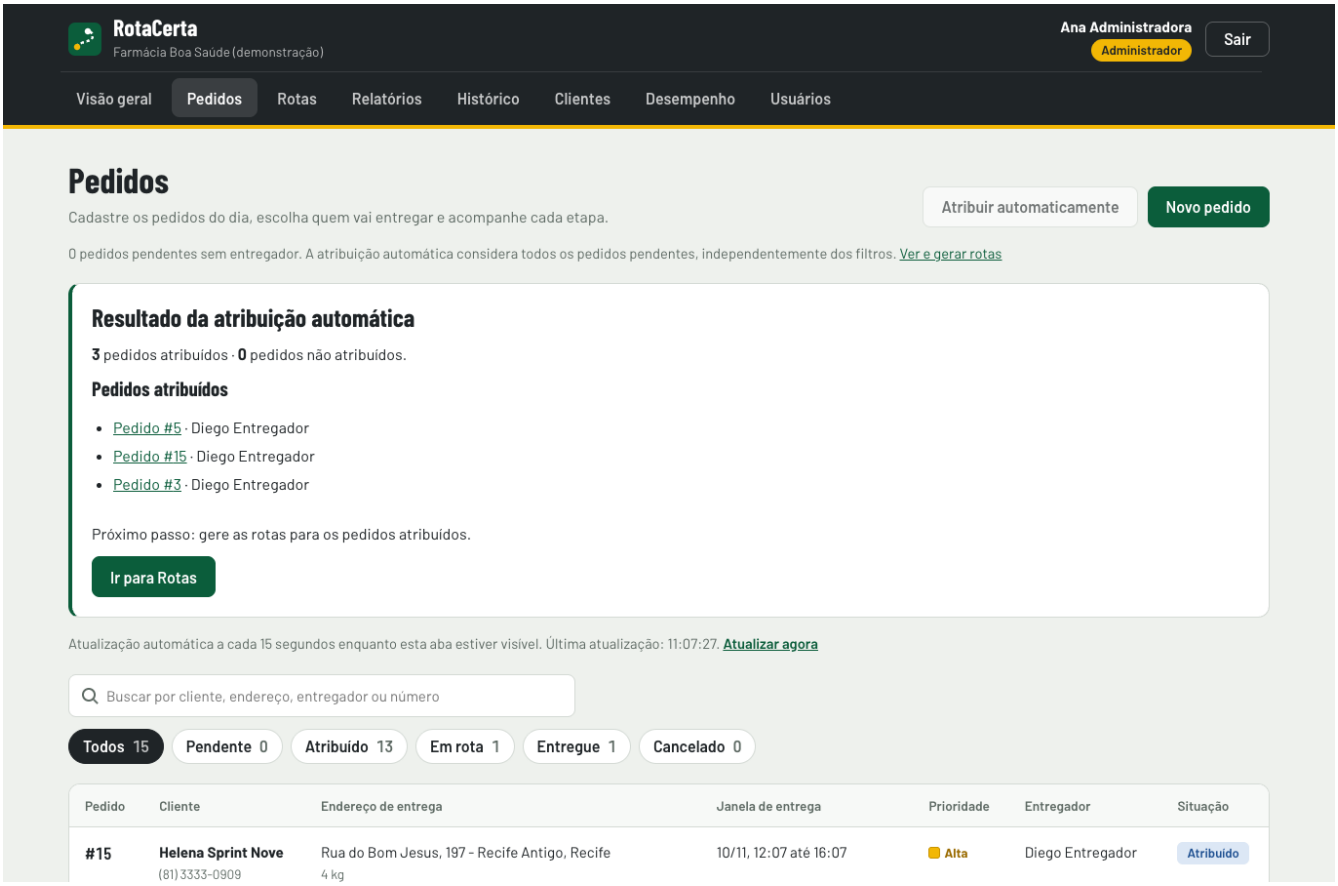
Clique em Editar pedido para corrigir um pedido aberto e depois em Salvar alterações. Pedidos entregues ou cancelados não podem ser editados diretamente. Depois da saída para entrega, não é permitido trocar cliente, entregador, endereço, posição ou peso de uma entrega pertencente a uma rota iniciada. Para uma nova entrega com outros dados, cadastre outro pedido.

Para cancelar, clique em Cancelar pedido, informe o motivo e confirme. O motivo deve ter pelo menos cinco caracteres úteis. O cancelamento fica no histórico. A exclusão é exclusiva do administrador e pode ser impedida pelo vínculo a uma rota iniciada; nesse caso, use o cancelamento quando ele for permitido.

O fluxo usual é Pendente, Atribuído, Em rota e Entregue. Atribuídos podem voltar a pendentes pela gestão; encerrados só podem ser reabertos quando as regras da rota permitem. Reabrir remove a atribuição e volta a Pendente. Uma rota já iniciada ou concluída impede essa reabertura para preservar seu histórico.

9 Atribuir os pedidos automaticamente

1. Confira se existem entregadores ativos, disponíveis e com capacidade.
2. Em Pedidos, clique em Atribuir automaticamente.
3. Leia a confirmação e clique em Atribuir pedidos.
4. Confira o resumo de atribuídos e não atribuídos e seus motivos.
5. Acesse Rotas para calcular os trajetos dos pedidos atribuídos.



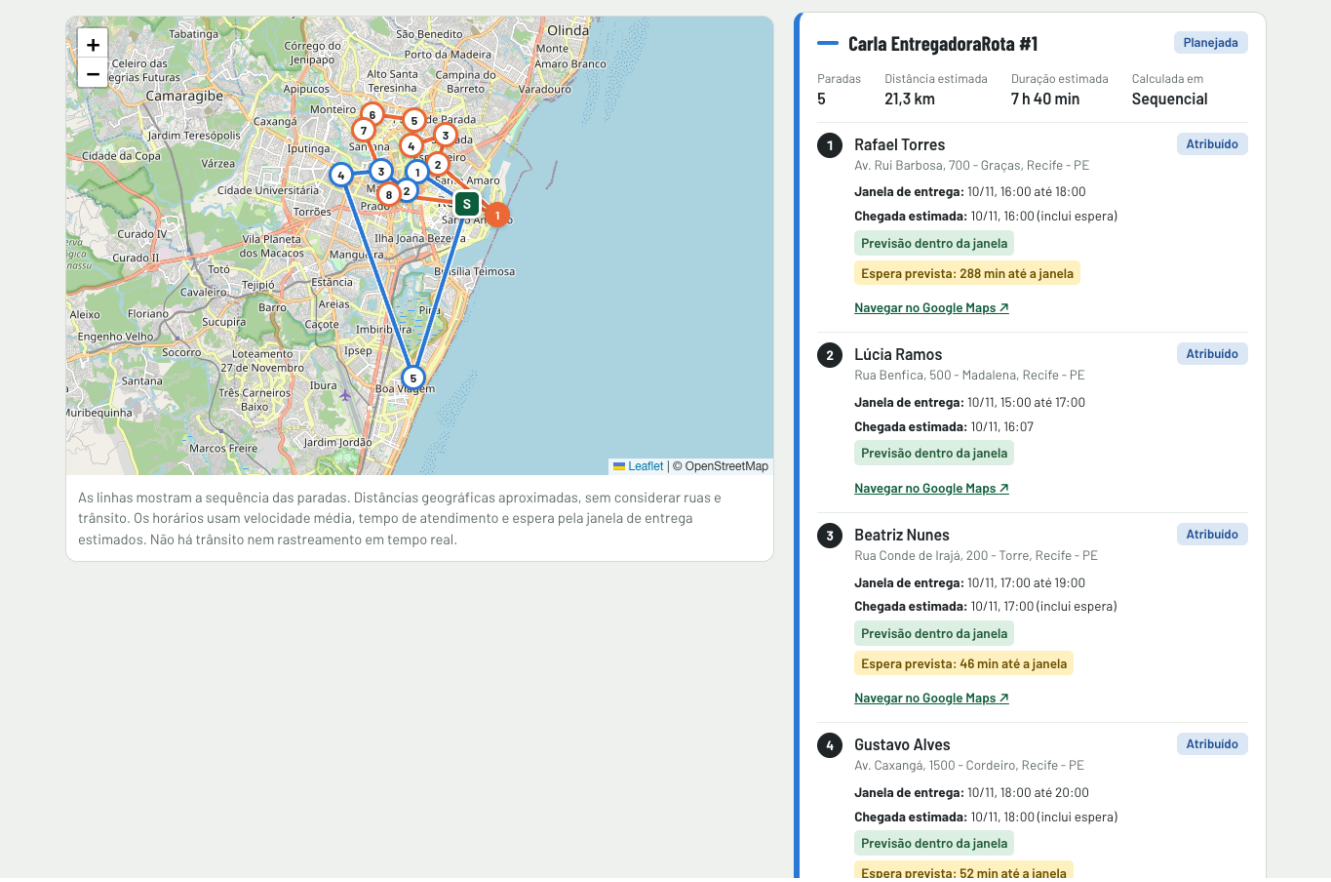
Resultado da atribuição automática com pedidos, entregadores e acesso a Rotas.

A ação considera todos os pedidos pendentes sem entregador, inclusive os ocultos por filtros. A escolha avalia prioridade, prazo e capacidade. Quem está com rota ou pedido em andamento fica fora da seleção automática. Um pedido fora do raio, com prazo vencido ou sem entregador elegível permanece pendente, com o motivo. Revise o pedido ou a disponibilidade antes de tentar novamente.

Para atribuição manual, abra o pedido, escolha Editar pedido e selecione Quem vai entregar. Salve. As validações de capacidade e acesso continuam valendo.

10 Gerar e consultar rotas

Abra Rotas depois da atribuição. Confira o ponto de saída, escolha Sequencial ou Paralelo em CPU e clique em Gerar rotas. O modo GPU só fica disponível em máquina compatível. Confira a quantidade de rotas geradas, as paradas, a distância e a duração prevista. O mapa usa uma cor para cada entregador e números nas paradas.



Mapa das rotas e lista de paradas do entregador, com janelas e estimativas.

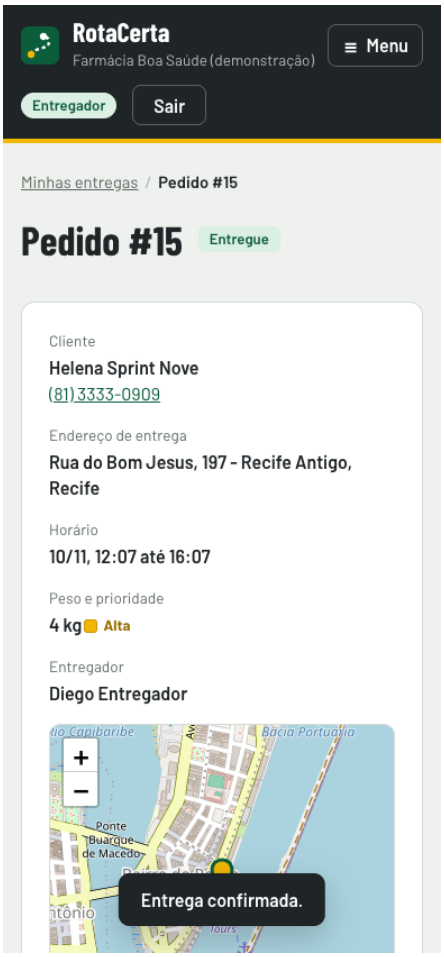
Gerar novamente substitui rotas ainda planejadas; rotas em andamento são preservadas. Pedidos novos de um entregador ocupado aguardam a próxima rota. O sistema explica quando algum entregador não entrou na geração.

O mapa liga as posições geográficas, sem acompanhar ruas ou trânsito. As previsões usam velocidade média e tempo de atendimento. Espera prevista indica o tempo até o início da janela; atraso previsto indica risco de ultrapassar o fim. Esses avisos ajudam a equipe a agir, mas não garantem o horário de chegada.

11 Executar a entrega pelo celular

Entre com a conta do entregador. Minhas entregas apresenta somente os seus pedidos. Confira sua disponibilidade, a rota e a sequência das paradas.

1. Fique Disponível para receber novos pedidos.
2. Quando a rota estiver pronta e você for sair, toque em Iniciar rota.
3. Siga as paradas. O botão de navegação abre o Google Maps em outra aba quando disponível.
4. Abra o pedido da parada e confira endereço e cliente.
5. Após entregar, toque em Confirmar entrega.
6. Confira a situação Entregue. Repita nas próximas paradas.



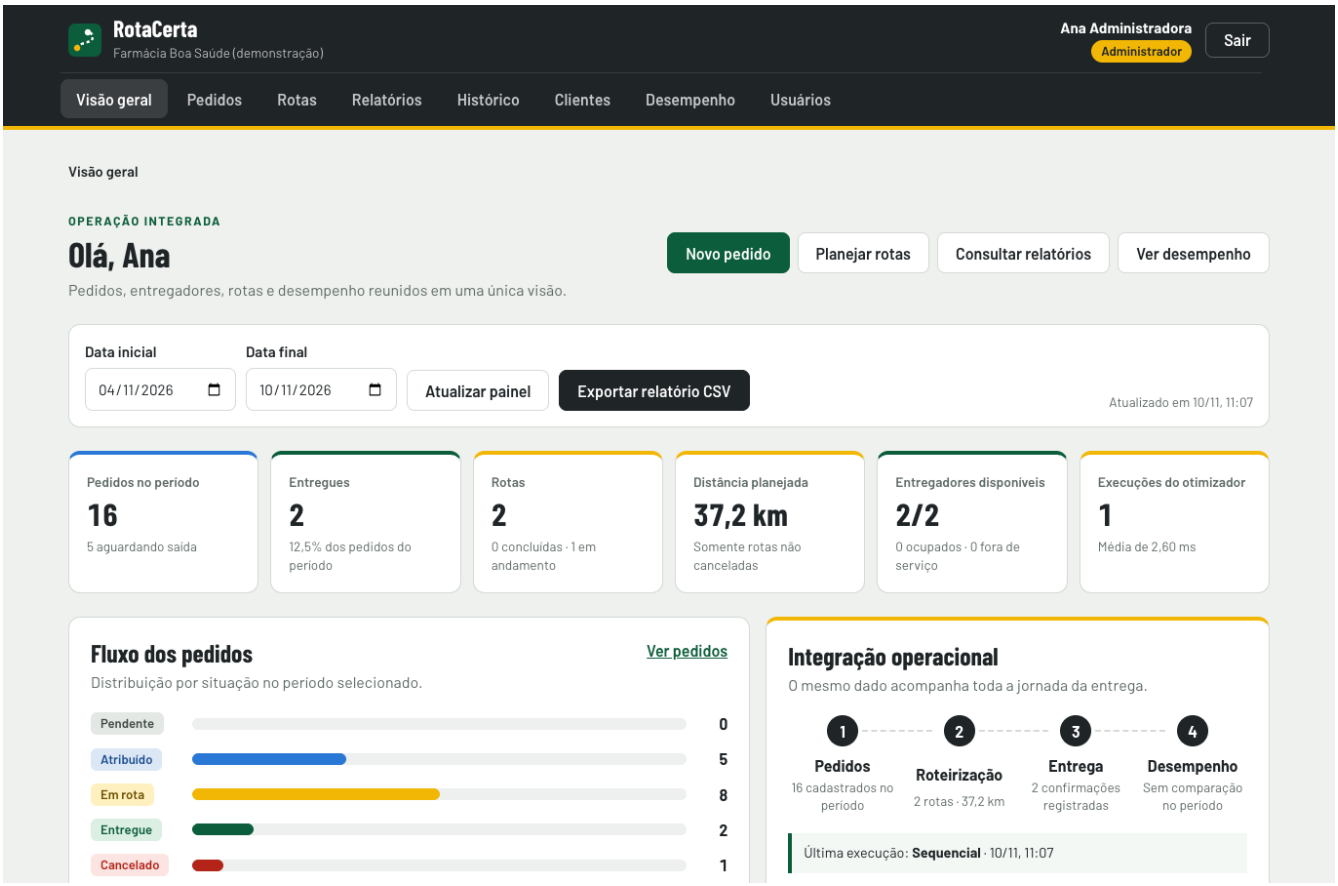
Pedido confirmado como Entregue na tela do entregador no celular.

Iniciar rota coloca seus pedidos em Em rota. Quando todas as paradas estiverem resolvidas, a rota é encerrada. Para pedidos atribuídos sem rota gerada, a tela também permite avançar a saída para entrega e a confirmação na ordem correta. O entregador não cancela nem reabre pedidos: peça apoio à gestão quando houver problema.

Fora de serviço impede o recebimento de novos pedidos; essa alteração é bloqueada se houver entrega em rota. Se sua lista parecer desatualizada, use Atualizar agora. No fim do uso em aparelho compartilhado, toque em Sair.

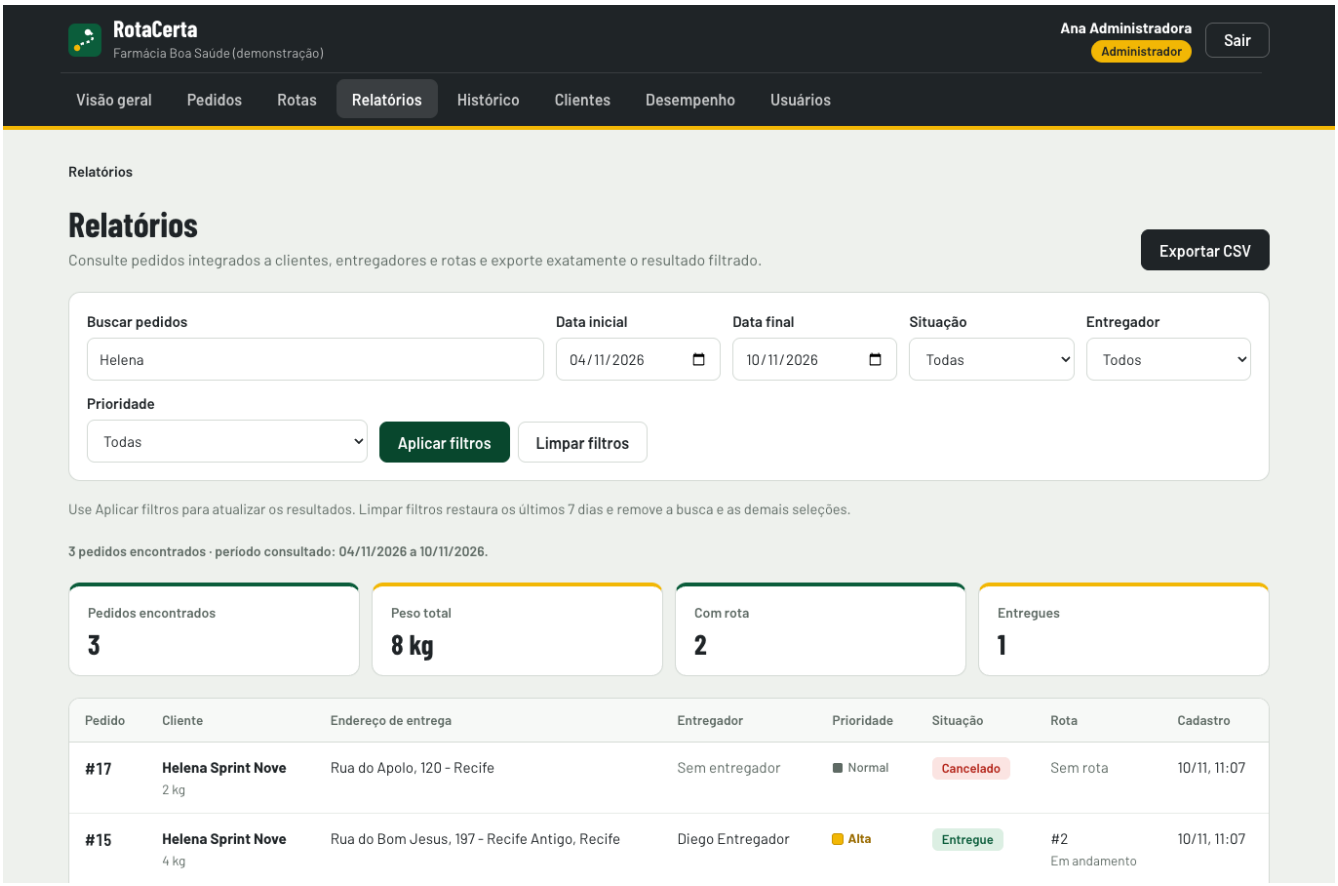
12 Usar a Visão geral e os relatórios

Administradores e atendentes usam Visão geral para acompanhar pedidos, entregadores, rotas e alertas. Escolha as datas e clique em Atualizar painel. Os atalhos levam ao cadastro de pedidos, ao planejamento e às consultas.



Visão geral com indicadores e atalhos da operação.

Em Relatórios, escolha período, busca, situação, entregador ou prioridade e clique em Aplicar filtros. A lista e os totais passam a representar esse resultado. Clique em Exportar CSV para baixar as mesmas linhas. O arquivo pode ser aberto em uma planilha; usa ponto e vírgula como separador. Não há exportação em XLSX.

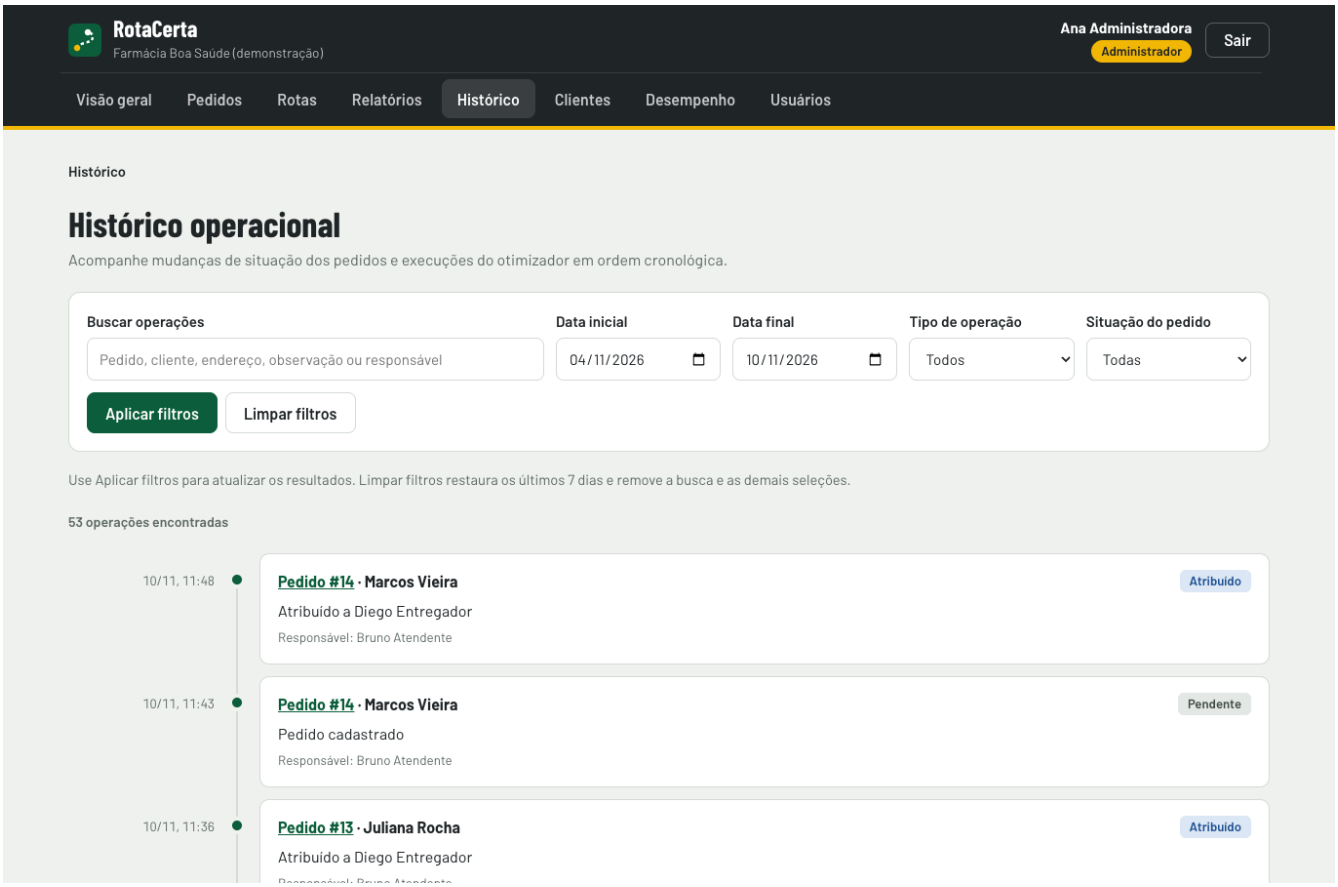


Relatório com filtros, resumo e botão Exportar CSV.

Limpar filtros volta aos últimos sete dias e remove as demais seleções. Um período pode ter até 366 dias. A consulta retorna até 2.000 pedidos; para volumes maiores, divida as datas. Se uma entrega não aparecer, confira o período e a situação antes de concluir que o cadastro foi perdido. Os limites diários da consulta são definidos em UTC; amplie as datas ao conferir registros perto da meia-noite.

13 Consultar o histórico operacional

Abra Histórico, escolha período e, se desejar, tipo da operação, situação do pedido e termo de busca. Clique em Aplicar filtros. A linha do tempo reúne mudanças de situação de pedidos e execuções do cálculo de rotas, com horário, responsável e observação. Ela não é um registro de todas as ações da aplicação.



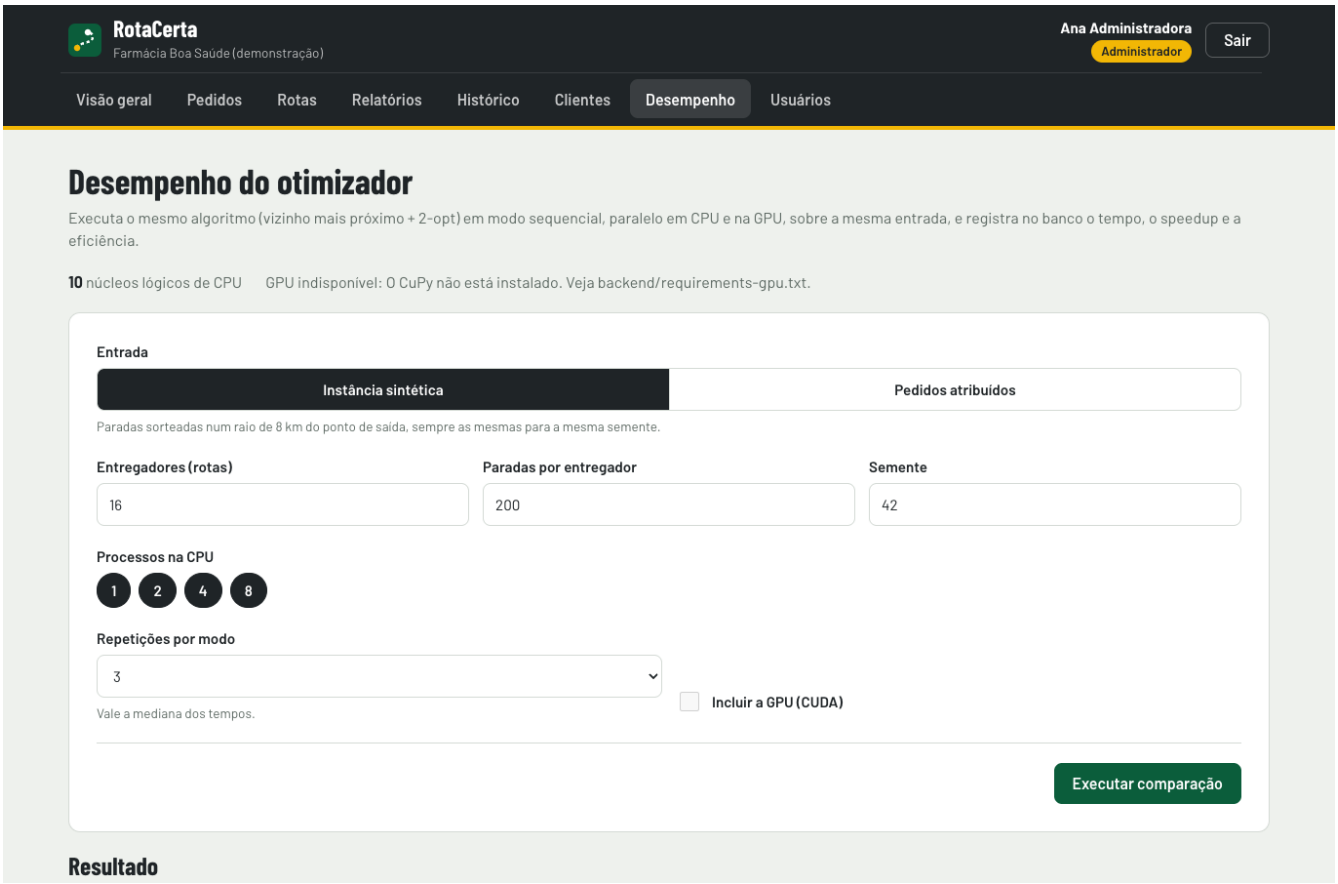
Histórico operacional com filtros e registros por responsável e horário.

A situação filtra eventos de pedidos; para consultar cálculos do otimizador, deixe esse filtro sem uma situação de pedido. A lista apresenta até os 500 eventos mais recentes do resultado. Reduza o período se precisar localizar eventos anteriores. Limpar filtros restaura a consulta padrão.

14 Comparar o desempenho do cálculo

Somente o administrador acessa Desempenho. Essa tela compara formas de executar o mesmo cálculo; ela não confirma entregas nem cria pedidos fictícios na operação. A entrada sintética serve apenas para os experimentos.

1. Escolha Instância sintética ou Pedidos atribuídos.
2. Na sintética, informe entregadores, paradas por entregador e semente.
3. Escolha a quantidade de processos de CPU e as repetições.
4. Inclua GPU apenas quando a tela informar que ela está disponível.
5. Clique em Executar comparação e aguarde a conclusão.
6. Confira os tempos, o gráfico e se as sequências são idênticas.

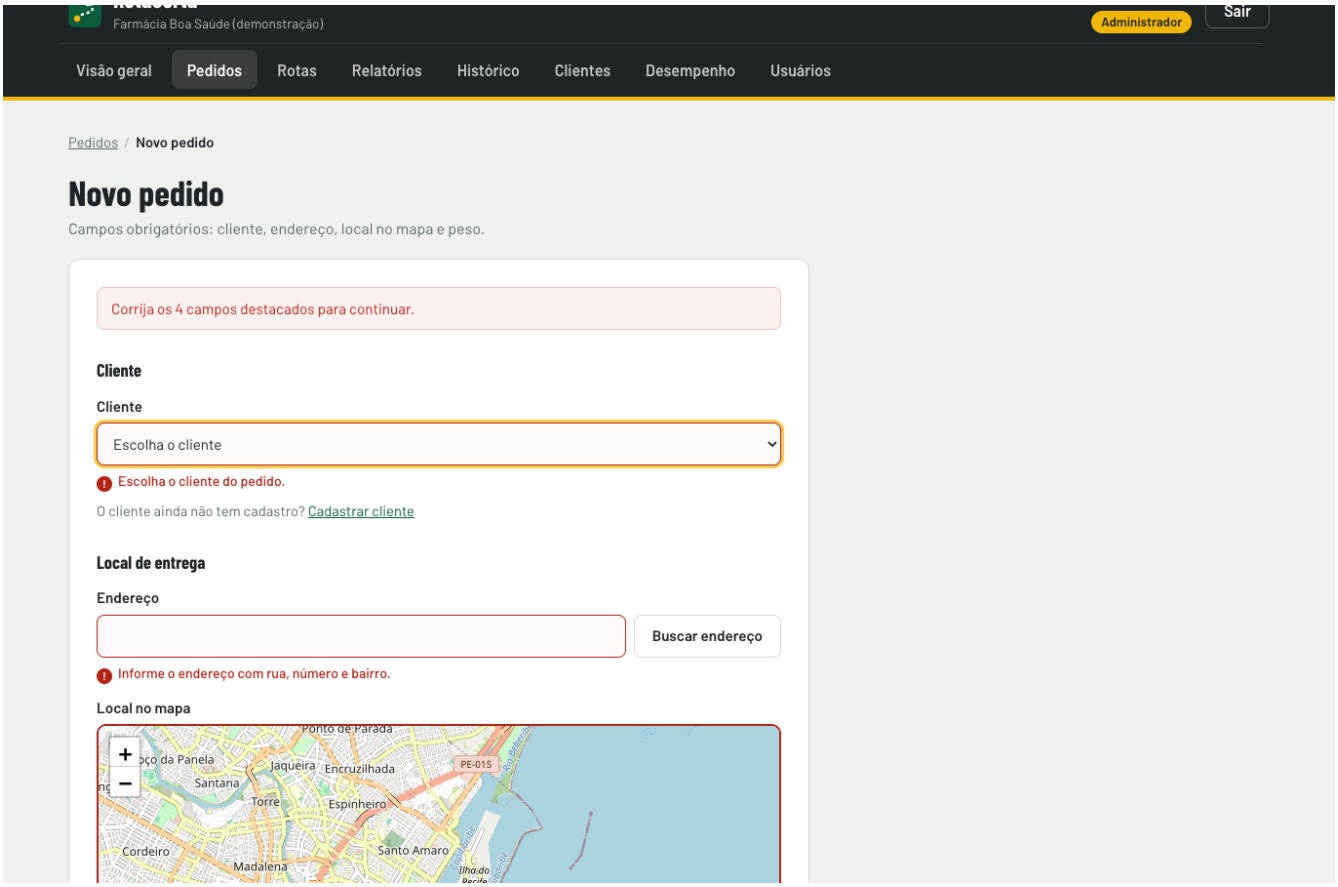


Desempenho com configuração de entrada, modos e resultados registrados.

Speedup acima de 1 indica execução mais rápida que a sequencial; abaixo de 1, mais lenta. Eficiência compara o ganho com o número de processos de CPU. Poucos pedidos podem não compensar o custo de distribuir o trabalho. Semente igual e mesma configuração permitem repetir a mesma entrada sintética. O resultado de uma máquina não é uma promessa de desempenho em outra.

15 Mensagens e situações comuns

Mensagem ou situação	Como proceder
E-mail ou senha inválidos	Confira os dados; peça ao administrador para verificar sua conta.
Usuário desativado	Solicite a reativação ao administrador.
Sua sessão expirou	Entre novamente; confirme em qual conta está acessando.
Corrija os campos destacados	Leia cada mensagem, ajuste os campos e salve novamente.
Busca de endereço indisponível	Marque o local diretamente no mapa e mantenha o endereço completo.
Capacidade de carga excedida	Escolha outro entregador ou reorganize os pedidos; não informe um peso incorreto.
Prazo já passou ou janela inválida	Informe uma nova janela futura; ao corrigir outro dado, uma janela antiga pode ser preservada.
Nenhum pedido atribuído aguardando rota	Atribua pedidos antes de gerar, ou confira se já pertencem a uma rota.
A atribuição mudou	Atualize a lista e gere o planejamento novamente quando permitido.
Não é possível desativar entregador	Resolva os pedidos ativos indicados na tela.
Nenhum resultado encontrado	Reveja busca, período e demais filtros; use Limpar filtros.
Não foi possível falar com o servidor	Confira a conexão e peça ao responsável para verificar se a aplicação está ligada.



Exemplo de validação de formulário, com mensagens junto aos campos e resumo.

Se o problema continuar, informe ao administrador a tela, o número do pedido, o horário e o que tentou fazer. Não compartilhe sua senha. A versão não envia mensagens por WhatsApp, SMS ou e-mail e não oferece rastreamento ao vivo por GPS.

16 Roteiro rápido para o primeiro uso

O administrador cadastra um entregador disponível e define o ponto de saída. O atendente cadastra um cliente e um pedido com local próximo desse ponto, sem janela de horário para simplificar o primeiro exercício. Depois atribui o pedido e gera a rota. O entregador entra em outra sessão do navegador, inicia a rota e confirma a entrega. A gestão retorna ao pedido e ao histórico para conferir o resultado e exporta o relatório do período.

Esse roteiro pode orientar o treinamento de uma pessoa nova. A validação de aceitação com usuário externo deve ser registrada pela equipe; as capturas deste manual são da verificação técnica automatizada.

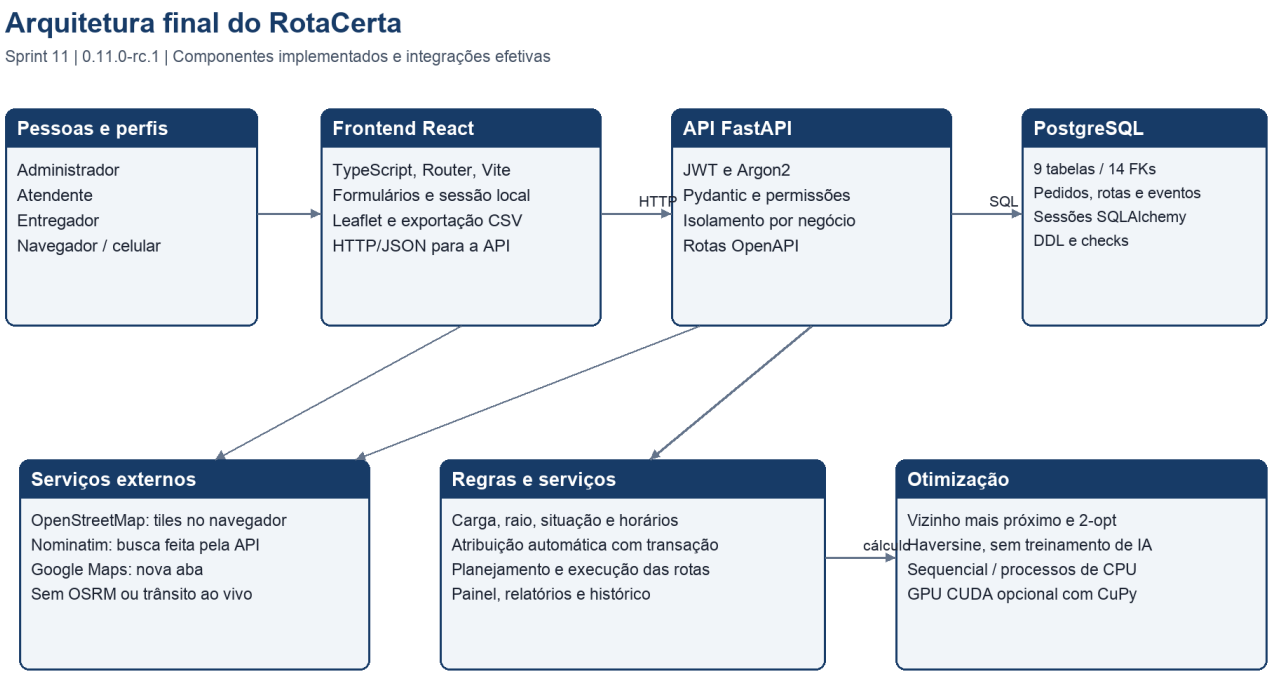
Manual Técnico do RotaCerta

Grupo 04 | UNINASSAU | Ciência da Computação | Turma 8NA | 2026.2. Versão 0.11.0-rc.1, revisada em 10/11/2026.

1 Visão geral da solução

O RotaCerta é uma aplicação web para organizar entregas de pequenos negócios. O frontend React consulta uma API HTTP FastAPI. A API autentica a conta, aplica regras de negócio e usa SQLAlchemy para persistir dados em PostgreSQL. O estabelecimento da conta delimita os dados operacionais; entregadores têm acesso restrito aos próprios pedidos e rotas.

O núcleo de otimização calcula sequências de paradas por vizinho mais próximo e 2-opt. Ele pode executar sequencialmente, em processos de CPU ou em GPU CUDA. O sistema não contém modelo de IA treinado, serviço de LLM ou pipeline de aprendizado de máquina: seu componente computacional é uma heurística de otimização.



Arquitetura final revisada na Sprint 11, com os componentes e as integrações efetivas.

O mapa usa Leaflet e tiles do OpenStreetMap. A API consulta Nominatim para sugerir posições de endereços. O Google Maps é aberto pelo navegador para navegação externa. Nenhum desses serviços participa do cálculo das distâncias do otimizador; não há OSRM ou trânsito em tempo real na versão entregue.

2 Tecnologias e dependências

Camada	Tecnologias da versão
Frontend	React 19.1.1, TypeScript 5.9.2, Vite 7.3.7 no lockfile, React Router 7.18.4, Leaflet 1.9.4
API	Python, FastAPI 0.116.1 e Uvicorn 0.35.0
Persistência	SQLAlchemy 2.1.1, psycopg 3.3.6, PostgreSQL 16+
Autenticação	PyJWT 2.15.0, Argon2 com argon2-cffi 25.1.0
Validação e HTTP	Pydantic, pydantic-settings 2.10.1, email-validator 2.3.0, httpx 0.28.1
Cálculo	NumPy 2.5.3; CuPy CUDA opcional em requirements-gpu.txt
Evidências e documentos	Playwright, pypdf, ReportLab e Pillow em requirements-docs.txt

Os arquivos de referência são `backend/requirements.txt`, `requirements-gpu.txt`, `requirements-docs.txt` e `frontend/package-lock.json`. O lockfile do frontend deve ser usado com `npm ci`. Dependências transitivas Python não têm um lockfile completo; ao reproduzir medições, registre também as versões efetivamente instaladas.

3 Organização dos módulos

Caminho	Responsabilidade
backend/app/main.py	Aplicação FastAPI, ciclo de vida, roteadores, tratamento de erros e health check
backend/app/config.py e database.py	Configuração, engine/sessões, DDL inicial e atualizações incrementais
backend/app/models.py e schemas.py	Mapeamento relacional e contratos de entrada/saída
backend/app/security.py	Hash e verificação de senha, JWT e dependências de autorização
backend/app/routers	Autenticação, usuários, clientes, pedidos, atribuição, rotas, consultas e desempenho
backend/app/rules.py e assignment.py	Regras, carga ativa, raio e atribuição automática transacional
backend/app/planning.py	Geração, agendamento, sincronização e execução das rotas
backend/app/reporting.py e benchmark.py	Indicadores, relatórios, histórico e medições de desempenho
backend/app/optimizer	Núcleo numérico, processos CPU e kernel CUDA opcional
frontend/src/App.tsx, session.tsx e api.ts	Rotas da interface, sessão, comunicação HTTP e adaptação de erros
frontend/src/pages e components	Telas por módulo, formulários, mapas, confirmações e navegação
backend/tests e frontend/src/*.test.mjs	Testes de regras, API, navegação, validação e utilitários
tools e tools/documentation	Ambientes locais, casos de navegador e geração de documentos
database, docs e output/pdf	DDL, manuais/documentação e PDFs cumulativos

4 Requisitos do ambiente

Use Python 3.12 ou superior, Node.js 20.19+ ou 22.12+, npm, Git e PostgreSQL 16+. O sistema é acessado em navegador moderno. Google Chrome é necessário para os roteiros de evidência, não para todo uso normal. Internet é necessária para instalar dependências e carregar mapa/busca externa. Sem GPU NVIDIA, os modos sequencial e paralelo em CPU continuam disponíveis.

O Compose é a alternativa para subir os três serviços sem instalar o PostgreSQL e o Python diretamente no host. Ele usa uma imagem PostgreSQL 16, API Python 3.12 e frontend Node 22. O serviço frontend do Compose usa Vite em modo de desenvolvimento para a demonstração acadêmica; não é um pacote de hospedagem de produção com proxy, HTTPS e distribuição de arquivos estáticos.

5 Obter o código e preparar a instalação manual

Na raiz de um terminal Bash em macOS ou Linux:

```
git clone https://github.com/alvccpj/rota-certa.git
cd rota-certa
git switch codex/sprint-11-preparacao-entrega
python3 -m venv backend/.venv
source backend/.venv/bin/activate
pip install -r backend/requirements.txt
npm ci --prefix frontend
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env
```

A branch identifica esta entrega; após sua integração, a `master` também passa a contê-la. Se o repositório já existe, use `git fetch origin` e selecione a branch correspondente sem substituir alterações locais não salvas.

No PowerShell, crie o ambiente com `python -m venv backend/.venv` e ative com `backend\.venv\Scripts\Activate.ps1`; execute os mesmos comandos pip e npm. O guia `COMO-RODAR.md` traz as variações completas de terminal.

6 Banco de dados e configuração local

Crie um usuário de aplicação e um banco no PostgreSQL usando pgAdmin ou psql. Use uma senha própria, definida fora do repositório. Pelo psql, conectado com permissão administrativa, substitua o nome de exemplo se necessário:

```
CREATE ROLE rotacerta_app LOGIN;
\password rotacerta_app
CREATE DATABASE rotacerta OWNER rotacerta_app;
```

Execute CREATE DATABASE separadamente de uma transação. O comando `\password` pede a senha sem incluí-la na linha SQL do manual. Não copie marcadores como `<SENHA>` literalmente para uma configuração final.

Edite `backend/.env` com a conexão do seu ambiente, no formato:

```
DATABASE_URL=postgresql+psycopg://<USUARIO>:<SENHA>@<HOST>:<PORTA>/<BANCO>
JWT_SECRET=<CHAVE_ALEATORIA_PRIVADA>
SEED_DEMO_DATA=true
```

Codifique caracteres reservados de usuário/senha para uso em URL. Guarde `.env` somente no ambiente de execução; ele é ignorado pelo Git. Os `.env.example` existentes contêm valores públicos de desenvolvimento, não credenciais de um serviço real. Substitua-os antes de disponibilizar uma instalação com dados reais. O frontend recebe apenas o endereço público da API em `VITE_API_URL`, nunca um segredo.

Na primeira inicialização, a API aplica `database/schema.sql` em PostgreSQL sem a tabela `users`. Depois, cria tabelas incrementais e aplica as alterações previstas em `database.py`. Se `SEED_DEMO_DATA=true` e não houver

estabelecimento, cria os dados acadêmicos de exemplo. Para começar sem exemplos, use `false` e cadastre um negócio pela interface.

7 Variáveis de ambiente

Variável	Uso e orientação
DATABASE_URL	Conexão SQLAlchemy; segredo se incluir senha; necessária para o banco escolhido
JWT_SECRET	Chave privada de assinatura HS256; defina uma chave aleatória e não a publique
ACCESS_TOKEN_MINUTES	Validade da sessão; padrão 480 minutos
CORS_ORIGINS	Origens do frontend separadas por vírgula; padrão localhost/127.0.0.1 na porta 5173
AUTO_CREATE_SCHEMA	Habilita criação/atualizações previstas no início; padrão true
SEED_DEMO_DATA	Dados de demonstração no banco vazio; padrão true no ambiente acadêmico
SCHEMA_PATH	Caminho alternativo do DDL, se o arquivo não estiver no caminho padrão do projeto
MAX_DELIVERY_RADIUS_KM	Raio de operação a partir do ponto de saída; padrão 30 km
AVERAGE_SPEED_KMH	Velocidade média da previsão; padrão 25; configure um valor positivo
SERVICE_MINUTES_PER_STOP	Atendimento por parada; padrão 5 minutos; use valor não negativo
LOCAL_UTC_OFFSET_HOURS	Deslocamento do dia operacional das rotas; padrão -3
GEOCODER_URL	Endpoint de busca de endereços; padrão Nominatim/OpenStreetMap
GEOCODER_USER_AGENT	Identificação HTTP da aplicação ao geocodificador
VITE_API_URL	Endereço da API acessível ao navegador; padrão http://localhost:8000
POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD	Banco, usuário e senha do serviço db no Compose
POSTGRES_PORT, API_PORT, FRONTEND_PORT	Portas publicadas pelo Compose; padrões 5432, 8000 e 5173

O backend lê `.env` a partir do diretório de trabalho. Por isso, na execução manual, inicie Uvicorn dentro de `backend`. Variáveis exportadas no processo prevalecem sobre o arquivo. O Vite lê a configuração ao iniciar ou compilar; reinicie após mudar `frontend/.env`. Uma instalação com portas diferentes também precisa ajustar CORS e o endereço da API no navegador.

8 Executar sem Docker

Terminal 1, na raiz e com o ambiente virtual ativo:

```
cd backend
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Terminal 2, na raiz:

```
cd frontend
npm run dev -- --host 127.0.0.1 --port 5173
```

Abra http://localhost:5173 e confira http://localhost:8000/health. A resposta normal inclui `status: ok` e `database: connected`. Se o banco falhar, o health check retorna 503. A documentação interativa fica em `/docs` e `/redoc` na API. Use Ctrl+C para encerrar os servidores. Para desenvolvimento, é possível usar `--reload` no

Uvicorn. O build é gerado por `npm run build --prefix frontend`; `npm run preview --prefix frontend` permite inspecioná-lo localmente.

Para uma demonstração SQLite, após instalar as dependências, execute `python tools/run_local_demo.py`. Os dados ficam em `backend/rotacerta-demo.db`. Esse modo não comprova os checks e bloqueios do PostgreSQL; a validação final do banco usa o PostgreSQL real.

9 Executar com Docker Compose

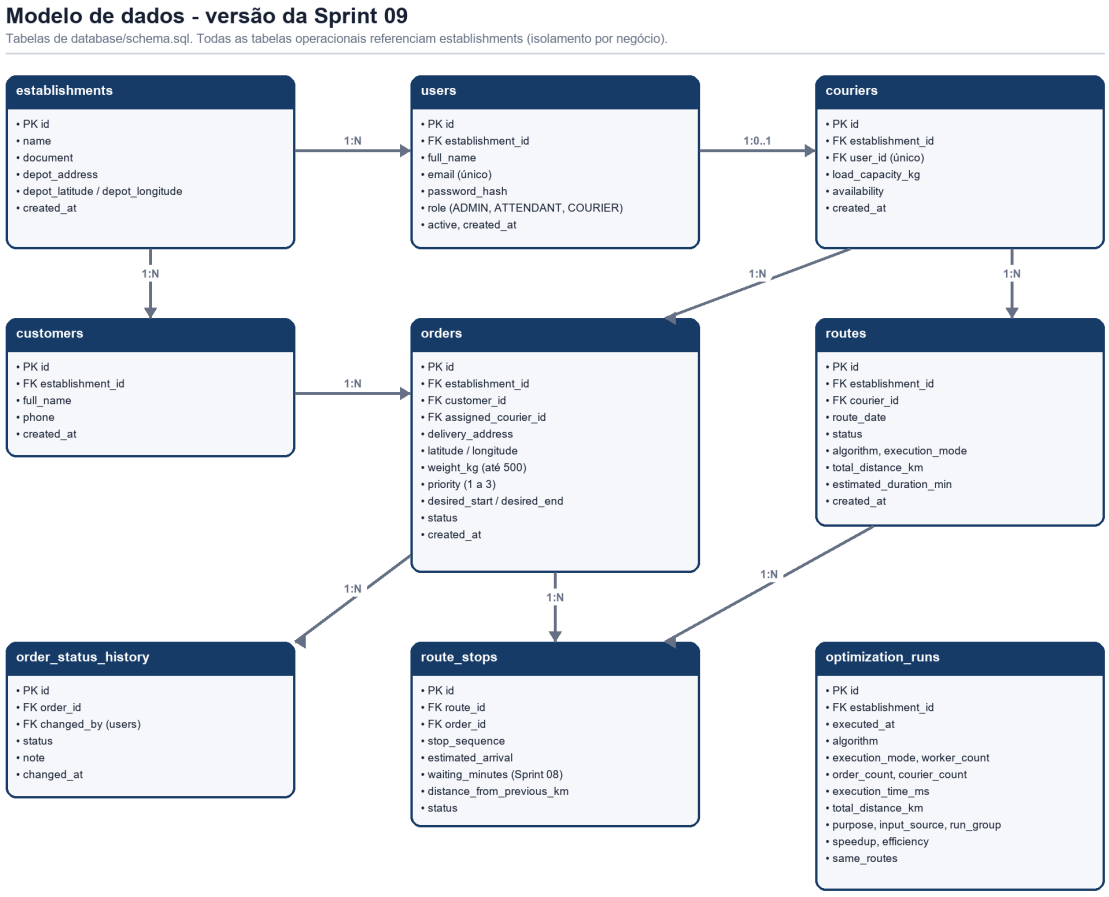
Na raiz, copie `.env.example` para `.env`, revise os valores locais e execute:

```
docker compose config --quiet
docker compose up --build
```

O primeiro comando verifica a configuração; o segundo constrói e inicia db, api e frontend. O banco aplica o DDL na criação inicial do volume. A API usa `db` como host dentro da rede Compose; o navegador precisa de um endereço acessível pelo host, como localhost. Aguardar o health check do banco faz parte da inicialização.

`docker compose down` mantém o volume. `docker compose down -v` apaga os dados e só deve ser usado para reiniciar um ambiente descartável. O YAML foi validado na Sprint 11; a subida dos containers não foi reexecutada nesta rodada. A execução e os testes atuais foram realizados na instalação manual PostgreSQL.

10 Modelo relacional e atualizações



Modelo relacional vigente com as nove tabelas.

Tabela	Papel e integridade principal
establishments	Negócio e ponto de saída; referência para o isolamento operacional
users	Usuários por negócio; e-mail único global; senha em hash e perfil
couriers	Entregador por usuário único, negócio, capacidade e disponibilidade
customers	Clientes do negócio, com nome e telefone opcional
orders	Cliente, entregador opcional, endereço, posição, peso, prioridade, janela e situação
order_status_history	Situação do pedido, observação, autor e instante; cascade na exclusão do pedido
routes	Entregador, negócio, data, situação, algoritmo, modo e métricas
route_stops	Pedido único por parada e sequência única por rota; cascade na exclusão da rota
optimization_runs	Execuções por negócio, origem, grupo, tempo, distância, speedup e eficiência

As PKs são BIGSERIAL, as referências BIGINT, medidas NUMERIC, eventos TIMESTAMPTZ e a data da rota DATE. O DDL possui checks de situações, perfis, modos, pesos/capacidades positivos, prioridade, janela, espera e métricas. Os índices atendem pedidos por negócio/situação e entregador, histórico, rotas por entregador/data e otimizações por modo/grupo.

O PostgreSQL é a fonte de referência do esquema; `models.py` mapeia suas entidades, mas nem todas as restrições do DDL estão declaradas no ORM. As FKs não garantem sozinhas que duas entidades pertencem ao mesmo negócio; a API aplica essa regra. A inspeção atual compara tabelas/colunas, lê as restrições e procura órfãos e relações entre negócios diferentes.

Na Sprint 11 não houve alteração no banco. Atualizações históricas contemplam o histórico de pedidos, métricas/modo GPU e `waiting_minutes`. Não há ferramenta de migração com revisões, como Alembic, nem rollback automático. Antes de alterar o DDL ou inicializar uma base antiga, faça backup e valide em cópia.

11 APIs e tratamento de erros

O contrato OpenAPI e o catálogo estão em `docs/api`. São 37 operações HTTP, sem prefixo `/api`. Corpos usam JSON; respostas 204 não têm corpo. `PUT` recebe um modelo completo. O login recebe e-mail/senha e retorna token Bearer; a API confere ativação e perfil no banco em cada requisição.

Os grupos são `/auth`, `/users`, `/couriers`, `/customers`, `/orders`, `/establishment/depot`, `/routes`, `/optimizer`, `/dashboard`, `/geocode` e `/health`. O catálogo lista métodos, parâmetros, modelos e perfis. Há exemplos completos em `docs/api/README.md`; o Swagger permite testar com Authorize.

401 indica sessão inválida, 403 falta de acesso, 404 registro inexistente/não visível, 409 conflito de negócio e 422 entrada inválida. `/health` e `/geocode` podem retornar 503. O frontend mostra mensagens e associa validações aos campos. Resposta inválida do geocodificador também é tratada como indisponibilidade.

`POST /optimizer/compare` é público, não persiste e usa entrada livre limitada a 64 rotas, 500 paradas por rota e 12.000 no total. `/optimizer/benchmark` é restrito ao administrador e persiste medições. Relatórios retornam JSON; o frontend produz o CSV, com escape de aspas e neutralização de prefixos de fórmula.

Datas das janelas devem ser enviadas em ISO 8601 com fuso; sem fuso são interpretadas como UTC. Relatório e histórico usam intervalos diários em UTC, com datas inclusivas e limite de 366 dias. O período padrão vai de seis dias antes até hoje do servidor. A data operacional da rota usa o deslocamento configurado. Esses critérios precisam ser considerados em consultas perto da meia-noite; não há seleção individual de fuso na interface.

O relatório retorna até 2.000 linhas, o histórico até 500 eventos, e o histórico do otimizador aceita limite entre 1 e 500. Não há paginação geral das listas.

12 Componente de otimização e desempenho

Para cada entregador, a entrada inclui depósito e pedidos atribuídos. A matriz de distâncias usa Haversine. O vizinho mais próximo constrói a sequência inicial; 2-opt inverte trechos quando reduz a distância, com desempate determinístico. O trajeto inclui a volta ao depósito. A heurística não garante o ótimo global.

No modo sequencial, o processo calcula uma rota por vez. Em CPU paralela, `ProcessPoolExecutor` distribui rotas independentes, reutilizando um pool por vez protegido por lock. O pool é aquecido nas comparações; threads BLAS por processo são limitadas para evitar excesso de paralelismo.

No modo CUDA, CuPy compila um RawKernel com NVRTC; cada rota usa um bloco de 256 threads para matriz, seleção de vizinhos e trocas 2-opt. A GPU é opcional, detectada por `/optimizer/capabilities`. A instalação específica está em `backend/requirements-gpu.txt`; requer NVIDIA e suporte ao CUDA correspondente. A Sprint 11 não revalidou esse modo no hardware ARM utilizado.

O benchmark registra mediana dos tempos, número efetivo de processos, distância e igualdade de rotas. Speedup é tempo sequencial dividido pelo tempo do modo; eficiência CPU é speedup dividido pelos processos. A eficiência GPU não é apresentada como se fosse eficiência de processos. Transferência e preparação podem tornar a GPU mais lenta em entradas pequenas.

O agendamento posterior usa velocidade média, atendimento e espera até a janela. Não altera a ordem para garantir todos os prazos. A atribuição automática prioriza urgência/prazo e balanceia carga elegível; não é uma otimização conjunta completa de frota e janelas.

13 Testes e reprodução das evidências

Ative o ambiente virtual e execute da raiz:

```
python -m unittest discover backend/tests -v
npm test --prefix frontend
npm run build --prefix frontend
pip install -r backend/requirements-docs.txt
```

Para a mesma suíte no PostgreSQL, configure `TEST_DATABASE_URL` com a conexão do banco de teste. Cada caso recria somente o schema `rotacerta_testes_automatizados`. Não execute suítes concorrentes que usem esse mesmo schema. Os testes de atribuição usam uma barreira e oito chamadas simultâneas, conferindo carga e histórico no PostgreSQL real.

Para os 61 casos de navegador, configure `POSTGRES_URL` para um banco de teste e inicie o ambiente descartável:

```
python tools/run_test_environment.py --schema sprint11_testes --api-port 8021 --web-port 5191
```

Em outro terminal, com o mesmo ambiente virtual ativo:

```
python tools/documentation/capture_sprint11_evidence.py
python tools/documentation/export_api_docs.py --evidence-dir docs/sprint-11/evidencias
```

O ambiente recria `sprint11_testes`. O roteiro usa API 8021 e frontend 5191; as variáveis `API_URL`, `APP_URL` e `EVIDENCE_DIR` permitem outro destino. Com `DATABASE_URL` apontando ao mesmo schema e `API_URL` à porta 8021, execute a inspeção sem alterar a base:

```
python tools/documentation/verify_release.py --evidence-dir docs/sprint-11/evidencias
```

O contrato exportado deve ser idêntico ao servido. As evidências ficam em `docs/sprint-11/evidencias` e as capturas em `assets`. O teste CUDA é dispensado sem GPU; concorrência PostgreSQL é dispensada na

rodada SQLite. A ausência de uma falha não equivale à homologação de carga ou à aceitação com usuários externos.

14 Proteção das informações e operação

Senhas de contas são armazenadas com Argon2; JWT usa HS256 e expiração. O navegador guarda a sessão no armazenamento local, sem refresh token. Sair remove a sessão do navegador, mas não revoga antecipadamente uma cópia de token ainda válida. Desativar a conta impede novas chamadas autenticadas.

Configure chave própria, acesso ao banco e origens antes de usar dados reais. Uma publicação precisa de HTTPS, controle de acesso à infraestrutura e política de backups. O projeto não oferece, nesta versão, limitação de tentativas de login, recuperação de senha por e-mail ou proteção completa contra abuso de requisições. Os limites do comparador não substituem controles de infraestrutura.

Não registre tokens, senhas ou URLs privadas completas em logs, issues ou PDFs. O log de criação do seed deixou de exibir a senha nesta Sprint. Arquivos `.env`, ambientes virtuais, bases SQLite, node_modules, dist e tmp são ignorados pelo Git. As credenciais dos exemplos históricos são públicas e restritas à demonstração; nenhuma credencial de serviço real deve ser incluída em uma entrega.

15 Backup e restauração

Use as ferramentas PostgreSQL com os parâmetros do ambiente autorizados. Defina `DB_HOST`, `DB_PORT`, `DB_USER` e `DB_NAME` localmente, sem gravar senha no repositório. O cliente pode solicitar a senha ou usar um arquivo de acesso protegido do operador. Exemplo de backup na raiz:

```
mkdir -p tmp/backups
pg_dump -h "$DB_HOST" -p "$DB_PORT" -U "$DB_USER" \
  --format=custom --no-owner --no-acl --file=tmp/backups/rotacerta.dump "$DB_NAME"
```

Teste a restauração em um banco novo, sem substituir o banco da operação:

```
createdb -h "$DB_HOST" -p "$DB_PORT" -U "$DB_USER" rotacerta_validacao_restore
pg_restore -h "$DB_HOST" -p "$DB_PORT" -U "$DB_USER" \
  --no-owner --no-acl --dbname=rotacerta_validacao_restore tmp/backups/rotacerta.dump
```

Confira tabelas, contagens e uma consulta da aplicação antes de considerar o backup utilizável. Armazene cópias em local protegido e defina retenção com o responsável pela operação. O diretório tmp é ignorado e não substitui uma política de backup. O ensaio desta Sprint utilizou somente o banco descartável.

16 Manutenção e continuidade

Parta da branch principal atualizada, crie uma branch curta e preserve o padrão dos módulos. Acrescente regra no backend, exponha-a pelo roteador e reflita os erros no formulário. Toda correção funcional deve ter teste que reproduza o problema. Para uma alteração de schema, mantenha DDL, modelos e estratégia de atualização coerentes e valide instalação nova e base anterior em cópias.

Antes do pull request, rode a suíte afetada, build e verificações necessárias. Atualize README, manuais, contrato e documentação final quando mudar comportamento. Use `export_api_docs.py` para regenerar as APIs. Não grave novas evidências em pastas de Sprints encerradas; use `--evidence-dir` ou `EVIDENCE_DIR`.

O PDF da Sprint 11 é reconstruído por `tools/documentation/build_sprint11_document.py`. As imagens e manuais são fontes versionadas; renderize o PDF e revise as páginas antes da entrega. O histórico das Sprints anteriores deve permanecer preservado. Siga `CONTRIBUTING.md` para commits e revisão por outro integrante.

17 Diagnóstico básico

Sintoma	Verificação recomendada
Health check 503	Conferir PostgreSQL, conexão, schema e logs do backend
Navegador sem conexão com a API	Conferir VITE_API_URL, porta da API, CORS e reiniciar o Vite
Tabelas ausentes em base nova	Conferir permissões, AUTO_CREATE_SCHEMA e caminho de schema.sql
Rota não gerada	Conferir ponto de saída, pedidos atribuídos, carga, disponibilidade e rota em andamento
Busca de endereço falha	Conferir serviço externo; o usuário pode marcar o mapa manualmente
GPU indisponível	Consultar capabilities e documentação do modo CUDA; usar CPU enquanto isso
Resultado não aparece no relatório	Conferir período UTC, filtros e limite de linhas
Testes PostgreSQL interferem entre si	Encerrar outra suíte que esteja recriando o mesmo schema descartável

Registre versão, cenário e mensagem ao abrir uma correção. Use dados fictícios para reproduzir. Escalabilidade com grande volume, combinações de escritas concorrentes e aceitação externa continuam exigindo validações específicas.

Documentação final consolidada do RotaCerta

Grupo 04 | Versão 0.11.0-rc.1 | Revisão em 10/11/2026. Esta referência descreve a implementação preparada na Sprint 11. Os documentos anteriores permanecem como histórico do planejamento e da evolução.

1 Objetivo e produto implementado

O sistema organiza a operação de entregas de pequenos negócios. Autenticação, clientes, pedidos, entregadores, atribuição, rotas, execução das entregas, indicadores, histórico e comparação de desempenho estão integrados ao banco. O público é composto por administrador, atendente e entregador; o cliente final não acessa uma área própria. As instruções operacionais estão no Manual do Usuário, e a instalação/manutenção, no Manual Técnico.

2 Requisitos funcionais e rastreabilidade

Os identificadores RF01 a RF10 são preservados desde o planejamento. A coluna abaixo registra como cada requisito se materializou e onde foi verificado. As referências F, V e N são os casos repetidos na Sprint 11, em `docs/sprint-11/evidencias/casos_de_teste.json`.

ID	Implementação vigente	Evidência
RF01 Autenticação e perfis	Cadastro de negócio, login JWT, usuários, perfis e isolamento por negócio	F01-F03, F11-F12, V01-V03, V20, N01-N08, N16
RF02 Pedidos	Cadastro, edição permitida, consulta, filtros, janela, peso, prioridade e cancelamento com motivo	F07-F10, F18, V06-V11, V14-V15
RF03 Entregadores	Conta de usuário com perfil, capacidade, disponibilidade e controle de acesso ativo	F11-F12, V13, V19 e testes de regras
RF04 Rotas otimizadas	Vizinho mais próximo e 2-opt; sequencial, CPU paralela e CUDA opcional	F15, F22; F16 depende de hardware NVIDIA
RF05 Atribuição	Manual no pedido e automática por elegibilidade, urgência e carga	F14, suíte test_assignment e concorrência PostgreSQL
RF06 Lista e mapa	Rotas com paradas numeradas, trajetos e navegação externa por coordenadas	F15, F17, N06 e capturas atuais
RF07 Situação da entrega	Transições controladas e sincronização com paradas/rotas	F17-F18, V11 e test_order_rules/test_routes
RF08 Histórico	Situações por pedido e linha do tempo com eventos de pedidos e otimização	F17-F18, F21 e test_history
RF09 Relatórios e comparação	CSV operacional filtrado; tempo, distância, speedup e eficiência na área Desempenho	F20-F22, V18, testes de CSV
RF10 Painel	Indicadores do negócio, período, alertas e atalhos	F19 e test_dashboard

O CSV operacional e a comparação do otimizador são recursos diferentes. O sistema não promete um único arquivo CSV com todas as métricas de benchmark. CUDA foi demonstrado na Sprint 09 e não reexecutado no hardware ARM desta rodada.

3 Requisitos não funcionais e limites da comprovação

ID	Situação atual	Limite ou validação adicional
RNF01 Desempenho	Processamento sequencial/paralelo e medições persistidas; igualdade de rotas conferida em CPU	Não há SLA numérico de produção; ganho depende da entrada e da máquina
RNF02 Segurança	Argon2, JWT, autorização no backend, isolamento, validação de dados e proteção do CSV	Sem rate limit de login, recuperação por e-mail ou revogação individual de token
RNF03 Usabilidade	Formulários com mensagens, foco, confirmações, menus e manual ilustrado	Aceitação com usuário externo continua pendente
RNF04 Portabilidade	Interface web e responsividade verificadas no Chrome em computador e celular simulado	Não houve matriz completa de navegadores/dispositivos físicos
RNF05 Escalabilidade	Banco relacional e paralelização; ensaio de oito atribuições simultâneas	Não homologado para volume elevado ou combinações de escritas concorrentes
RNF06 Confiabilidade	Suíte PostgreSQL, 14 FKs sem órfãos, persistência e ensaio de backup/restauração	Sem solução implementada de alta disponibilidade ou backup automático
RNF07 Manutenibilidade	Separação de módulos, testes, manuais, contrato e versionamento	CI de navegador e migrações com revisões formais ainda não implementados

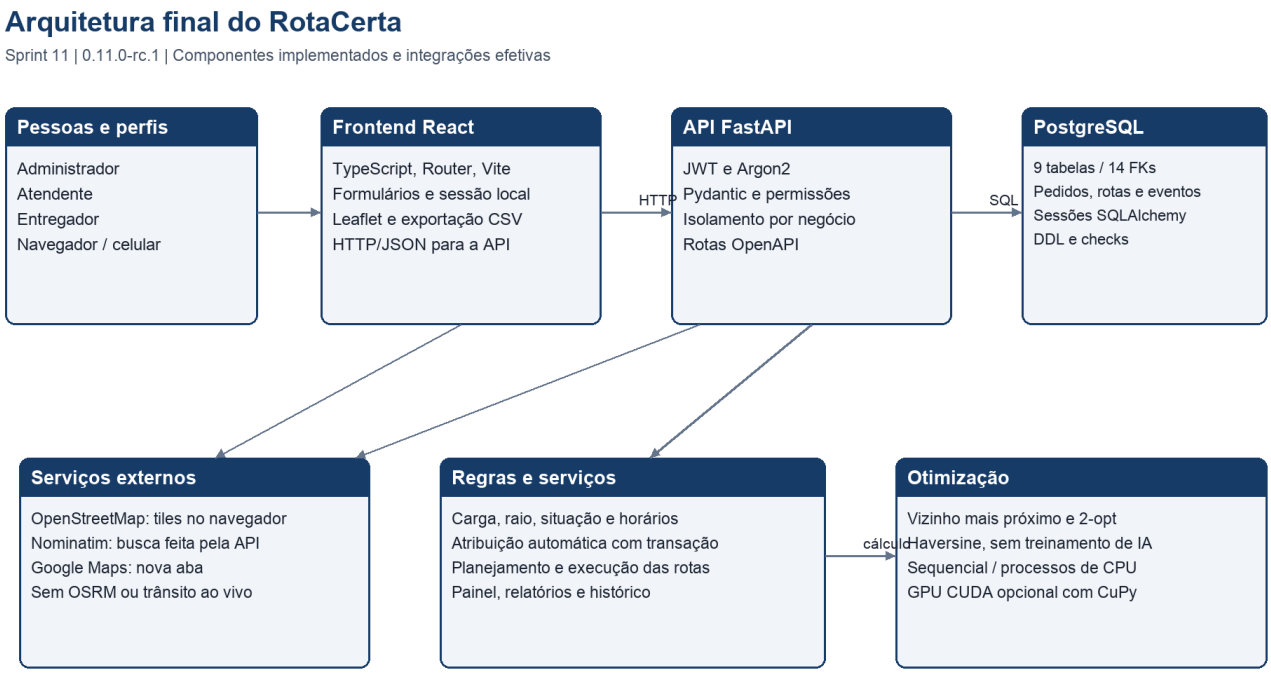
4 Casos de uso finais

Os identificadores UC01 a UC07 também são mantidos. O perfil Atendente passou a compartilhar com Administrador a geração de rotas e as consultas operacionais; gestão de usuários e benchmark continuam exclusivos do administrador.

Caso	Ator e pré-condição	Fluxo e resultado	Alternativas
UC01 Cadastrar pedido	Administrador/atendente autenticado; cliente existente	Escolher cliente, endereço/local, peso e prioridade; salvar; pedido fica pendente ou atribuído	Campo inválido, raio, janela ou carga impedem salvar e mostram orientação
UC02 Cadastrar entregador	Administrador autenticado	Novo usuário, perfil Entregador e capacidade; salvar; conta e registro de entregador criados	E-mail duplicado ou capacidade ausente/inválida recusados
UC03 Gerar rotas do dia	Administrador/atendente; ponto de saída e pedidos atribuídos	Escolher modo e gerar; trajetos, paradas e métricas persistidos	Ocupados/fora de serviço podem ser ignorados com motivo; sem elegíveis retorna conflito
UC04 Visualizar rota	Entregador ativo com atribuição própria	Abrir Minhas entregas; consultar mapa e sequência; abrir navegação externa	Sem rota mostra situação vazia; não acessa rota de outro entregador
UC05 Atualizar entrega	Entregador com pedido próprio atribuído/em rota	Iniciar rota ou saída; confirmar entrega; histórico e parada sincronizados	Pular etapa é recusado; cancelamento/reabertura cabem à gestão
UC06 Consultar métricas	Administrador; atendente tem consultas operacionais	Filtrar painel/relatório e, para administrador, executar/consultar benchmark	Período inválido recusado; sem dados mostra estado vazio; CUDA depende de hardware

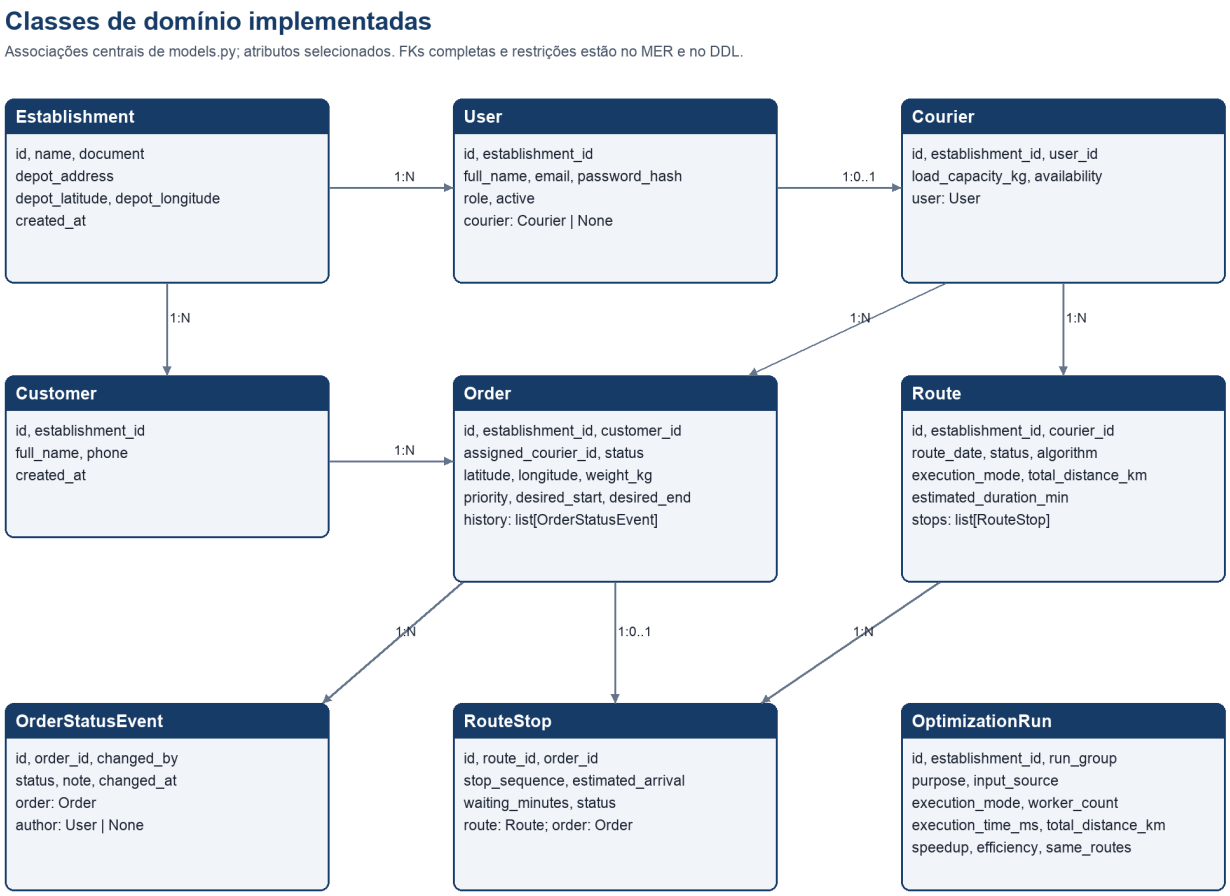
Caso	Ator e pré-condição	Fluxo e resultado	Alternativas
UC07 Autenticar	Qualquer perfil com conta ativa	Informar e-mail/senha; receber sessão e abrir a página permitida	Credenciais inválidas, conta desativada ou sessão expirada recebem tratamento específico

5 Arquitetura e classes implementadas



Arquitetura final com HTTP local, persistência, serviços e otimização.

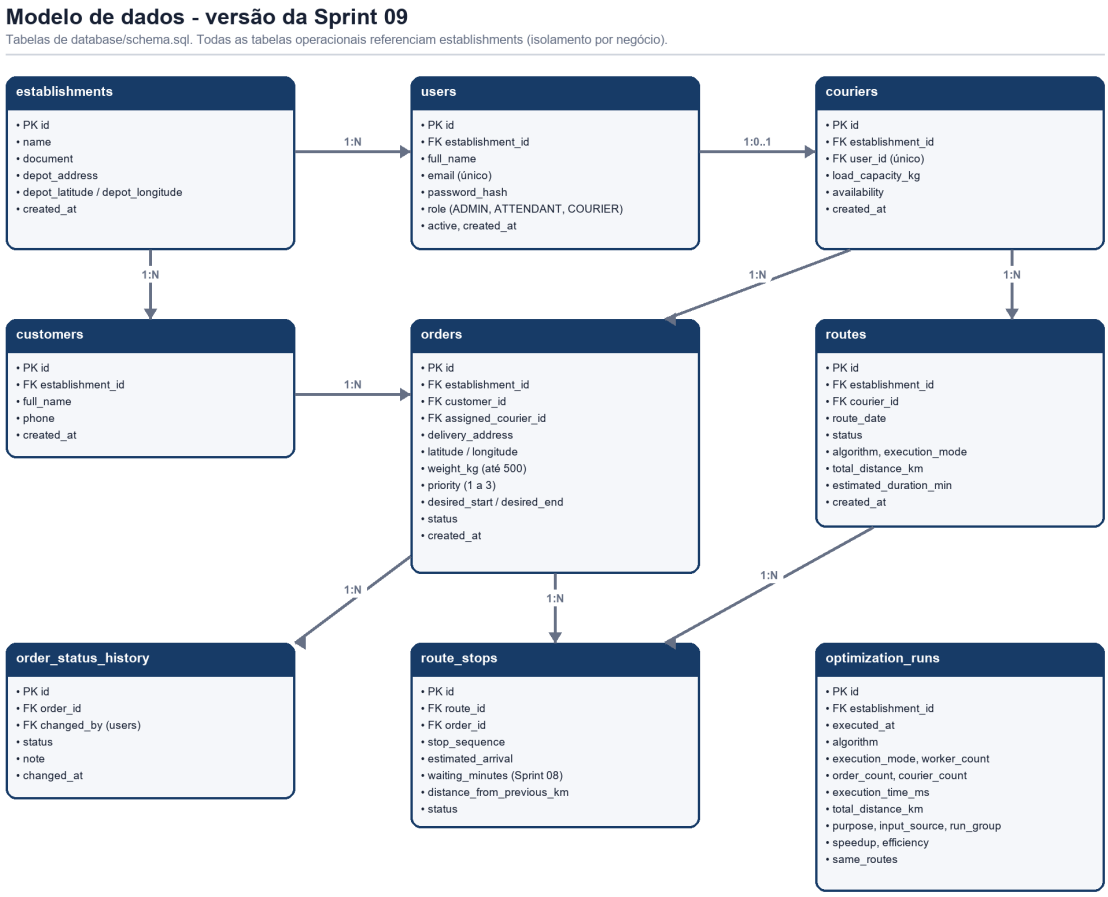
O frontend recebe interação e prepara as chamadas; o backend decide permissões e regras; os serviços de domínio coordenam a persistência e o cálculo. As regras não dependem apenas dos controles visuais. O contrato de 37 operações está em `docs/api/openapi.json`, acompanhado de guia e catálogo.



Classes de domínio mapeadas no SQLAlchemy com associações centrais e atributos selecionados.

O diagrama de classes da Sprint 02 é preservado como planejamento. A referência acima usa as nove classes realmente mapeadas em `backend/app/models.py`. Os atributos e associações completos estão no código e no esquema relacional; os serviços de cálculo e regras ficam em módulos separados, não em métodos fictícios nas entidades.

6 Modelo de dados vigente



Modelo relacional conferido com o DDL, sem mudança estrutural nas Sprints 10 e 11.

O DDL final está em `database/schema.sql`. São nove tabelas e 14 FKs. A inspeção da Sprint 11 conferiu as colunas com o ORM, PKs, FKs, checks, índices e unicidades. Não houve nova migração estrutural nesta Sprint. Medidas são NUMERIC, eventos TIMESTAMPTZ e a data operacional da rota DATE. A API complementa o DDL com isolamento por estabelecimento, capacidade, raio e regras de transição.

7 Regras de negócio consolidadas

Regra	Comportamento implementado
RN01 Isolamento	O estabelecimento da conta limita os registros; o entregador consulta somente seus pedidos/rotas
RN02 Dados e carga	Pedido com peso maior que zero e até 500 kg; capacidade do entregador maior que zero e até 1.000 kg
RN03 Local de entrega	Coordenadas válidas e raio padrão de 30 km quando o ponto de saída está definido
RN04 Janela	Nova janela termina no futuro e depois do início; uma janela antiga pode ser preservada para corrigir outros dados
RN05 Atribuição automática	Pendentes sem entregador; prioridade/prazo/ID; menor carga/ID entre ativos disponíveis, com capacidade e sem operação em andamento
RN06 Concorrência	Bloqueio do estabelecimento serializa atribuição automática no PostgreSQL; o ensaio não cobre todas as escritas
RN07 Transições	Pendente, atribuído, em rota e entregue; cancelamento por gestão com motivo; entregador apenas avança suas etapas
RN08 Preservação da rota	Após início, dados essenciais e reabertura do pedido têm restrições; novo pedido preserva a trilha anterior
RN09 Usuários	Não retirar acesso de entregador com pedidos ativos; não colocar fora de serviço com pedido em rota; não remover o próprio acesso administrativo
RN10 Planejamento	Rotas planejadas podem ser substituídas; rotas em andamento permanecem; mudança do ponto de saída é bloqueada com rota ativa
RN11 Previsões	Haversine, média de velocidade/atendimento e espera; atraso é sinalizado sem garantia das janelas
RN12 Consultas	Período de até 366 dias; relatório até 2.000 linhas e histórico até 500 eventos; CSV é gerado no frontend
RN13 Integrações	Busca de endereço indisponível/inválida oferece marcação manual; mapa e navegação externa dependem dos serviços disponíveis
RN14 Comparação livre	Até 64 rotas, 500 paradas por rota e 12.000 no total; entrada maior é recusada antes do cálculo

8 Decisões e mudanças em relação ao planejamento

O projeto adotou Python/NumPy para o cálculo de CPU e CUDA C via CuPy para GPU. A avaliação de C++/OpenMP não se tornou um módulo do produto. O mapa usa distâncias geográficas, e o adaptador OSRM foi retirado do escopo. As janelas resultam em previsão/espera/aviso, sem solver que garanta seu atendimento. Notificações externas e rastreamento GPS ao vivo ficaram fora do escopo.

A exportação implementada é CSV. O PDF cumulativo é material de entrega da disciplina, produzido pelos roteiros de documentação; não é um relatório PDF gerado por um botão da aplicação. A distribuição de pedidos é uma etapa separada da otimização da sequência de cada entregador.

Na Sprint 11, os limites do comparador e a validação da resposta de geocodificação passaram a constar do contrato atual. O DDL e o algoritmo principal não foram alterados. Nenhum recurso ainda não implementado é apresentado nesta documentação como funcionalidade disponível.

9 Fontes oficiais da entrega

Manual do Usuário: `docs/manuais/MANUAL_DO_USUARIO.md`. Manual Técnico: `docs/manuais/MANUAL_TECNICO.md`. APIs: `docs/api/README.md`, `ENDPOINTS.md` e `openapi.json`. Revisão e correções: `docs/sprint-11/REVISAO_CODIGO.md`. Resultados executados: `docs/sprint-11/evidencias`. Instruções de execução e equipe: `README.md` e `COMO-RODAR.md`.

O índice `docs/README.md` aponta para essas referências. As partes anteriores do PDF mantêm contexto histórico; a Parte XI descreve esta versão. A equipe deve atualizar as fontes e repetir os testes afetados antes de qualquer nova correção posterior à consolidação.

Revisão de código da Sprint 11

Grupo 04 - RotaCerta. Revisão técnica em 10/11/2026 sobre a base `043b6a4`, com as correções da versão 0.11.0-rc.1. O objetivo foi revisar organização, legibilidade, consistência, tratamento de erros e proteção das informações nos módulos entregues e confirmar o funcionamento após os ajustes.

1 Escopo e método

A revisão combinou leitura dos módulos principais, rastreamento dos fluxos da interface até as regras e o banco, revisão de configuração e documentação, execução das suítes e repetição dos casos na aplicação real. Os achados foram reproduzidos antes de alterar o comportamento. Não se trata de certificação de segurança, teste de invasão ou prova de ausência de todos os defeitos.

Área revisada	Arquivos e pontos observados	Resultado
Entrada e erros da API	main.py, schemas.py e routers; validação, códigos HTTP e respostas	Contratos mantidos; resposta externa inválida passou a ser tratada
Autenticação e autorização	security.py, auth.py, users.py; Argon2, JWT, ativação e perfil no banco	Regras cobertas pela suíte; senha retirada dos logs de inicialização
Isolamento dos negócios	Filtros de pedidos/clientes/usuários, referências e propriedade de rotas	Testes entre negócios e entregadores repetidos sem falha
Pedidos e entregadores	rules.py, orders.py e users.py; carga, raio, janela, transições e histórico	Restrições e mensagens conferidas com os fluxos
Atribuição e transações	assignment.py e test_release_s10.py; bloqueios e commit	Oito chamadas simultâneas preservaram capacidade e unicidade de histórico
Planejamento e execução	planning.py e routes.py; substituição de planos, início, paradas e sincronização	Fluxos repetidos; sem alteração no algoritmo de planejamento
Otimização CPU/GPU	routing.py, gpu.py, benchmark.py e modelos; limites, pool, desempates e métricas	Limites adicionados ao comparador livre; CUDA lido, mas não reexecutado nesta máquina
Consultas e CSV	reporting.py e frontend/reporting.ts; filtros, períodos, limites e escape	Prefixos de fórmula precedidos por espaços passaram a ser neutralizados
Interface e sessão	App, session, access, guards, formulários e páginas; navegação e erros	Menus por perfil, redirecionamentos e validações repetidos no navegador
Componentes e mapas	Field, ConfirmDialog, AutoRefresh, ErrorBoundary, MapPicker e RouteMap	Foco, confirmações, escape de texto e responsividade conferidos
Banco e inicialização	models.py, database.py, schema.sql, seed e configuração	Nove tabelas, 14 FKs sem órfãos, DDL/ORM e persistência conferidos
Dependências e execução	requirements, package/lockfile, Dockerfiles, Compose e exemplos	Build e npm audit passaram; Compose validado sintaticamente
Organização e documentação	README, manuais, fontes finais, geradores, .gitignore e contribuição	Índice vigente, configuração frontend e destinos de evidência organizados

2 Problemas reproduzidos e corrigidos

BUG 11 Resposta externa inválida na geocodificação

O código tratava erros HTTP e JSON inválido, mas supunha que o JSON válido sempre era uma lista de endereços com coordenadas. Uma resposta como um objeto de erro, item sem latitude ou endereço interno fora do formato podia causar exceção não tratada. Coordenadas como longitude 181 também eram aceitas e enviadas ao mapa.

O processamento agora verifica a estrutura da lista e do endereço, converte os campos dentro do tratamento de erros e valida coordenadas nos limites geográficos. Formatos inválidos resultam em GeocodingUnavailable, e o roteador retorna 503 com a orientação de marcar o local manualmente. Lista vazia e coordenadas zero continuam válidas.

Validação: `GeocoderBoundaryTestCase` em `backend/tests/test_delivery_s11.py`, com respostas simuladas, sem depender da internet. O registro anterior às correções está em `evidencias/regressoes_antes_backend.txt`; a suíte final passou nos dois bancos. Situação: corrigido; severidade média.

BUG 12 Fórmula precedida por espaços no utilitário CSV

O utilitário `csvCell` protegia prefixos de fórmula no primeiro caractere, mas preservava, sem neutralização, textos como espaço seguido de `=1+1`. A proteção foi ampliada para considerar espaços antes dos sinais de fórmula, mantendo aspas, separadores, acentos e demais textos.

Os cinco casos adicionados em `frontend/src/sprint08.test.mjs` falharam antes da correção e passaram depois. O registro está em `evidencias/regressoes_antes_frontend.txt`. Trata-se de reforço do utilitário de exportação; vários campos já são normalizados no cadastro. Não foi constatado incidente nem execução de fórmula em dados reais. Situação: corrigido; severidade média na função de exportação.

3 Melhorias de robustez e organização

ID	Melhoria	Verificação
AJ11-01	Comparador público limitado a 64 rotas, 500 paradas por rota e 12.000 no total, antes de iniciar o cálculo	Quatro testes de limites, incluindo aceitação da fronteira; contrato atualizado
AJ11-02	Remoção da senha de demonstração dos logs e do import que só era usado para imprimi-la	Revisão de main.py e dos executores; sem alteração da autenticação
AJ11-03	Exportador de API registra JSON de evidência apenas quando recebe destino explícito	Nova exportação na pasta da Sprint 11, sem sobrescrever a Sprint 10
AJ11-04	Inspeção do banco aceita destino de evidência por parâmetro	Nova inspeção no schema sprint11_testes
AJ11-05	Adição de frontend/.env.example e índice de documentação; revisão do modelo de pull request existente	Links locais e organização conferidos; nenhum segredo no exemplo frontend
AJ11-06	Manuais completos, documentação final e novos diagramas de arquitetura/classes	Conferência com telas, rotas e modelos implementados; PDF renderizado

Os limites do comparador reduzem entradas desproporcionais, mas não substituem controle de taxa ou de recursos da infraestrutura. Não houve mudança do DDL, do desempate do algoritmo ou da regra de atribuição automática.

4 Legibilidade e código desnecessário

A separação entre roteadores, serviços, regras, modelos e interface foi mantida. As correções usam os mecanismos de exceção e validação existentes, com mudanças pequenas e testes focados no comportamento. O import da senha no inicializador foi removido junto da impressão desnecessária. Não foram eliminadas funcionalidades válidas apenas para reduzir arquivos.

Geradores de documentos anteriores, PDFs cumulativos e registros de Sprint são histórico intencional. Os módulos de GPU permanecem porque fazem parte do produto, mesmo sem execução nesta máquina. Caches, builds e bases locais não são materiais da entrega e estão cobertos pelo .gitignore.

5 Verificação das informações e do repositório

O procedimento `tools/documentation/audit_repository.py` verifica arquivos locais indevidos, proteção do .gitignore, links Markdown relativos e padrões explícitos de chaves privadas/tokens em arquivos atuais de texto. O resultado completo está em `evidencias/organizacao_github.json`. Valores encontrados não são impressos pelo verificador.

Os exemplos de conexão/conta de desenvolvimento já presentes no projeto são públicos e identificados como demonstração. Os novos manuais não publicam senhas reais, tokens ou chaves de ambiente. Essa revisão não percorre todo o histórico Git nem serviços externos e não permite afirmar que qualquer formato de segredo seria detectado automaticamente.

O repositório é público; manuais e figuras ficam versionados para avaliação. A branch da Sprint é `codex/sprint-11-preparacao-entrega`. O histórico deve apresentar a correção técnica e a documentação em commits identificáveis, com pull request para revisão humana antes da integração.

6 Resultados após as correções

Verificação	Resultado
Backend PostgreSQL 17.10	159 descobertos: 158 aprovados e 1 dispensado por falta de CUDA
Backend SQLite	159 descobertos: 156 aprovados e 3 dispensados, incluindo os dois de concorrência PostgreSQL
Frontend	55 aprovados, sem falhas
Navegador	60 aprovados, nenhum reprovado e F16 não executado por falta de CUDA
Interface adicional	320, 390 e 768 px sem overflow e ações alcançáveis por rolagem interna
Build e auditoria npm	Build concluído e zero vulnerabilidades reportadas nesta execução
Banco e contrato	Nove tabelas, 14 FKs sem órfãos e 37 operações com contrato sincronizado
Backup e restauração	Banco descartável exportado e restaurado em outro banco; dados conferidos

Os sete novos testes de backend e os cinco casos adicionais de frontend já estão incluídos nesses totais. A execução que reproduz defeitos antes de corrigi-los é mantida separada; os resultados finais são os arquivos `testes_*` e `casos_de_teste.json`. Os testes foram realizados pela equipe por ferramentas, sem simular uma aceitação externa inexistente.

7 Pendências para continuidade

Permanecem pendentes a sessão guiada com usuário externo, a revalidação CUDA em máquina NVIDIA e a revisão humana do material. Não foram homologados publicação de produção, carga elevada, todos os navegadores ou combinações de escritas concorrentes. A subida do Compose não foi reexecutada, apenas sua configuração foi validada.

Como continuidade técnica, a equipe pode adotar migrações formais, CI de navegador, limites gerais de requisição e estratégia de publicação. Esses itens são registrados como próximos passos, não como entregas já implementadas.